

Fortran

de.wikibooks.org

20. Juni 2026

On the 28th of April 2012 the contents of the English as well as German Wikibooks and Wikipedia projects were licensed under Creative Commons Attribution-ShareAlike 3.0 Unported license. A URI to this license is given in the list of figures on page 473. If this document is a derived work from the contents of one of these projects and the content was still licensed by the project under this license at the time of derivation this document has to be licensed under the same, a similar or a compatible license, as stated in section 4b of the license. The list of contributors is included in chapter Contributors on page 471. The licenses GPL, LGPL and GFDL are included in chapter Licenses on page 479, since this book and/or parts of it may or may not be licensed under one or more of these licenses, and thus require inclusion of these licenses. The licenses of the figures are given in the list of figures on page 473. This PDF was generated by the $\text{\LaTeX}$ typesetting software. The $\text{\LaTeX}$ source code is included as an attachment (`source.7z.txt`) in this PDF file. To extract the source from the PDF file, you can use the `pdfdetach` tool including in the `poppler` suite, or the `http://www.pdfplabs.com/tools/pdftk-the-pdf-toolkit/` utility. Some PDF viewers may also let you save the attachment to a file. After extracting it from the PDF file you have to rename it to `source.7z`. To uncompress the resulting archive we recommend the use of `http://www.7-zip.org/`. The $\text{\LaTeX}$ source itself was generated by a program written by Dirk Hünninger, which is freely available under an open source license from `http://de.wikibooks.org/wiki/Benutzer:Dirk\_Huenniger/wb2pdf`.

Inhaltsverzeichnis

1.	Einleitung	3
1.1.	Geschichte	3
1.2.	Eigenschaften	5
1.3.	Referenzen	5
I.	Compiler	7
1.4.	Was ist ein Compiler?	9
1.5.	Aufbau eines Compilers	10
1.6.	Geschichte	11
2.	Gfortran	13
2.1.	Was ist gfortran?	13
2.2.	Installation	13
2.3.	Starten des gfortran-Compilers	13
2.4.	Dateiendungen für Quelldateien	15
2.5.	Anwendung	16
2.6.	Prüfung des Quellcodes auf Standardkonformität	16
2.7.	Historisches	17
2.8.	gfortran-Weblinks	17
3.	Der Intel Fortran Compiler	19
3.1.	Allgemeines	19
3.2.	Installation	19
3.3.	Dateiendungen für Quelldateien	19
3.4.	Anwendung	20
3.5.	Weblinks	20
4.	Der Silverfrost Compiler	21
4.1.	Weblinks	21
5.	Der AMD Fortran Compiler	23
5.1.	Weblinks	23
II.	Die Programmiersprache	25
5.2.	Was ist eine Programmiersprache?	27
5.3.	Einordnung von Fortran	27
5.4.	Ein Überblick über die historische Entwicklung von Fortran	28

III.	FORTRAN 77	31
6.	Programmaufbau	33
6.1.	Beispiel: Hallo Welt	33
6.2.	Das Zeilenformat	33
6.3.	Die Programmstruktur für das Hauptprogramm	35
6.4.	Der FORTRAN-Zeichenvorrat	35
6.5.	Symbolische Namen	36
7.	Datentypen, Variablen, Wertzuweisungen, Konstanten	39
7.1.	Datentypen	39
7.2.	Variablen	40
7.3.	Benannte Konstanten	42
7.4.	Wertzuweisung	43
8.	Felder	45
8.1.	Eindimensionale Felder	45
8.2.	Mehrdimensionale Felder	46
9.	Arithmetische Ausdrücke	49
9.1.	Arithmetische Operatoren	49
9.2.	Ergebnisdatentyp	50
10.	Logische Ausdrücke	53
10.1.	Logische Operatoren	53
10.2.	Wahrheitstafel	53
11.	Vergleichsausdrücke	55
11.1.	Vergleichsoperatoren für arithmetische Typen	55
11.2.	Zeichenkettenvergleiche	56
11.3.	Operatorenpriorität	57
12.	Stringoperationen	59
12.1.	Verknüpfungsoperator	59
12.2.	Teilketten	59
13.	Verzweigungen und Schleifen	61
13.1.	GOTO	61
13.2.	CONTINUE	61
13.3.	Bedingtes GOTO	62
13.4.	IF-Verzweigungen	62
13.5.	DO-Schleifen	64
13.6.	Weitere Schleifen	65
13.7.	Implizite Schleife	66
13.8.	STOP	66
14.	Standardfunktionen	69
14.1.	Datentypumwandlung	69
14.2.	Mathematische Funktionen	70

14.3.	Zeichenketten-Funktionen	73
15.	Unterprogramme	75
15.1.	Funktionsanweisung	75
15.2.	FUNCTION	76
15.3.	SUBROUTINE	78
15.4.	Felder als Parameter	79
15.5.	Prozeduren als Parameter	82
15.6.	COMMON	83
15.7.	ENTRY	84
15.8.	SAVE	85
15.9.	DATA	86
16.	Ein- und Ausgabe	89
16.1.	READ	89
16.2.	WRITE	91
16.3.	Formatierung	92
16.4.	Dateien	94
17.	Anhang	105
17.1.	PAUSE	105
17.2.	INCLUDE	105
17.3.	Hollerith-Konstanten	106
17.4.	Arithmetisches IF	107
17.5.	ASSIGN und Assigned GOTO	108
17.6.	MilStd 1753, DoD Supplement To American National Standard X3.9-1978	109
IV.	Fortran 95	111
18.	Programmaufbau	113
18.1.	Beispiel: Hallo Welt	113
18.2.	Das Zeilenformat	113
18.3.	Die Programmstruktur für das Hauptprogramm	115
18.4.	Der Fortran-Zeichenvorrat	115
18.5.	Symbolische Namen	115
18.6.	Reservierte Schlüsselwörter?	115
18.7.	Details zur Anordnungsreihenfolge von Anweisungen	116
19.	Datentypen, Variablen, Wertzuweisungen, Konstanten	119
19.1.	Datentypen	119
19.2.	Variablen	120
19.3.	Benannte Konstanten	122
19.4.	Wertzuweisung	122
20.	Felder	125
20.1.	Beispiel	125
20.2.	Eindimensionale Felder	125
20.3.	Mehrdimensionale Felder	127

20.4.	Feldinitialisierung	128
20.5.	Teilfelder	129
21.	Arithmetische Ausdrücke	131
21.1.	Arithmetische Operatoren	131
21.2.	Ergebnisdatentyp	131
22.	Logische Ausdrücke	133
22.1.	Logische Operatoren	133
22.2.	Wahrheitstafel	133
22.3.	Operatorenpriorität	134
23.	Vergleichsausdrücke	135
23.1.	Vergleichsoperatoren	135
23.2.	Operatorenpriorität	136
24.	Stringoperationen	137
24.1.	Verknüpfungsoperator	137
24.2.	Teilketten	137
25.	Verzweigungen und Schleifen	139
25.1.	Einleitung	139
25.2.	if-Verzweigungen	139
25.3.	Die select case-Verzweigung	144
25.4.	do-Schleifen	145
25.5.	Spezialkonstrukte für Felder	149
25.6.	Terminierung	153
26.	Datentypen höherer Genauigkeit	157
26.1.	Einfache Variante	157
26.2.	System- und compilerunabhängige Variante	158
27.	Datenverbund	161
27.1.	Einleitung	161
27.2.	Einen Datenverbund anlegen	161
27.3.	Eine Variable vom Typ "Datenverbund" anlegen	162
27.4.	Zugriff auf Einzelkomponenten eines Datenverbunds	162
27.5.	Zugriff auf einen Datenverbund als Ganzes	163
27.6.	Elemente eines Datenverbunds initialisieren	163
27.7.	Das Attribut <code>sequence</code>	165
27.8.	Die Verwendung eines Datenverbunds in separaten Programmeinheiten	166
28.	Standardfunktionen	167
28.1.	Allgemeine Hinweise	167
28.2.	Tabellenlegende	167
28.3.	Datentypfunktionen	167
28.4.	Mathematische Funktionen	168
28.5.	Stringfunktionen	168
28.6.	Feldfunktionen	168

28.7.	Zeigerfunktionen	168
28.8.	Bitfunktionen	169
28.9.	Weitere Funktionen	169
28.10.	Intrinsische Subroutinen	169
29.	Unterprogramme	171
29.1.	Das <code>function</code> Unterprogramm	171
29.2.	Das <code>subroutine</code> Unterprogramm	173
29.3.	Weitere Anweisungen für die Unterprogrammtechnik	174
29.4.	Felder als Parameter	177
29.5.	Prozeduren als Parameter	178
29.6.	Optionale Parameter	179
29.7.	Module	180
29.8.	Rekursiver Unterprogrammaufruf	181
30.	Ein- und Ausgabe	183
30.1.	<code>read</code>	183
30.2.	<code>write</code>	184
30.3.	Kürzer: <code>print</code> , <code>read</code> , <code>write</code> und Namenslisten	186
30.4.	Formatierung	187
30.5.	Dateien	190
31.	Zeiger	201
31.1.	Was sind Zeiger?	201
31.2.	Zeiger in Fortran 95	201
31.3.	Assoziationsstatus	202
31.4.	Speicherplatz dynamisch reservieren und freigeben	203
31.5.	Zeiger und Felder	204
31.6.	Verkettete Listen	206
32.	Vektoren- und Matrizenrechnung	211
33.	Systemroutinen	213
33.1.	Datum und Zeit	213
33.2.	Zufallszahlen	213
33.3.	Kommandozeilenargumente	213
34.	Von der modularen zur objektorientierten Programmierung	215
34.1.	Module im Detail	215
34.2.	Objektorientierte Programmierung	228
34.3.	Ausblick	233
34.4.	Literatur	234
35.	Offizielle Fortran 95-Erweiterungen	235
35.1.	Floating-point Exceptions	235
35.2.	Allocatable Components	235

V.	Fortran 2003	239
36.	Programmaufbau	241
36.1.	Programmaufbau und Zeilenformat	241
36.2.	Zeichenvorrat	241
36.3.	Anwendungsgebiet der neu hinzugekommenen Zeichen	241
36.4.	Der Block	242
37.	Datentypen	243
37.1.	Intrinsische Datentypen	243
37.2.	Enumeration	244
37.3.	Derived Type	245
37.4.	Felder	246
38.	Zeiger	247
38.1.	Prozedurenzeiger	247
38.2.	Zeiger und das <code>intent</code> -Attribut	249
38.3.	Zeiger und Felder	250
39.	Ein- und Ausgabe	251
39.1.	Streams	251
39.2.	Asynchrone I/O	253
39.3.	Rekursive I/O	254
39.4.	Sonstiges	254
40.	Intrinsische Funktionen und Subroutinen	255
40.1.	Neu	255
40.2.	Erweitert	260
41.	Intrinsische Module	263
41.1.	Grundlegendes	263
41.2.	Das intrinsische Modul <code>iso_fortran_env</code>	263
41.3.	Das intrinsische Modul <code>iso_c_binding</code>	265
41.4.	Die intrinsischen IEEE-Module	266
VI.	Bibliotheken	273
41.5.	Quelltextbibliotheken	275
41.6.	Statische Bibliotheken	275
41.7.	Dynamische Bibliotheken	275
41.8.	Bibliotheken in verschiedenen Programmiersprachen	276
41.9.	Bibliotheken bei verschiedenen Betriebssystemen	276
42.	Grafik und GUI	279
42.1.	DISLIN	279
43.	Allgemeines	281

44.	Beispiele	283
44.1.	Beispiel 1: Strings und Zahlen	283
44.2.	Beispiel 2: Zeichnen von Kurven und Funktionen	284
44.3.	Beispiel 3: Ein Pie-Chart	285
44.4.	Beispiel 4: Ein Meldungsfenster	286
45.	Weblinks	289
45.1.	f03gl	289
45.2.	Allgemeines	289
45.3.	Beispiel	289
45.4.	Besonderheiten	291
45.5.	Weblinks	292
45.6.	gtk-fortran	293
46.	Mathematik	297
46.1.	BLAS und ATLAS	297
47.	Allgemeines	299
48.	Installation von BLAS	301
49.	Beispiele	303
49.1.	Beispiel: Die Level 1-Funktionen <i>sdot</i> und <i>dnrm2</i>	303
49.2.	Beispiel: Die Level 2-Subroutine <i>sger</i>	303
50.	Weblinks	305
50.1.	FGSL	305
50.2.	Allgemeines	305
50.3.	Beispiele	306
50.4.	Weblinks	310
50.5.	LAPACK	310
50.6.	Allgemeines	310
50.7.	Beispiel: Lösen eines einfachen Gleichungssystems	311
50.8.	Beispiel: Inverse Matrix	312
50.9.	Weblinks	313
51.	Parallele Programmierung	315
51.1.	OpenMP	315
51.2.	Was ist OpenMP	315
51.3.	Ein einfaches Beispiel	315
51.4.	Thread-Erzeugung: Die parallel -Direktive	316
51.5.	Thread-Anzahl bestimmen	316
51.6.	Sichtbarkeit/Gültigkeit von Daten	317
51.7.	Die do -Direktive	318
51.8.	Die sections -Direktive	320
51.9.	Weitere Direktiven	321
51.10.	Kombinierte Direktiven	321
51.11.	Synchronisation	322
51.12.	Das Modul omp_lib	324

51.13.	Literatur	326
51.14.	Weblinks	326
VII. Fortran in Kombination mit anderen Programmiersprachen		327
52.	Fortran und Tcl	329
52.1.	Beispiel	329
52.2.	Alternativen	331
53.	Weblinks	333
54.	Fortran und C	335
54.1.	Interface	335
54.2.	Fortran 90/95	335
54.3.	gfortran und gcc	335
54.4.	ifx und gcc	337
54.5.	Fortran 2003	339
54.6.	Ein einfaches Beispiel	340
54.7.	Datentyp-Zuordnung	341
54.8.	„call by value“ vs. „call by reference“	342
54.9.	Globale C-Variablen	343
54.10.	Felder	343
54.11.	Übergabe von Zeichenketten	343
54.12.	Enumerationen	345
54.13.	Zeiger	346
54.14.	Datenverbund	348
54.15.	Problematische Kamelhöckerschreibweise?	353
54.16.	Sonstiges	354
55.	Fortran und Python	355
55.1.	Voraussetzungen	355
55.2.	Beispiel	355
55.3.	Weblinks	356
VIII. Fortran-„Dialekte“		357
56.	F	359
56.1.	Was ist F?	359
56.2.	Einige Unterschiede zu Fortran 90/95	359
56.3.	Weblinks	361
57.	Ratfor	363
57.1.	Allgemeines	363
57.2.	Ratfor 77	363
57.3.	Ratfor 90	364
57.4.	Weblinks	364

IX.	Sonstiges	365
58.	Cray Pointer	367
58.1.	Einleitung	367
58.2.	Grundlagen	367
58.3.	Beispiel	368
58.4.	Weblinks	369
X.	Anhang	371
59.	Eine Gegenüberstellung von grundlegenden Fortran- und C-Sprachelementen	373
60.	Programmgrundstruktur	375
60.1.	Groß-, Kleinschreibung	375
60.2.	Kommentare	375
60.3.	Das Ende einer Anweisung	375
60.4.	Hauptprogramm	375
60.5.	Unterprogramme	375
61.	Einfache Datentypen	377
62.	Variablendeklaration	379
63.	Konstanten	381
64.	Benannte Konstanten	383
65.	Operatoren	385
65.1.	Arithmetische Operatoren	385
65.2.	Vergleichsoperatoren	385
65.3.	Logische Operatoren	385
65.4.	Stringverknüpfung	385
65.5.	Bitoperatoren	386
65.6.	Weitere Operatoren	386
66.	Zusammengesetzte Datentypen	387
67.	Felder	389
68.	Zeichenketten	391
69.	Blöcke, Verzweigungen, Schleifen	393
69.1.	Block	393
69.2.	Verzweigungen	393
69.3.	Schleifen	393
69.4.	Sonstiges	393
70.	Ein-, Ausgabe	395

71.	Dateien	397
72.	Zeiger	399
73.	Anwendungsbeispiele	401
73.1.	Dreiecksberechnung	401
74.	Aufgabe	403
75.	Grundlagen	405
75.1.	Koordinatenwerte —> Richtungsvektoren	406
75.2.	Seitenlängen und Umfang	406
75.3.	Winkel	407
75.4.	Fläche	408
75.5.	Umkreis	409
75.6.	Inkreis	410
75.7.	Schwerpunkt	412
76.	Code	413
77.	Screenshots	415
77.1.	Kettenlinie	416
77.2.	Aufgabe	416
77.3.	Grundlagen	416
77.4.	Code	416
77.5.	Screenshots	417
78.	Debugger	419
79.	Der GNU Debugger	421
79.1.	gdb und gfortran	421
79.2.	gdb und Intels ifx	421
80.	Quellcodedokumentation	423
80.1.	ROBODoc	423
80.2.	Ein einfaches Beispiel	423
80.3.	Weblinks	426
80.4.	Natural Docs	426
80.5.	Ein einfaches Beispiel	427
80.6.	Kurze Erläuterung	428
80.7.	Weblinks	429
80.8.	Doxygen	429
80.9.	Ein einfaches Beispiel	429
80.10.	Weblinks	430
81.	Make & Co.	431
81.1.	Einleitung	432
81.2.	Explizite Bekanntgabe von Modulpfaden	432
81.3.	GNU Make	432

81.4.	CMake	437
81.5.	SCons	438
82.	Style Guides, Best Practices	441
XI.	Weblinks	443
82.1.	Wikis	445
82.2.	Standards	445
82.3.	Skripten, Tutorials, Bücher	445
82.4.	Compiler	446
82.5.	Debugger	446
82.6.	Amüsantes	446
82.7.	Sonstiges	447
XII.	Stichwortverzeichnis	449
82.8.	A	451
82.9.	B	453
82.10.	C	453
82.11.	D	455
82.12.	E	457
82.13.	F	458
82.14.	G	459
82.15.	H	459
82.16.	I	460
82.17.	J	462
82.18.	K	462
82.19.	L	462
82.20.	M	463
82.21.	N	464
82.22.	O	465
82.23.	P	465
82.24.	Q	465
82.25.	R	465
82.26.	S	466
82.27.	T	467
82.28.	U	468
82.29.	V	468
82.30.	W	469
82.31.	X	469
82.32.	Y	469
82.33.	Z	469
83.	Autoren	471
	Abbildungsverzeichnis	473

84.	Licenses	479
84.1.	GNU GENERAL PUBLIC LICENSE	479
84.2.	GNU Free Documentation License	480
84.3.	GNU Lesser General Public License	481

1. Einleitung

w: Fortran¹

Fortran ist eine Programmiersprache, die insbesondere für numerische Berechnungen eingesetzt wird. Der Name entstand aus *FORMula TRANslation* und wurde bis zur Version FORTRAN 77 mit Großbuchstaben geschrieben.

1.1. Geschichte

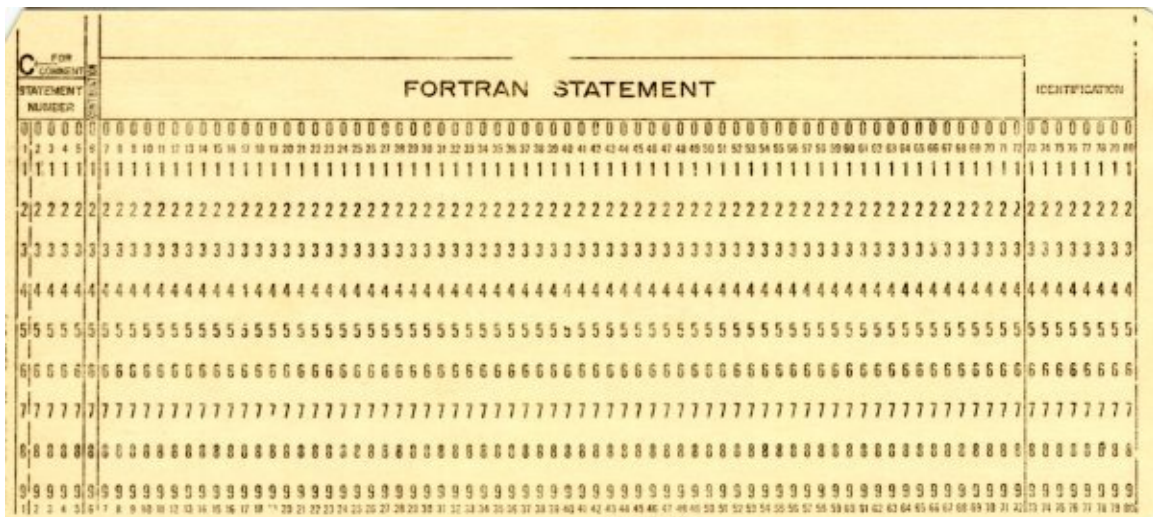


Abb. 1 Eine FORTRAN-Lochkarte aus den Anfangstagen des Computerzeitalters

Fortran gilt als die erste jemals tatsächlich realisierte höhere Programmiersprache. Sie geht zurück auf einen Vorschlag, den John W. Backus, Programmierer bei IBM, 1953 seinen Vorgesetzten unterbreitete.

Dem Entwurf der Sprache folgte die Entwicklung eines Compilers durch ein IBM-Team unter Leitung von John W. Backus. Das Projekt begann 1954 und war ursprünglich auf sechs Monate ausgelegt. Tatsächlich konnte Harlan Herrick, der Erfinder der später heftig kritisierten Goto-Anweisung, am 20. September 1954 das erste Fortran-Programm ausführen. Doch erst 1957 wurde der Compiler für marktreif befunden und mit jedem IBM 704-System ausgeliefert. Backus hatte darauf bestanden, den Compiler von Anfang an mit der Fähigkeit zu Optimierungen auszustatten: er sah voraus, dass sich Fortran nur dann durchsetzen würde,

¹ <https://de.wikipedia.org/wiki/%20Fortran>

wenn ähnliche Ausführungsgeschwindigkeiten wie mit bisherigen Assembler-Programmen erzielt würden.

1.1.1. Versionen

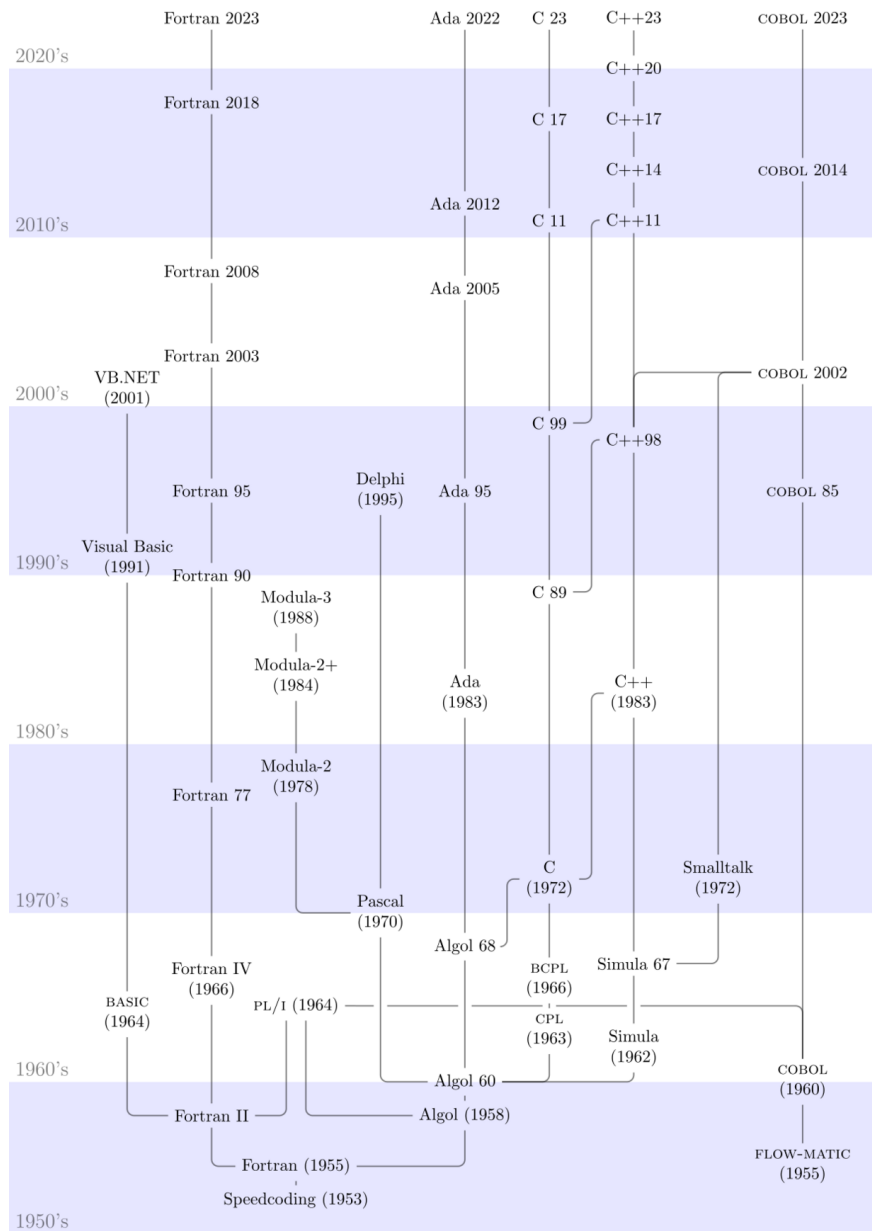


Abb. 2 Genealogie

Fortran wurde mehrmals erweitert. Viele neue Sprachelemente wurden zunächst von einem einzelnen Hersteller eingeführt und später in den internationalen Standard übernommen. Als Versionen folgten aufeinander FORTRAN I, FORTRAN II, FORTRAN IV, FORTRAN 66, FORTRAN 77, Fortran 90, Fortran 95, Fortran 2003, Fortran 2008, Fortran 2018, Fortran

2023. Ab FORTRAN 66 ist Fortran von einer internationalen Organisation standardisiert. Die Fortschreibung der Standards ist ein komplizierter Prozess, der oft wesentlich länger dauert als zunächst angestrebt: Der Nachfolger des 1978 erschienenen Standards FORTRAN 77, der als Fortran 8x bezeichnet wurde, war ursprünglich für das Jahr 1982 geplant, später dann für das Jahr 1985, und wurde schließlich unter der Bezeichnung Fortran 90 erst am 11. April 1991 als neuer Standard und Nachfolger von FORTRAN 77 angenommen.²

Im Laufe dieser Erweiterungen wurden zahlreiche Sprachelemente aus neueren Programmiersprachen übernommen. Beruhte früher Fortran-Stil noch ganz auf Goto-Anweisungen, kann man seit FORTRAN 77 uneingeschränkt strukturiert programmieren. Mit Fortran 90 wurde das aus der Lochkartenzeit stammende Zeilenformat freigegeben. Ab Fortran 90 wurden interessante Elemente eingeführt, die auch z.B. in Ada vorhanden sind, beispielsweise optionale Parameter und die Möglichkeit, Prozedurparameter nicht nur über die Position in der Parameterliste zu identifizieren, sondern über ihren Namen.

1.1.2. Varianten

Einige von Fortran abgeleitete Programmiersprachen bzw. Dialekte von Fortran sind beispielsweise Ratfor, F und HPF (High Performance Fortran). Auf Fortran aufgesetzt ist das Finite-Elemente-Programmpaket Nastran.

1.2. Eigenschaften

Fortran war und ist für numerische Berechnungen vorgesehen und optimiert. Von Anfang an hatte Fortran den Potenz-Operator **. Dieser ist in vielen anderen Hochsprachen nicht vorhanden. Weiters kennt Fortran einen Datentyp für komplexe Zahlen. Mit Fortran 90 wurden Vektor- und Matrix-Operationen standardisiert. Insbesondere für wissenschaftliche und numerische Berechnungen gibt es in FORTRAN umfangreiche Bibliotheken, die immer noch weit verbreitet sind, auch wenn eine zunehmende Menge an Routinen inzwischen nach C und C++ portiert wurde.

1.3. Referenzen

2 Vorwort von Michael Metcalf in: W. H. Press, S. A. Teukolsky, W. T. Vetterling, B. P. Flannery: *Numerical Recipes in Fortran 90*. Cambridge University Press, 1999, ISBN 0-521-57439-0.

Teil I.

Compiler

1.4. Was ist ein Compiler?

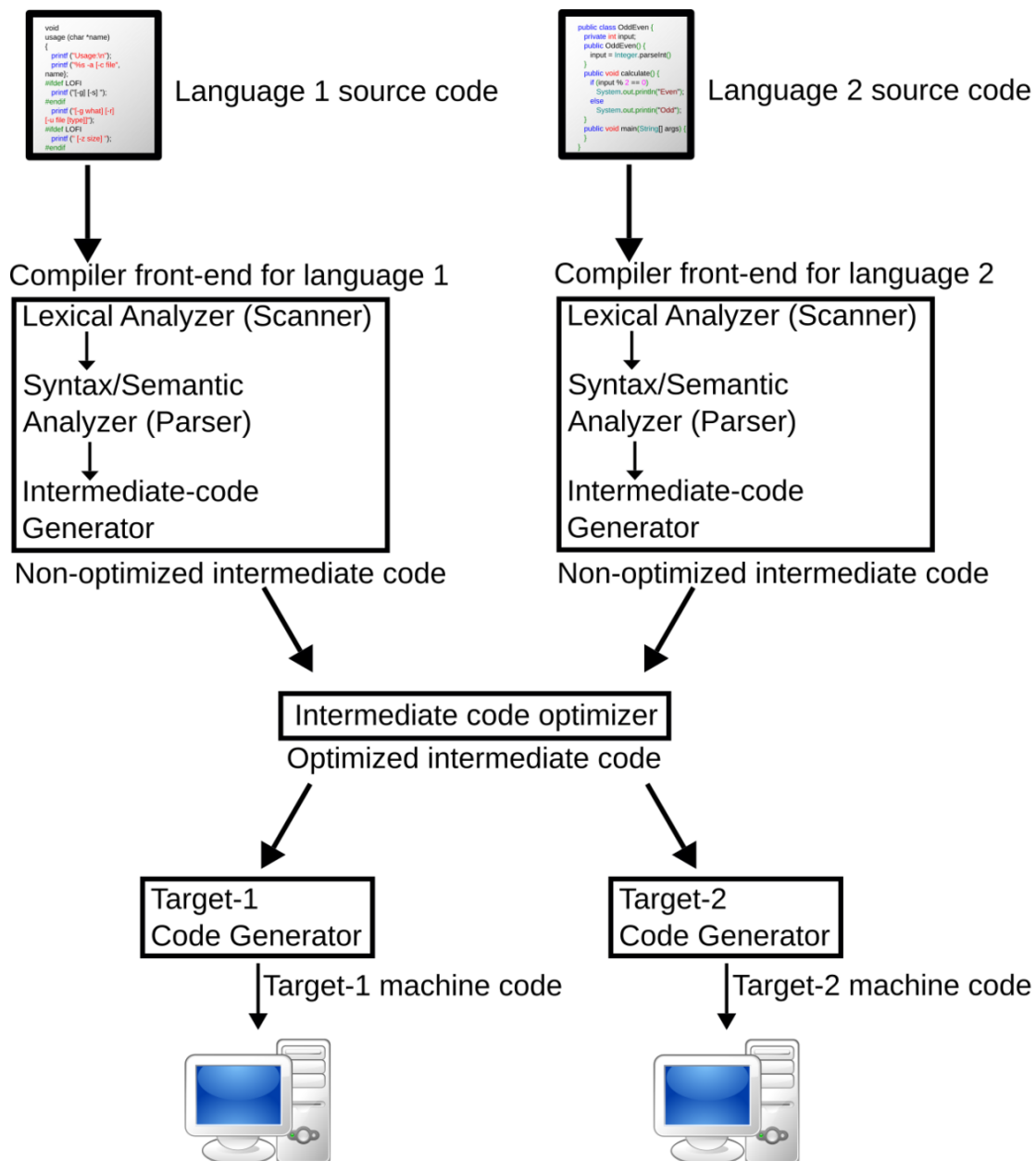


Abb. 3 Compilerschema

Ein Compiler (auch Kompilierer oder Übersetzer) ist ein Computerprogramm, das ein in einer Quellsprache geschriebenes Programm - genannt Quellprogramm - in ein semantisch äquivalentes Programm einer Zielsprache (Zielprogramm) umwandelt. Üblicherweise handelt es sich dabei um die Übersetzung eines von einem Programmierer in einer Programmiersprache geschriebenen Quelltextes in Assemblersprache, Bytecode oder Maschinensprache. Das Übersetzen eines Quellprogramms in ein Zielprogramm durch einen Compiler wird als Kompilierung oder auch als Übersetzung bezeichnet.

Die Bezeichnung *Compiler* (engl. to compile: zusammenstellen) ist eigentlich irreführend. Ursprünglich bezeichnete das Wort Compiler Programme, die Unterprogramme zusammenfügen (etwa mit heutigen Linkern vergleichbar). Dies geht an der heutigen Kernaufgabe eines Compilers vorbei.

Verwandt mit einem Compiler ist ein Interpreter, der ein Programm nicht in die Zielsprache übersetzt, sondern Schritt für Schritt direkt ausführt.

1.5. Aufbau eines Compilers

Compiler werden in verschiedene Phasen gegliedert, die jeweils verschiedene Teilaufgaben des Compilers übernehmen. Sie werden sequentiell ausgeführt. Im Wesentlichen lassen sich zwei Phasen unterscheiden: das Frontend (auch *Analysephase*), das den Quelltext analysiert und daraus einen attributierten Syntaxbaum erzeugt, sowie das Backend (auch *Synthesephase*), das daraus das Zielformat erzeugt.

1.5.1. Frontend (auch „Analysephase“)

Im Frontend wird der Code analysiert, strukturiert und auf Fehler geprüft. Es ist auch selbst wieder in Phasen gegliedert:

Lexikalische Analyse

Die lexikalische Analyse zerteilt den eingelesenen Quelltext in zusammengehörende Token verschiedener Klassen, z. B. Schlüsselwörter, Bezeichner, Zahlen und Operatoren. Dieser Teil des Compilers heißt Scanner oder Lexer.

Ein Scanner benutzt gelegentlich einen separaten Screener, um Whitespace (also Leerzeichen, Tabulatorzeichen, Zeilenenden) und Kommentare zu überspringen.

Syntaktische Analyse

Die syntaktische Analyse überprüft, ob der eingelesene Quellcode ein korrektes Programm der zu übersetzenden Quellsprache ist, d. h. der Syntax (Grammatik) der Quellsprache entspricht. Dabei wird die Eingabe in einen Syntaxbaum umgewandelt. Dieser Teil wird auch als Parser bezeichnet.

Semantische Analyse

Die semantische Analyse überprüft die statische Semantik, also über die syntaktische Analyse hinausgehende Bedingungen an das Programm. Zum Beispiel muss eine Variable in der Regel deklariert worden sein, bevor sie verwendet wird, und Zuweisungen müssen mit kompatiblen (verträglichen) Datentypen erfolgen. Dies kann mit Hilfe von Attributgrammatiken realisiert werden. Dabei werden die Knoten des vom Parser generierten Syntaxbaums mit Attributen versehen, die Informationen enthalten. So kann zum Beispiel eine Liste aller deklarierten Variablen erstellt werden. Die Ausgabe der semantischen Analyse nennt man dann dekorierten oder attributierten Syntaxbaum.

1.5.2. Backend (auch „Synthesephase“)

Das Backend erzeugt aus dem vom Frontend erstellten attribuierten Syntaxbaum den Programmcode der Zielsprache.

Zwischencodeerzeugung

Viele moderne Compiler erzeugen aus dem Syntaxbaum einen Zwischencode, der schon relativ maschinennah sein kann und führen auf diesem Zwischencode z. B. Programmoptimierungen durch. Das bietet sich besonders bei Compilern an, die mehrere Quellsprachen oder verschiedene Zielplattformen unterstützen. Hier kann der Zwischencode auch ein Austauschformat sein.

Programmoptimierung

Der Zwischencode ist Basis vieler Programmoptimierungen.

Codegenerierung

Bei der Codegenerierung wird der Programmcode der Zielsprache entweder direkt aus dem Syntaxbaum oder aus dem Zwischencode erzeugt. Falls die Zielsprache eine Maschinensprache ist, kann das Ergebnis direkt ein ausführbares Programm sein oder eine so genannte Objektdatei, die durch das Linken mit der Laufzeitbibliothek und evtl. weiteren Objektdateien zu einer Bibliothek oder einem ausführbaren Programm führt.

1.6. Geschichte

Die Geschichte des Compilerbaus wurde von den jeweils aktuellen Programmiersprachen und Hardwarearchitekturen geprägt. Der erste Compiler (A-0) wurde 1952 von der Mathematikerin Grace Hopper entwickelt. Weitere frühe Meilensteine sind 1954 der erste FORTRAN-Compiler und 1960 der erste COBOL-Compiler. Viele Architekturmerkmale heutiger Compiler wurden aber erst in den 1960er Jahren entwickelt.

2. Gfortran

2.1. Was ist gfortran?

gfortran (oder *GNU Fortran*) ist ein Fortran-Compiler-Frontend für die GNU Compiler Collection (GCC).

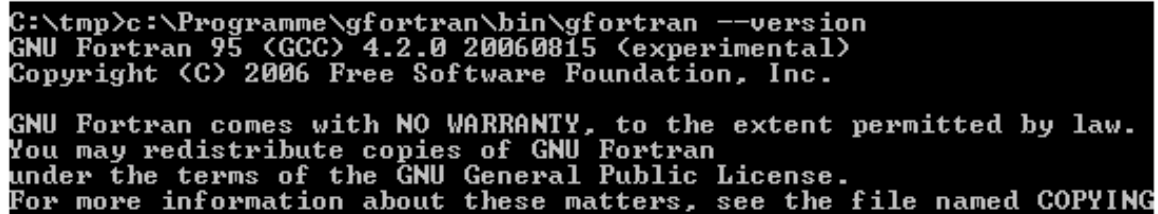
2.2. Installation

Vorcompilierte Pakete und Anleitungen zur Installation von *gfortran* finden sich gegliedert nach Betriebssystem und Prozessortyp auf: <http://gcc.gnu.org/wiki/GFortranBinaries>

2.3. Starten des gfortran-Compilers

2.3.1. MS Windows

Nach erfolgter Installation befindet sich das *gfortran*-Softwarepaket beispielsweise im Verzeichnis `C:\Programme\gfortran`. Zwecks Funktionstest wird die *Eingabeaufforderung* von MS Windows gestartet und der *gfortran*-Compiler aufgerufen:



```
C:\tmp>c:\Programme\gfortran\bin\gfortran --version
GNU Fortran 95 (GCC) 4.2.0 20060815 (experimental)
Copyright (C) 2006 Free Software Foundation, Inc.

GNU Fortran comes with NO WARRANTY, to the extent permitted by law.
You may redistribute copies of GNU Fortran
under the terms of the GNU General Public License.
For more information about these matters, see the file named COPYING
```

Abb. 4

Das Fortran-Programm für einen ersten konkreten Compilertest könnte so aussehen

```
program test
  write (*,*) 'Hallo Welt!'
end program test
```

Dieses Quellprogramm werde via Texteditor für dieses Beispiel unter dem Dateinamen `test.f90` im Verzeichnis `c:\tmp` gespeichert. Nun wird das Programm kompiliert und gebunden

```
C:\tmp>c:\Programme\gfortran\bin\gfortran test.f90
```

Abb. 5

und liegt dann unter dem Programmnamen `a.exe` als ausführbares Programm vor. Die Startanweisung für das Programm `a.exe` inklusive unmittelbar danach folgender Programmausgabe sieht so aus.

A screenshot of a Windows command prompt window with a black background and white text. The prompt shows the command 'C:\tmp>a.exe' followed by the output 'Hallo Welt!' on the next line.

```
C:\tmp>a.exe  
Hallo Welt!
```

Abb. 6

Alternativ kann `a.exe` natürlich auch konventionell mittels *Windows-Explorer* gestartet werden. In diesem speziellen Fall wäre aber bis auf ein kurzes Aufblinken des Eingabeaufforderungs-Fensters nicht viel zu sehen, da das Programm nach der Textausgabe beendet wird.

2.3.2. Linux

Für die Linux-Variante des *gfortran*-Compilers gilt prinzipiell dieselbe Vorgehensweise wie für die Windows-Variante.

Compilieren des Beispielprogrammes:

```
:/tmp> /disc1/programme/gfortran/bin/gfortran test.f90
```

Abb. 7

Starten des ausführbaren Programms `a.out` und Anzeige der Programmausgabe:

```
:/tmp> ./a.out
```

Abb. 8

```
Hallo Welt!
```

Abb. 9

Durch entsprechende Nutzung des Linux-PATH-Mechanismus (z.B. symbolischer Link in ein PATH-Verzeichnis oder Aufnahme des `./gfortran/bin/-`Verzeichnisses in den PATH) kann die Angabe des gesamten Compilerpfades bei jedem `gfortran`-Compileraufruf entfallen.

```
:/tmp> gfortran test.f90
```

Abb. 10

2.4. Dateiendungen für Quelldateien

Mit `gfortran` lassen sich Programme verschiedener Fortran-Sprachstandardversionen kompilieren. Der Fortran-Typ wird üblicherweise durch die Dateiendung der Quelldatei festgelegt.

Dateiendung	Fortran-Version
.f	FORTTRAN 77 (fixes Zeilenformat)
.f90	Fortran 90 (freies Zeilenformat)
.f95	Fortran 95 (freies Zeilenformat)
.f03	Fortran 2003 (freies Zeilenformat)
.f08	Fortran 2008 (freies Zeilenformat)
.F	FORTTRAN 77 (fixes Zeilenformat) mit Preprocessing durch cpp
.F90	Fortran 90 (freies Zeilenformat) mit Preprocessing durch cpp
.F95	Fortran 95 (freies Zeilenformat) mit Preprocessing durch cpp
.F03	Fortran 2003 (freies Zeilenformat) mit Preprocessing durch cpp
.F08	Fortran 2008 (freies Zeilenformat) mit Preprocessing durch cpp

Da andere Compiler (z.B. Intels ifx) die Endungen `.f95` und höher nicht verstehen, kann es aus Kompatibilitätsgründen sinnvoll sein auch Fortran-Programme neueren Standards generell mit der Dateiendung `.f90` zu versehen.

2.5. Anwendung

In der Anwendung gleicht *gfortran* anderen GCC-Frontends (z. B. *gcc*, *g++* oder *g77*).

- Übersetzung einer Quelldatei in die ausführbare Datei *a.out* (bzw. *a.exe* auf MS Windows-Systemen):

```
gfortran bsp.f90
```

- Übersetzung einer Quelldatei in eine Objektdatei *bsp.o*:

```
gfortran -c bsp.f90
```

- Übersetzung einer Quelldatei in die ausführbare Datei *bsp*:

```
gfortran -o bsp bsp.f90
```

- Statisches Linken:

```
gfortran -static -o bsp bsp.f90
```

- Mehrere Quelldateien kompilieren und zu einer ausführbaren Datei linkern:

```
gfortran -c bsp1.f90
```

```
gfortran -c bsp2.f90
```

```
gfortran -o bsp bsp1.o bsp2.o
```

- Mehrere Quelldateien in einer Anweisung kompilieren und zu einer ausführbaren Datei linkern:

```
gfortran -o bsp bsp1.f90 bsp2.f90
```

2.6. Prüfung des Quellcodes auf Standardkonformität

Zur Sicherstellung einer strikten Standardkonformität kann die Option `std` mit folgenden Parametern verwendet werden (Stand: *gfortran* 14.2.0 von 2024):

Parameter	Kommentar
f95	Fortran 95
f2003	Fortran 2003
f2008	Fortran 2008
f2018	Fortran 2018
f2023	Fortran 2023
gnu	Fortran mit GNU-Erweiterungen (letzter Fortran-Standard; wird nichts angegeben, so wird diese Option standardmäßig verwendet)
legacy	

- Beispiele:

```
gfortran -std=f95 bsp.f
```

```
gfortran -std=f95 -W -Wall -pedantic bsp.f
```

2.7. Historisches

Mit g95 <https://g95.sourceforge.net/> stand dem Fortran-Programmierer ein weiterer freier (im Sinne von Freier Software) Kommandozeilen-Compiler zur Verfügung, der für eine Vielzahl von Plattformen erhältlich war. g95 und gfortran basierten auf dem selben Programmcode. Im Jahr 2003 gabelten sich die Entwicklungszweige. gfortran ist nun Teil der GCC. Die Entwicklung von g95 wurde vom ursprünglichen Programmautor Andrew Vaught unabhängig davon weitergeführt. Laut <http://fortranwiki.org/fortran/show/Compilers> wird g95 seit dem Jahr 2013 nicht mehr weiterentwickelt.

2.8. gfortran-Weblinks

- GNU Fortran³
- GNU Fortran-Wiki⁴

³ <http://gcc.gnu.org/fortran/>

⁴ <http://gcc.gnu.org/wiki/GFortran>

3. Der Intel Fortran Compiler

3.1. Allgemeines

Intel Fortran Essentials ist ein Fortran-Compiler-Paket, das dzt. kostenfrei verfügbar ist.

3.2. Installation

1. Der Download kann aktuell über die Seite <https://www.intel.com/content/www/us/en/developer/tools/oneapi/hpc-toolkit-download.html?packages=fortran-essentials&fortran-essentials-os=windows&fortran-essentials-win=offline> erfolgen (Continue as a Guest (download starts immediately)).
2. Eigentliche Installation:
 - a) MS Windows: Die Installation des *Intel Fortran Compilers* erfolgt, indem man das heruntergeladene komprimierte Software-Paket im Explorer doppelklickt.
 - b) Unter OpenSUSE Leap 15.6 oder Tumbleweed: Die heruntergeladene sh-Datei z.B. in einer Konsole ausführen und den Anweisungen des Programms folgen. Erhält man später beim Compilieren einer Fortran-Datei eine Fehlermeldung, dass eine Bibliotheksdatei nicht gefunden wird, so muss man ggf. den Bibliothekspfad entsprechend hinzufügen, z.B. in der jeweiligen Konsole folgendes ausführen (oder dauerhaft in die .bashrc-Datei des Benutzers einfügen. Die vier Punkte stehen stellvertretend für den entsprechenden Pfad, der natürlich korrekt einzusetzen ist):

```
LD_LIBRARY_PATH="....:LD_LIBRARY_PATH"
export LD_LIBRARY_PATH
```

3.3. Dateiendungen für Quelldateien

Mit dem *Intel Fortran Compiler* lassen sich Programme verschiedener Fortran-Sprachstandardversionen kompilieren. Der Fortran-Typ wird üblicherweise durch die Dateieindung der Quelldatei festgelegt.

Dateieindung	Fortran-Version
.f, .for, .ftn, .i	FORTTRAN 77 (fixes Zeilenformat)
.f90, .i90	Fortran 90/95 (freies Zeilenformat)
.F, .FOR, .FTN, .FPP, .fpp	FORTTRAN 77 (fixes Zeilenformat) mit Preprocessing
.F90	Fortran 90/95 (freies Zeilenformat) mit Preprocessing

3.4. Anwendung

In der Anwendung gleicht der *Intel Fortran Compiler* dem *GNU Fortran Compiler*. Die offensichtlichsten Unterschiede sind:

- Die Intel Fortran Compiler-Software wird mittels *ifx* gestartet
- Die Intel Fortran Compiler-Software kennt die Dateierendungen *.f95* und *.F95* (oder höher) nicht

- Übersetzung einer Quelldatei in die ausführbare Datei *a.out*:

```
ifx bsp.f90
```

- Übersetzung einer Quelldatei in eine Objektdatei *bsp.o*:

```
ifx -c bsp.f90
```

- Übersetzung einer Quelldatei in die ausführbare Datei *bsp*:

```
ifx -o bsp bsp.f90
```

- Mehrere Quelldateien kompilieren und zu einer ausführbaren Datei link:

```
ifx -c bsp1.f90
```

```
ifx -c bsp2.f90
```

```
ifx -o bsp bsp1.o bsp2.o
```

- Mehrere Quelldateien in einer Anweisung kompilieren und zu einer ausführbaren Datei link:

```
ifx -o bsp bsp1.f90 bsp2.f90
```

3.5. Weblinks

- Developer Toolkits⁵

⁵ <https://www.intel.com/content/www/us/en/developer/tools/oneapi/toolkits.html#gs.11z5sq>

4. Der Silverfrost Compiler

Der Silverfrost FTN95-Compiler ist ein Fortran-Programmpaket für die Entwicklung von Fortran Programmen unter MS Windows. Es gibt eine "Personal Edition" für den Privatgebrauch und zu Evaluierungszwecke. Diese ist kostenlos erhältlich, hat aber Einschränkungen. Neben dem Compiler bietet Silverfrost auch eine IDE namens Plato und andere Gimmicks. Die uneingeschränkten Versionen (akademische bzw. kommerzielle Lizenz) kosten dann doch schon einen beträchtlichen Betrag (das geht in die Hunderte von Euros).

Gestartet werden kann der Compiler mittels Plato-IDE oder über den Befehl `ftn95.exe` im Silverfrost-Installationsverzeichnis ... \FTN95\ftn95.exe.

4.1. Weblinks

- Silverfrost Homepage⁶

⁶ <https://www.silverfrost.com/>

5. Der AMD Fortran Compiler

Das AMD-Compiler-Paket (AOCC, AMD Optimizing C/C++ and Fortran Compilers) gibt es für verschiedene Linux-Distributionen. Es basiert auf w:LLVM⁷. Für OpenSUSE Leap oder Tumbleweed ist z.B. das .rpm-Paket für die *SUSE Linux Enterprise Server distribution* geeignet. Installiert werden kann es z.B. mit dem üblichen `sudo rpm -Uvh ...` (die drei Punkte stehen für den Dateinamen des heruntergeladenen rpm-Pakets). Installiert wird es unter dem Verzeichnis `/opt/AMD/aocc-compiler-...` (auch hier muss man sich für die drei Punkte die entsprechende Version dazudenken). Gestartet wird der AMD-Fortran Compiler mittels `flang`.

5.1. Weblinks

- AMD Optimizing C/C++ and Fortran Compilers (AOCC)⁸

⁷ <https://de.wikipedia.org/wiki/LLVM>

⁸ <https://www.amd.com/en/developer/aocc.html>

Teil II.

Die Programmiersprache

5.2. Was ist eine Programmiersprache?

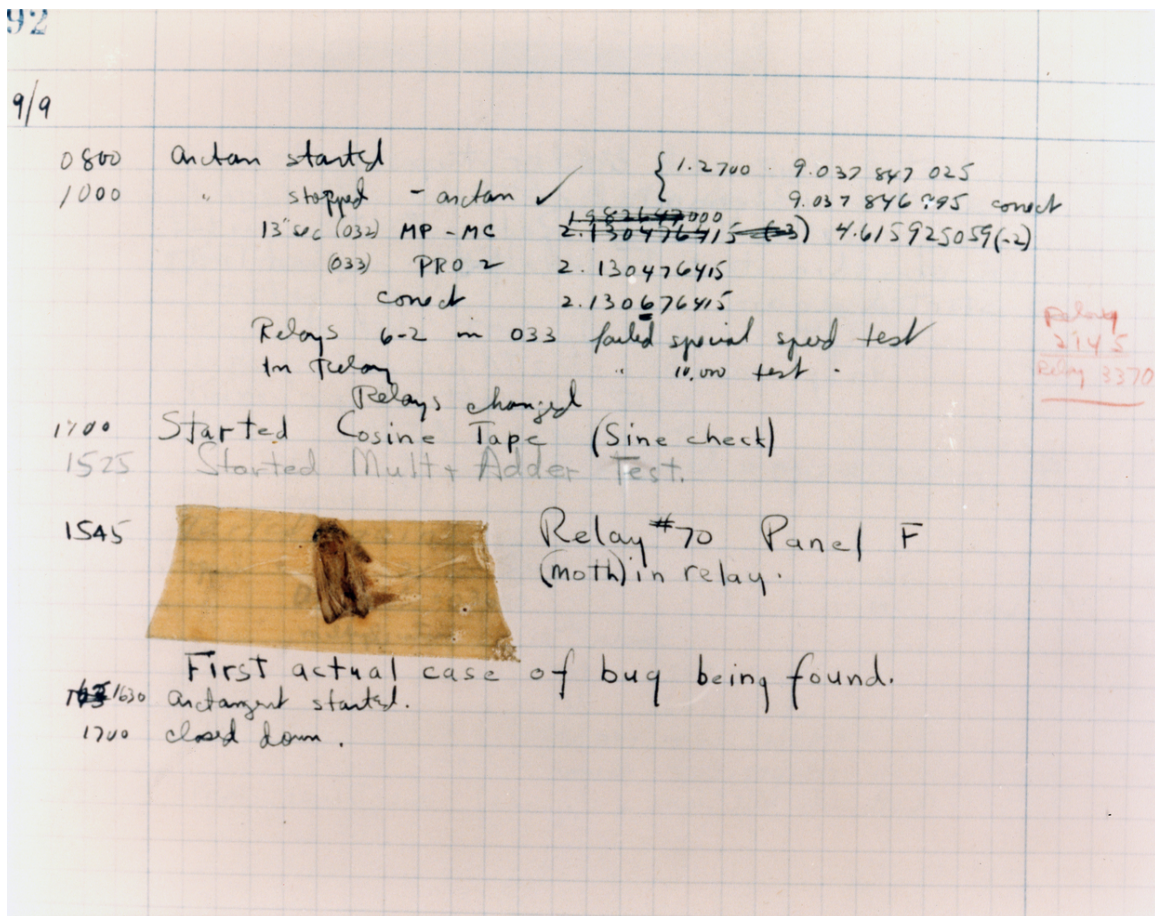


Abb. 11 Früher wurden Computer "debugged", heutzutage Computerprogramme

w:Programmiersprache⁹

In Kurzform: *Eine Programmiersprache ist eine Sprache zwecks Abfassung von Computerprogrammen.*

5.3. Einordnung von Fortran

5.3.1. Generell

- Fortran ist eine **höhere Programmiersprache** (Higher Level Language, HLL, Programmiersprache der 3. Generation)
- Fortran ist eine **prozedurale Programmiersprache** (w:Prozedurale Programmierung¹⁰)

⁹ <https://de.wikipedia.org/wiki/Programmiersprache>

¹⁰ <https://de.wikipedia.org/wiki/Prozedurale%20Programmierung>

- Fortran ist eine **imperative Programmiersprache** (w:Imperative_Programmierung¹¹)
- Fortran ist eine **objektorientierte Programmiersprache** (ab Fortran 2003, w:Objektorientierte_Programmierung¹²).
- Fortran verwendet das Konzept der **starken Typisierung**. Fortran kennt **explizite und implizite Typisierung** (w:Typisierung (Informatik)¹³)
- Fortran ist eine sehr alte Programmiersprache, die aber laufend weiterentwickelt und somit den modernen Trends immer wieder angepasst wurde und wird.

5.3.2. Popularität

Die Popularität einer Programmiersprache einigermaßen fundiert zu bestimmen ist nicht einfach. Dennoch gibt es Institutionen, die das versuchen, sei es über die Anzahl von Einträgen in Suchmaschinen, Zugriffstatistiken für Internetseiten, Nutzerbefragungen oder auch zeitliche Veränderungen bei Buchverkäufen. Hier werden zwei Statistiken angeführt:

1. Lt. TIOBE lag Fortran im Juli 2021 hinsichtlich Popularität mit einem Rating von 1,12% an 14. Stelle von insgesamt 100 gelisteten Programmiersprachen (mit einer stark steigenden Tendenz im letzten Jahr). Das war zwar weit hinter dem führenden Dreigespann Python, C++ und Java. Dennoch rangierte Fortran in dieser Statistik vor anderen bekannten Programmiersprachen, wie z.B. Ruby, Perl, Lisp oder Haskell. Im Mai 2022 rutschte Fortran auf die 30. Stelle ab und erklomm im Oktober 2024 wieder die 9. Stelle. Im März 2026 lag Fortran an 13. Stelle. (TPCI - TIOBE Programming Community Index¹⁴).
2. Lt. GitHub lag Fortran im Q3/2025 auf dem 42. Rang von 50 gelisteten Programmiersprachen (GitHub Top 50 Programming Languages Globally¹⁵).

Solche Statistiken sind natürlich mit Vorsicht zu genießen, aber in Form eines groben Richtwerts "Daumen mal Pi" können sie schon einen ersten Eindruck von Verbreitung und Popularität einer Programmiersprache geben.

Im Bereich der numerische Datenverarbeitung, insbesondere auf Hochleistungsrechnern, ist Fortran gemeinsam mit C/C++ nach wie vor führend.

5.4. Ein Überblick über die historische Entwicklung von Fortran

Simulation von Galaxien mittels Fortran-Programm¹⁶

John W. Backus (1924-2007), US-amerikanischer Informatiker, Erfinder von Fortran¹⁷

Fortran-Logo¹⁸

¹¹ https://de.wikipedia.org/wiki/Imperative_Programmierung

¹² <https://de.wikipedia.org/wiki/Objektorientierte%20Programmierung>

¹³ <https://de.wikipedia.org/wiki/Typisierung%20%28Informatik%29>

¹⁴ <https://www.tiobe.com/tiobe-index/>

¹⁵ <https://innovationgraph.github.com/global-metrics/programming-languages>

Jahr	Version	Anmerkungen
1954 bis 1957	FORTTRAN, FORTRAN I	entwickelt von einem IBM-Team unter Leitung von John W. Backus ¹⁹ , war dies die erste wirklich erfolgreiche höhere Programmiersprache
1957/58	FORTTRAN II	Inline-Assembler, Kommentare, u.a.
1958	FORTTRAN III	einige kleinere Änderungen, wurde aber nie offiziell freigegeben
1961/62	FORTTRAN IV	eine verbesserte und erweiterte Version
1966	FORTTRAN 66	die erste standardisierte Fortran-Version und gleichzeitig überhaupt die erste standardisierte höhere Programmiersprache
1978	FORTTRAN 77	DO-Schleife, IF THEN-ELSE IF-ELSE-END IF, CHARACTER-Datentyp, u.a.
1991	Fortran 90	free form style, Module, Zeiger, Datenverbund, u.v.m.
1997	Fortran 95	kleinere Änderungen
2003	Fortran 2003	OOP, C-Binding, u.a.
2010	Fortran 2008	Co-Arrays, Submodules, diverse neue Funktionen und viele andere kleinere Änderungen
2018	Fortran 2018	viele kleinere Änderungen (z.B. weitergehende Interoperabilität mit C, zusätzliche parallele Features, Konformität mit ISO/IEC/IEEE 60559:2011)
2023	Fortran 2023	viele kleinere Änderungen
	Fortran 202y	

Siehe auch:

- Fortran 202y²⁰
- Fortran 2023²¹
- Programmiersprache Fortran 2018 veröffentlicht²²
- The historical development of Fortran²³
- A Brief History of FORTRAN/Fortran²⁴
- FORTRAN I²⁵

¹⁹ <https://de.wikipedia.org/wiki/John%20Warner%20Backus>

²⁰ <https://wg5-fortran.org/f202y.html>

²¹ <https://wg5-fortran.org/f2023.html>

²² <https://www.heise.de/developer/meldung/Programmiersprache-Fortran-2018-veroeffentlicht-4241463.html>

²³ <http://www.nsc.liu.se/~boein/f77to90/a7.html>

²⁴ <http://www.ibiblio.org/pub/languages/fortran/ch1-1.html>

²⁵ <http://www.paulgraham.com/history.html>

Teil III.

FORTRAN 77

6. Programmaufbau

6.1. Beispiel: Hallo Welt

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
      PROGRAM HALLO
C
C      Das typische Hallo Welt-Programm
C
      WRITE (*,*) 'Hallo Welt!'
      END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Wie Sie an diesem Beispiel ersehen, weist FORTRAN-77-Code eine charakteristische Struktur auf. Etwas seltsam mutet an, dass die Programmanweisungen nicht am Zeilenanfang stehen, sondern mehrere Leerzeichen eingerückt sind. Das ist obligatorisch, ansonsten würde sich das Programm nicht kompilieren lassen. Die Nummerierungen oberhalb und unterhalb der Programmanweisungen (12345678901...) gehören übrigens nicht zum FORTRAN-Code, sondern sollen nur die Spaltenstruktur von FORTRAN-77-Programmen verdeutlichen und die Orientierung etwas erleichtern.

Weiter ist zu erkennen, dass das Beispiel sehr kurz und aussagekräftig ausfällt. Die Anweisung in der ersten Zeile kennzeichnet die Programmeinheit als Hauptprogramm und gibt ihr die Bezeichnung HALLO. Die nächsten drei Zeilen sind Kommentarzeilen, erkennbar am Buchstaben in der ersten Spalte der Zeile. Dann folgt die Anweisung, einen String auf die Standardausgabe zu schreiben. Und schließlich signalisiert die END-Anweisung das Programmende.

6.2. Das Zeilenformat

Normalerweise gilt, dass jede FORTRAN-Anweisung in einer eigenen Zeile steht. Eine Spezialität von FORTRAN 77 ist die Spaltenorganisation eines Programmes: das fixe Zeilenformat. Es gibt fixe Spalten (Zeichenpositionen) in denen bestimmte Inhalte stehen müssen bzw. dürfen. Diese Art der Codeanordnung rührt von den Anforderungen der Lochkarte her. Bei den damaligen Großrechenanlagen war die Eingabe von Programmen häufig nur in Form von Lochkartenstapeln möglich. FORTRAN 77 nimmt auf diese Beschränkung Rücksicht.

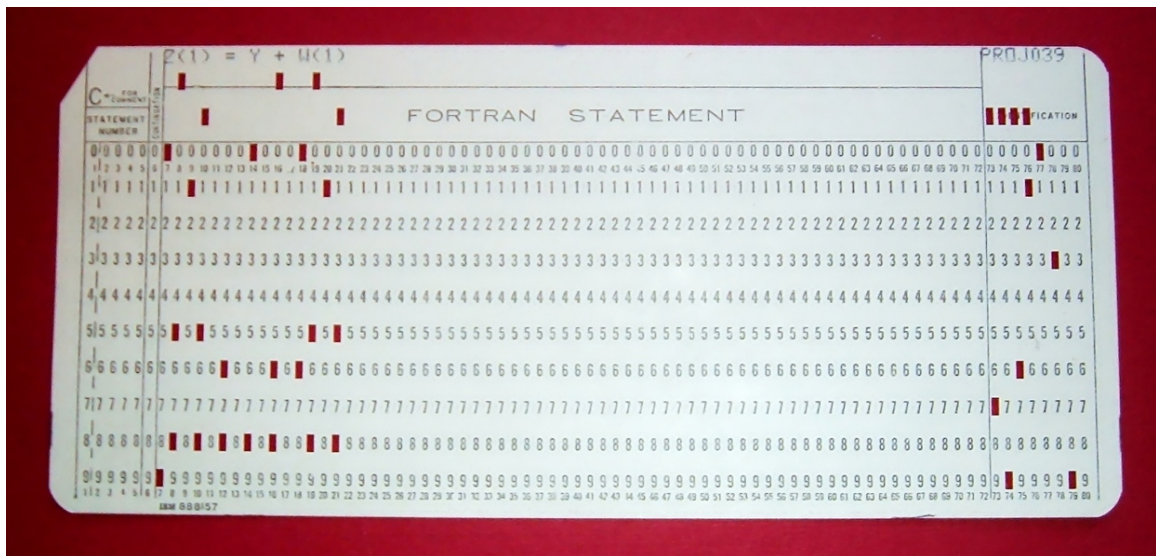


Abb. 12 Lochkarte mit Fortran-Statements

Der generelle Aufbau des fixen Zeilenformates von FORTRAN 77 ist wie folgt:

Spaltennummer																												
1	2	3	4	5	6	7	8	9	10		69	70	71	72	73	74	75	76	77	78	79	80						
Marke					Markierung Fortsetzungszelle	Anweisung										Lochkarten-Sequenznummer (Kommentar)												
Markierung Kommentarzeile																												

Abb. 13

In der nachstehenden Tabelle wird die Bedeutung der einzelnen Spalten detaillierter dargestellt.

Spalte	Inhalt	Bedeutung
1	C oder *	Kennzeichnet eine Kommentarzeile
1 bis 5	Eine Zahl 1 bis 99999	Anweisungsnummer (Marke)
6	Leerzeichen oder 0 (Null)	Beginn einer Anweisung (das ist der Normalfall)

6	sonstiges Zeichen	Fortsetzungszeile (standardmäßig sind bis zu 19 Fortsetzungszeilen erlaubt)
7 bis 72		FORTRAN-Befehl (Anweisung)
73 bis 80	beliebige Zeichen	Kommentar (ursprünglich für Lochkarten-Sequenznummern)

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP
* Leerzeilen werden übrigens wie Kommentarzeilen behandelt

* Eine 0 (Null) an der sechsten Position entspricht einem Leerzeichen,
* foerdert aber nicht gerade die Uebersichtlichkeit
  OA = 5
  B = 7
  C = A +
* Und jetzt kommt eine Fortsetzungszeile
  $B
  WRITE (*,*) C
  END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Besondere Vorsicht ist bei langen Anweisungen geboten. Alles nach der 72 Zeichenposition wird nicht mehr als Teil der Anweisung aufgefasst. Im günstigsten Fall wirft der Compiler in einem solchen Fall bei der Übersetzung einen Syntaxfehler aus.

6.3. Die Programmstruktur für das Hauptprogramm

Ein Hauptprogramm weist immer folgende Struktur auf

1. **PROGRAM** *prname*
2. Vereinbarungsteil
3. Aktionsteil
4. **END**

prname ist ein symbolischer Name für das Hauptprogramm und kann mehr oder minder willkürlich festgelegt werden. Das erste Zeichen muss immer ein Buchstabe sein. Im Vereinbarungsteil werden z. B. die Variablen deklariert. Im Aktionsteil wird dann der eigentliche Programmablauf festgelegt. **END** kennzeichnet das Programmende. Theoretisch könnte im Hauptprogramm die erste Zeile (**PROGRAM prname**) auch komplett entfallen. In älteren Programmcodes wurde das durchaus auch so gehandhabt. Allerdings leidet darunter die Übersichtlichkeit des Programmes. Die **END**-Anweisung muss auf jeden Fall angegeben werden.

6.4. Der FORTRAN-Zeichenvorrat

FORTRAN-77-Programme bestehen standardmäßig nur aus folgenden Zeichen

- Großbuchstaben: A bis Z
- Ziffern: 0 bis 9
- 13 Sonderzeichen: + - * / = () : , . ' \$ und dem Leerzeichen

Viele FORTRAN-77-Compiler akzeptieren auch Kleinbuchstaben. Zeichenkettenlitterale können natürlich alle ASCII-Zeichen beinhalten.

6.5. Symbolische Namen

Standardmäßig dürfen symbolische Namen maximal sechs Zeichen lang sein. Als erstes Zeichen muss immer ein Buchstabe (A-Z) stehen, der Rest muss alphanumerisch sein (Buchstabe oder Ziffer). „Lustigerweise“ dürfen bei FORTRAN 77 Leerzeichen auch innerhalb eines symbolischen Namens auftreten.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```

      PR O GRA M BSP
*
* gfortran, ifx etc. kompilieren diesen Code und
* bei der Programmausfuehrung wird auch das richtige
* Ergebnis angezeigt.
*
      ALPHA = 5
      BETA = 7
      GAMM a = A L PH    A + B E    TA
      WRI TE (*,*) G A MM    A
      EN D
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Besonders unangenehm kann dieses Verhalten werden, wenn in einer Zählschleife anstelle eines Kommas irrtümlicherweise ein Punkt gesetzt wird, wenn also z. B. anstelle

```
DO 10 I = 1, 3
```

fälschlicherweise

```
DO 10 I = 1. 3
```

steht. Letzteres entspricht nämlich der Zuweisung der Zahl 1.3 an die Variable D010I.

Gleiches gilt, wenn ähnlich wie in den Programmiersprachen C, C++ oder Java versucht wird, einer Variablen gleich in einer Deklarationszeile einen Wert zuzuweisen, z. B.

```
REAL A = 10.5
WRITE(*,*) A
```

Hier wird irgendein Wert ausgegeben, aber mit an Sicherheit grenzender Wahrscheinlichkeit nicht 10.500000, denn die Wertzuweisung erfolgte an die Variable `REALA` und nicht an `A`.

7. Datentypen, Variablen, Wertzuweisungen, Konstanten

Dieses Kapitel handelt von Datentypen, Variablen, Konstanten und der Wertzuweisung.

7.1. Datentypen

7.1.1. Arithmetische Datentypen

FORTRAN 77 unterscheidet standardmäßig zwischen vier arithmetischen Datentypen:

Datentyp	Kommentar	Literale (Beispiele)
INTEGER	Ganzzahlen	15, -6500, 200 000 000
REAL	Reelle Zahlen einfacher Genauigkeit	3.1415, -5.5, .7E3, 12.5E-5
DOUBLE PRECISION	Reelle Zahlen doppelter Genauigkeit	3.1415D0, -5.5D0, .7D3, 12.5D-5
COMPLEX	Komplexe Zahlen (zwei REAL-Zahlen)	(3.1415, -5.5), (1.4, 7.1E4)

7.1.2. Logischer Datentyp

Datentyp	Kommentar	Literale (alle)
LOGICAL	Logischer Datentyp (wahr oder falsch)	.TRUE., .FALSE.

Manchmal sind in alten FORTRAN-Programmen auch folgende Schreibweisen zu finden, welche jedoch nicht standardkonform sind:

- **INTEGER*4, REAL*4, LOGICAL*4** (Standardgrößen)
- **INTEGER*1, LOGICAL*1**
- **INTEGER*2, LOGICAL*2**
- **REAL*8** (entspricht DOUBLE PRECISION)
- **COMPLEX*16** (komplexe Zahlen mit zwei DOUBLE-PRECISION-Elementen)

Die Zahlen geben den Speicherplatzbedarf in Byte an.

7.1.3. Zeichenketten

Datentyp	Kommentar	Beispiel (Konstante)
CHARACTER* <i>n</i>	Zeichenkette (String) mit einer Länge von <i>n</i> Zeichen	'Hallo, Welt!'
CHARACTER	Zeichenkette mit einer Länge von einem Zeichen (entspricht CHARACTER*1)	'A'

Beachte: Im Gegensatz zu vielen anderen Programmiersprachen werden Zeichenketten in FORTRAN 77 nicht in Anführungszeichen eingeschlossen, sondern in Apostrophe.

Tritt in einem String ein Apostroph auf, so muss dieses verdoppelt werden, z. B.

```
'Wie geht''s?'
```

Beispiel:

```
CHARACTER*5 STR
STR = 'Hallo'
```

Alternative Schreibweise:

```
CHARACTER STR*5
STR = 'Hallo'
```

7.2. Variablen

Eine Variable ist charakterisiert durch einen

- symbolischen Namen
- Datentyp
- Wert
- Speicherplatz

Beim Programmstart hat eine Variable keinen definierten Wert. Eine Variable kann ihren Datentyp auf zwei Arten erhalten, durch implizite oder explizite Typanweisung.

7.2.1. Implizite Typanweisung

Bei der impliziten Typanweisung bestimmt der Anfangsbuchstabe des Variablenbezeichners den Datentyp.

Anfangsbuchstabe der Variablen
I, J, K, L, M oder N
alle restliche Buchstaben

Impliziter Datentyp
INTEGER
REAL

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

  B1 = 8.9
  C1 = 3.
  I1 = B1/C1

  WRITE (*,*) I1
C  Das Ergebnis ist 2, da I1 implizit als INTEGER definiert ist
  END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Die Standardzuordnung der Anfangsbuchstaben kann durch das Schlüsselwort `IMPLICIT` auch geändert werden.

Es gibt noch ein weiteres Problem dieser impliziten Datentypzuordnung. Durch Tippfehler können unbeabsichtigt neue Variablen entstehen. Entschärft werden kann diese Tatsache durch folgende Festlegung im Vereinbarungsteil des Programms:

```
IMPLICIT LOGICAL (A-Z)
```

Dadurch werden alle Variablen mit implizit festgelegtem Datentyp automatisch zu Variablen mit logischem Datentyp. In vielen Fällen konnten so Tippfehler bei Variablennamen schnell eingegrenzt werden.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

  IMPLICIT LOGICAL (A-Z)
  REAL REE

  REE = 5.8

C  Tippfehler: REA anstatt REE
  WRITE (*,*) REA + 2.1

  END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Der Compilierungsversuch mit *gfortran* endet mit einer Fehlermeldung. Das sieht so aus

```
In file test.f:9
```

```
WRITE (*,*) REA + 2.1
1
Error: Operands of binary numeric operator '+' at (1) are LOGICAL(4)/REAL(4)
```

Doch Intels *ifx* und AMDs AOCC (*flang*) zeigen die Grenzen der Sinnhaftigkeit der IMPLICIT-Anweisung in der dargestellten Art und Weise beim heutigen Einsatz von FORTRAN-77-Code auf. Diese Compiler akzeptieren den Beispielcode warnhinweislos und liefern bei Programmausführung den Wert 2.1. Wirklich Abhilfe schafft also erst die IMPLICIT NONE-Anweisung. Diese legt eindeutig fest, dass ausschließlich die explizite Datentypfestlegung Verwendung finden soll. Allerdings ist IMPLICIT NONE erst ab Fortran-90/95 Standard.

7.2.2. Explizite Typanweisung

Durch die Vorgabe von

datentyp variablenbezeichner

im Vereinbarungsteil des Programms wird der Datentyp einer Variablen explizit festgelegt. Die explizite Typanweisung hat gegenüber der impliziten Typanweisung Vorrang.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

IMPLICIT LOGICAL (A-Z)
REAL B
REAL C
REAL I

B = 8.9
C = 3.
I = B/C

WRITE (*,*) I
C Das Ergebnis ist 2.966666
END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

7.3. Benannte Konstanten

Benannte Konstanten können durch das PARAMETER-Schlüsselwort festgelegt werden.

```
CHARACTER*5 STR
PARAMETER (PI=3.1415, PIFAK=PI/2., STR='Hallo')
```

Der zugewiesene Wert kann eine Konstante (Literal) oder eine schon definierte benannte Konstante sein. Der Datentyp muss vorher vereinbart werden oder ist implizit bekannt.

Für Zeichenketten ist im Zusammenwirken mit `PARAMETER` auch eine `*`-Schreibweise möglich. Dies erspart die explizite Angabe der Stringlänge.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CHARACTER*(*) A
PARAMETER (A = 'Jetzt wird auch die Stringlaenge festgelegt')

WRITE (*,*) A
C  Ausgabe: Jetzt wird auch die Stringlaenge festgelegt

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

7.4. Wertzuweisung

Wertzuweisungen haben wir schon kennengelernt:

`variable = ausdruck`

Beispiel:

```
K = 1
K = K + 2
```

Die Wertzuweisung an eine Variable ist, wie am vorigen Beispiel und auch nachfolgend zu sehen, nicht zu verwechseln mit einer mathematischen Gleichung. Der Ausdruck

```
K + 2 = 5
```

wäre zwar mathematisch korrekt. Als Wertzuweisung in einem FORTRAN-Programm ist dies aber keine gültige Formulierung. `K + 2` ist kein zulässiger Ausdruck auf der linken Seite des Zuweisungsoperators (L-Wert).

Beachte: In FORTRAN 77 ist auch keine Kette von Wertzuweisungen möglich. Der folgende Ausdruck ist in FORTRAN 77 **nicht** erlaubt und liefert eine Fehlermeldung.

```
I = J = K = 1.5
C Fehler!
```


8. Felder

Bei allem, was mehr oder weniger wie ein Vektor, eine Matrix oder eine sonstige Aneinanderreihung von gleichartigen Elementen aussieht, kann der Einsatz von Feldern (Arrays) sinnvoll sein.

8.1. Eindimensionale Felder

Für die Deklaration von eindimensionalen Feldern gibt es mehrere Möglichkeiten. Die Feldgrenzen müssen konstante Werte sein. Die Varianten werden nun anhand von Beispielen gezeigt.

8.1.1. Variante 1: Einfach

```
REAL ARR(10)
```

Beachte: Der Feldindex läuft hier von 1 bis 10 und nicht von 0 bis 9, wie es bei vielen modernen Hochsprachen der Fall ist.

8.1.2. Variante 2: Das DIMENSION-Schlüsselwort

```
REAL ARR  
DIMENSION ARR(10)
```

8.1.3. Variante 3: Verwendung von benannten Konstanten

```
INTEGER MAXIND  
PARAMETER (MAXIND=10)  
REAL ARR(MAXIND)
```

Hier erfolgt die Festlegung der Feldgröße über eine benannte Konstante.

8.1.4. Variante 4: Explizite Angabe der Indexgrenzen

```
REAL ARR(0:9)
```

Hier wird Unter- und Obergrenze explizit angegeben. Der Index läuft nun von 0 bis 9. Auch negative Werte für die Indizes sind möglich, z. B.

```
REAL ARR(-4:5)
```

8.1.5. Beispiel

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```

PROGRAM BSP

INTEGER MAXIND
PARAMETER (MAXIND=10)
REAL ARR(MAXIND)
C ACHTUNG! Array startet mit dem Index 1
C ARR(0) waere ein Fehler!
  ARR(1) = 1.5
  ARR(2) = 2.5
  ARR(10) = 10.5

  WRITE (*,*) ARR(1)
C 1.5 wird ausgegeben

  WRITE (*,*) ARR(10)
C 10.5 wird ausgegeben

END

```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Ein

```
WRITE (*,*) ARR
```

listet den gesamten Feldinhalt.

```

1.500000      2.500000      0.000000      3.9876625E-34    0.000000
3.9902670E-34 -2.7682916E-05
-2.7269103E-05 -2.9040850E-05    10.50000

```

Im Beispielsfall wurden die Feldelemente `ARR(3)` bis `ARR(9)` nicht explizit vorbelegt. Sie sind deshalb undefinierten Inhalts und können bei jedem Programmaufruf andere Werte annehmen.

8.2. Mehrdimensionale Felder

Für mehrdimensionale Felder gelten die gleichen Varianten wie für eindimensionale Felder. Standardmäßig kann ein Feld bis zu sieben Dimensionen besitzen. Die Speicherreihenfolge ist spaltenorientiert. Das bedeutet, der erste Index variiert am schnellsten:

$$a_{11}, a_{21}, \dots, a_{(n-1)m}, a_{nm}$$

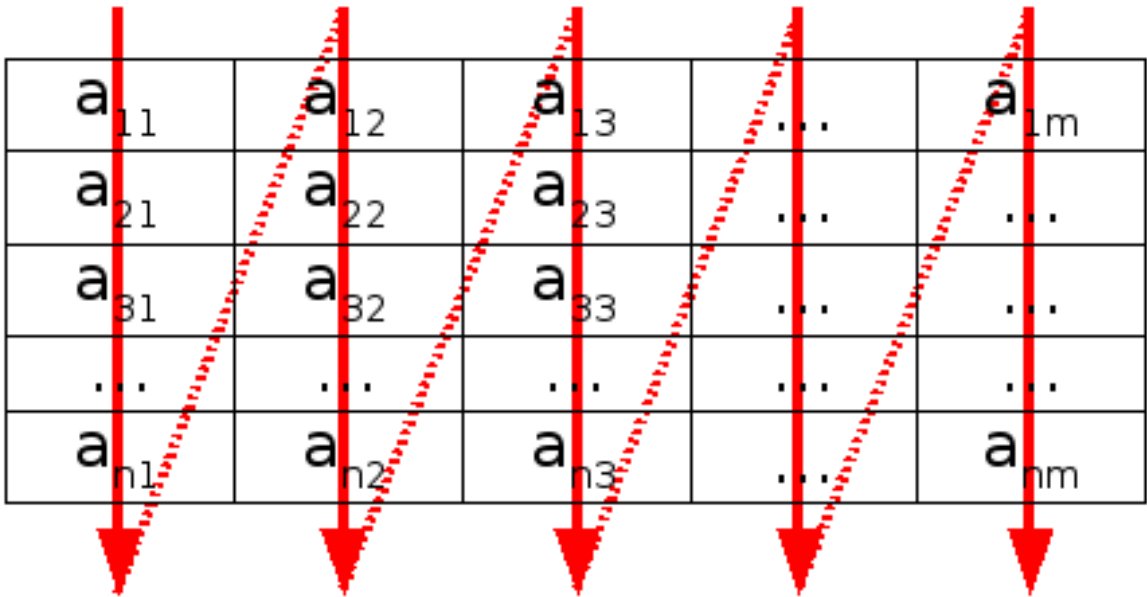


Abb. 14

8.2.1. Beispiel: Ein 2-dimensionales Feld

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*10 ARR(0:9, 2:5)

ARR(0, 2) = 'Hallo'
ARR(1, 2) = 'Welt'
C ...
ARR(9, 5) = 'Universum'

WRITE (*,*) ARR(0, 2)
C Hallo

WRITE (*,*) ARR(9, 5)
C Universum

END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

8.2.2. Beispiel: Spaltenorientierte Speicherreihenfolge

Die 3×3 -Matrix $A = \begin{pmatrix} 1 & -5 & 0 \\ 40 & 3 & -2 \\ -1 & 9 & 65 \end{pmatrix}$ soll in ein Fortran-Programm eingelesen und wieder komplett ausgegeben werden. Zusätzlich soll auch der Wert des Feldelementes a_{23} (2. Zeile, 3. Spalte, Wert = -2) separat ausgegeben werden.

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER ARR(3,3)

C Feldelemente einlesen
WRITE (*,*) 'Werte (spaltenorientierte Eingabe):'
READ (*,*) ARR

C Komplettes Feld ausgeben
WRITE (*,*) 'Gesamtfeld = ' , ARR

C a23 ausgeben
WRITE (*,*) 'a23 = ', ARR(2,3)

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Ein-/Ausgabe:

```
Werte (spaltenorientierte Eingabe):
1
40
-1
-5
3
9
0
-2
65
Gesamtfeld =      1      40      -1      -5      3
              9      0
              -2     65
a23 =          -2
```

9. Arithmetische Ausdrücke

9.1. Arithmetische Operatoren

FORTRAN 77 kennt folgende arithmetische Operatoren

Operator	Kommentar	Mathematische Entsprechung
$A + B$	Addition	$A + B$
$A - B$	Subtraktion	$A - B$
$A * B$	Multiplikation	AB
A / B	Division	$\frac{A}{B}$
$A ** B$	Exponentiation	A^B

Mit dem Exponentiationsoperator (Potenzierung) war und ist FORTRAN 77 anderen Programmiersprachen einen Schritt voraus. Andererseits kennt FORTRAN 77 den aus vielen anderen Programmiersprachen bekannten Modulo-Operator nicht. Als Überkompensation gibt es für diesen Zweck die `MOD()`²⁶-Funktion sowohl für Ganzzahlen, wie auch für Fließkommazahlen.

9.1.1. Operatorenpriorität

Die Priorität der arithmetischen Operatoren entspricht den mathematischen Gesetzmäßigkeiten.

- Klammerung vor allem anderen, z. B. $(A+B)*C \Leftrightarrow A*C+B*C$
- Exponentiation vor Punktrechnung, z. B. $A*B**C \Leftrightarrow A*(B**C)$
- Punktrechnung vor Strichrechnung, z. B. $A+B*C \Leftrightarrow A+(B*C)$

9.1.2. Berechnungsfolge bei gleicher Priorität

- Klammerung, Punktrechnung und Strichrechnung: $\rightarrow$
Beispiel: $A*B/C*D \Leftrightarrow ((A*B)/C)*D$
- Exponentiation: $\leftarrow$
Beispiel: $A**B**C \Leftrightarrow A**(B**C)$

Außerdem ist zu beachten, dass niemals zwei Operatoren direkt aufeinander folgen dürfen.

Beispiel: Der Ausdruck $1.5**-1$ ist in FORTRAN 77 falsch und führt zu einer Fehlermeldung. Richtig ist $1.5**(-1)$

²⁶ Kapitel 14.2.5 auf Seite 71

9.2. Ergebnisdatentyp

9.2.1. Operanden gleichen Datentyps

Bei Operanden gleichen Datentyps erhält das Ergebnis den Datentyp der Operanden.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

REAL A

A = 3/2
C 3 ist ein INTEGER und 2 ist auch ein INTEGER,
C daher muss das Ergebnis auch ein INTEGER sein, also 1.
C Die Zuweisung an die REAL-Variable A stellt das
C Ergebnis nicht mehr richtig.

WRITE (*,*) A
C Ausgabe: 1.00000

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

9.2.2. Implizite Typumwandlung bei Operanden gemischten Datentyps

Weisen die Operanden unterschiedliche Datentypen auf, so wird bei jeder Operation, falls nötig, das Ergebnis dem höherwertigen Datentyp angepasst.

INTEGER → REAL → DOUBLE PRECISION → COMPLEX

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

REAL A

A = 3/2.
C 2. ist ein REAL. Jetzt stimmt das Ergebnis.

WRITE (*,*) A
C Ausgabe: 1.500000

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

9.2.3. Explizite Typumwandlung

FORTRAN 77 besitzt auch Funktionen zur expliziten Umwandlung des Datentyps. Diese werden im Kapitel Standardfunktionen²⁷ näher beschrieben.

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

REAL R
COMPLEX C

R = 2
C = CMPLX(R)

WRITE (*,*) C
C Ausgabe: ( 2.000000 , 0.000000 )

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

²⁷ Kapitel 14.1 auf Seite 69

10. Logische Ausdrücke

Logische Ausdrücke können zwei Zustände annehmen, *wahr* oder *falsch*. Diese werden in FORTRAN 77 durch die Literale **.TRUE.** oder **.FALSE.** dargestellt.

10.1. Logische Operatoren

Folgende Tabelle enthält alle in FORTRAN 77 bekannte logische Operatoren. Sie sind in der Reihenfolge ihrer Prioritäten absteigend geordnet.

Operator	Kommentar	Mathematische Entsprechung
.NOT. A	logisches NICHT	$\neg A$
A .AND. B	logisches UND	$A \wedge B$
A .OR. B	logisches ODER	$A \vee B$
A .EQV. B	logische Äquivalenz (XNOR)	$\overline{A \oplus B}$
A .NEQV. B	logische Antivalenz (XOR)	$A \oplus B$

10.2. Wahrheitstafel

A	B	.NOT. A	A .AND. B	A .OR. B	A .EQV. B	A .NE- QV. B
.TRUE.	.TRUE.	.FALSE.	.TRUE.	.TRUE.	.TRUE.	.FALSE.
.TRUE.	.FALSE.	.FALSE.	.FALSE.	.TRUE.	.FALSE.	.TRUE.
.FALSE.	.TRUE.	.TRUE.	.FALSE.	.TRUE.	.FALSE.	.TRUE.
.FALSE.	.FALSE.	.TRUE.	.FALSE.	.FALSE.	.TRUE.	.FALSE.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

LOGICAL L

L = .TRUE.

WRITE (*,*) .NOT. L
C Ausgabe: F

END
```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

Beispiel:

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

LOGICAL A, B

A = .TRUE.
B = .FALSE.

WRITE (*,*) A .NEQV. B
C  Ausgabe: T

END
```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

11. Vergleichsausdrücke

Zum Vergleichen zweier arithmetischer Ausdrücke oder von Strings gibt es in FORTRAN 77 Vergleichsoperatoren. Das Ergebnis eines Vergleichs ist immer logischer Wert (.TRUE. oder .FALSE.).

11.1. Vergleichsoperatoren für arithmetische Typen

Operator	Kommentar	Mathematische Entsprechung
A .LT. B	less than (kleiner als)	$A < B$
A .LE. B	less equal (kleiner gleich)	$A \leq B$
A .GT. B	greater than (größer als)	$A > B$
A .GE. B	greater equal (größer gleich)	$A \geq B$
A .EQ. B	equal (gleich)	$A = B$
A .NE. B	not equal (ungleich)	$A \neq B$

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER A, B

A = 5
B = 6

WRITE (*,*) A .LT. B
C Ausgabe: T

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*5 A, B

A = 'Halli'
B = 'Hallo'
```



```

        WRITE (*,*) A .LT. B
C  Ausgabe: T

        END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Beim Rechnen mit Fließkommazahlen (Datentypen: `REAL`, `DOUBLE PRECISION`, `COMPLEX`) sind die systemimmanenten Rechenungenauigkeiten zu beachten. Aus diesem Grund sollten Fließkommazahlen nicht auf strikte (Un)Gleichheit geprüft werden, sondern Vergleiche sollten einen kleinen Toleranzbereich aufweisen: $x \pm \epsilon = y$.

Beispiel (hier mit $\epsilon = 0,00001$ und $y = 2$):

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

        X = LOG(A)

C  Statt...
        IF (X .EQ. 2)

C  besser
        IF (ABS(X - 2) .LT. .00001)

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

11.2. Zeichenkettenvergleiche

Das Ergebnis eines Vergleichs von Zeichenketten mittels Vergleichsoperatoren ist teilweise systemabhängig. Ausnahmen sind `.EQ.` und `.NE.` Systemunabhängige Resultate sind durch Verwendung der entsprechenden lexikalischen Standardfunktionen²⁸ erhältlich. Dort wird immer die Reihenfolge im ASCII-Zeichensatz verwendet.

Beispiel:

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

PROGRAM BSP

IMPLICIT LOGICAL(A-Z)

CHARACTER*15 A, B

A = 'Hallö'
B = 'hallo'

```

²⁸ Kapitel 14.3.3 auf Seite 73

```

        WRITE (*,*) 'A gleich B? ', A .EQ. B
        WRITE (*,*) 'A kleiner als B (Operator)? ', A .LT. B
        WRITE (*,*) 'A kleiner als B (Funktion)? ', LLT (A, B)
C  Ausgabe:
C  A gleich B?  F
C  A kleiner als B (Operator)?  T
C  A kleiner als B (Funktion)?  T

        END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

11.3. Operatorenpriorität

1. Klammerung
2. Arithmetische Operatoren
3. Vergleichsoperatoren
4. Logische Operatoren
5. Zuweisungsoperator

Beispiel:

```

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```

```

PROGRAM BSP

IMPLICIT LOGICAL(A-Z)

REAL A, B, C
LOGICAL X, RES

A = 5.5
B = -1.2
C = 8.6
X = .FALSE.

RES = X .AND. A - B .GT. C .OR. A .LE. C
C entspricht infolge Op.priorität:
C  RES = ((X .AND. ((A-B) .GT. C)) .OR. (A .LE. C))

        WRITE (*,*) RES
C Ausgabe: T

        END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Vergleichsoperatoren und logische Operatoren finden in erster Linie bei Verzweigungen und Schleifenbedingungen²⁹ Verwendung.

²⁹ Kapitel 13 auf Seite 61

12. Stringoperationen

FORTRAN 77 bietet vergleichsweise komfortable Operatoren zur Behandlung von Zeichenketten.

12.1. Verknüpfungsoperator

Operator	Kommentar
$A // B$	Operator zum Verknüpfen von Zeichenketten

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CHARACTER*4 A, B*10

A='How '
B='do you do.'

WRITE (*,*) A // B
C Ausgabe: How do you do.

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

12.2. Teilketten

Ein String ist ein CHARACTER-Feld. Auch Teilketten einer solchen Zeichenkette können adressiert werden.

Prinzip	Beschreibung
$stringname (anfang:ende)$	von <i>anfang</i> bis <i>ende</i>
$stringname (:ende)$	vom ersten Zeichen bis <i>ende</i>
$stringname (anfang:)$	von <i>anfang</i> bis zum letzten Zeichen
$stringname (index:index)$	genau ein Zeichen an der Position <i>index</i>

Dabei muss *anfang* stets größer oder gleich Eins sein. *ende* darf nicht größer als die Länge der Zeichenkette sein. *index* muss sich stets zwischen Eins und der Länge der Zeichenkette befinden.

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*10 A

A='Hallo Welt'

WRITE (*,*) A(2:4)
C  Ausgabe: all

WRITE (*,*) A(5:)
C  Ausgabe: o Welt

WRITE (*,*) A(:3)
C  Ausgabe: Hal

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*10 A

A='Hallo Welt'
A(7:) = 'XYZ'

WRITE (*,*) A
C  Ausgabe: Hallo XYZ

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Neben diesen Möglichkeiten sind in FORTRAN 77 auch einige Standardfunktionen für das Hantieren mit Zeichenketten vorgesehen. Diese sind im Kapitel Standardfunktionen³⁰ beschrieben.

30 Kapitel 14.3 auf Seite 73

13. Verzweigungen und Schleifen

13.1. GOTO

GOTO bewirkt einen Sprung zu einer bestimmten Anweisungsnummer.

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

      GOTO 100
      WRITE (*,*) 'Hallo'
100   WRITE (*,*) 'Welt'
C   Ausgabe: Welt

      END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.2. CONTINUE

CONTINUE ermöglicht bei Anweisungsnummern eine „leere“ Anweisung.

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

      GOTO 100
      WRITE (*,*) 'Hallo'
100   CONTINUE
C   keine Ausgabe

      END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.3. Bedingtes GOTO

Beim bedingten GOTO ist in Abhängigkeit von einer Integer-Variablen der Sprung zu einer bestimmten Anweisungsnummer möglich.

Beispiel: Eine „Switch“-Verzweigung

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER I

I=2
GOTO (100, 200, 300), I
100 WRITE (*,*) 'Hallo 1'
    GOTO 1000
200 WRITE (*,*) 'Hallo 2'
    GOTO 1000
300 WRITE (*,*) 'Hallo 3'
1000 CONTINUE
C  Ausgabe: Hallo 2

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.4. IF-Verzweigungen

13.4.1. Der IF-Einzeiler

IF (logischer ausdruck) anweisung

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER I

I=2
IF (I .EQ. 2) WRITE (*,*) 'Hallo'
C  Ausgabe: Hallo

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.4.2. IF-THEN

```
IF (logischer ausdruck) THEN
  anweisungsblock
END IF
```

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP
```

```
INTEGER I
```

```
I=2
```

```
IF (I .EQ. 2) THEN
```

```
  WRITE (*,*) 'Hallo'
```

```
END IF
```

```
C Ausgabe: Hallo
```

```
END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
0.|1.2.3.4.5.6.7|.8
```

13.4.3. IF-THEN-ELSE

```
IF (logischer ausdruck) THEN
  if-anweisungsblock
ELSE
  else-anweisungsblock
END IF
```

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP
```

```
INTEGER I
```

```
I=333
```

```
IF (I .GE. 444) THEN
```

```
  WRITE (*,*) 'Hallo'
```

```
ELSE
```

```
  WRITE (*,*) 'Hola'
```

```
END IF
```

```
C Ausgabe: Hola
```

```
END
```



```
123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

13.4.4. ELSE-IF

```
IF (logischer ausdruck 1) THEN
  if-anweisungsblock 1
ELSE IF (logischerAusdruck 2) THEN
  if-anweisungsblock 2
ELSE IF (logischerAusdruck n) THEN
  if-anweisungsblock n
ELSE
  else-anweisungsblock
END IF
```

```
0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

INTEGER I

I = 2
IF (I .EQ. 1) THEN
  WRITE (*,*) 'I ist eins'
ELSE IF (I .EQ. 2) THEN
  WRITE (*,*) 'I ist zwei'
ELSE
  WRITE (*,*) 'Ich weiß nicht was I ist'
END IF
C Ausgabe: I ist zwei

END
```

```
123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

13.5. DO-Schleifen

Die DO-Schleife (Zählschleife) ist die einzige Schleife die FORTRAN 77 standardmäßig kennt.

```
DO nr zählvariable = startwert, endwert [, schrittweite]
  anweisungsblock
nr CONTINUE
```

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890
```

```

PROGRAM BSP

INTEGER I

DO 100 I = 1, 10
    WRITE (*,*) I
100 CONTINUE
C Zeilenweise Ausgabe der Zahlen 1 bis 10
END

```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.6. Weitere Schleifen

Da FORTRAN 77 keine weiteren Schleifen kennt, müssen diese mit Hilfe einer IF-Verzweigung und einem GOTO-Befehl nachgebildet werden. Prominente Beispiele aus anderen Programmiersprachen sind die While-Schleife und die Do-While-Schleife.

13.6.1. While-Schleife (kopfgesteuerte Schleife)

Beispiel:

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```

PROGRAM BSP

INTEGER I

I=0

10 IF(I .LT. 5) THEN
    WRITE (*,*) I
    I = I + 1
    GOTO 10
END IF
C Die Zahlen 0 bis 4 werden ausgegeben

END

```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

13.6.2. Do-While-Schleife (fußgesteuerte Schleife)

Beispiel:

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```

PROGRAM BSP

```

```
INTEGER I

I=6

10  CONTINUE
    WRITE (*,*) I
    I = I + 1
    IF (I .LT. 5) GOTO 10
C   Die Zahl 6 wird ausgegeben

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

13.7. Implizite Schleife

Bei Eingabe oder Ausgabe ist die Angabe einer impliziten Schleife möglich.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

INTEGER I

WRITE (*,*) ('Hallo', I = 1, 10)
C   Ausgabe: HalloHalloHalloHalloHalloHalloHalloHalloHalloHallo

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

13.8. STOP

Die STOP-Anweisung beendet das Programm.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

WRITE (*,*) 'Vor Stop-Statement'
STOP
WRITE (*,*) 'Nach Stop-Statement'
C   Ausgabe: Vor Stop-Statement

END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

14. Standardfunktionen

In diesem Kapitel werden für die Funktionsargumente und Rückgabewerte nachfolgende Kürzel verwendet, die Aufschluss über die möglichen Datentypen geben.

Kürzel	Datentypen
i	INTEGER
r	REAL
d	DOUBLE PRECISION
x	COMPLEX
c	CHARACTER*n
l	LOGICAL

14.1. Datentypumwandlung

14.1.1. Umwandlung in INTEGER

- $i = \text{INT}(i)$

Abschneiden des Nachkommaanteils:

- $i = \text{INT}(r)$
- $i = \text{IFIX}(r)$
- $i = \text{IDINT}(d)$
- $i = \text{INT}(x)$

ASCII-Wert des Zeichens c:

- $i = \text{ICHAR}(c)$

14.1.2. Umwandlung in REAL

- $r = \text{REAL}(i)$
- $r = \text{FLOAT}(i)$
- $r = \text{REAL}(r)$
- $r = \text{SNGL}(d)$
- $r = \text{REAL}(x)$

14.1.3. Umwandlung in DOUBLE

- $d = \text{DBLE}(i)$
- $d = \text{DBLE}(r)$

- $d = \text{DBLE}(d)$
- $d = \text{DBLE}(x)$

14.1.4. Umwandlung in COMPLEX

- $x = \text{CMPLX}(i)$
- $x = \text{CMPLX}(r)$
- $x = \text{CMPLX}(d)$
- $x = \text{CMPLX}(x)$

14.1.5. Umwandlung in CHARACTER

Gibt das Zeichen zum ASCII-Wert i zurück.

- $c = \text{CHAR}(i)$

14.2. Mathematische Funktionen

14.2.1. Abschneiden des Nachkommaanteils

Ist das Argument größer Null, wird die nächstkleinere ganze Zahl zurückgegeben. Ist das Argument kleiner Null, wird die nächstgrößere ganze Zahl zurückgegeben.

- $r = \text{AINT}(r)$
- $d = \text{DINT}(d)$

14.2.2. Runden

Ist das Argument größer oder gleich Null, ist der Rückgabewert $\text{INT}(X+0.5)$. Ist das Argument kleiner Null, ist der Rückgabewert $\text{INT}(X-0.5)$

- $r = \text{ANINT}(r)$
- $d = \text{DNINT}(d)$
- $i = \text{NINT}(r)$
- $i = \text{IDNINT}(d)$

14.2.3. Absolutwert

- $i = \text{IABS}(i)$
- $r = \text{ABS}(r)$
- $d = \text{DABS}(d)$
- $r = \text{CABS}(x)$

14.2.4. Double Precision-Produkt

Rückgabewert ist $r1 \times r2$ mit Datentyp `DOUBLE PRECISION`

- $d = \text{DPROD}(r1, r2)$

14.2.5. Modulo

Rückgabewert ist $\text{zahl1} - \text{INT}(\text{zahl1} / \text{zahl2}) * \text{zahl2}$

- $i = \text{MOD}(i1, i2)$
- $r = \text{AMOD}(r1, r2)$
- $d = \text{DMOD}(d1, d2)$

14.2.6. Vorzeichentransfer

Wenn die $\text{zahl2} \geq 0$ ist, dann wird $|\text{zahl1}|$ zurückgegeben. Wenn die $\text{zahl2} < 0$ ist, dann wird $-|\text{zahl1}|$ zurückgegeben.

- $i = \text{ISIGN}(i1, i2)$
- $r = \text{SIGN}(r1, r2)$
- $d = \text{DSIGN}(d1, d2)$

14.2.7. Positive Differenz

Für $\text{zahl1} > \text{zahl2}$ ist der Rückgabewert $\text{zahl1} - \text{zahl2}$. Für $\text{zahl1} \leq \text{zahl2}$ wird Null zurückgegeben.

- $i = \text{IDIM}(i1, i2)$
- $r = \text{DIM}(r1, r2)$
- $d = \text{DDIM}(d1, d2)$

14.2.8. Maximum

Gibt den größten Argumentwert zurück.

- $i = \text{MAX0}(i1, i2, \dots)$
- $r = \text{AMAX1}(r1, r2, \dots)$
- $d = \text{DMAX1}(d1, d2, \dots)$
- $r = \text{AMAX0}(i1, i2, \dots)$
- $i = \text{MAX1}(r1, r2, \dots)$

14.2.9. Minimum

Gibt den kleinsten Argumentwert zurück.

- $i = \text{MIN0}(i1, i2, \dots)$
- $r = \text{AMIN1}(r1, r2, \dots)$
- $d = \text{DMIN1}(d1, d2, \dots)$
- $r = \text{AMIN0}(i1, i2, \dots)$
- $i = \text{MIN1}(r1, r2, \dots)$

14.2.10. Komplexe Zahlen

Gibt den Imaginärteil zurück:

- $r = \text{AIMAG}(x)$

Gibt die konjugiert komplexe Zahl zurück:

- $x = \text{CONJG}(x)$

14.2.11. Quadratwurzel

Gibt die Quadratwurzel zurück:

- $r = \text{SQRT}(r)$
- $d = \text{DSQRT}(d)$
- $x = \text{CSQRT}(x)$

14.2.12. Exponentialfunktion

Gibt natürliche Exponentialfunktion zurück:

- $r = \text{EXP}(r)$
- $d = \text{DEXP}(d)$
- $x = \text{CEXP}(x)$

14.2.13. Logarithmus naturalis

Gibt den natürlichen Logarithmus zurück:

- $r = \text{ALOG}(r)$
- $d = \text{DLOG}(d)$
- $x = \text{CLOG}(x)$

14.2.14. Dekadischer Logarithmus

Gibt den dekadischen Logarithmus zurück:

- $r = \text{ALOG10}(r)$
- $d = \text{DLOG10}(d)$

14.2.15. Winkelfunktionen

- $r = \text{SIN}(r)$
- $d = \text{DSIN}(d)$
- $x = \text{CSIN}(x)$
- $r = \text{COS}(r)$
- $d = \text{DCOS}(d)$
- $x = \text{CCOS}(x)$
- $r = \text{TAN}(r)$
- $d = \text{DTAN}(d)$

14.2.16. Arkusfunktionen

- $r = \text{ASIN}(r)$
- $d = \text{DASIN}(d)$
- $r = \text{ACOS}(r)$
- $d = \text{DACOS}(d)$
- $r = \text{ATAN}(r)$
- $d = \text{DATAN}(d)$

Gibt $\arctan(\frac{r1}{r2})$ zurück:

- $r = \text{ATAN2}(r1, r2)$
- $d = \text{DATAN2}(d1, d2)$

Diese Funktionen sind für ähnliche Werte der beiden Argumente erheblich genauer.

14.2.17. Hyperbelfunktionen

- $r = \text{SINH}(r)$
- $d = \text{DSINH}(d)$
- $r = \text{COSH}(r)$
- $d = \text{DCOSH}(d)$
- $r = \text{TANH}(r)$
- $d = \text{DTANH}(d)$

14.3. Zeichenketten-Funktionen

14.3.1. Länge

- $i = \text{LEN}(c)$

14.3.2. Index eines Teilstrings

Gibt die erste Position des Auftretens eines Teilstrings $c2$ in $c1$ zurück. $c2$ muss eine (benannte) Konstante sein.

- $i = \text{INDEX}(c1, c2)$

14.3.3. Lexikalische Funktionen

Hier wird unabhängig von der Plattform immer der ASCII-Zeichensatz als Grundlage verwendet.

Lexikalisch größer oder gleich (Rückgabewert ist `.TRUE.` wenn $c1 \geq c2$):

- $l = \text{LGE}(c1, c2)$

Lexikalisch größer als (Rückgabewert ist `.TRUE.` wenn $c1 > c2$):

- $l = \text{LGT}(c1, c2)$

Lexikalisch kleiner oder gleich (Rückgabewert ist `.TRUE.` wenn $c1 \leq c2$):

- $l = \text{LLE}(c1, c2)$

Lexikalisch kleiner als (Rückgabewert ist `.TRUE.` wenn $c1 < c2$):

- $l = \text{LLT}(c1, c2)$

Obige Funktionenaufstellung basiert auf dem *Fortran 77 Sprachstandard X3J3/90.4, Kap.15: Functions and Subroutines*.

15. Unterprogramme

Natürlich können in FORTRAN 77 auch eigene Unterprogramme erstellt werden.

15.1. Funktionsanweisung

Eine Funktionsanweisung (auch Anweisungsfunktion genannt) stellt die einfachste Möglichkeit dar, ein Unterprogramm in FORTRAN 77 zu realisieren. Eine Funktionsanweisung kann nur einen Ausdruck umfassen und gilt nur in der Programmeinheit in der sie definiert wurde.

Definieren einer Funktionsanweisung:

funktionsname([formale parameter]) = *ausdruck*

Aufruf der Funktion:

[*variable* =] *funktionsname*([aktuelle parameter])

Beispiel:

0. |1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

FUNK() = 5

WRITE (*,*) FUNK ()
C  Ausgabe: 5.000000

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0. |1.2.3.4.5.6.7|.8

Beispiel:

0. |1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

REAL A, B, C
FUNK(A, B) = COS(A) * LOG(B)

C = FUNK(3.1415, 2.)
```

```

      WRITE (*,*) C
C   Ausgabe: -0.6931472

      END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

15.2. FUNCTION

Soll eine Funktion mehrere Anweisungen umfassen, so genügt das Konzept der Funktionsanweisung nicht mehr. FORTRAN 77 kennt zu diesem Zweck das Schlüsselwort **FUNCTION**.

*[datentyp] FUNCTION funktionsname([formale parameter] anweisungen***END**

Aufgerufen wird eine derartige Funktion gleich wie eine Funktionsanweisung.

Beispiel:

Datei *bsp.f*:

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

      PROGRAM BSP

C   Funktionsaufruf
      WRITE(*,*) FUNK()
C   Ausgabe: 27.50000

      END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Datei *funk.f*:

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

      REAL FUNCTION FUNK()

      REAL TMP

      DO 10 I = 1,10
        TMP = TMP + I*0.5
10    CONTINUE

      FUNK = TMP
      END

```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Übersetzung mit *gfortran*:

```
gfortran bsp.f funk.f
```

Eine Funktion muss einen Wert zurückgeben. Welcher Wert das ist, wird durch eine Zuweisung an den Funktionsnamen erreicht. Wird am Anfang des Funktionskopfes kein Datentyp explizit vorgegeben, so gelten die Regeln für die implizite Datentypvergabe.

Mit Hilfe des Schlüsselwortes `RETURN` kann eine Funktion auch vor dem Funktionsende verlassen werden.

Beispiel:

Datei *bsp.f*:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

C Funktionsaufruf
  WRITE(*,*) FUNK(3)
C Ausgabe: 1.500000

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Datei *funk.f*:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
FUNCTION FUNK(I1)

IF(I1 .LE. 5) THEN
  FUNK = 1.5
  RETURN
END IF

FUNK = SIN(I1*0.5)

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

15.3. SUBROUTINE

Eine Subroutine besitzt im Gegensatz zu einer Funktion keinen Datentyp und gibt keinen Wert zurück.

SUBROUTINE *subroutinenname*(*[formale parameter]*) *anweisungen***END**

Aufruf der Subroutine:

CALL *subroutinenname*(*[aktuelle parameter]*)

Beispiel:

Datei *test.f*

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP
CALL SUB
END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Datei *sub.f*

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
SUBROUTINE SUB
WRITE (*,*) 'Hallo Welt!'
END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Übersetzung mittels *gfortran*:

```
gfortran -c sub.f
gfortran -c test.f
gfortran test.o sub.o
```

Anzeige auf der Standardausgabe:

```
Hallo Welt!
```

Auch eine Subroutine kann mittels **RETURN** vorzeitig verlassen werden.

Die aktuellen und formalen Parameter müssen hinsichtlich Datentyp, Anzahl, Reihenfolge übereinstimmen. Alle Namen und Variablen in einer Programmeinheit (Subroutine, Funktion oder Hauptprogramm) sind grundsätzlich nur lokal in der jeweiligen Programmeinheit bekannt. Über die Unterprogrammparameter können aber sehr wohl Werte in der aufrufenden Programmeinheit geändert werden.

Beispiel:

Datei *bsp.f*:

0. |1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

REAL A = 2.0

CALL SUB(A)

WRITE(*,*) A
C Ausgabe: 10

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0. |1.2.3.4.5.6.7|.8

Datei *sub.f*:

0. |1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
SUBROUTINE SUB(X)

REAL X
REAL A

C Unterprogrammparameter
X = 10

C lokale Variable
A = 500

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0. |1.2.3.4.5.6.7|.8

15.4. Felder als Parameter

Beispiel: Übergabe eines ganzen Feldes

Datei *bsp.f*:

0. |1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890


```
PROGRAM BSP

INTEGER FELD(3,3)
INTEGER CNT

CNT = 1

DO 10 I = 1, 3
  DO 20 J = 1, 3
    FELD(J,I) = CNT
    CNT = 1 + CNT
20  CONTINUE
10  CONTINUE

C  Unterprogrammaufruf
   CALL SUB(FELD)
C  Ausgabe: 1   2   3   4   5   6   7   8   9

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Datei *sub.f*

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
SUBROUTINE SUB(ARR)

INTEGER ARR(3, 3)

WRITE(*,*) ARR

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Beispiel: Übergabe einer Feld-Teilkette

Datei *bsp.f*:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

INTEGER FELD(3,3)
INTEGER CNT

CNT = 1

DO 10 I = 1, 3
  DO 20 J = 1, 3
    FELD(J,I) = CNT
    CNT = 1 + CNT
20  CONTINUE
10  CONTINUE
```

```

C Unterprogrammaufruf
  CALL SUB(FELD(1:2,2:3))
C Ausgabe: 4      5      7      8

      END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Datei *sub.f*

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

      SUBROUTINE SUB(ARR)

      INTEGER ARR(0:1, 0:1)

      WRITE(*,*) ARR

      END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Beispiel: Übergabe eines Feld-Einzelements

Datei *bsp.f*

```

0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890

```

```

      PROGRAM BSP

      INTEGER FELD(3,3)
      INTEGER CNT

      CNT = 1

      DO 10 I = 1, 3
        DO 20 J = 1, 3
          FELD(J,I) = CNT
          CNT = 1 + CNT
        20 CONTINUE
      10 CONTINUE

C Unterprogrammaufruf
  CALL SUB(FELD(1,2))
C Ausgabe: 4

      END

```

```

123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Datei *sub.f*

```

0.|1.2.3.4.5.6.7|.8

```

1234567890123456789012345678901234567890123456789012345678901234567890

```
SUBROUTINE SUB(ARR)

INTEGER ARR

WRITE(*,*) ARR

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

15.5. Prozeduren als Parameter

Auch Prozeduren können als Parameter übergeben werden.

Standardfunktionen werden dazu folgendermaßen im Vereinbarungsteil gekennzeichnet:

Aufruf der Subroutine:

INTRINSIC *namensliste*

Eigene Funktionen oder Subroutinen mit:

EXTERNAL *namensliste*

Beispiel:

Datei *bsp.f*:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

REAL PI
PARAMETER(PI=3.1415927)

C intrinsic functions
  INTRINSIC SIN, COS

C Unterprogrammaufrufe
  CALL SUB(SIN, PI)
C Ausgabe: 0.000000
  CALL SUB(COS, PI)
C Ausgabe: -1.000000

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Datei *sub.f*:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
SUBROUTINE SUB(FUNK, X)

REAL FUNK, X

WRITE(*,*) NINT(FUNK(X)*1000)/1000.0

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

15.6. COMMON

Mit **COMMON** lässt sich ein gemeinsamer Datenbereich für mehrere Programmeinheiten realisieren.

Unbenannter COMMON:

COMMON *variablenliste*

COMMON */name/ variablenliste*

Beispiel:

Datei *bsp.f*:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

REAL A, B, C, D
COMMON A, B, C
COMMON /C1/ D

A = 4.0
B = 5.0
C = 6.0

CALL SUB

WRITE (*,*) A, B, C, D
C Ausgabe: 3.330000  4.440000  6.000000  5.550000

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Datei *sub.f*:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
SUBROUTINE SUB
```

```
REAL X, Y, Z
```

```
COMMON X, Y
```

```
COMMON /C1/ Z
```

```
X = 3.33
```

```
Y = 4.44
```

```
Z = 5.55
```

```
END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

15.7. ENTRY

Mittels **ENTRY** kann gezielt in ein Unterprogramm gesprungen werden. Dieses Konstrukt widerspricht aber einer strukturierten Programmierung und sollte nicht verwendet werden.

ENTRY *entryname*[(*formale parameter*)]

Der Aufruf entspricht dem einer Subroutine:

CALL *entryname*[(*aktuelle parameter*)]

Beispiel:

Datei *bsp.f*

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP
```

```
CALL SUB
```

```
C  Ausgabe: Hallo
```

```
C           Welt!
```

```
CALL E1
```

```
C  Ausgabe: Welt!
```

```
END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Datei *sub.f*:

0.|1.2.3.4.5.6.7|.8

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
SUBROUTINE SUB

WRITE(*,*) 'Hallo'

ENTRY E1
WRITE (*,*) 'Welt!'

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
0.|1.2.3.4.5.6.7|.8
```

15.8. SAVE

Durch ein **SAVE**-Statement in Unterprogrammen behalten die lokalen Variablen ihren jeweiligen Wert auch nach Verlassen des Unterprogrammes. Dieses Konstrukt ist meist nicht notwendig, da die meisten FORTRAN-Compiler dieses Verhalten ohnehin automatisch aufweisen (siehe auch Kapitel DATA³¹ zwecks Initialisierung von Variablen).

SAVE [*variablenliste*]

Beispiel:

Datei *bsp.f*:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CALL SUB
C Ausgabe: 1.000000

CALL SUB
C Ausgabe: 2.000000

CALL SUB
C Ausgabe: 3.000000

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
0.|1.2.3.4.5.6.7|.8
```

Datei *sub.f*:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

³¹ Kapitel 15.9 auf Seite 86

```
SUBROUTINE SUB

REAL A
SAVE

A = A + 1

WRITE(*,*) A

END
```

```
123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

15.9. DATA

DATA dient zur Wertinitialisierung von Variablen vor der Programmeinheitsausführung. Diese Anweisung ist also nicht gleichzusetzen mit einer Wertzuweisung.

Beispiel:

DATA [*variablenliste*] /*variablenwerte*/

Beispiel:

Datei *bsp.f*:

```
0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CALL SUB
C  Ausgabe: 1.000000

CALL SUB
C  Ausgabe: 2.000000

CALL SUB
C  Ausgabe: 3.000000

END
```

```
123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Datei *sub.f*:

```
0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890
```

```
SUBROUTINE SUB

REAL A
```

```
DATA A /0.0/
```

```
A = A + 1
```

```
WRITE(*,*) A
```

```
END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
0.|1.2.3.4.5.6.7|.8
```

Unterschied zwischen Wertinitialisierung und Wertzuweisung:

	Wertinitialisierung	Wertzuweisung
Code	PROGRAM BSP CALL SUB CALL SUB CALL SUB END SUB- ROUTINE SUB REAL A DATA A /3.0/ A = A + 1 WRITE(*,*) A END	PROGRAM BSP CALL SUB CALL SUB CALL SUB END SUB- ROUTINE SUB REAL A A = 3.0 A = A + 1 WRITE(*,*) A END
Ausgabe	4.000000 5.000000 6.000000	4.000000 4.000000 4.000000

16. Ein- und Ausgabe

16.1. READ



Abb. 15

Die **READ**-Anweisung dient dem Einlesen von Daten. Typisches Beispiel ist die Dateneingabe mittels Tastatur. Formal sieht eine **READ**-Anweisung so aus:

```
READ([UNIT=]unit, [FMT=]fmt [, ERR=err] [, END=end] [, IOSTAT=iostat])  
[eingabeliste]
```

- unit ... Nummer der Eingabeeinheit (ist systemabhängig), Sternoperator oder auch die einer Datei mittels **OPEN**-Anweisung zugeordnete Nummer.
- fmt ... Anweisungsnummer zu einer **FORMAT**-Anweisung oder Sternoperator
- err ... Tritt während der Eingabe ein Fehler auf, so wird zu dieser Anweisungsnummer gesprungen
- end ... Nach dem Einlesen des letzten Datensatzes wird zu dieser Anweisungsnummer gesprungen
- iostat ... **READ**-Status

Listengesteuerte Eingabe auf der Standardeingabe (normalerweise die Tastatur):

```
READ (*,*) A, B, C
```

Alternativ kann das auch so geschrieben werden:

```
READ (UNIT=*, FMT=*) A, B, C
```

Beim *Intel Fortran Compiler*, *gfortran* und *g95* ist auch **UNIT = 5** als **stdin** (Tastatur) vorgelegt. Das Einlesen aus Dateien und die Einstellung des Formates werden später erläutert.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER I(5)

C Einlesen in ein Feld (UNIT ... Standardeingabe, FMT ... listengesteuert)
  READ (*,*) I
C ...

END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

Kurze Erläuterung zu IOSTAT:

Wert	Erläuterung
0	kein Fehler
positiver Wert (systemabhängig)	Fehler
negativer Wert (systemabhängig)	<i>End Of File</i> und kein Fehler

Beispiel:

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER I
INTEGER ST

C Einlesen eines Wertes
  READ (*, *, IOSTAT=ST) I

C Ausgabe des IO-Status
  WRITE (*,*) 'IO-Status:', ST

END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

Ausgabe:

Für Eingabe: 5 $\rightarrow$ 0

Für Eingabe: 5.3 $\rightarrow$ Positiver Wert = Fehler

16.2. WRITE



Abb. 16

Die `WRITE`-Anweisung dient der Datenausgabe. Typisches Beispiel ist die Anzeige von Daten auf dem Bildschirm. Formal sieht eine `WRITE`-Anweisung so aus:

```
WRITE([UNIT=]unit, [FMT=]fmt [, ERR=err] [, IOSTAT=iostat]) [ausgabeliste]
```

- `unit` ... Nummer der Ausgabeeinheit (ist systemabhängig), Sternoperator oder auch die einer Datei mittels `OPEN`-Anweisung zugeordnete Nummer.
- `fmt` ... Anweisungsnummer zu einer `FORMAT`-Anweisung oder Sternoperator
- `err` ... Tritt während der Ausgabe ein Fehler auf, so wird zu dieser Anweisungsnummer gesprungen
- `iostat` ... `WRITE`-Status

Listengesteuerte Ausgabe auf der Standardausgabe (normalerweise der Bildschirm):

```
WRITE (*,*) A, B, C
```

Alternativ kann das auch so geschrieben werden:

```
WRITE (UNIT=*, FMT=*) A, B, C
```

Beim *Intel Fortran Compiler*, *gfortran* und *g95* sind auch

- `unit=0` als `stderr` (Bildschirm) und
- `unit=6` als `stdout` (Bildschirm)

vorbelegt. Bezüglich `IOSTAT` gilt auch hier der im vorigen Abschnitt kurz geschilderte Sachverhalt. Die Ausgabe in Dateien und die Einstellung des Formates werden nachfolgend erläutert.

Beispiel:

```
0. | 1. 2. 3. 4. 5. 6. 7 | .8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP
```

```
INTEGER I(5)
```

```

C      I(1) = ...
C      ...

C Ausgabe der Feldwerte (UNIT ... Standardausgabe, FMT ... listengesteuert)
      WRITE (*,*) I
C      ...

      END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

16.3. Formatierung



Abb. 17

Die Ein- und Ausgabeformatierung kann beeinflusst werden. Zu diesem Zweck gibt es die **FORMAT**-Anweisung.

... (..., FMT = *marke*, ...) ... *marke* **FORMAT** (*formatliste*)

Alternativ dazu kann auch direkt bei der **FMT**-Option die Formatliste bekanntgemacht werden.

... (..., FMT = '(formatliste)', ...) ...

16.3.1. Formatlistenelemente

Formatspezifizierer	Kommentar
I <i>x</i> [. <i>z</i>]	Ganzzahl mit einer Feldlänge von <i>x</i> Zeichen. <i>z</i> gibt die Mindestanzahl der auszugebenden Zeichen an (Feld wird, wenn nötig, mit führenden Nullen aufgefüllt).
F <i>x</i> . <i>y</i>	Fixkommazahl mit einer Gesamtfeldlänge von <i>x</i> Zeichen. <i>y</i> ist die Anzahl der Nachkommastellen (Vorzeichen und Dezimalpunkt müssen in der Gesamtfeldlänge berücksichtigt werden).

Ex.y	Gleitkommazahl mit einer Gesamtfeldlänge von x Zeichen. y ist die Anzahl der Nachkommastellen. (Vorzeichen, Dezimalpunkt und die Zeichen für den Exponenten müssen in der Gesamtfeldlänge berücksichtigt werden).
Dx.y	„-“
A	Ein Zeichenkette.
Ax	Eine Zeichenkette mit x Zeichen.
Lx	Ein logischer Wert, T bzw. F
xX	x Leerzeichen.

Obige Tabelle der Formatlistenelemente ist nicht vollständig. Fortran kennt noch weitere Formatierungsmöglichkeiten. Die Ausgabe erfolgt normalerweise rechtsbündig. Reicht die Gesamtfeldlänge bei numerischen Werten nicht aus, so werden anstelle einer Zahl Sternchen angezeigt.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

INTEGER A

A = 999
WRITE (*, 3333) A
C Ausgabe: 999

A = -999
WRITE (*, 3333) A
C Ausgabe: ***

3333 FORMAT (I3)

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
0.|1.2.3.4.5.6.7|.8
```

Weitere Formatierungsbeispiele:

Code	Ausgabe
WRITE(*, 999) 1234 WRITE(*, 999) 1234567 WRITE(*, 999) 1234567890 999 FORMAT(I9.6)	001234 1234567 *****
WRITE(*, 999) 555.6666 WRITE(*, 999) +5.6 WRITE(*, 999) -55.666E7 WRITE(*, 999) -55555.666 999 FORMAT(F9.3)	555.667 5.600 ***** *****
WRITE(*, 999) 555.6666 WRITE(*, 999) +5.6 WRITE(*, 999) -55.666E7 WRITE(*, 999) -55555.666 999 FORMAT(E9.3)	0.556E+03 0.560E+01 -.557E+09 - .556E+05

WRITE(*, 999) 'Hallo' WRITE(*, 999) 'ABCDEFGHIJKL' WRITE(*, 888) 'ABCDEFGHIJKL' 888 FORMAT(A) 999 FORMAT(A10)	Hallo ABCDEFGHIJ ABCDEFGHIJKL
WRITE(*, *) 'FORTRAN', '77' WRITE(*, 999) 'FORTRAN', '77' 999 FORMAT(A, 1X, A)	FORTRAN77 FORTRAN 77
WRITE(*, 888) 'FORTRAN', '77' WRITE(*, 999) 'FORTRAN', '77' 888 FORMAT(A, T3, A) 999 FORMAT(A, T20, A)	FO77RAN FORTRAN 77
WRITE(*, 999) 'FORTRAN', '77' 999 FORMAT(A, /, A)	FORTRAN 77
WRITE(*, 999) 34.56 WRITE(*, *) 34.56 C SP ... Sign Plus (+) 999 FORMAT(SP, F12.3)	+34.560 34.56

16.3.2. Wiederholung von Formatteilen

Beispiel:

<pre> WRITE (*, 100) 'abc', 10.3, 'xxx', 23.4 100 FORMAT (2(A3, F6.1)) </pre>

16.3.3. WRITE etwas anders

Beispiel:

<pre> WRITE (*, 100) 100 FORMAT ('Hallo', 1X, 'Welt!') </pre>

16.4. Dateien



Abb. 18

16.4.1. Datensatz

Datensätze können in folgender Form auftreten:

- Formatierter Datensatz: Textdatensatz
- Unformatierter Datensatz: Datensatz in einer maschineninternen Form.
- Dateiendensatz

16.4.2. Datei

Für FORTRAN 77 ist alles eine Datei, das durch READ oder WRITE bearbeitbar ist.

Zugriffsmethoden:

- Sequentieller Zugriff: Lesen ab Beginn der Datei (file) und dann immer den nächsten Datensatz einlesen. Geschrieben wird jeweils ans Dateiende. Auf interne Dateien kann nur sequentiell zugegriffen werden.
- Direkter Zugriff: Bearbeiten in beliebiger Reihenfolge durch Angabe der Satznummer.

Dateitypen:

- Externe Datei: Eine konventionelle Datei
- Interne Datei: CHARACTER-Variable oder -Feld.

Dateien haben im Betriebssystem einen Dateinamen. In FORTRAN wird eine Datei über eine Dateinummer (unit) angesprochen. Die Zuordnung erfolgt mit dem Befehl OPEN.

16.4.3. OPEN

Zum Öffnen einer externen Datei dient die OPEN -Anweisung.

OPEN (liste)

mit folgender *liste*

Element	Kommentar
[UNIT =] x	x ist eine Dateinummer (0 bis 99)
FILE = x	x ist der externe Dateiname
IOSTAT = x	x ist 0 wenn OPEN fehlerfrei ausgeführt wurde, ansonsten eine systemabhängige Fehlernummer
ERR = x	Bei Fehler Sprung zur Anweisungsnummer x
STATUS = x	Dateistatus: 'OLD' ... Datei existiert bereits 'NEW' ... Datei wird neu erzeugt 'REPLACE' ... Wenn die Datei existiert, wird sie überschrieben, ansonsten neu erzeugt. 'SCRATCH' ... namenlose temporäre Datei 'UNKNOWN' ... System bestimmt Dateistatus selbst
ACCESS = x	Zugriffsmethode: 'SEQUENTIAL' ... Sequentielle Datei 'DIRECT' ... direkter Zugriff

FORM = x	Format: 'FORMATTED' oder 'UNFORMATTED'
RECL = x	Datensatzlänge (positive Zahl, ACCESS='DIRECT', in Bytes bzw. bei formatierten Dateien in Characters)
BLANK = x	'NULL' (ignorieren von Leerzeichen bei numerischen Werten) oder 'ZERO' (Leerzeichen bei numerischen Werten als 0 interpretieren)

Eingestellte Vorgabewerte sind:

- STATUS = 'UNKNOWN'
- ACCESS = 'SEQUENTIAL'
- FORM = 'FORMATTED'
- BLANK = 'NULL'

Wird ACCESS='DIRECT' gesetzt, so gilt FORM='UNFORMATTED' als Vorgabewert.

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

OPEN (20, FILE='/tmp/testdatei.txt', STATUS='OLD', ERR=222)

WRITE (*,*) 'Voller Erfolg'

CLOSE(20)
GOTO 333

222 WRITE(*,*) 'Fehler beim Öffnen der Datei'
333 END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

16.4.4. CLOSE

Geschlossen wird die Verbindung zur externen Datei mit dem CLOSE-Befehl.

CLOSE (liste)

liste:

Element	Kommentar
[UNIT =] x	wie bei OPEN
IOSTAT = x	wie bei OPEN
ERR = x	wie bei OPEN
STATUS = x	KEEP ... Datei erhalten
	DELETE ... Datei löschen

16.4.5. Lesen und Schreiben

Aus Dateien gelesen oder in Dateien geschrieben wird mit den bereits bekannten READ- und WRITE-Anweisungen.

Element	Kommentar
[UNIT =] x	Unit 0 bis 99 bzw. CHARACTER-Variable oder Feld (interne Datei)
[FMT =] x	siehe Formatierung ³²
REC = x	Datensatznummer bei Direktzugriff (siehe Abschnitt Direktzugriff ³³)
IOSTAT =	wie bei READ ³⁴
x	
ERR = x	Bei Fehler Sprung zur Anweisungsnummer x
END = x	Bei Dateiende Sprung zur Anweisungsnummer x (nicht erlaubt bei Direktzugriff)

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```

PROGRAM BSP

CHARACTER*80 A

OPEN (20, FILE='/tmp/testdatei.txt', STATUS='OLD', ERR=222)

10  CONTINUE
C  Aus Datei lesen
    READ (20, 888, END=20) A
C  Auf Standardausgabe schreiben
    WRITE (*,*) A
    GOTO 10

20  CLOSE(20)
    GOTO 333

222 WRITE(*,*) 'Fehler beim Öffnen der Datei'
888  FORMAT(A)
333  END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

```

Direktzugriff

OPEN:

Element	Kommentar
ACCESS =	x ... 'DIRECT'
x	

³² Kapitel 16.3 auf Seite 92

³³ Kapitel 16.4.5 auf Seite 97

³⁴ Kapitel 16.1 auf Seite 89

RECL = x x ... Datensatzlänge (positive Zahl, ACCESS='DIRECT', in Bytes
bzw. bei formatierten Dateien in Characters)

READ/WRITE:

Element	Kommentar
REC = x	x ... Satznummer bei Direktzugriff

Beispiel: Gegeben ist die Textdatei /tmp/testdatei.txt mit dem Inhalt

Die WRITE-Anweisung dient der Datenausgabe aus einem FORTRAN-Programm auf ein externes Gerät. Typisches Beispiel ist die Anzeige von Daten auf dem Bildschirm.
Formal sieht eine WRITE-Anweisung so aus:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CHARACTER*10 C

OPEN (20, FILE='/tmp/testdatei.txt',
&      STATUS='OLD',
&      ACCESS='DIRECT',
&      RECL=10,
&      ERR=222)

READ (20, REC=4, ERR=333) C
WRITE (*,*) C

READ (20, REC=25, ERR=333) C
WRITE (*,*) C

CLOSE (20)
GOTO 444

222 WRITE (*,*) 'Fehler beim Öffnen der Datei'
333 WRITE (*,*) 'Fehler beim Lesen des Datensatzes'
444 CONTINUE

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Ausgabe:

Datenausga
Fehler beim Lesen des Datensatzes

Positionieren bei sequentiellen Dateien

Datensatzzeiger um einen Datensatz zurücksetzen:

```
BACKSPACE ([UNIT=]x [,IOSTAT=y] [,ERR=z])
```

Positionieren an den Dateibeginn:

```
REWIND ([UNIT=]x [,IOSTAT=y] [,ERR=z])
```

Schreiben eines Dateiendsatzes:

```
ENDFILE ([UNIT=]x [,IOSTAT=y] [,ERR=z])
```

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

CHARACTER*100 C(3)

OPEN (20, FILE='/tmp/testx.txt',
&      STATUS='NEW',
&      ERR=222)

WRITE (20,*) 'Das ist eine Testdatei'
WRITE (20,*) 'Dies ist Zeile 2 der Testdatei'
WRITE (20,*) 'Jenes die Zeile 3 der Testdatei'
WRITE (20,*) 'Jetzt ist''s aber genug'
ENDFILE (20, ERR=444)

REWIND (20, ERR=444)
READ (20, FMT=555, ERR=333) C
WRITE (*, FMT=555) C

BACKSPACE (20, ERR=444)
READ (20, FMT=555, ERR=333) C(1)
WRITE (*, FMT=555) C(1)

GOTO 999

222 WRITE (*,*) 'Fehler beim Öffnen der Datei'
GOTO 999

333 WRITE (*,*) 'Fehler beim Lesen des Datensatzes'
GOTO 999

444 WRITE (*,*) 'Sonstiger Fehler'
GOTO 999

555 FORMAT (A)

999 CLOSE (20)

C Ausgabe:
C Das ist eine Testdatei
C Dies ist Zeile 2 der Testdatei
C Jenes die Zeile 3 der Testdatei
C Jenes die Zeile 3 der Testdatei
```

END

1234567890123456789012345678901234567890123456789012345678901234567890
 0.|1.2.3.4.5.6.7|.8

16.4.6. INQUIRE

Die Anweisung INQUIRE dient der Abfrage einiger Eigenschaften von Dateien oder I/O-Units.

INQUIRE (FILE = *x*, *liste*)

mit *x* ... Dateiname (inkl. Pfad)

INQUIRE ([UNIT =] *x*, *liste*)

mit *x* ... Nummer der I/O-Unit.

liste:

Element	Kommentar
ACCESS = <i>x</i>	<i>x</i> : • 'SEQ' ... sequentieller Dateizugriff • 'DIRECT' ... Direktzugriff
BLANK = <i>x</i>	<i>x</i> : • 'NULL' • 'ZERO'
DIRECT = <i>x</i>	<i>x</i> : • 'YES' ... Direktzugriff • 'NO' ... kein Direktzugriff für diese Datei erlaubt • 'UNKNOWN' ... unbekannt
ERR = <i>x</i>	Bei Fehler Sprung zur Anweisungsnummer <i>x</i>
EXIST = <i>x</i>	<i>x</i> : • .TRUE. ... Datei existiert • .FALSE. ... Datei existiert nicht
FORM = <i>x</i>	<i>x</i> : • 'FORMATTED' ... Datei geöffnet für formatierte Datensätze • 'UNFORMATTED' ... Datei geöffnet für unformatierte Datensätze

FORMATTED = x	• 'YES' ... formatierte Datensätze sind erlaubt • 'NO' ... formatierte Datensätze sind nicht erlaubt • 'UNKNOWN' ... unbekannt
IOSTAT = x	x ist 0 wenn INQUIRE fehlerfrei ausgeführt wurde, ansonsten eine systemabhängige positive Fehlernummer
NAME = x	Der Dateiname wird der Zeichenketten-Variablen x zugewiesen. Hat die Datei keinen Namen, dann ist das Ergebnis undefiniert.
NAMED = x	x: • .TRUE. ... Datei besitzt Namen • .FALSE. ... Datei besitzt keinen Namen
NEXTREC = x	x ... Nummer des nächsten Datensatzes
NUMBER = x	x ... Nummer der mit einer externen Datei verbundenen I/O-Unit.
OPENED = x	x: • .TRUE. ... Datei ist geöffnet • .FALSE. ... Datei ist nicht geöffnet
RECL = x	x ... Datensatzlänge bei Direktzugriff
SEQUENTIAL = x	x: • 'YES' ... sequentiell • 'NO' ... nicht sequentiell • 'UNKNOWN' ... unbekannt
UNFORMATTED = x	• 'YES' ... unformatierte Datensätze sind erlaubt • 'NO' ... unformatierte Datensätze sind nicht erlaubt • 'UNKNOWN' ... unbekannt

Beispiel: Datei vorhanden?

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```

PROGRAM BSP

LOGICAL L

INQUIRE (FILE='/tmp/testdatei.txt', EXIST=L, ERR=222)
WRITE (*,*) L
C  Ausgabe:
C    wenn Datei existiert:      T
C    wenn Datei nicht existiert: F

GOTO 999

222 WRITE (*,*) 'Fehler!'
999 CONTINUE

```

END

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

Beispiel: Infos zu einer geöffneten Datei

0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

LOGICAL EX
CHARACTER*15, DI, FO, AC, SE
INTEGER NU

OPEN (25, FILE='/tmp/testdatei.txt', STATUS='OLD', ERR=222)

INQUIRE (25, EXIST = EX,
&         DIRECT = DI,
&         SEQUENTIAL = SE,
&         FORMATTED = FO,
&         ACCESS = AC,
&         NUMBER = NU,
&         ERR=222)

WRITE (*,*) 'EXIST? ', EX
WRITE (*,*) 'DIRECT? ', DI
WRITE (*,*) 'SEQUENTIAL? ', SE
WRITE (*,*) 'FORMATTED? ', FO
WRITE (*,*) 'ACCESS? ', AC
WRITE (*,*) 'NUMBER? ', NU

C  Ausgabe, z.B.
C  EXIST?  T
C  DIRECT? YES
C  SEQUENTIAL? YES
C  FORMATTED? YES
C  ACCESS? SEQUENTIAL
C  NUMBER?          25

GOTO 999

222 WRITE (*,*) 'Fehler!'
999 CLOSE (25)

END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

16.4.7. Interne Dateien

- Interne Dateien sind vom Datentyp CHARACTER (Zeichen oder Zeichenketten)
- Das Lesen aus bzw. das Schreiben in interne Dateien erfolgt immer sequentiell

Beispiel: Schreiben in eine interne Datei

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*15 CH
REAL R

R = 12.5678

C  Interne Datei "CH"
  WRITE (CH, *) R

  WRITE (*,*) 'R lexikalisch groesser als Buchstabe "A"? ',
&            LGE(CH, 'A')

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

Beispiel: Lesen aus einer internen Datei

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*15 CH
REAL R

CH = '12.5678'

C  Interne Datei "CH"
  READ (CH, '(F15.5)') R

  WRITE (*,*) 'R      = ', R
  WRITE (*,*) 'R**2 = ', R**2

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

17. Anhang

17.1. PAUSE

Die PAUSE-Anweisung unterbricht die Programmausführung. Diese Anweisung wurde mit dem Fortran 90/95-Standard aus dem Fortran-Sprachumfang verbannt. Moderne Compiler geben deshalb teilweise bei Verwendung dieser Anweisung eine Warnmeldung (deleted feature o.ä.) aus. Das Verfahren zur normalen Fortsetzung des infolge PAUSE-Anweisung angehaltenen Programmes ist compilerabhängig, z. B.:

- *gfortran*: go `RETURN`
- *Intels ifx 2025.2*: continue `RETURN`

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

WRITE (*,*) 'Hallo '
PAUSE
WRITE (*,*) 'Welt!'

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Ausgabe bei Compilierung mit *gfortran*:

```
Hallo
PAUSE
To resume execution, type go.  Other input will terminate the job.
```

Ausgabe bei Compilierung mit *ifx*:

```
Hallo
FORTRAN PAUSE
PAUSE prompt>
```

17.2. INCLUDE

INCLUDE ermöglicht das Einbinden einer Datei. INCLUDE ist nicht explizit im FORTRAN 77-Standard angeführt, sondern wurde 1978 als Erweiterung des FORTRAN-Standards vom US-amerikanischen DoD im MilStd 1753 festgelegt (siehe auch Abschnitt MilStd 1753³⁵).

Beispiel:

Datei *bsp.f*:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
PROGRAM BSP

INCLUDE 'inc.f'

WRITE(*,*) 'Fläche ist', 10**2*PI, 'cm2'
C Ausgabe: Fläche ist    314.1593    cm2

END
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

Datei *inc.f*:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
REAL PI
PARAMETER(PI=3.141593)
```

```
1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

inc.f ist nicht explizit zu kompilieren! Es genügt z.B. Folgendes:

```
gfortran bsp.f
```

Das bindet *inc.f* automatisch ein.

17.3. Hollerith-Konstanten

Eine veraltete Möglichkeit Zeichenketten anzugeben stellen die Hollerith³⁷-Konstanten dar. Eine Hollerith-Konstante besteht aus

1. Einer positiven Ganzzahl, welche die Zeichenkettenlänge angibt
2. Dem Buchstaben H

³⁵ Kapitel 17.6 auf Seite 109

³⁷ Herman Hollerith (* 1860, † 1929), Erfinder des Hollerith-Lochkartenverfahrens, w:Herman Hollerith
[^]{<https://de.wikipedia.org/wiki/Herman%20Hollerith>}

3. Der Zeichenkette

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

CHARACTER*5 C
DATA C /5HUralt/

WRITE (*,*) C
C  Ausgabe: Uralt

END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

17.4. Arithmetisches IF

Schon in grauer Vorzeit fanden Programmierer arithmetische IF-Anweisungen sehr unterhaltsam, weil damit der Programmcode interessanter gestaltet werden konnte (Quelle: Ed Post - Real Programmers Don't Use PASCAL³⁸). Diese Aussage ist begreiflicherweise auch heute noch uneingeschränkt gültig.

IF(ausdruck) ziel1, ziel2, ziel3

mit

ausdruck

< 0

= 0

> 0

springe zu

ziel1

ziel2

ziel3

Beispiel:

0.|1.2.3.4.5.6.7|.8

1234567890123456789012345678901234567890123456789012345678901234567890

```
PROGRAM BSP

INTEGER I

I = -55

IF(I) 100, 200, 300
GOTO 999

100 WRITE(*,*) "Negative Zahl"
```

38 <http://www.ee.ryerson.ca/~elf/hack/realmen.html>

```
C Ausgabe: Negative Zahl
      GOTO 999
```

```
200  WRITE(*,*) "Null"
      GOTO 999
```

```
300  WRITE(*,*) "Positive Zahl"
```

```
999  CONTINUE
      END
```

```
123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8
```

17.5. ASSIGN und Assigned GOTO

Noch wesentlich amüsanter als bei der unachtsamen Verwendung des arithmetischen IF können die Ergebnisse durch Verwendung von Assign-Anweisungen ausfallen.

ASSIGN weist eine Zielmarke einer INTEGER-Variablen zu:

ASSIGN zielmarke TO variable

Verwendung finden kann dies neben anderem beim Assigned GOTO:

GOTO variable (liste)

Beispiel:

```
0.|1.2.3.4.5.6.7|.8
123456789012345678901234567890123456789012345678901234567890
```

```
      PROGRAM BSP
      INTEGER VAR
C ...
      ASSIGN 200 TO VAR
C ...
C ...
C ...
      GOTO VAR (100, 200, 300)
100  WRITE(*,*) "Negative Zahl"
      GOTO 999
200  WRITE(*,*) "Null"
C Ausgabe: Null
      GOTO 999
300  WRITE(*,*) "Positive Zahl"
999  CONTINUE
      END
```

1234567890123456789012345678901234567890123456789012345678901234567890

0.|1.2.3.4.5.6.7|.8

17.6. MilStd 1753, DoD Supplement To American National Standard X3.9-1978

Nachfolgend stichwortartig die vom amerikanischen Department of Defense im Jahr 1978 ergänzend zu FORTRAN 77 geforderten Spracheigenschaften

- END DO (gelabelt, für DO-Schleifen)
- DO WHILE-Schleife
- INCLUDE
- IMPLICIT NONE
- Manipulation von Bitfeldern (IOR, IAND, ISHFT, ...)
- READ/WRITE-Verhalten nach EOF.

Teil IV.

Fortran 95

18. Programmaufbau

18.1. Beispiel: Hallo Welt

Fortran 90/95-Code (free source form)

```
program hallo
  ! Das typische "Hallo Welt"-Programm
  write( *, * ) 'Hallo Welt!'
end program hallo
```

Dieses Programm besteht aus den gleichen Anweisungen wie ein entsprechendes FORTRAN 77-Programm und wurde in der sogenannten "free source form" verfasst. Aus diesem Grund ist die erste Spalte nicht mehr dem Kommentarzeichen vorbehalten. Kommentare werden hier durch ein Rufzeichen eingeleitet. Alles rechts vom Rufzeichen wird als Kommentar behandelt. Ein Kommentar kann also auch nach einer Anweisung stehen. Im Gegensatz zu einem FORTRAN 77-Programm wird in Fortran 90/95 oft die Kleinschreibung bevorzugt. Dies ist aber nicht zwingend erforderlich, da die Fortran-Compiler case-insensitiv sind. D.h. sie unterscheiden nicht zwischen Groß- und Kleinschreibung. Eine Einrückung von Blöcken ist nicht zwingend erforderlich, fördert aber die Übersichtlichkeit des Programmcodes.

Die Anweisung in der ersten Zeile kennzeichnet die Programmeinheit als Hauptprogramm und gibt ihr die Bezeichnung `hallo`. Es folgt eine Kommentarzeile. Dann folgt die Anweisung, einen String auf die Standardausgabe zu schreiben. Und schließlich signalisiert die `end`-Anweisung das Programmende.

18.2. Das Zeilenformat

Fortran 90/95 bietet zwei verschiedene Programmaufbaumöglichkeiten:

- free source form
- fixed source form

Die "free source form" ist neu in Fortran 90/95. Sie bietet die Möglichkeit Programme ohne fixe Spaltenzuordnung zu schreiben, wie dies auch in den meisten anderen gebräuchlichen Programmiersprachen üblich ist. Zusätzlich ist in Fortran 90/95 aus Kompatibilitätsgründen auch das alte FORTRAN 77-Zeilenformat (fixed source form) enthalten. Dieses sollte in neuen Programmen aber nicht mehr verwendet werden.

Normalerweise gilt, dass jede Fortran-Anweisung in einer eigenen Zeile steht. Bei Verwendung der "free source form" gelten folgende Bedingungen. Eine Zeile darf maximal 132 Zeichen lang sein. Als Zeilenumbruchzeichen dient das Kaufmanns-Und (&). Das Kaufmanns-Und steht in diesem Fall immer am Ende der fortzuführenden Zeile, optional auch zusätz-

lich am Beginn der Fortsetzungszeile. Standardmäßig sind maximal 40 Fortsetzungszeilen erlaubt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  ! Leerzeilen werden vom Compiler ignoriert

  ! Vereinbarungsteil
  implicit none                ! implizite Datentypvereinbarung ausschalten

  integer      :: a, b, c      ! Variablendeklaration (Ganzzahlen)
  character(25) :: str          ! Variablendeklaration (String mit einer Länge von
                                25 Zeichen)

  ! Aktionsteil
  a = 5
  b = 7
  c = a + &
  ! und jetzt kommt eine Fortsetzungszeile
  b

  write( *, * ) c

  ! auch Strings oder Schlüsselwörter können auf der nächsten Zeile fortgesetzt
  werden.
  ! Dabei ist unbedingt darauf zu achten, dass die Fortsetzungszeile auch mit
  einem
  ! Kaufmanns-Und eingeleitet wird, da ansonsten einige Compiler Fehlermeldungen
  liefern
  str = "Hal&
  &lo Welt!"

  wr&
  &rite( *, * ) str

  ! Ausgabe:
  ! 12
  ! Hallo Welt!
end program bsp
```

Mehrere Anweisungen können in eine Zeile geschrieben werden, wenn eine Trennung durch jeweils ein Semikolon erfolgt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: a, b, c

  a = 5; b = 7; c = a + b

  write( *, * ) c

  ! Ausgabe:
  ! 12
end program bsp
```

18.3. Die Programmstruktur für das Hauptprogramm

Ein Hauptprogramm weist in einfachster Darstellung immer folgende Struktur auf:

1. `program Name`
2. Vereinbarungsteil
3. Aktionsteil
4. `end [program [Name]]`

Die in eckigen Klammern angegebenen Bestandteile sind optional. Sie sollten dennoch stets mit angegeben werden. Genau eine `end`-Anweisung (mit oder ohne optionale Bestandteile) ist obligatorisch. Bei Hauptprogramm und Unterprogrammen ist `end` eine ausführbare Anweisung. Ein Fortran-Programm darf genau ein Hauptprogramm enthalten, es ist der Startbereich für die Programmausführung.

18.4. Der Fortran-Zeichenvorrat

Fortran 95-Programme bestehen standardmäßig aus folgenden Zeichen:

- Großbuchstaben: A bis Z
- Kleinbuchstaben: a bis z
- Ziffern: 0 bis 9
- Den 13 FORTRAN 77-Sonderzeichen: `+ - * / = ( ) : , . ' $` und dem Leerzeichen
- Unterstrich (Underscore): `_`
- Weitere Sonderzeichen: `! ? " ' & ; < >`

Ein Fortran 90/95-Compiler ist case-insensitiv: er unterscheidet **nicht** zwischen Groß- und Kleinbuchstaben. Stringkonstanten können natürlich alle ASCII-Zeichen beinhalten.

18.5. Symbolische Namen

Standardmäßig dürfen symbolische Namen maximal 31 Zeichen lang sein. Das erste Zeichen muss immer ein Buchstabe sein. Anschließend sind alphanumerische Zeichen (Buchstabe oder Ziffer) und Unterstriche erlaubt. Im Gegensatz zu FORTRAN 77 dürfen nun keine Leerzeichen innerhalb eines symbolischen Namens auftreten.

18.6. Reservierte Schlüsselwörter?

Im Fortran 95-Standard ist zwar die Rede von Schlüsselwörtern (statement keywords), wie z.B. `if`, `do`, `real`, `write`. Allerdings wird eindeutig darauf hingewiesen, dass diese nicht reserviert sind. Das heißt, solche Schlüsselwörter können auch für eigene Bezeichner verwendet werden.

Beispiel: Folgender Programmcode ist in Fortran 90/95 gültig, aber nicht empfehlenswert

Fortran 90/95-Code (free source form)

```
program bsp
```

```
implicit none

integer :: integer, write, if

integer = 6
write = 5
if = 1

write( *, * ) integer + write + if
end program bsp
```

18.7. Details zur Anordnungsreihenfolge von Anweisungen

Fortran schreibt eine gewisse Anordnungsstruktur der einzelnen Programmelemente vor. So ist das folgende Programm

Fortran 90/95-Code (free source form)

```
program hallo
  write( *, * ) "Hallo Welt"
  implicit none
end program hallo
```

nicht standardkonform, da die `implicit none`-Vereinbarung nicht nach einer ausführbaren Anweisung folgen darf. So müsste es richtig lauten

Fortran 90/95-Code (free source form)

```
program hallo
  implicit none
  write( *, * ) "Hallo Welt"
end program hallo
```

Im Fortran 90/95-Standard ist genau festgelegt wann bestimmte Programmelemente auftreten dürfen. Prinzipiell gilt, dass Vereinbarungs-Anweisungen (nichtausführbare Anweisungen) vor Aktions-Anweisungen (ausführbaren Anweisungen) stehen. Doch es gibt Ausnahmen, z.B. darf die nichtexekutierbare `format`-Anweisung auch zwischen oder nach ausführbaren Anweisungen stehen. Eindeutig ist Folgendes geregelt. Die Programmeinheit beginnt mit dem charakteristischen Schlüsselwort, z.B. `function`. Werden Module eingebunden (`use ...`), so erfolgt dies vor jeder anderen Vereinbarung oder ausführbaren Anweisung. Danach folgt ein "Mischbereich" (im Fortran-Standard wird das natürlich nicht "Mischbereich" genannt). Vor dem `end`-Statement darf ein `contains`-Abschnitt mit Unterprogrammen stehen.

Fortran 90/95-Code (free source form)

```
program (, function, subroutine, ...) ...
  use ...

  ! Hier folgt nun der so genannte "Mischbereich"

contains
```

```

    interne Unterprogramme oder Modul-Unterprogramme
end ...

```

Im "Mischbereich" ist die Anordnung einzelner Elemente zwar auch im Standard geregelt, jedoch gibt es Elemente die an jeder Stelle des Mischbereichs auftreten dürfen, bei anderen ist wiederum klar festgelegt, dass sie nur nach oder vor anderen Elementen auftreten dürfen. Hier soll deshalb nur eine vereinfachte Variante gezeigt werden. Für genauere Informationen wird auf den Fortran-Working-Draft verwiesen.

Fortran 90/95-Code (free source form)

```

program (, function, subroutine, ...) ...
  use ...

! ----- Beginn "Mischbereich" (mögliche Anordnungsreihenfolge) -----
  implicit ...

  ! Definiton von Datenverbunden, Interface-Blöcke, benannte Konstanten,
  Variablendeklarationen

  ! ausführbare Anweisungen

  ... format ...
! ----- Ende "Mischbereich" -----

  contains
    Interne Unterprogramme bzw. Modul-Unterprogramme
end ...

```

Nicht jede Anweisung ist in jeder Programmeinheit erlaubt. So dürfen z.B. keine **format**-Anweisungen im Hauptteil eines Moduls (wohl aber in Modul-Unterprogrammen) verwendet werden.

19. Datentypen, Variablen, Wertzuweisungen, Konstanten

Dieses Kapitel handelt von Datentypen, Variablen, Konstanten und der Wertzuweisung.

19.1. Datentypen

19.1.1. Arithmetische Datentypen

Datentyp	Kommentar	Beispiele (Konstanten)
<code>integer</code>	Ganzzahlen	15, -6500, 200000000
<code>real</code>	Gleitkommazahlen einfacher Genauigkeit	3.1415, -5.5, .7e3, 12.5E-5
<code>(double precision)</code>	Gleitkommazahlen doppelter Genauigkeit (aus FORTRAN 77)	3.1415D0, -5.5D0, .7d3, 12.5D-5
<code>complex</code>	Komplexe Zahlen (zwei <code>real</code> -Zahlen)	(3.1415, -5.5), (1.4, 7.1E4)

`double precision` ist in Fortran 95 nur noch aus historischen Gründen (FORTRAN 77) vorhanden. Fortran 95 bietet für Datentypen höherer Genauigkeit bzw. mit größerem Zahlenbereich andere Sprachmittel. Diese Thematik wird später erläutert (Datentypen höherer Genauigkeit³⁹).

Binär-, Oktal- oder Hexadezimalzahlen können als "boz literal constants" angegeben werden:

Schreibweise 1	Schreibweise 2	Kommentar
<code>B'zahl'</code>	<code>B'zahl'</code>	Binäre Zahl
<code>O'zahl'</code>	<code>O'zahl'</code>	Oktalzahl
<code>Z'zahl'</code>	<code>Z'zahl'</code>	Hexadezimalzahl

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: a, b, c

  data a /Z"AB1C"/
```

39 Kapitel 26 auf Seite 157


```

print *, a
! Ausgabe: 43804

data b /0"7134"/
print *, b
! Ausgabe: 3676

data c /B'101110110001'/
print *, c
! Ausgabe: 2993
end program bsp

```

19.1.2. Logischer Datentyp

Datentyp	Kommentar	Mögliche Werte
logical	Logischer Datentyp (wahr oder falsch)	.TRUE., .FALSE.

19.1.3. Zeichenketten

Datentyp	Kommentar	Beispiel (Konstante)
character(n)	Zeichenkette (String) mit einer Länge von n Zeichen	'Hallo, Welt!', "Hallo, Welt!"
character(len=n)	-"-	
character*n	-"- (FORTRAN 77-Stil, sollte nicht mehr verwendet werden)	
character	Zeichenkette (String) mit einer Länge von einem Zeichen	'H', "h"

Beachte: Zeichenketten können in Fortran 95 in Apostrophe oder Anführungszeichen eingeschlossen werden.

Beispiel:

"Wie geht's?"
'Er sagte: "Hallo"'

19.2. Variablen

Eine Variable ist charakterisiert durch einen

- symbolischen Namen
- Datentyp
- Wert
- Speicherplatz

Beim Programmstart hat eine Variable keinen definierten Wert. Eine Variable kann ihren Datentyp auf zwei Arten erhalten, durch implizite oder explizite Typanweisung.

19.2.1. Implizite Typanweisung

Bei der impliziten Typanweisung bestimmt der Anfangsbuchstabe des Variablenbezeichners den Datentyp.

Datentyp	Anfangsbuchstabe der Variablen
integer	Buchstaben I bis N (oder i bis n)
real	restliche Buchstaben

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  b1 = 8.9
  c1 = 3.
  i1 = B1/C1

  write (*,*) i1
  ! Das Ergebnis ist 2, da i1 implizit als integer definiert ist
end program bsp
```

Die Standardzuordnung der Anfangsbuchstaben kann durch das Schlüsselwort `implicit` auch geändert werden. Infolge der durch implizite Typanweisung entstehenden Fehlermöglichkeiten ist es sinnvoll, die implizite Typanweisung komplett auszuschalten. Dies wird durch die Anweisung

```
implicit none
```

gleich nach der `program`-Anweisung erreicht. Dann muss der Datentyp jeder Variablen explizit festgelegt werden.

19.2.2. Explizite Typanweisung

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real :: b
  real :: c
  real :: i
  ! alternativ auch als real :: b, c, i

  b = 8.9
  c = 3.
  i = b/c

  write (*,*) i
  ! Das Ergebnis ist 2.966666
end program bsp
```

19.3. Benannte Konstanten

Benannte Konstanten werden in Fortran 95 folgendermaßen festgelegt:

datentyp, parameter :: symname = wert

Beispiele:

```
real, parameter:: PI = 3.1415, PIFAC = PI/2.0
```

```
character(5), parameter :: str = 'Hallo'
```

Der zugewiesene Wert kann eine Konstante (Literal) oder eine schon definierte benannte Konstante sein.

Für die Zeichenkettenlänge ist bei benannten Konstanten auch eine *-Schreibweise möglich. Dies erspart die explizite Angabe der Stringlänge.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(*), parameter :: a = 'Jetzt wird auch die Stringlaenge festgelegt'

  write(*,*) a
  ! Ausgabe: Jetzt wird auch die Stringlaenge festgelegt
end program bsp
```

19.4. Wertzuweisung

Wertzuweisungen haben wir schon kennengelernt:

variable = ausdruck

Beispiel:

```
k = 1
k = k + 2
```

Die Wertzuweisung an eine Variable ist, wie am vorigen und auch am nächsten Beispiel zu sehen, nicht zu verwechseln mit einer mathematischen Gleichung. Der Ausdruck

```
k + 2 = 5
```

wäre zwar mathematisch korrekt. Als Wertzuweisung in einem Fortran-Programm ist dies aber keine mögliche Formulierung. `k+2` ist kein zulässiger Ausdruck auf der linken Seite des Zuweisungsoperators (L-Wert).

In Fortran ist auch keine Kette von Wertzuweisungen möglich. Der folgende Ausdruck ist **nicht** erlaubt und liefert eine Fehlermeldung.

```
i = j = k = 1.5  
! Fehler
```


20. Felder

20.1. Beispiel

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real, dimension(10) :: arr
  ! ACHTUNG! Array startet mit dem Index 1
  ! arr(0) waere ein Fehler!

  arr(1) = 1.5
  arr(2) = 2.5
  arr(10) = 10.5

  write(*,*) arr(1)
  ! 1.500000 wird ausgegeben

  write(*,*) arr(10)
  ! 10.500000 wird ausgegeben
end program bsp
```

20.2. Eindimensionale Felder

20.2.1. Statische Speicherallokation

Variante 1: Die dimension-Anweisung

```
real, dimension(10) :: arr
```

Der Feldindex läuft von 1 bis 10.

Variante 2: Einfach

```
real :: var(10)
```

Variante 3: Verwendung von benannten Konstanten

```
integer, parameter :: MAXIND = 10
real, dimension(MAXIND) :: arr
```

Hier erfolgt die Festlegung der Feldgröße über eine benannte Konstante.

Variante 4: Explizite Angabe der Feldgrenzen

```
real, dimension(0:9) :: arr
```

Hier wird Unter- und Obergrenze explizit angegeben. Der Index läuft nun von 0 bis 9. Die Feldgrenzen können auch negativ sein.

20.2.2. Dynamische Speicherallokation

Mit Fortran 90/95 kann der benötigte Speicherplatz für ein Feld dynamisch angefordert werden. Zu diesem Zweck gibt es das Schlüsselwort `allocatable`, welches unmittelbar bei der Felddeklaration die beabsichtigte dynamische Speicherallokation signalisiert.

datentyp, dimension(:), allocatable :: variablenbezeichner

Die intrinsische Funktion `allocate` dient dann der konkreten Speicherallokation im Akti-onsteil des Programmes, also u.a. der Festlegung der konkreten Feldgröße. Der reservierte Speicherplatz kann mit der Funktion `deallocate` auch wieder freigegeben werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real, dimension(:), allocatable :: arr  ! dynamisches eindimensionales Feld

  integer :: status

  ! Allokation
  allocate(arr(0:9), stat=status)

  ! Feld mit Werten belegen,
  ! ....
  ! ....

  ! Deallokation
  deallocate(arr, stat=status)
end program bsp
```

Die Statusvariable sollte nach Ausführung der Allokationsfunktionen jeweils den Wert 0 aufweisen. Andere Werte stehen für eine Fehlermeldung.

Die Funktion `allocated()` hilft bei der Überprüfung des Allokationsstatus eines Feldes. Ist für das abgefragte Feld bereits Speicherplatz allokiert, dann liefert die Funktion `.TRUE.`, ansonsten `.FALSE.`

```
! ...
if (.not. allocated(arr)) then
  allocate(arr(groesse), stat=s)
end if
! ...
```

Weitere Funktionen bezüglich Feldern sind im Kapitel Standardfunktionen⁴⁰ aufgelistet.

20.3. Mehrdimensionale Felder

Für mehrdimensionale Felder gelten die gleichen Varianten wie für eindimensionale Felder. Die Speicherreihenfolge ist spaltenorientiert. Das bedeutet, der erste Index variiert am schnellsten: $a_{11}, a_{21}, \dots, a_{(n-1)m}, a_{nm}$

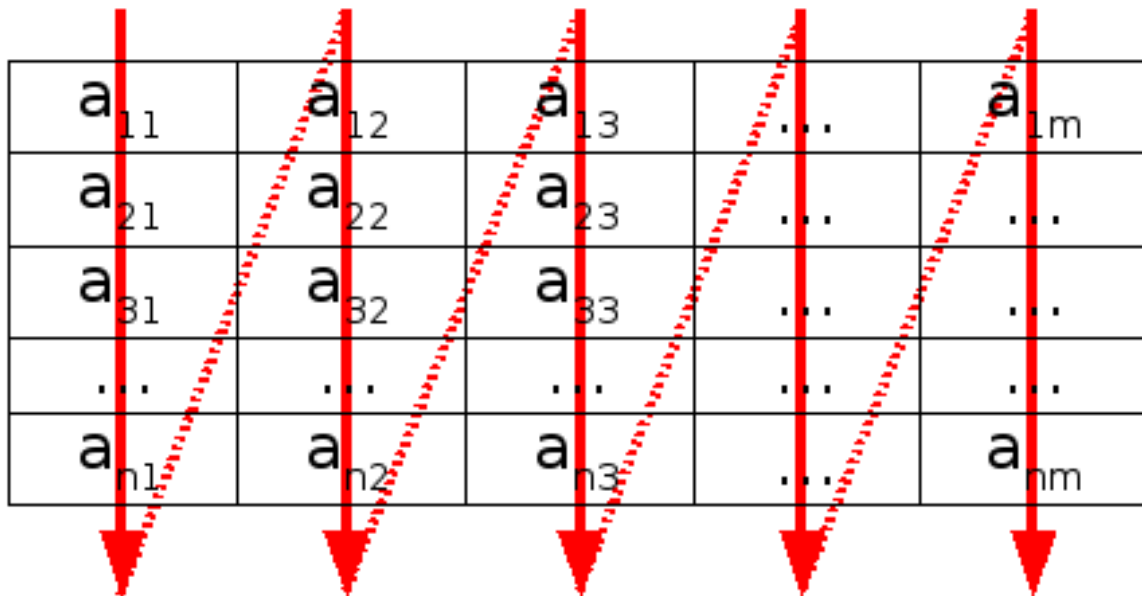


Abb. 19

Ein mehrdimensionales Feld kann auch in Fortran 90/95 maximal 7-dimensional gestaltet werden.

20.3.1. Beispiel: Ein 2-dimensionales Feld

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  character(10), dimension(0:9, 2:5) :: arr

  arr(0, 2) = 'Hallo'
  arr(1, 2) = 'Welt'
  ! ...
  arr(9, 5) = 'Universum'
```

⁴⁰ Kapitel 28.6 auf Seite 168


```
write (*,*) arr(0, 2)
! Ausgabe: Hallo

write (*,*) arr(9, 5)
! Ausgabe: Universum
end program bsp
```

20.3.2. Beispiel: Spaltenorientierte Speicherreihenfolge

Die 3x3-Matrix $A = \begin{pmatrix} 1 & -5 & 0 \\ 40 & 3 & -2 \\ -1 & 9 & 65 \end{pmatrix}$ soll in ein Fortran-Programm eingelesen und wieder komplett ausgegeben werden. Zusätzlich soll auch der Wert des Feldelementes a_{23} (2. Zeile, 3. Spalte, Wert=-2) separat ausgegeben werden.

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  integer, dimension(3,3) :: arr(3,3)

  ! Feldelemente einlesen
  write (*,*) 'Werte (spaltenorientierte Eingabe):'
  read (*,*) arr

  ! Komplettes Feld ausgeben
  write (*,*) 'Gesamtfeld = ' , arr

  ! a23 ausgeben
  write (*,*) 'a23 = ', ARR(2,3)
end program bsp
```

Ein-/Ausgabe:

```
Werte (spaltenorientierte Eingabe):
1
40
-1
-5
3
9
0
-2
65
Gesamtfeld =           1          40          -1          -5          3
          9          0
          -2          65
a23 =           -2
```

20.4. Feldinitialisierung

Felder lassen sich auch gleich bei der Deklaration mit Werten initialisieren.

20.4.1. Array constructor

Bei eindimensionalen Feldern kann die Feldinitialisierung direkt mittels *array constructor* erfolgen.

Beispiel:

```
real, dimension(5) :: arr = (/1.1, 1.2, 1.3, 2.1, 2.4/)
```

20.4.2. Reshape

Für mehrdimensionale Felder besteht die Möglichkeit der direkten Verwendung des *array constructors* nicht. Stattdessen kann eine derartige Initialisierung über den Umweg der *reshape*-Funktion geschehen.

Beispiel: Die Matrix

```
7 8 9
7 8 9
```

soll in einem 2D-Feld gespeichert werden.

```
integer, dimension(2,3) :: arr = reshape( (/7, 7, 8, 8, 9, 9/), (/2, 3/) )
```

20.5. Teilfelder

Ähnlich wie das schon in FORTRAN 77 bei Zeichenketten möglich war, können nun auch Teilfelder direkt angesprochen werden.

Prinzip

feldname (*anfang:ende*)

feldname (:*ende*)

feldname (*anfang*:)

Beschreibung

von *anfang* bis *ende*

vom 1. Feldelement bis *ende*

von *anfang* bis zum letzten Feldelement

- *anfang* $\geq$ untere Indexgrenze
- *ende* $\leq$ obere Indexgrenze

Beispiel: Eindimensionale Felder **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(2), dimension(5) :: arr = (/ 'A1', 'A2', 'A3', 'A4', 'A5' /)
  integer, dimension(-3:1) :: i = (/ -30, -20, -10, 0, 10 /)
```

```
write (*,*) arr(2:4)
! Ausgabe: A2A3A4

write (*,*) arr(2:)
! Ausgabe: A2A3A4A5

write (*,*) arr(:3)
! Ausgabe: A1A2A3

write (*,*) arr(1)
! Ausgabe: A1

write (*,*) arr
! Ausgabe: A1A2A3A4A5

write (*,*) i(-2:0)
! Ausgabe: -20   -10    0

end program bsp
```

21. Arithmetische Ausdrücke

21.1. Arithmetische Operatoren

Fortran 95 kennt wie FORTRAN 77 folgende arithmetische Operatoren

Operator	Kommentar
+	Addition
-	Subtraktion
*	Multiplikation
/	Division
**	Exponentiation

21.1.1. Operatorenpriorität

Die Priorität der arithmetischen Operatoren entspricht den mathematischen Konventionen.

- Klammerung vor allem anderen, z.B. $(a+b)*c \Leftrightarrow a*c+b*c$
- Exponentiation vor Punktrechnung, z.B. $a*b**c \Leftrightarrow a*(b**c)$
- Punktrechnung vor Strichrechnung, z.B. $a+b*c \Leftrightarrow a+(b*c)$

21.1.2. Berechnungsfolge bei gleicher Priorität

- Klammerung, Punktrechnung und Strichrechnung: $\rightarrow$
Beispiel: $a*b/c*d \Leftrightarrow ((a*b)/c)*d$
- Exponentiation: $\leftarrow$
Beispiel: $a**b**c \Leftrightarrow a**(b**c)$

Außerdem ist zu beachten, dass niemals zwei Operatoren direkt aufeinander folgen dürfen.

Beispiel: Der Ausdruck $1.5**-1$ ist in Fortran 95 falsch und führt zu einer Fehlermeldung.
Richtig ist $1.5**(-1)$

21.2. Ergebnisdatentyp

21.2.1. Operanden gleichen Datentyps

Bei Operanden gleichen Datentyps erhält das Ergebnis den Datentyp der Operanden.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none
```

```
real :: a

a = 3/2
! 3 ist ein integer und 2 ist auch ein integer,
! daher muss das Ergebnis auch ein integer sein, also 1.
! Die Zuweisung an die real-Variable a stellt das
! Ergebnis nicht mehr richtig.

write(*,*) a
! Ausgabe: 1.00000
end program bsp
```

21.2.2. Implizite Typumwandlung bei Operanden gemischten Datentyps

Weisen die Operanden unterschiedliche Datentypen auf, so wird bei jeder Operation, falls nötig, das Ergebnis dem höherwertigen Datentyp angepasst.

integer → real → complex

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real :: a

  a = 3/2.
  ! 2. ist ein real. Jetzt stimmt das Ergebnis.

  write (*,*) a
  ! Ausgabe: 1.500000
end program bsp
```

21.2.3. Explizite Typumwandlung

Fortran 95 besitzt auch Funktionen zur expliziten Umwandlung des Datentyps. Diese werden im Kapitel Standardfunktionen⁴¹ näher beschrieben.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real :: r
  complex :: c

  r = 2.
  c = cmplx(r)

  write (*,*) c
  ! Ausgabe: ( 2.000000 , 0.000000 )
end program bsp
```

⁴¹ Kapitel 28 auf Seite 167

22. Logische Ausdrücke

Logische Ausdrücke können zwei Zustände annehmen, `.TRUE.` oder `.FALSE.`.

22.1. Logische Operatoren

Operator	Kommentar
<code>.NOT.</code>	logisches NICHT
<code>.AND.</code>	logisches UND
<code>.OR.</code>	logisches ODER
<code>.EQV.</code>	logische Äquivalenz
<code>.NEQV.</code>	logische Antivalenz

22.2. Wahrheitstafel

a	b	<code>.NOT.</code> a	a <code>.AND.</code> b	a <code>.OR.</code> b	a <code>.EQV.</code> b	a <code>.NEQV.</code> b
<code>.TRUE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.TRUE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>
<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.FALSE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>
<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>
<code>.FALSE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>	<code>.FALSE.</code>	<code>.TRUE.</code>	<code>.FALSE.</code>

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  logical :: l

  l = .TRUE.

  write(*,*) .NOT. l
  ! Ausgabe: F
end program bsp
```

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  logical :: a, b
```

```
a = .TRUE.  
b = .FALSE.  
  
write (*,*) a .NEQV. b  
! Ausgabe: T  
end program bsp
```

22.3. Operatorenpriorität

1. Klammerung () bindet am stärksten
2. .NOT.
3. .AND.
4. .OR.
5. .EQV., bzw. .NEQV.

23. Vergleichsausdrücke

23.1. Vergleichsoperatoren

Zum Vergleichen zweier arithmetischer Ausdrücke oder von Strings gibt es Vergleichsoperatoren. Das Ergebnis eines Vergleichs ist ein logischer Wert (.TRUE. oder .FALSE.).

Operator in Fortran 95	Operator in FORTRAN 77	Kommentar
<	.LT.	less than (kleiner als, <)
<=	.LE.	less equal (kleiner gleich, <=)
>	.GT.	greater than (größer als, >)
>=	.GE.	greater equal (größer gleich, >=)
==	.EQ.	equal (gleich, ==)
/=	.NE.	not equal (ungleich, !=)

Beispiel: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  integer :: a, b

  a = 5
  b = 6

  write (*,*) A < B
  ! Ausgabe: T
end program bsp
```

Beispiel: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  character(len=5) :: a, b

  a = "Halli"
  b = "Hallo"

  write (*,*) a < b
  ! Ausgabe: T
end program bsp
```


23.2. Operatorenpriorität

1. Klammerung
2. Arithmetische Operatoren
3. Vergleichsoperatoren
4. Logische Operatoren

24. Stringoperationen

24.1. Verknüpfungsoperator

Operator	Kommentar
//	Operator zum Verknüpfen von Strings

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(len=4) :: a
  character(len=10) :: b

  a = 'How '
  b = 'do you do.'

  write(*,*) a // b
  ! Ausgabe: How do you do.
end program bsp
```

24.2. Teilketten

Ein String ist ein `character`-Feld. Auf die Stringelemente kann wie in einem Feld zugegriffen werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(10) :: a

  a='Hallo Welt'

  write(*,*) a(2:4)
  ! Ausgabe: all

  write(*,*) a(5:)
  ! Ausgabe: o Welt

  write (*,*) a(:3)
  ! Ausgabe: Hal
end program bsp
```

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(10) :: a

  a='Hallo Welt'
  a(7:) = 'XYZ'

  write(*,*) a
  ! Ausgabe: Hallo XYZ
end program bsp
```

25. Verzweigungen und Schleifen

25.1. Einleitung

Neben der sequentiellen Programmausführung bietet Fortran 90/95 andere grundlegende Kontrollstrukturen zur Steuerung des Programmablaufs.

Verzweigungen 	Schleifen 	Terminierung 
<code>if</code> <code>select case</code>	<code>do</code>	<code>stop</code> <code>exit</code> <code>cycle</code>

Zwecks komfortabler Manipulation von Feldinhalten stehen zusätzliche Spezialkonstrukte zur Verfügung:

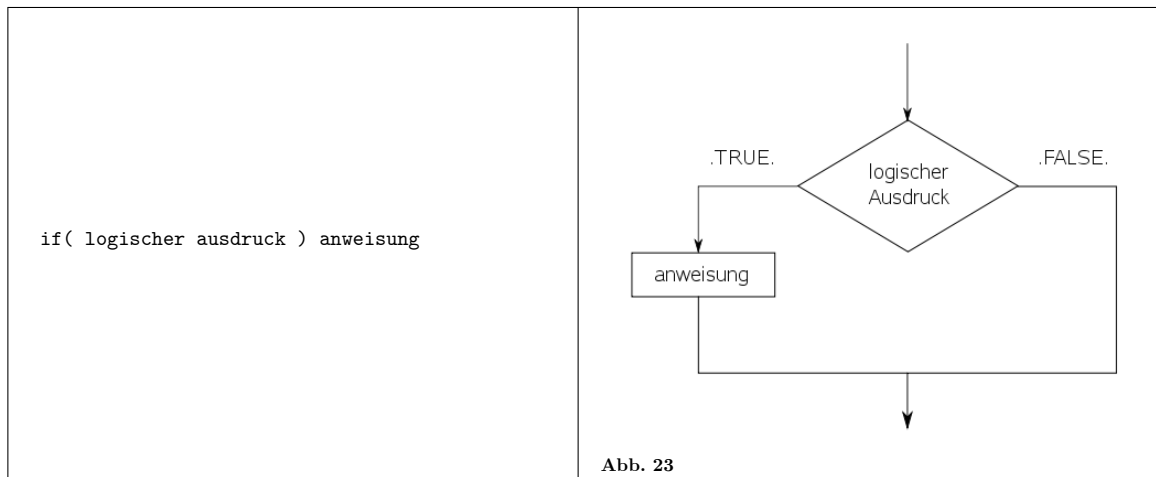
<code>where</code> <code>forall</code>

All diese Programmelemente werden in diesem Kapitel detailliert dargestellt. Neben den genannten Steuerkonstrukten sind in Fortran 90/95 auch noch das berüchtigte `goto` und einige als veraltet gekennzeichnete Sprunganweisungen möglich. Fortran 90 und Fortran 95 unterscheiden sich in Details bei Schleifen und Verzweigungen. Da Fortran 90 durch Fortran 95 bzw. beide durch den Fortran 2003-Standard abgelöst wurden, wird auf die Unterschiede nicht näher eingegangen.

25.2. if-Verzweigungen

25.2.1. Der if-Einzeiler

Form	Strukturbild
------	--------------



Der if-Einzeiler oder das "if statement" ist die einfachste Form, um in Fortran-Programmen eine Verzweigung zu realisieren. Ist der angegebene logische Ausdruck wahr, dann wird die Anweisung ausgeführt, sonst nach dem if-Einzeiler mit der Programmausführung fortgesetzt. Dieses Konstrukt kann nur eine einzelne Anweisung verdauen! Die gesamte if-Struktur inklusive Anweisung muss in einer Zeile angegeben werden. Eine konventionelle Zeilenfortsetzung mit dem &-Zeichen ist aber möglich. Einige Anweisungen sind nicht erlaubt. So kann die Anweisung nicht wieder ein "if statement" sein (geschachtelte if-Einzeiler). Ebenso darf der Anweisungsteil nicht aus `end program`, `end function` oder `end subroutine` bestehen.

Beispiel:

```

program bsp
  implicit none

  integer :: i

  i = 2

  ! Der logische Ausdruck ist .true. , "Hallo" wird ausgegeben
  if (i == 2) print *, 'Hallo'

  ! Das funktioniert nicht und ist deshalb auskommentiert
  ! if (i == 2)
  !   print *, 'Hallo'

  ! So geht's aber
  if (i == 2)                &
    print *, 'Hallo'

  ! Folgendes wiederum wird nicht das ev. erhoffte Resultat zeigen (= keine
  ! Ausgabe, da der logische Ausdruck .false.) Die durch Strichpunkt
  ! abgetrennte write-Anweisung gehört nicht mehr zum "if statement",
  ! sondern ist schon eine eigenständige Anweisung ausserhalb des
  ! if-statements
  if (i == 3) print *, 'Hallo 1'; print *, 'Hallo 2'

  ! Ausgabe:
  !   Hallo
  !   Hallo
  !   Hallo 2
end program

```

25.2.2. if-then

Kurzform	Langform	Strukturbild
<pre>if(logischer ausdruck) then anweisungsblock end if</pre>	<pre>name: if(logischer ausdruck) then anweisungsblock end if name</pre>	Abb. 24

Das if-then-Konstrukt erlaubt eine konventionelle einseitige Verzweigung. Im Gegensatz zum if-Einzeiler ist hier ein Anweisungsblock mit beliebig vielen Anweisungen erlaubt. Einem if-then-Konstrukt, wie auch den meisten nachfolgenden Verzweigungs- und Schleifentypen, kann eine Bezeichnung mitgegeben werden.

Beispiel: `<syntaxhighlight lang=fortran> program bsp`

```
implicit none

integer :: i

i = 2

if (i == 2) then
  print *, 'Hallo1'
end if
```

```
CHECKIT: if (i /= 3) then
  print *, 'Hallo2'
  exit CHECKIT
  print *, 'Hallo3'      ! <- wird nicht mehr ausgefuehrt
end if CHECKIT
```

```
! Ausgabe:
!  Hallo1
!  Hallo2
```

`end program </syntaxhighlight>`

25.2.3. if-then-else

Kurzform	Langform	Strukturbild
----------	----------	--------------

<pre> if(logischer ausdruck) then if-anweisungsblock else else-anweisungsblock end if </pre>	<pre> name: if(logischer ausdruck) then if-anweisungsblock else name else-anweisungsblock end if name </pre>	Abb. 25
--	--	----------------

Dies ist eine typische zweiseitige Verzweigung. Je nachdem, ob die Bedingung zu wahr oder falsch ausgewertet wird, wird der if- oder der else-Anweisungsblock ausgeführt, um nach dem Verzweigungsende wieder den gleichen Programmcode abzuarbeiten. Bei der Langform (benanntes if-then-else-Konstrukt) ist übrigens auch noch eine andere Variante möglich. So könnte der Bezeichner **name** nach **else** auch weggelassen werden. Nach dem **end if** ist er, sofern die benannte Form gewählt wurde, aber auf jeden Fall anzugeben.

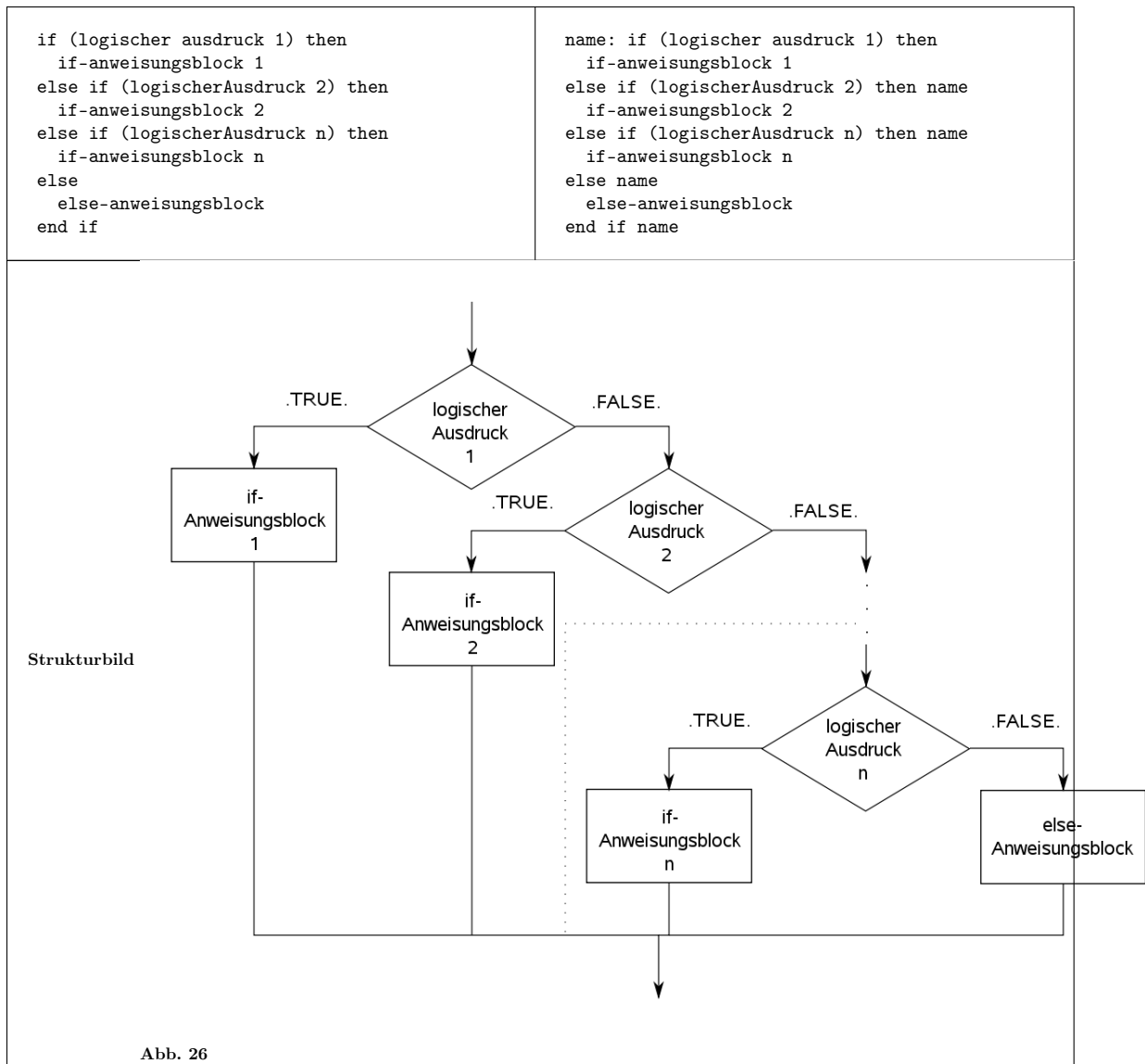
Beispiel: `<syntaxhighlight lang=fortran> program bsp`

<pre> implicit none real :: t, tc, tr character :: conv </pre>
<pre> print *, "Gib Temperatur in Kelvin ein:" read (*,*) t </pre>
<pre> CHECKRANGE: if (t >= 0.0) then print *, "Umrechnung (c -> in Celsius, anderes Zeichen -> in Rankine):" read (*,*) conv </pre>
<pre> if (conv == "c") then tc = t - 273.15 print *, tc else tr = 1.8 * t print *, tr end if else CHECKRANGE print *, "Fehler: t < 0.0 K" end if CHECKRANGE </pre>

`end program </syntaxhighlight>`

25.2.4. else-if

Kurzform	Langform
----------	----------



Das ist die allgemeinste Form der if-Verzweigung und wird im Fortran-Standard als "if construct" bezeichnet. Die vorher beschriebenen Formen sind nur vereinfachte Spezialfälle dieses Verzweigungstyps.

Beispiel: `<syntaxhighlight lang=fortran> program bsp`

```

implicit none

integer :: i

print *, "Gib eine natuerliche Zahl ein:"
read (*,*) i

! *** Langform ***
I999: if (i == 1) then
  print *, "A"

```



```

else if ((i > 1) .and. (i <= 5)) then I999
  print *, "BCDE"
else if ((i > 5) .and. (i <= 11)) then I999
  print *, "FGHIJK"
else I999
  print *, "L-Z"
end if I999

```

```

! *** Das Gleiche in Kurzform ***
if (i == 1) then
  print *, "A"
else if ((i > 1) .and. (i <= 5)) then
  print *, "BCDE"
else if ((i > 5) .and. (i <= 11)) then
  print *, "FGHIJK"
else
  print *, "L-Z"
end if

```

end program </syntaxhighlight>

25.3. Die select case-Verzweigung

Kurzform	Langform	Strukturbild
<pre> select case(variable) case(fall1) anweisungsblock1 <nowiki>!</nowiki> ... case(falln) anweisungsblockn [case default anweisungsblockdefault] end select </pre>	<pre> name: select case(variable) case(fall1) name anweisungsblock1 <nowiki>!</nowiki> ... case(falln) name anweisungsblockn [case default name anweisungsblockdefault] end select name </pre>	Abb. 27

Eine andere Möglichkeit zur Erstellung von Verzweigungen stellt die "select case"-Steueranweisung (das "case construct") bereit. Die **variable** kann vom Typ **integer**, **logical** oder **character** sein. Die Fälle werden durch Konstanten repräsentiert. Es muss nicht jeder Fall einzeln angeschrieben werden. Fälle, die zwar die gleichen Aktionen ausführen sollen, aber durch verschiedene Konstanten dargestellt werden, können zu einem Fall zusammengezogen werden. Optional kann auch noch ein "Default"-Fall angegeben werden.

Möglichkeiten um Fälle festzulegen:

n	Einzelne Konstante	gleich n
n1, n2, n3	Fallliste	n1 oder n2 oder n3
n:	Bereich	von n bis ...

:m	Bereich	von ... bis m
n:m	Bereich	von n bis m

Die in der Tabelle gelisteten Alternativen können natürlich auch kombiniert werden, z.B. eine Liste aus Einzelkonstanten und Bereichen. Bei logischem Datentyp ist die Angabe von Bereichen natürlich unsinnig.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer :: i

  read( *, * ) i

  select case( i )
    case( :111 )
      write( *, * ) 'Fall 1'
    case( 112:332, 334 )
      write( *, * ) 'Fall 2'
    case( 333 )
      write( *, * ) 'Fall 3'
    case default
      write( *, * ) 'unspezifiziertes Ereignis'
  end select

! Ausgabe (Eingabe: 222):
!   Fall 2
! Ausgabe (Eingabe: -5):
!   Fall 1
! Ausgabe (Eingabe: 333):
!   Fall 3
! Ausgabe (Eingabe: 5000):
!   unspezifiziertes Ereignis
end program bsp

```

25.4. do-Schleifen

Fortran kennt nur einen allgemeinen Schleifentyp – die do-Schleife. Diese do-Schleife gibt es aber in verschiedenen Ausprägungen, so dass damit ziemlich alle denkbaren Einsatzfälle abgedeckt sind.

Im Prinzip haben alle do-Schleifen die Form

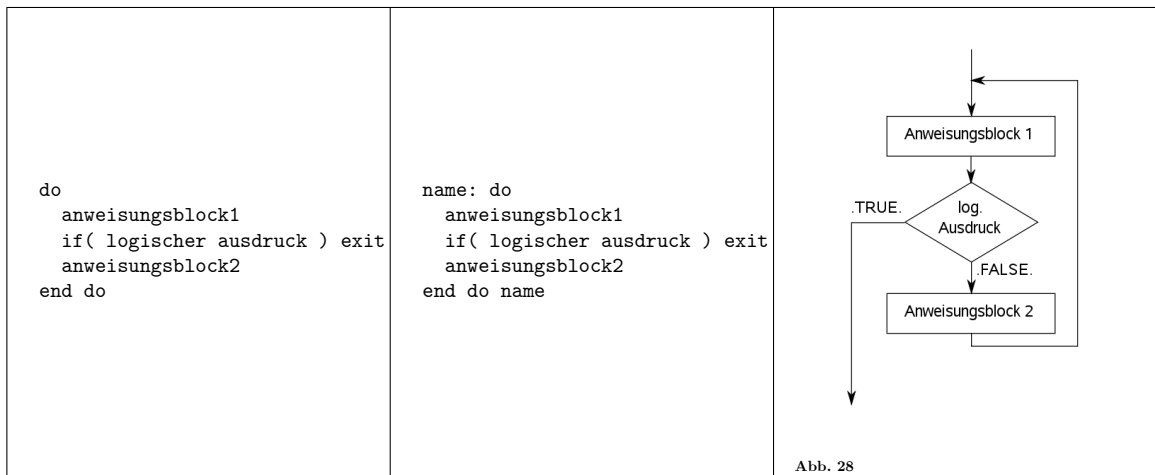
```

do [...]
  ...
end do

```

25.4.1. do-if-exit

Kurzform	Langform	Strukturbild
----------	----------	--------------



Lässt man den `anweisungsblock1` weg, dann erhält man eine kopfgesteuerte Schleife. Entfällt der `anweisungsblock2`, so ist die Schleife fußgesteuert. Ohne Abbruchbedingung ist dies eine Endlosschleife.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i = 1

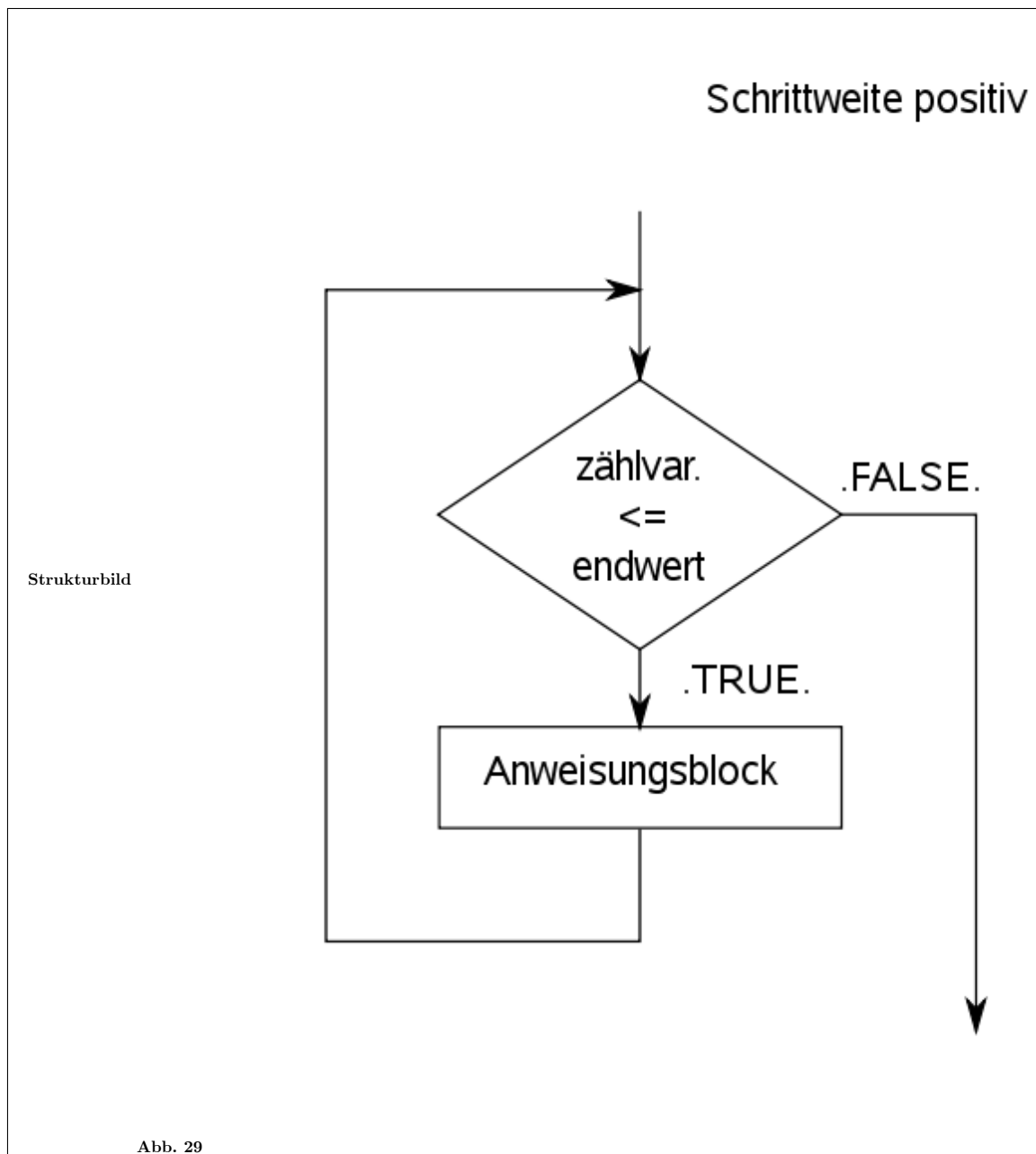
  do
    write( *, * ) i
    i = i + 1
    if( i > 10 ) exit
  end do

  write( *, * ) "Die do-Schleife wurde beendet"

! Ausgabe
! 1
! 2
! ...
! 10
! Die do-Schleife wurde beendet
end program bsp
```

25.4.2. Die do-Zählschleife

Kurzform	Langform
<pre>do zählvariable = startwert, endwert [, schrittweite] anweisungsblock end do</pre>	<pre>name: do zählvariable = startwert, endwert [, schrittweite] anweisungsblock end do name</pre>



Zählvariable, Startwert, Endwert und Schrittweite müssen vom Typ `integer` sein. Die Zählvariable darf in der Schleife nicht manipuliert werden. Wird die Schrittweite nicht explizit vorgegeben, so hat sie den Wert 1.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer :: i

```

```
do i = 1, 10
  write( *, * ) i
end do

! Zeilenweise Ausgabe der Zahlen 1 bis 10
end program bsp
```

25.4.3. do while

Kurzform	Langform	Strukturbild
<pre>do while(logischer ausdruck) anweisungsblock end do</pre>	<pre>name: do while(logischer ausdruck) anweisungsblock end do name</pre>	<pre> graph TD Entry(()) --> Decision{log. Ausdruck} Decision -- TRUE. --> Block[Anweisungsblock] Block --> Entry Decision -- FALSE. --> Exit(()) </pre> Abb. 30

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i

  i = 0

  do while( i < 5 )
    write( *, * ) i
    i = i + 1
  end do

! Die Zahlen 0 bis 4 werden ausgegeben
end program bsp
```

25.4.4. Die implizite do-Liste

Bei Eingabe oder Ausgabe ist die Angabe einer impliziten do-Liste möglich.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i
```

```

    write( *, * ) ( 'Hallo', i = 1, 10 )
! Ausgabe: HalloHalloHalloHalloHalloHalloHalloHalloHallo
end program bsp

```

25.5. Spezialkonstrukte für Felder

25.5.1. where

Sollen in Feldern Elemente manipuliert werden, so können die **where**-Anweisungen (das "masked array assignment") hilfreich sein. Dabei lassen sich Feldelementen in Abhängigkeit von vorgegebenen Bedingungen (Maske) neue Werte zuweisen.

Der where-Einzeiler

Form
<pre>where(bedingung) variable = ausdruck</pre>

Das "where statement" muss komplett in eine Zeile geschrieben werden, wobei Zeilenumbrüche mit dem &-Zeilenumbruchsmarker jedoch erlaubt sind. Für komplexere Probleme ist das im nächsten Abschnitt vorgestellte "where construct" (where-elsewhere) vorgesehen.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer, dimension(5) :: arr = (/ 5, 1, -1, 1, 7 /)

  write( *, * ) arr
! Ausgabe: 5  1 -1  1  7

  where( arr >= 3 ) arr = 99999
  write(*,*) arr
! Ausgabe: 99999 1  -1  1  99999
end program bsp

```

where-elsewhere

Kurzform	Langform
----------	----------

<pre> where(bedingung1) anweisungsblock [elsewhere(bedingung2) anweisungsblock2] [elsewhere(bedingungn) anweisungsblockn] [elsewhere anweisungsblockm] end where </pre>	<pre> name: where(bedingung1) anweisungsblock [elsewhere(bedingung2) name anweisungsblock2] [elsewhere(bedingungn) name anweisungsblockn] [elsewhere name anweisungsblockm] end where name </pre>
---	---

Beispiel: Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  integer, dimension(10) :: arr = (/ 5, 1, -1, 1, 7, -3, 7, 6, -9, 8 /)

  where( arr <= 0 )
    arr = 0
  elsewhere( arr > 0 .and. arr <= 3 )
    arr = 1
  elsewhere
    arr = 2
  end where

  write(*,*) arr
! Ausgabe: 2  1  0  1  2  0  2  2  0  2
end program bsp

```

25.5.2. forall

Auch die **forall**-Schleife ist für den Einsatz bei Feldern gedacht. Im Gegensatz zum "where construct" werden hier die ausgewählten Feldelemente in erster Linie über die Indizes und erst in zweiter Linie über den Wert bestimmt. Mit Fortran 2018 ist **forall** als *obsolescent feature* gekennzeichnet und sollte nicht mehr verwendet werden.

Der forall-Einzeiler

Formen
<pre>forall(index = subscript:subscript[:stride]) anweisung</pre>
<pre>forall(ind1 = subs1a:subs1b[:str1], ind2 = subs2a:subs2b[:str2] [, indn = subsna:subsnb[:strn]]) anweisung</pre>

Die erste angegebene Form ist für eindimensionale Felder gedacht, die zweite für mehrdimensionale Felder. Über **subscript** kann der Indexbereich festgelegt werden. Das optionale **stride** gibt die Schrittweite vor. Wird dieser Wert nicht angegeben, so ist die Schrittweite = 1.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer, dimension(5)      :: a = (/ 1, 2, 3, 4, 5 /)
  integer, dimension(2, 2)   :: b = reshape( (/ 0, 2, -1, 5 /), (/ 2, 2 /) )

  integer                    :: i, j

  forall( i = 1:3 ) a(i) = 0
  write( *, * ) a
  ! Ausgabe: 0  0  0  4  5

  forall( i = 1:5:2 ) a(i) = -1
  write( *, * ) a
  ! Ausgabe: -1  0  -1  4  -1

  forall( i = 1:5 ) a(i) = max ( a(i), 3 )
  write( *, * ) a
  ! Ausgabe: 3  3  3  4  3

  forall( i = 1:2, j = 2:2 ) b(i, j) = -9
  write( *, * ) b
  ! Ausgabe:  0  2  -9  -9
end program bsp

```

Aber das ist nicht alles, was der forall-Einzeiler zustande bringt. Zusätzlich zur Feldelementbestimmung über Indexbereiche kann auch eine Maske vorgegeben werden. Die forall-Schleife ist also auch eine erweiterte where-Anweisung.

Form

```
forall( ind1 = s1a:s1b[:s1] [, indn = sna:snb[:sn] [, maske] ) anweisung
```

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer, dimension(5)      :: a = (/ 1, 2, 3, 4, 5 /)

  integer                    :: i, j

  forall( i = 1:4, a(i) > 2 ) a(i) = 0
  write( *, * ) a
  ! Ausgabe: 1  2  0  0  5
end program bsp

```

Bei der Verwendung von forall-Anweisungen sind einige Prämissen zu beachten. So müssen nebst anderem in Masken oder im Anweisungsbereich verwendete Funktionen **pure** sein. Was das genau bedeutet wird später im Kapitel Unterprogramme⁴² erläutert

42 Kapitel 29.3 auf Seite 174

Beispiel: Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  integer, dimension(5)    :: a = (/ 1, 2, 3, 4, 5 /)

  integer                  :: i

  ! Die Funktion berechne_1() darf nicht innerhalb einer forall-Anweisung
  ! aufgerufen werden, da sie nicht "pure" ist
  ! forall( i = 1:4, a(i) > 2 ) a(i) = berechne_1( a(i) ) ! FEHLERMELDUNG

  ! Die Funktion berechne_2 macht das Gleiche wie berechne_1(),
  ! ist aber als "pure" gekennzeichnet und darf in der forall-Anweisung
  ! aufgerufen werden
  forall( i = 1:4, a(i) > 2 ) a(i) = berechne_2( a(i) ) ! OK

  write( *, * ) a
  ! Ausgabe: 1  2  9 12  5

contains
  integer function berechne_1( val )
    integer, intent( in ) :: val
    integer                :: res

    res = val * 3
    berechne_1 = res
  end function berechne_1

  pure integer function berechne_2( val )
    integer, intent( in ) :: val
    integer                :: res

    res = val * 3
    berechne_2 = res
  end function berechne_2
end program bsp

```

Das "forall construct"

Kurz- form	<pre>forall(index = subscript:subscript[:stride]) anweisungsblock end forall</pre>
Lang- form	<pre>name: forall(index = subscript:subscript[:stride]) anweisungsblock end forall name</pre>

Es gilt im Prinzip das gleiche wie für den forall-Einzeiler. Es sind innerhalb der Schleife eben mehrere Anweisungen erlaubt. Es gelten auch die Formen für mehrdimensionale Felder und mit Vorgabe einer Maske. Diese sind in diesem Abschnitt aber nicht mehr explizit angeführt.

25.6. Terminierung

25.6.1. stop

Die `stop`-Anweisung beendet das Programm.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  write( *, * ) 'Vor Stop-Statement'
  stop
  write( *, * ) 'Nach Stop-Statement'

! Ausgabe:
!   Vor Stop-Statement
end program bsp
```

Der `stop`-Anweisung kann auch ein "stop-code" mitgegeben werden. Dieser "stop-code" kann eine Ganzzahl (maximal 5-stellig) oder eine Zeichenkette sein. Der "stop-code" wird bei der Programmtermination ausgegeben. Wo die Ausgabe erfolgt und wie sie aussieht ist aber betriebssystem- und compilerabhängig.

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  integer :: i

  read( *, * ) i

  if( i == 1 ) then
    stop 999
  else
    stop "Sonstiger Stop"
  end if

! Ausgabe (bei Eingabe von 1):
! ifort (Linux):
!   999
! gfortran, g95 (Linux):
!   STOP 999
! Sun f95 (Linux):
!   STOP: 999
end program bsp
```

25.6.2. exit

Mit der Anweisung `exit` kann eine `do`-Schleife verlassen werden. Wird eine `do`-Schleife mit einer Bezeichnung versehen, so kann bei der `exit`-Anweisung explizit auf die benannte Schleife Bezug genommen werden. Wird kein Schleifenname angegeben, so bezieht sich die `exit`-Anweisung immer auf die innerste Schleife, mit der sie im Zusammenhang steht.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i, j

  outerloop: do i = 1, 5
    write( *, * ) "Outer", i, "Beginn"

    innerloop: do j = 1, 3
      if( i == 1 ) exit
      if( i == 2 ) exit innerloop
      if( i == 4 ) exit outerloop
      write( *, * ) "Inner", j
    end do innerloop

    write( *, * ) "Outer", i, "Ende"
  end do outerloop

! Ausgabe:
! Outer 1 Beginn
! Outer 1 Ende
! Outer 2 Beginn
! Outer 2 Ende
! Outer 3 Beginn
! Inner 1
! Inner 2
! Inner 3
! Outer 3 Ende
! Outer 4 Beginn
end program bsp
```

25.6.3. cycle

Mit `cycle` wird der aktuelle do-Schleifendurchlauf beendet und wieder zum Schleifenkopf gesprungen. Wird eine do-Schleife mit einer Bezeichnung versehen, so kann bei der `cycle`-Anweisung explizit auf die benannte Schleife Bezug genommen werden. Wird kein Schleifenname angegeben, so bezieht sich die `cycle`-Anweisung immer auf die innerste Schleife, mit der sie im Zusammenhang steht.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i, j

  outerloop: do i = 1, 5
    write( *, * ) "Outer", i, "Beginn"

    innerloop: do j = 1, 3
      if( i == 1 ) cycle outerloop
      if( i == 4 ) cycle innerloop
      if( i == 5 ) cycle
      write( *, * ) "Inner", j
    end do innerloop

    write( *, * ) "Outer", i, "Ende"
  end do outerloop

! Ausgabe:
! Outer      1 Beginn
```

```
! Outer      2 Beginn
! Inner      1
! Inner      2
! Inner      3
! Outer      2 Ende
! Outer      3 Beginn
! Inner      1
! Inner      2
! Inner      3
! Outer      3 Ende
! Outer      4 Beginn
! Outer      4 Ende
! Outer      5 Beginn
! Outer      5 Ende
end program bsp
```


26. Datentypen höherer Genauigkeit

Fortran 95 bietet für die Festlegung von Datentypen höherer Genauigkeit einen neuen Ansatz.

26.1. Einfache Variante

Mittels `kind` (dt.: Art, Sorte, Gattung, Wesen) kann die Genauigkeit bzw. der Wertebereich eines Datentyps festgelegt werden. Die einfache Variante ist aber system- und compilerabhängig.

`datentyp (kind=wert) :: var`

Kurzform:

`datentyp (wert) :: var`

Für `wert` ist meist (aber nicht immer) die Anzahl der Bytes einzusetzen. Literale eines solchen Datentyps sind mit einem entsprechenden Anhängsel zu kennzeichnen, z.B.:

- `3.141592654_8`
- `-4.9e55_8`

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real(kind=8) :: variable
  ! variable ist nun vom Datentyp "double precision"

  variable = 1.555555555555_8

  write(*,*) variable
  ! Ausgabe: 1.555555555555000

  ! Hier wird nur eine gewöhnliche real-Zahl mit 7 Nachkommastellen
  ! zugewiesen
  variable = 1.555555555555

  write(*,*) variable
  ! Ausgabe: 1.55555558204651
end program bsp
```

26.2. System- und compilerunabhängige Variante

Das `kind`-Attribut darf über eine Konstante oder eine benannte Konstante belegt werden. So kann statt

```
real(kind=8) :: var
```

auch

```
integer, parameter :: dp = 8  
real(kind=dp)      :: var
```

geschrieben werden. Der entsprechende `kind`-Wert ist auch per Funktion ermittelbar. Die Variable `var`

```
integer, parameter :: dp = kind(0.0d0)  
real(kind=dp)      :: var
```

ist in diesem Beispiel somit vom Datentyp `double precision`. Das ist schon compilerunabhängig verwendbarer Fortran-Code.

Nachfolgend wird die `selected_real_kind(p, r)`-Funktion vorgestellt, die durch Vorgabe der gewünschten Nachkommastellen und/oder des Exponentenmaximalwertes einen potentiell geeigneten `kind`-Wert für Gleitkommazahlen ermittelt.

26.2.1. `selected_real_kind`

```
integer, parameter :: name = selected_real_kind(anzahl_nachkommastellen,  
max_exponentenwert)
```

Die Funktion `selected_real_kind` gibt einen Wert zurück, der alle Gleitkommazahlen mit mindestens `anzahl_nachkommastellen` Dezimalstellen Genauigkeit und einem Exponentenbereich von `max_exponentenwert` berücksichtigt. Gibt es keinen solchen systemspezifischen Wert, so wird einer der folgenden Werte zurückgegeben:

- -1 ... die geforderte Anzahl an Dezimalstellen ist nicht verfügbar
- -2 ... der Exponentenbereich ist nicht verfügbar
- -3 ... nichts von beidem ist verfügbar

Wird eine solche Konstante mit negativem Wert nachfolgend als `kind`-Wert bei der Deklaration einer Variablen, Konstanten etc. verwendet, so führt dies in aller Regel zu einer Fehlermeldung bereits beim Kompilervorgang.

Es ist auch möglich die `selected_real_kind`-Funktion mit nur einem Argument aufzurufen. Die Argumente sind mit `p` (für *precision*) bzw. `r` (für *range*) benannt.

Beispiele:

```
integer, parameter :: dp = selected_real_kind(r=60)
```

```
integer, parameter :: dp = selected_real_kind(p=15)
```

```
integer, parameter :: ultrap = selected_real_kind(1000, 5000) ! liefert
höchstwahrscheinlich                                     ! die negative
Zahl -3
```

26.2.2. Variablendeklaration

```
real (kind=name) :: variable
```

Kurzform:

```
real (name) :: variable
```

Beispiel:

```
integer, parameter :: dp = selected_real_kind(r=60)
real(kind=dp)      :: var
```

26.2.3. Konstanten

```
zahl_name
```

Literale eines solchen Datentyps sind mit dem im Programmcode festgelegten charakteristischen Anhängsel zu kennzeichnen, z.B.:

- 3.141592654_dp
- -4.9e55_dp

26.2.4. Beispiel

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  integer, parameter :: dp = selected_real_kind(15, 300)

  real(kind=dp) :: variable

  variable = 1.5555555555_dp / 2_dp

  write(*,*) variable
  ! Ausgabe: 0.777777777750000
```



```
write(*,*) kind(variable)
! Ausgabe: 8
end program bsp
```

27. Datenverbund

27.1. Einleitung

Datenelemente lassen sich auch zu eigenen zusammengesetzten Datentypen verbinden. Der Datenverbund ist in anderen Programmiersprachen auch als Struktur (structure, struct) bekannt. Der Fortran 95-Standard spricht vom "derived type", also einem abgeleiteten Datentyp. Zulässig als Komponenten eines Datenverbundes sind Skalare, Felder, Zeiger oder auch andere Datenverbunde. Ein Datenverbund ist durch das Schlüsselwort `type` gekennzeichnet und kann in folgenden Formen auftreten

<code>type name</code>	<code>type [,zugriffsspezifizierer] :: name</code>
<code>...</code>	<code>...</code>
<code>end type [name]</code>	<code>end type [name]</code>

Die Elemente in eckigen Klammern sind optional. Die Angabe eines Zugriffsspezifizierers (`public`, `private`) ist nur bei Verwendung von Datenverbunden in Modulen erlaubt.

27.2. Einen Datenverbund anlegen

```
type :: name
  [sequence] ! optional sequentielle Speicherplatzablage

  datentyp :: komponentenname_1
  <nowiki>!</nowiki> ...
  datentyp :: komponentenname_n
end type [name]
```

Beispiel (Programmausschnitt): **Fortran 90/95-Code (free source form)**

```
! ...

type :: koord
  integer          :: id
  real, dimension(3) :: x
end type koord

! ...
```

27.3. Eine Variable vom Typ "Datenverbund" anlegen

Die Variablendeklaration erfolgt in der Form

```
type( name ) :: var
```

Beispiel (Programmausschnitt): **Fortran 90/95-Code (free source form)**

```
! ...  
  
type :: koord  
  integer          :: id  
  real, dimension(3) :: x  
end type koord  
  
type( koord ) :: k  
  
! ...
```

27.4. Zugriff auf Einzelkomponenten eines Datenverbunds

Eine Einzelkomponente in einem Datenverbund lässt sich als Kombination des Datenverbundvariablenbezeichners und des Komponentennamen ansprechen. Verknüpft werden diese beiden Bezeichner mit dem %-Zeichen.

```
var%komponentenname
```

Beispiel (Programmausschnitt): **Fortran 90/95-Code (free source form)**

```
! ...  
  
type :: koord  
  integer          :: id  
  real, dimension(3) :: x  
end type koord  
  
type( koord ) :: k  
  
k%id = 999  
k%x(1) = 0.0  
k%x(2) = 20.5  
k%x(3) = 10.0  
  
write( *, * ) k%x(2)  
  
! ...
```

27.5. Zugriff auf einen Datenverbund als Ganzes

Ein Datenverbund kann in bestimmten Situation nicht nur komponentenweise angesprochen werden, sondern auch als Einheit, z.B. bei der Ein-/Ausgabe oder beim Zuweisen der Werte einer Datenverbundvariablen an eine andere.

Beispiel (Programmausschnitt): **Fortran 90/95-Code (free source form)**

```
! ...

type :: koord
  integer          :: id
  real, dimension(3) :: x
end type koord

type( koord ) :: k1, k2

read( *, * ) k1
k2 = k1
write( *, * ) k2

! ...
```

27.6. Elemente eines Datenverbunds initialisieren

27.6.1. Default-Werte im Datenverbund setzen

Innerhalb eines Datenverbundes lassen sich die einzelnen Variablen mit Vorgabewerten belegen, z.B.

```
integer          :: i = 5
character, dimension( 2, 2 ) :: c = reshape( (/ "A", "B", "C", "D" /), (/ 2, 2
/) )
```

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  type :: koord
    integer          :: id = -999999
    real, dimension( 3 ) :: x = 0.0
  end type koord

  type( koord ) :: k

  write( *, * ) k

! Ausgabe:
!  -999999  0.0  0.0  0.0
end program bsp
```

27.6.2. Der "structure constructor"

Die Initialisierung der Datenverbundelemente bzw. eine spätere Wertzuweisung kann auch per "structure constructor" ("derived type value constructor") vorgenommen werden.

```
name( wertliste )
```

Beispiel: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  type :: koord
    integer      :: id
    real, dimension( 3 ) :: x
  end type koord

  type( koord ) :: k = koord( 12, (/ 0.0, 1.5, 20.5 /) )
  write( *, * ) k

  k = koord( 13, (/ 5.5, 0.0, 0.5 /) )
  write( *, * ) k

! Ausgabe:
!  12  0.0  1.5  20.5
!  13  5.5  0.0   0.5
end program bsp
```

Enthält ein Datenverbund bereits Variablen mit Vorgabewerten, so werden diese durch die Werte im "structure constructor" überschrieben.

Beispiel: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  type :: person
    character(25) :: vorname = 'NN'
    character(25) :: nachname = 'NN'
    integer      :: alter
  end type person

  type(person), dimension(5) :: p

  p(1) = person('Harri', 'P.', 41)
  p(5) = person('Creszenzia', 'T.', 18)

  write(*, fmt='111') p
! Ausgabe:
!   Harri                P.                41
!   NN                   NN                 0
!   NN                   NN                 0
!   NN                   NN                 0
!   Creszenzia           T.                 18

  write(*,*) p(1)%vorname
! Ausgabe:
!   Harri
```

```

p(2)%vorname = 'Holger'
write(*, '(A)') p%vorname
! Ausgabe:
!   Harri
!   Holger
!   NN
!   NN
!   Creszenzia

111 format(2A, I3)
end program bsp

```

Beispiel: Verwendung eines Datenverbunds in einem anderen Datenverbund **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  type :: tripel
    real :: x, y, z
  end type tripel

  type :: koerper
    integer :: id
    type( tripel ) :: bezugscoordinate
    type( tripel ) :: orientierung
  end type koerper

  type( koerper ) :: k

  k = koerper( 1005, tripel( 10.5, 0.0, -6.5 ), tripel( 0.0, 0.0, -0.8 ) )

  write( *, * ) k
! Ausgabe:
!   1005   10.50000      0.000000      -6.500000      0.000000
!   0.000000      -0.8000000
end program bsp

```

27.7. Das Attribut `sequence`

Im Datenverbund kann zu Beginn auch das Attribut `sequence` angegeben werden. Dieses spezifiziert, dass die Werte eines Datenverbunds sequentiell gespeichert werden. Wird dieses Attribut weggelassen, so ist nicht gewährleistet, dass die Werte von Datenverbundvariablen in der selben Reihenfolge, wie sie im Datenverbund gereiht wurden im Speicher wiederzufinden sind, oder dass sie zusammenhängend gespeichert werden. Oft wird die Speicherreihenfolge keine wesentliche Rolle spielen. Ein Spezialfall, wo die Speicherreihenfolge jedoch essenziell ist, wird anschließend dargestellt.

27.8. Die Verwendung eines Datenverbunds in separaten Programmeinheiten

Ein Datenverbund kann über ein Modul verschiedenen Programmeinheiten bekannt gemacht werden. Dazu aber später. Es ist nämlich auch so möglich in komplett getrennten Programmeinheiten auf ein und denselben Datenverbund-Datentyp zuzugreifen.

Beispiel: Ein ähnliches Beispiel findet sich auch im J3/97-007R2 Working Draft, Note 4.31 **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  type :: koord
    sequence
    integer          :: id
    real, dimension( 3 ) :: x
  end type koord

  type( koord ) :: k = koord( 555, (/ 3.0, 4.0, 5.5 /) )

  call ausgabe( k )

! Ausgabe
!   555   3.0000000   4.0000000   5.5000000
end program bsp

subroutine ausgabe( coord )
  type :: koord
    sequence
    integer          :: id
    real, dimension( 3 ) :: x
  end type koord

  type( koord ), intent( in ) :: coord

  write( *, * ) coord
end subroutine ausgabe
```

In diesem speziellen Fall muss der Datenverbund lt. Standard mit dem Attribut **sequence** versehen werden. Des Weiteren muss der im Hauptprogramm deklarierte Datenverbund komplett identisch mit jenem im Unterprogramm sein (Datenverbundname, Variablenbezeichner, Reihenfolge, Datentypen). In der Praxis sehen das viele Compiler nicht so eng. Allerdings können bei Nichtbeachtung dieser Vorgaben spätestens beim Wechsel des Compilers oder auf andere Rechnerarchitekturen größere Probleme auftreten.

28. Standardfunktionen

28.1. Allgemeine Hinweise

Die in den nachfolgenden Abschnitten angeführten Tabellen und Listen geben die im Fortran 95-Working Draft gelisteten intrinsischen Funktionen in simplifizierter Form wieder. Einzelheiten stehen jeweils in Unterkapiteln.

Auch die Standard-Subroutinen wurden berücksichtigt. Die einzelnen Compiler kennen zum Teil wesentlich mehr Funktionen als im Standard vorgegeben. Bei Verwendung solcher Funktionen sind die Programme jedoch nicht mehr quellcodekompatibel. Auf die Wiedergabe solcher compilerspezifischen Funktionen wird hier deshalb verzichtet.

28.2. Tabellenlegende

Abkürzung	Beschreibung
i	Integer-Datentyp (integer)
r	Real-Datentyp (real)
x	Complex-Datentyp (complex)
d	Double-precision-Datentyp (real(z, kind(0.0D0)))
z	beliebiger numerischer Datentyp (integer, real, complex)
c	Zeichen (character)
l	Logical-Datentyp (logical)
any	beliebiger intrinsischer Datentyp
arr	Feld (Array)
aarr	dynamisches Feld
ptr	Zeiger (Pointer)

28.3. Datentypfunktionen

- Umwandlung in INTEGER
- Umwandlung in REAL
- Umwandlung in DOUBLE PRECISION
- Umwandlung in COMPLEX
- Umwandlung in CHARACTER
- kind-Parameter

Einzelheiten siehe Datentypfunktionen⁴³

⁴³ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Datentypfunktionen%20>

28.4. Mathematische Funktionen

- Rundung
- Absolutwert
- Modulo
- Vorzeichentransfer
- Positive Differenz
- Maximum
- Minimum
- Komplexe Zahlen
- Quadratwurzel
- Exponentialfunktion
- Logarithmen
- Winkelfunktionen
- Arkusfunktionen
- Hyperbelfunktionen

Einzelheiten siehe Mathematische Funktionen⁴⁴

28.5. Stringfunktionen

- Lexikalische Funktionen
- Sonstige

Einzelheiten siehe Stringfunktionen⁴⁵

28.6. Feldfunktionen

- Konstruktion und Umgestaltung von Feldern
- Abfragen von Feldstatus, Felddaten und Feldmetadaten
- Funktionen für Vektoren und Matrizen
- Sonstige

Einzelheiten siehe Feldfunktionen⁴⁶

28.7. Zeigerfunktionen

Einzelheiten siehe Zeigerfunktionen⁴⁷

⁴⁴ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Mathematische%20Funktionen%20>

⁴⁵ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Stringfunktionen%20>

⁴⁶ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Feldfunktionen%20>

⁴⁷ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Zeigerfunktionen%20>

28.8. Bitfunktionen

Einzelheiten siehe Bitfunktionen⁴⁸

28.9. Weitere Funktionen

Einzelheiten siehe Weitere Funktionen⁴⁹

28.10. Intrinsische Subroutinen

Einzelheiten siehe Intrinsische Subroutinen⁵⁰

⁴⁸ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Bitfunktionen%20>
⁴⁹ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Weitere%20Funktionen%20>
⁵⁰ <https://de.wikibooks.org/wiki/Fortran%3A%20Fortran%2095%3A%20Standardfunktionen%3A%20Intrinsische%20Subroutinen%20>

29. Unterprogramme

Werden Programme umfangreicher und komplexer oder werden einzelne Programmabschnitte öfter verwendet, dann ist es sinnvoll Unterprogramme (Prozeduren) zu verwenden. Fortran 95 kennt zwei Typen von Unterprogrammen:

- Funktionen (**function**)
- Subroutinen (**subroutine**)

Darüber hinaus existiert in Fortran die Möglichkeit Unterprogramme und Daten mittels Modulen in das Programm einzubeziehen.

29.1. Das **function** Unterprogramm

Eine Funktion ist ein Unterprogramm, das genau einen Wert zurück gibt. Welcher Art von Datentyp der Wert ist, wird durch eine Zuweisung an den Funktionsnamen erreicht (z. B. **real**).

Die allgemeine Funktionsdeklaration lautet:

```
datentyp function funktionsname ( [formale parameter] )  
  Vereinbarungsteil  
  Anweisungsteil  
  funktionsname = wert  
  [return]  
end function funktionsname
```

Wird der Datentyp bei der Funktionsdeklaration nicht angegeben, so muss der Datentyp der Funktion im Vereinbarungsteil festgelegt werden:

```
function funktionsname ( [formale parameter] )  
  Vereinbarungsteil  
  datentyp :: funktionsname  
  Anweisungsteil  
  funktionsname = wert  
  [return]  
end function funktionsname
```

Mittels **return** kann zur aufrufenden Programmeinheit zurückgekehrt werden. Ein **return** unmittelbar vor der **end**-Anweisung ist nicht unbedingt erforderlich, da das Programm beim Erreichen der **end**-Anweisung zur aufrufenden Programmeinheit zurück springt. Deswegen

sollte bei allen neueren Programmen darauf verzichtet werden. Wichtig ist die **return**-Anweisung für alternative Rücksprünge zur aufrufenden Programmeinheit.

Eine Funktion wird meist mittels folgender allgemeiner Anweisung von der ausführenden Programmeinheit aufgerufen:

```
variable = funktionsname( aktuelle parameter )
```

Dabei orientiert sich diese Unterprogrammtechnik direkt an mathematischen Funktionen. Im folgenden Beispiel ruft das Programm **test** die Funktion **funk** auf, führt das Unterprogramm aus und gibt den entsprechenden Rückgabewert an das Programm **test** zurück:

Datei *bsp.f95*: **Fortran 90/95-Code (free source form)**

```
program test
!
implicit none

real :: funk
real :: x, y

write(*,*) 'x -> '
read(*,*) x

y = funk( x )

write(*,*) 'funk -> ', y
end program test

real function funk( a )
!
implicit none

real :: a

funk = a**2
end function funk
```

Das Unterprogramm **funk** kann sich direkt unter der aufrufenden Programmeinheit in der *selben* Textdatei oder in einer *separaten* Datei befinden. Befindet sich das Unterprogramm in einer separaten Datei, so muss diese und die aufrufende Programmeinheit zusammen kompiliert werden:

Übersetzung mit *gfortran* im vorliegenden Fall:

```
gfortran -o bsp bsp.f95
```

Übersetzung mit *gfortran* bei separaten Dateien (bspw.):

```
gfortran -o bsp bsp.f95 funk.f95
```

Es ist aber auch möglich eine Funktion nur durch ihren Namen aufzurufen. Der Rückgabewert der Funktion wird dann direkt ausgegeben:

Datei *bsp.f95*: **Fortran 90/95-Code (free source form)**

```

program test
!
implicit none

real :: funk
real :: x

write(*,*) 'x -> '
read(*,*) x

write(*,*) 'funk -> ', funk( x )

end program test

real function funk( a )
!
implicit none

real :: a

funk = a**2

end function funk

```

29.2. Das subroutine Unterprogramm

Eine **subroutine** ist ein Unterprogramm, das mehr als einen Wert zurückgeben kann. Eine Subroutine besitzt im Gegensatz zu einer Funktion keinen Datentyp und Rückgabewert.

Die allgemeine Deklaration einer Subroutine lautet:

```

subroutine subroutinenname ([formale parameter])
  Vereinbarungsteil
  Anweisungsteil
  [return]
end subroutine subroutinenname

```

Mittels **return** kann zur aufrufenden Programmeinheit zurückgekehrt werden. Ein **return** unmittelbar vor der **end**-Anweisung ist nicht unbedingt erforderlich, da das Programm beim Erreichen der **end** zur aufrufenden Programmeinheit zurück springt. Deswegen sollte bei allen neueren Programmen darauf verzichtet werden. Wichtig ist die **return** Anweisung für alternative Rücksprünge zur aufrufenden Programmeinheit.

Aufgerufen wird eine Subroutine aus der aufrufenden Programmeinheit mittels der **call**-Anweisung:

```
call subroutinenname ([aktuelle parameter])
```

Im folgenden Beispiel wird die Subroutine `sub` aufgerufen. Diese gibt dann zwei Werte zurück:

Datei *bsp.f95* **Fortran 90/95-Code (free source form)**

```
program bsp
!
implicit none

real :: x, y, z

write( *, * ) 'x -> '
read( *, * ) x

call sub( x, y, z )

write( *, * ) 'y -> ', y, 'z -> ', z

end

subroutine sub( a, b, c )
!
implicit none

real :: a, b, c

b = a**2
c = a**2 + b

end subroutine sub
```

29.3. Weitere Anweisungen für die Unterprogrammtechnik

29.3.1. Das `intent`-Attribut

Im Vereinbarungsteil der Funktion wird der Parameterdeklaration das Schlüsselwort `intent` mitgegeben.

datentyp, intent(in) :: var

Mit `intent( in )` wird angezeigt, dass der Parameter `var` in der Funktion nicht geändert wird und kein Rückfluss der Information in die aufrufende Programmeinheit stattfindet. Ein `intent( out )` oder `intent( inout )` wie bei Subroutinen wäre meist auch möglich, widerspricht aber dem Grundgedanken des Fortran-Funktionsbegriffs und der strukturierten Programmierung. Bei einer Funktion soll der Informationsrückfluss über den Rückgabewert stattfinden und nicht über Parameter.

Bei der Parameterübergabe bietet eine Subroutine folgende `intent`-Möglichkeiten:

- *datentyp*, intent(in) :: var ... Informationsfluss von der aufrufenden Programmeinheit in die Funktion
- *datentyp*, intent(out) :: var ... Informationsfluss von der Subroutine zur aufrufenden Programmeinheit
- *datentyp*, intent(inout) :: var ... beidseitiger Informationsfluss

Beispiel: Fortran 90/95-Code (free source form)

```

program bsp
  implicit none
  real :: a, b, c

  a = 1.0
  b = 2.0
  c = 3.0

  call sub(a, b, c)

  write (*,*) 'Hauptprogramm: ', a, b, c
  ! Ausgabe: 1.000000      22.20000      33.30000
end

subroutine sub(x, y, z)
  implicit none
  real, intent(in)    :: x
  real, intent(out)   :: y
  real, intent(inout) :: z

  write (*,*) 'Unterprogramm: ', x, y, z
  ! Ausgabe: 1.000000      2.000000      3.000000

  y = 22.2
  z = 33.3
end subroutine sub

```

Die aktuellen und formalen Parameter müssen hinsichtlich Datentyp, Anzahl, Reihenfolge übereinstimmen.

29.3.2. Die pure Anweisung

Ein Unterprogramm welches keine Seiteneffekte hat ist ein bloßes bzw. reines (pure) Unterprogramm. Ein Unterprogramm erzeugt dann keine Seiteneffekte, wenn es weder seine Eingabedaten, noch die Daten verändert, die außerhalb des Unterprogrammes liegen, es sei denn, es wären seine Ausgabedaten. In einem reinen Unterprogramm haben die lokalen Variablen keine **save** Attribute, noch werden die lokalen Variablen in der Datendeklaration initialisiert.

Reine Unterprogramme sind für das **forall**-Konstrukt notwendig: das **forall**-Konstrukt wurde für das parallele Rechnen konzipiert, weshalb hier der Computer entscheidet, wie das Konstrukt abgearbeitet werden soll. Dazu ist es aber notwendig, dass es egal ist in welcher Reihenfolge das Konstrukt abgearbeitet wird. Gilt dies nicht - hat das Unterprogramm also Seiteneffekte - so kann das **forall**-Konstrukt nicht verwendet werden.

Jedes Ein- und Ausgabeargument in einem reinen Unterprogramm muss mittels des **intent**-Attributs deklariert werden. Darüberhinaus muss jedes Unterprogramm, das von einem reinen Unterprogramm aufgerufen werden soll, ebenfalls ein reines Unterprogramm sein. Sonst ist das aufrufende Unterprogramm kein reines Unterprogramm mehr.

29.3.3. Die elemental Anweisung

Ein Unterprogramm ist elementar, wenn es als Eingabewerte sowohl Skalare als auch Felder akzeptiert. Ist der Eingabewert ein Skalar, so liefert ein elementares Unterprogramm einen Skalar als Ausgabewert. Ist der Eingabewert ein Feld, so ist der Ausgabewert ebenfalls ein Feld.

29.3.4. Die return Anweisung

Eine **return**-Anweisung darf nur im Gültigkeitsbereich von Unterprogrammen verwendet werden. Sie bewirkt einen Abbruch der Unterprogrammausführung und eine Rückkehr zum Aufrufpunkt, wo mit der nächsten Anweisung fortgesetzt wird.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program main
  implicit none

  call xyz( -2 )
  write( *, * ) "UP-Ende"
  stop
  write( *, * ) "Programmende"

contains
  subroutine xyz( n )
    integer, intent( in ) :: n
    integer                :: k

    do k = n, n+20
      write( *, * ) k
      if( k == 5 ) return
    end do

    write( *, * ) "k_max =", k
  end subroutine xyz

! Ausgabe:
!          -2
!          -1
!           0
!           1
!           2
!           3
!           4
!           5
!   UP-Ende
end program main

```

Einige Compiler erlauben zwecks Programmabbruch auch ein **return** anstelle des **stop** im Hauptprogramm. Das ist aber nicht standardkonform. Andere Compiler würden solchen Code mit einer Fehlermeldung abweisen, das Programm wäre somit nicht mehr uneingeschränkt portabel.

Ein **exit** in der Schleife anstelle der **return**-Anweisung würde nur den Schleifendurchlauf abbrechen und die Unterprogrammausführung würde nach der Schleife fortgesetzt.

29.4. Felder als Parameter

Beispiel: Übergabe eines ganzen Feldes

Datei *bsp.f95*: Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  integer, dimension(3,3) :: feld
  integer :: cnt, i, j

  cnt = 1

  do i = 1, 3
    do j = 1, 3
      feld(j,i) = cnt
      cnt = 1 + cnt
    end do
  end do

  ! Unterprogrammaufruf
  call sub(feld)
  ! Ausgabe: 1  2  3  4  5  6  7  8  9
end program bsp

```

Datei *sub.f95*: Fortran 90/95-Code (free source form)

```

subroutine sub(arr)
  implicit none
  integer, dimension(3,3), intent(in) :: arr

  write(*,*) arr
end subroutine sub

```

Beispiel: Übergabe einer Feld-Teilkette

Datei *bsp.f95*: Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  integer, dimension(3,3) :: feld
  integer :: cnt, i, j

  cnt = 1

  do i = 1, 3
    do j = 1, 3
      feld(j,i) = cnt
      cnt = 1 + cnt
    end do
  end do

  ! Unterprogrammaufruf
  call sub(feld(1:2,2:3))
  ! Ausgabe: 4  5  7  8
end program bsp

```

Datei *sub.f95* **Fortran 90/95-Code (free source form)**

```
subroutine sub(arr)
  implicit none
  integer, dimension(0:1, 0:1), intent(in) :: arr

  write(*,*) arr
end subroutine sub
```

Beispiel: Übergabe eines Feld-Einzelements

Datei *bsp.f95*: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer, dimension(3,3) :: feld
  integer cnt, i, j

  cnt = 1

  do i = 1, 3
    do j = 1, 3
      feld(j,i) = cnt
      cnt = 1 + cnt
    end do
  end do

  ! Unterprogrammaufruf
  call sub(feld(1,2))
  ! Ausgabe: 4
end program bsp
```

Datei *sub.f95* **Fortran 90/95-Code (free source form)**

```
subroutine sub(arr)
  implicit none
  integer, intent(in) :: arr

  write(*,*) arr
end subroutine sub
```

29.5. Prozeduren als Parameter

Auch Prozeduren können als Parameter übergeben werden.

Standardfunktionen (intrinsic functions) werden dazu folgendermaßen im Vereinbarungsteil gekennzeichnet:

intrinsic *namensliste*

oder

datentyp, intrinsic :: *namensliste*

Eigene Unterprogramme oder Unterprogramme aus Bibliotheken mit:

external namensliste

oder

datentyp, external :: *namensliste*

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  real, parameter :: PI=3.1415927

  ! intrinsic functions
  intrinsic sin, cos

  ! Unterprogrammaufrufe
  call sub(sin, PI)
  ! Ausgabe: 0.000000
  call sub(cos, PI)
  ! Ausgabe: -1.000000
end program bsp

subroutine sub(funk, x)
  implicit none
  real :: funk, x

  write(*,*) nint(funk(x)*1000)/1000.0
end subroutine sub
```

29.6. Optionale Parameter

Ab Fortran 90 sind auch optionale Parameter für Unterprogramme erlaubt. Solche Parameter sind durch das Attribut `optional` zu kennzeichnen. Auch der aufrufenden Programmeinheit muss diese Parameter-Optionalität bekannt gegeben werden, z.B. über ein Interface. Das aktuelle Vorhandensein eines als optional markierten Parameters beim Unterprogrammaufruf kann im Unterprogramm selbst durch die Funktion `present` geprüft werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  interface
    subroutine ab(a, b)
      integer, intent( in )      :: a
      integer, intent( in ), optional :: b
    end subroutine ab
  end interface
```

```
call ab( 1 )
call ab( 8, 12 )

! Ausgabe:
!   Nur a gegeben  1
!   Beide Parameter gegeben  8 12
end program bsp

subroutine ab(a, b)
  integer, intent( in )          :: a
  integer, intent( in ), optional :: b

  if( present( b ) ) then
    write (*,*) "Beide Parameter gegeben", a, b
  else
    write (*,*) "Nur a gegeben", a
  end if
end subroutine ab
```

29.7. Module

Module ersetzen in Fortran 95 die FORTRAN 77-COMMON-Blöcke. In einem Modul können Variablen, Konstanten und auch Unterprogramme abgelegt werden. Diese können dann von verschiedenen Programmeinheiten angesprochen werden.

```
module modulname
  [implicit none]
  [save]

  Deklaration von Variablen, Konstanten

  [contains
    Unterprogramme
  ]
end module modulname
```

Das Einbinden eines Moduls in eine Programmeinheit geschieht mittels der Anweisung

```
use modulname [, only liste]
```

only signalisiert, dass nur die in **liste** angegebenen Variablen oder Konstanten verwendet werden sollen.

Beispiel: **Fortran 90/95-Code (free source form)**

```
module m1
  implicit none
  save

  real, parameter :: PI=3.1415
  real :: a
end module m1
```

```

program bsp
  use m1
  implicit none

  a = 1.5
  write(*,*) 'Hauptprogramm 1: ', PI, a
  ! Ausgabe: Hauptprogramm 1:      3.141500      1.500000
  call sub
  write(*,*) 'Hauptprogramm 2: ', PI, a
  ! Ausgabe: Hauptprogramm 2:      3.141500      2.500000

end program bsp

subroutine sub
  use m1
  implicit none

  write(*,*) 'Unterprogramm: ', PI, a
  ! Ausgabe: Unterprogramm:      3.141500      1.500000
  a = 2.5

end subroutine sub

```

Das **save**-Statement in Modulen stellt sicher, dass die aktuellen Werte von Modulvariablen beim Wechsel zwischen den verschiedenen Programmeinheiten sicher erhalten bleiben.

29.8. Rekursiver Unterprogrammaufruf

In Fortran 95 können Unterprogramme (Funktionen, Subroutinen) auch rekursiv aufgerufen werden. Rekursion bedeutet, dass ein Unterprogramm sich selbst wieder aufruft (lat. recurrere oder en. recur ... wiederkehren, zurückkommen).

Beispiel: Berechnung von $n!$ (vereinfacht) **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer :: zahl, ergebnis, fac

  write(*,*) "Gib eine Ganzzahl ein: "
  read(*,*) zahl

  ergebnis = fac(zahl)

  write(*,*) "Ergebnis ist: ", ergebnis
end program bsp

recursive function fac(n) result(zerg)
  implicit none
  integer, intent(in) :: n
  integer :: zerg

  ! Vereinfacht: Keine Überprüfung ob Überlauf, negative Zahl, etc.

  zerg = n

  if (n > 1) then
    zerg = n * fac(n-1)
  end if

```

```
    return  
end function fac
```

Der Funktionskopf ist bei diesem Beispiel etwas anders gestaltet als üblich. Während der Intel-Fortran-Compiler auch ein

```
recursive integer function fac(n)
```

oder ein

```
integer recursive function fac(n)
```

problemlos akzeptiert, wirft der *gfortran*-Compiler bei diesen Varianten einen Syntaxfehler.

30. Ein- und Ausgabe

30.1. read



Abb. 31

Die **read**-Anweisung dient dem Einlesen von Daten. Typisches Beispiel ist die Dateneingabe mittels Tastatur. Formal sieht eine **read**-Anweisung so aus:

```
read([unit=]unit, [fmt=]fmt [, iostat=iostat] [, advance=advance]) [eingabeliste]
```

- *unit* ... Nummer der Eingabeeinheit (ist systemabhängig), Sternoperator oder auch die einer Datei mittels **open**-Anweisung zugeordnete Nummer.
- *fmt* ... Anweisungsnummer zu einer **format**-Anweisung, Sternoperator oder Formatliste
- *iostat* ... read-Status
- *advance* ... siehe write⁵¹

Listengesteuerte Eingabe auf der Standardeingabe (normalerweise die Tastatur):

```
read (*,*) a, b, c
```

Alternativ kann das auch so geschrieben werden:

```
read (unit=*, fmt=*) a, b, c
```

Beim *Intel Fortran Compiler*, *gfortran* und *g95* ist auch **unit = 5** als **stdin** (Tastatur) vorgelegt. Das Einlesen aus Dateien und die Einstellung des Formates werden später erläutert.

Beispiel: **Fortran 90/95-Code (free source form)**

⁵¹ Kapitel 30.2 auf Seite 184


```
program bsp
  implicit none

  integer :: i(5)

  ! Einlesen in ein Feld (unit ... Standardeingabe, fmt ... listengesteuert)
  read (*,*) i
  ! ...

end program bsp
```

Kurze Erläuterung zu `iostat`:

Wert	Erläuterung
0	kein Fehler
positiver Wert (systemabhängig)	Fehler
negativer Wert (systemabhängig)	<i>End Of File</i> und kein Fehler

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i, st

  ! Einlesen eines Wertes
  read (*, *, iostat=st) i

  ! Ausgabe des IO-Status
  write (*,*) 'IO-Status:', st
end program bsp
```

Ausgabe:

Für Eingabe: 5 $\rightarrow$ 0

Für Eingabe: 5.3 $\rightarrow$ Positiver Wert = Fehler

Das Einlesen aus Dateien und die Einstellung des Formates werden später erläutert.

30.2. write



Abb. 32

Die **write**-Anweisung dient der Datenausgabe. Typisches Beispiel ist die Anzeige von Daten auf dem Bildschirm. Formal sieht eine **write**-Anweisung so aus:

```
write([unit=]unit, [fmt=]fmt [, iostat=iostat] [, advance=advance]) [ausgabeliste]
```

- *unit* ... Nummer der Ausgabeeinheit (ist systemabhängig), Sternoperator oder auch die einer Datei mittels **open**-Anweisung zugeordnete Nummer.
- *fmt* ... Anweisungsnummer zu einer **format**-Anweisung, Sternoperator oder Formatliste
- *iostat* ... write-Status
- *advance* ... nur bei sequentieller formatierter I/O. Formatspezifizierer muss explizit gegeben sein.
 - 'no' ... kein Vorschub des *file position pointers* zum nächsten Datensatz (z.B. kein Zeilenvorschub).
 - 'yes' ... mit Vorschub des *file position pointers* zum nächsten Datensatz (voreingestellt)

Listengesteuerte Ausgabe auf der Standardausgabe (normalerweise der Bildschirm):

```
write (*,*) a, b, c
```

Alternativ kann das auch so geschrieben werden:

```
write (unit=*, fmt=*) a, b, c
```

Beim *Intel Fortran Compiler*, *gfortran* und *g95* sind auch

- **unit=0** als stderr (Bildschirm) und
- **unit=6** als stdout (Bildschirm)

vorbelegt. Bezüglich *iostat* gilt auch hier der im vorigen Abschnitt kurz geschilderte Sachverhalt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: i(5) = (/ 5, 4, 1, -1, -7 /)

  ! ...

  ! Ausgabe der Feldwerte (unit ... Standardausgabe, fmt ... listengesteuert)
  write (*,*) i
  ! ...
end program bsp
```

Beispiel: **advanceFortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  character(10) :: ch
```

```
integer :: st

write(*, '(A)', advance='YES') "Hallo"
write(*, '(A)', advance='YES') "Welt"
write(*, *) "ABCDEFGHJKLMN"
end program bsp
```

Ausgabe:

advance='YES'	advance='NO'
Hallo Welt ABCDEFGHJKLMN	HalloWelt ABCDEFGHJKLMN

Die Ausgabe in Dateien und die Einstellung des Formates werden etwas später erläutert.

30.3. Kürzer: print, read, write und Namenslisten

Für die listengesteuerte Ein- und Ausgabe existieren auch vereinfachte Formen. Für Eingaben wird wieder der `read`-Befehl verwendet, für Ausgaben gibt es die `print`-Anweisung.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer          :: a, b
  real             :: r
  complex          :: z
  character( len = 10 ) :: str

  read *, a, b, r, z, str
! Eingabe per Tastatur:
! 10, 30, 55.5, (10.8,7.0), Hallo

  print *, str, a, b, r, z
! Ausgabe am Bildschirm:
! Hallo 10 30 55.5 (10.8,7.)
end program bsp
```

Bei mehrfachem Aufruf gleicher Ein- bzw. Ausgabeanweisungen kann durch Verwendung von Namenslisten der Programmcode kürzer gestaltet werden. Die Dateneingabe wird dadurch aber etwas komplizierter:

- eingeleitet wird die Eingabe durch ein `&`-Zeichen, unmittelbar gefolgt vom Namenslistenbezeichner
- danach folgen ein oder mehrere Leerzeichen
- es folgen die Zuweisungen von Werten zu den Variablennamen
- abgeschlossen wird die Eingabe durch einen Slash `/`

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer          :: a, b
```

```

real                :: r
complex             :: z
character( len = 10 ) :: str

namelist / LISTEA / a, b, r, z, str
namelist / LISTEB / str, r, z

read( *, nml = LISTEA )
! Eingabe per Tastatur:
!   &LISTEA b = 30, a = 10, r = 55.5, z = (10.8,7.0), str = "Hallo" /

write( *, nml = LISTEB )
! Ausgabe auf dem Bildschirm (Intel 9.1):
!   &LISTEB
!   STR      = Hallo      ,
!   R        = 55.50000   ,
!   Z        = (10.80000,7.00000)
!   /
end program bsp

```

30.4. Formatierung



Abb. 33

Die Ein- und Ausgabeformatierung kann beeinflusst werden. Zu diesem Zweck gibt es die `format`-Anweisung.

... (..., `fmt = marke`, ...) ... *marke* format (*formatliste*)

Alternativ kann die Formatliste auch direkt in die `read`- oder `write`-Anweisung eingebunden werden

... (..., `fmt = '(formatliste)'`, ...) ...

30.4.1. Formatlistenelemente

Formatspezifizierer **Kommentar**

`Ix[z]`

Ganzzahl mit einer Feldlänge von `x` Zeichen. `z` gibt die Mindestanzahl der auszugebenden Zeichen an (Feld wird, wenn nötig, mit führenden Nullen aufgefüllt).

Bx[.z]	Ganzzahl, Ausgabe als Binärzahl.
Ox[.z]	Ganzzahl, Ausgabe als Oktalzahl.
Zx[.z]	Ganzzahl, Ausgabe als Hexadezimalzahl.
Fx.y	Fixkommazahl mit einer Gesamtfeldlänge von x Zeichen. y ist die Anzahl der Nachkommastellen (Vorzeichen und Dezimalpunkt müssen in der Gesamtfeldlänge berücksichtigt werden). F0.y führt zu einer variablen Feldlänge in Abhängigkeit vom minimalen Platz der für die Vorkommastellen nötig ist.
Ex.y	Gleitkommazahl mit einer Gesamtfeldlänge von x Zeichen. y ist die Anzahl der Nachkommastellen. (Vorzeichen, Dezimalpunkt und die Zeichen für den Exponenten müssen in der Gesamtfeldlänge berücksichtigt werden).
Dx.y	Gleitkommazahl, doppelte Genauigkeit
A	Eine Zeichenkette.
Ax	Eine Zeichenkette mit x Zeichen.
Lx	Ein logischer Wert, T (wahr) bzw. F (falsch).
xX	x Leerzeichen.
/	Zeilenvorschub
Tx	Tabulator (an der Position x)

Obige Tabelle der Formatlistenelemente ist nicht vollständig; es gibt z.B. noch »G« für ein »generelles« Ausgabeformat, das aber wenig gebräuchlich ist. Die Ausgabe erfolgt normalerweise rechtsbündig. Reicht die Gesamtfeldlänge bei numerischen Werten nicht aus, so werden anstelle einer Zahl Sternchen angezeigt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer :: a

  a = 999
  write(*, 3333) a
  ! Ausgabe: 999

  a = -999
  write (*, 3333) a
  ! Ausgabe: ***

3333 FORMAT (I3, /, /)
! / ... nach jeder 3333-write-Anweisung werden zwei Leerzeilen eingefügt
end program bsp
```

Alternativ könnte die Formatliste auch so in die `write`-Anweisung eingebaut werden:

```
write(*, '(I3, /, /)') a
```

Oder mittels benannter Konstante auch so:

```
character(*), parameter :: fs = '(I3, /, /)'  
print fs, a
```

Hier wurde statt der `write`-Anweisung der `print`-Befehl verwendet. Der Effekt ist derselbe, es erfolgt eine formatierte Ausgabe.

Weitere Formatierungsbeispiele:

Code	Ausgabe
WRITE(*, 999) 1234 WRITE(*, 999) 1234567 WRITE(*, 999) 1234567890 999 FORMAT(I9.6)	001234 1234567 *****
WRITE(*, 999) 555.6666 WRITE(*, 999) +5.6 WRITE(*, 999) -55.666E7 WRITE(*, 999) -55555.666 999 FORMAT(F9.3)	555.667 5.600 *****
WRITE(*, 999) 555.6666 WRITE(*, 999) +5.6 WRITE(*, 999) -55.666E7 WRITE(*, 999) -55555.666 999 FORMAT(E9.3)	0.556E+03 0.560E+01 -.557E+09 - .556E+05
WRITE(*, 999) 'Hallo' WRITE(*, 999) 'ABCDEFGHijkl' WRITE(*, 888) 'ABCDEFGHijkl' 888 FORMAT(A) 999 FORMAT(A10)	Hallo ABCDEFGHIJ ABCDEFGHIJKL
WRITE(*, *) 'FORTRAN', '77' WRI- TE(*, 999) 'FORTRAN', '77' 999 FOR- MAT(A, 1X, A)	FORTRAN77 FORTRAN 77
WRITE(*, 888) 'FORTRAN', '77' WRI- TE(*, 999) 'FORTRAN', '77' 888 FOR- MAT(A, T3, A) 999 FORMAT(A, T20, A)	FO77RAN FORTRAN 77
WRITE(*, 999) 'FORTRAN', '77' 999 FORMAT(A, /, A)	FORTRAN 77
WRITE(*, 999) 34.56 WRITE(*, *) 34.56 C SP ... Sign Plus (+) 999 FORMAT(SP, F12.3)	+34.560 34.56

30.4.2. Wiederholung von Formatteilen

Beispiel:

```
write(*, 100) 'abc', 10.3, 'xxx', 23.4  
100 format (2(A3, F6.1))
```

30.4.3. write etwas anders

Beispiel:

```
write (*, 100)
100 format ('Hallo', X, 'Welt!')
```

30.4.4. Dynamische Mehrfachformatierung

Formatierungsanweisungen können auch als String bearbeitet werden, indem man z.B. die Anzahl der auszugebenden Variablen per **write**-Befehl in die Formatierung schreibt.

Mehrfachformatierung - Beispiel:

```
character(5) :: formatierung
integer, dimension(1:4) :: einsen = 1
integer :: anzahl
anzahl = 4
formatierung = '( I1)'
write(formatierung(2:2), '(I1)') anzahl
write(*, formatierung) einsen
```

30.5. Dateien

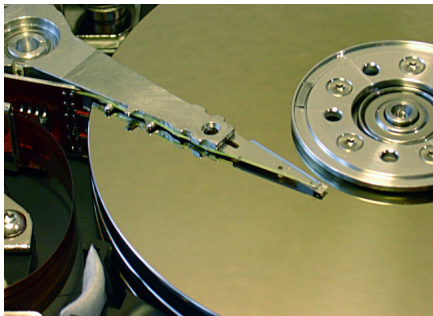


Abb. 34

30.5.1. Datensatz

Datensätze können in folgender Form auftreten:

- Formatierter Datensatz: Textdatensatz
- Unformatierter Datensatz: Datensatz in einer maschineninternen Form.
- Dateiendensatz

30.5.2. Datei

Für Fortran ist alles eine Datei, das durch **read** oder **write** bearbeitbar ist.

Zugriffsmethoden:

- Sequentieller Zugriff: Lesen ab Beginn der Datei (file) und dann immer den nächsten Datensatz einlesen. Geschrieben wird jeweils ans Dateiende. Auf interne Dateien kann nur sequentiell zugegriffen werden.
- Direkter Zugriff: Bearbeiten in beliebiger Reihenfolge durch Angabe der Satznummer.
- Binärer Zugriff: Bearbeiten von Dateien, die binäre Daten enthalten, z. B. Bilder von CCD-Kamera, Scilab/Matlab **save**-Dateien

Dateitypen:

- Externe Datei: Eine konventionelle Datei
- Interne Datei: character-Variable oder -Feld.

Dateien haben im Betriebssystem einen Dateinamen. In Fortran wird eine Datei über eine Dateinummer (unit) angesprochen. Die Zuordnung erfolgt mit dem Befehl **open**.

30.5.3. open

Zum Öffnen einer externen Datei dient die **open** -Anweisung.

open (liste)

mit folgender *liste*

Element	Kommentar
[unit =] x	x ist eine Dateinummer (Ganzzahl, sollte über 10 liegen, da oft Nummern unter 10 fix zugeordnet sind, z.B. der Standardein-, ausgabe).
file = x	x ist der externe Dateiname
iostat = x	x ist 0 wenn open fehlerfrei ausgeführt wurde, ansonsten eine systemabhängige Fehlernummer
status = x	Dateistatus: 'old' ... Datei existiert bereits 'new' ... Datei wird neu erzeugt 'scratch' ... namenlose temporäre Datei 'unknown' ... System bestimmt Dateistatus selbst 'replace' ... der Inhalt einer bereits vorhandenen Datei wird gelöscht.
access = x	Zugriffsmethode: 'sequential' ... Sequentielle Datei 'direct' ... direkter Zugriff 'stream' ... binärer Zugriff
position = x	Den Dateisatzzeiger beim Öffnen der Datei an eine bestimmte Position setzen. ('asis', 'rewind', 'append')
form = x	Format: 'formatted' oder 'unformatted'
action = x	'read' ... nur Lesezugriff 'write' ... nur Schreibzugriff 'readwrite' ... Lesen und Schreiben
recl = x	Datensatzlänge (positive Zahl, access='direct', in Bytes)

<code>err = x</code>	Im Fehlerfall Sprung zur Marke x
<code>blank = x</code>	'null' oder 'zero' (nur für <code>form='formatted'</code>)
<code>delim = x</code>	'apostrophe'
	'quote'
	'none'
<code>pad = x</code>	'yes' oder 'no' (nur für <code>form='formatted'</code>)

Eingestellte Vorgabewerte sind:

- `status = 'unknown'`
- `position = 'asis'`
- `access = 'sequential'`
- `form = 'formatted'`

Wird `access='direct'` gesetzt, so gilt `form='unformatted'` als Vorgabewert.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer :: stat
  character(80) :: str

  open(20, file='/tmp/testdatei.txt', iostat=stat)

  if(stat == 0) then
    write(*,*) 'Das Öffnen der Datei war ein voller Erfolg'

    do
      read(20, '(A)', iostat=stat) str
      ! Bei EOF wird stat /= 0
      if (stat /= 0) exit
      write(*,*) str
    end do
  else
    write(*,*) 'Datei konnte nicht geöffnet werden'
  end if

  close(20)
end program bsp

```

30.5.4. close

Geschlossen wird die Verbindung zur externen Datei mit dem `close`-Befehl.

`close (liste)`

liste:

Element	Kommentar
<code>[unit =] x</code>	wie bei <code>open</code>
<code>iostat = x</code>	wie bei <code>open</code>
<code>err = x</code>	wie bei <code>open</code>

status = x 'keep' ... Datei erhalten (voreingestellt)
 'delete' ... Datei löschen

30.5.5. Lesen und Schreiben

Gelesen oder geschrieben wird mit den bereits bekannten **read**- und **write**-Anweisungen.

Element	Kommentar
[unit =] x	Dateinummer bzw. CHARACTER-Variable oder Feld (interne Datei)
[fmt =] x	siehe Formatierung ⁵²
[nml] = x	x ... namelist-group-name
rec = x	Datensatznummer bei Direktzugriff (siehe Abschnitt Direktzugriff ⁵³)
iostat = x	wie bei read ⁵⁴
err = x	Bei Fehler Sprung zur Anweisungsnummer x
end = x	Bei Dateiarbeit Sprung zur Anweisungsnummer x (nicht erlaubt bei Direktzugriff, nicht bei write)
advance =	'yes' oder 'no'
x	
eor = x	Bei <i>End of Record</i> Sprung zur Marke x (nicht bei write)
size = x	x ... Zeichenzähler (nicht bei write, advance='no')

Es existiert eine Menge von Einschränkungen, wann welche Elemente erlaubt sind, bzw. welche nur kombiniert auftreten sollen, z.B.

- wenn der **rec**-Spezifizierer Verwendung findet, dann darf kein **end**-Element angegeben werden
- Bei Dateneingabe nur dann ein **size**-Spezifizierer, wenn **advance='no'** gesetzt ist.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  character (len = 80) :: a
  integer              :: st = 0

  open (20, file='/tmp/testdatei.txt', status='OLD', iostat=st)

  if (st /= 0) then
    stop "open-Fehler!"
  end if

  ! Aus Datei lesen
  do
    read (20, 888, iostat=st) a
    ! Auf Standardausgabe schreiben

    if (st == 0) then
      write (*, 888) a
    end if
  end do

```

⁵² Kapitel 30.4 auf Seite 187

⁵³ Kapitel 30.5.5 auf Seite 194

⁵⁴ Kapitel 30.1 auf Seite 183

```
        else if (st > 0) then
            write (*,*) "read-Fehler!"
            exit
        else if (st < 0) then
            exit
        end if
    end do

    close(20)

888  format(A)
end program bsp
```

Direktzugriff

OPEN:

Element	Kommentar
access =	x ... 'DIRECT'
x	
recl = x	x ... Datensatzlänge (positive Zahl, access='DIRECT', in Bytes bzw. bei formatierten Dateien in Characters)
fmt = x	x ... Formatangabe (wird eine Datei 'FORMATTED' geöffnet, dann muss auch eine konkrete Formatliste angegeben werden, ansonsten tut's auch der Sternoperator)

READ/WRITE:

Element	Kommentar
REC = x	x ... Satznummer bei Direktzugriff

Beispiel: Gegeben ist die Textdatei /tmp/testdatei.txt mit dem Inhalt

Die WRITE-Anweisung dient der Datenausgabe aus einem FORTRAN-Programm auf ein externes Gerät. Typisches Beispiel ist die Anzeige von Daten auf dem Bildschirm.
Formal sieht eine WRITE-Anweisung so aus:

Fortran 90/95-Code (free source form)

```
program bsp
    implicit none

    character (len = 10) :: c
    integer             :: st

    open (20, file='/tmp/testdatei.txt', status='OLD', form='FORMATTED', &
         access='DIRECT', recl=15, iostat=st)

    if (st /= 0) then
        stop "open-Fehler!"
    end if

    read (20, fmt='(A)', rec=4, iostat=st) c
```

```

if (st /= 0) then
  write (*,*) "read-Error"
else
  write (*,*) c
end if

close (20)
end program bsp

```

Ausgabe:

```
s einem F0
```

30.5.6. Positionieren bei sequentiellen Dateien

Datensatzzeiger um einen Datensatz zurücksetzen:

```
backspace ([unit=]x [,iostat=y] [,err=z])
```

Positionieren an den Dateibeginn:

```
rewind ([unit=]x [,iostat=y] [,err=z])
```

Schreiben eines Dateiendsatzes:

```
endfile ([unit=]x [,iostat=y] [,err=z])
```

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  character (len = 100), dimension(3) :: c
  integer :: st = 0

  open (20, file='/tmp/testx.txt', status='NEW', iostat=st)
  call checkStatus(st, "open-")

  write (20,*) 'Das ist eine Testdatei'
  write (20,*) 'Dies ist Zeile 2 der Testdatei'
  write (20,*) 'Jenes die Zeile 3 der Testdatei'
  write (20,*) "Jetzt ist's aber genug"
  endfile (20)

  rewind (20, iostat=st)
  call checkStatus(st, "rewind-")

  read (20, fmt=555, iostat=st) c
  call checkStatus(st, "read-")

  write (*, fmt=555) c

  backspace (20, iostat=st)
  call checkStatus(st, "backspace-")

```

```

read (20, fmt=555, iostat=st) c(1)
call checkStatus(st, "read-")

write (*, fmt=555) c(1)

close (20)

555 format (A)
end program bsp

subroutine checkStatus(st, ch)
  integer,          intent (in) :: st
  character (*), intent (in) :: ch

  if (st /= 0) then
    close(20)
    write (*,*) ch // "Fehler!"
    stop
  end if
end subroutine checkStatus

```

Ausgabe:

```

Das ist eine Testdatei
Dies ist Zeile 2 der Testdatei
Jenes die Zeile 3 der Testdatei
Jenes die Zeile 3 der Testdatei

```

30.5.7. inquire

Die Anweisung `inquire` dient der Abfrage einiger Eigenschaften von Dateien oder I/O-Units.

`inquire (file = x, liste)`

mit *x* ... Dateiname (inkl. Pfad)

`inquire ([unit =] x, liste)`

mit *x* ... Nummer der I/O-Unit.

liste:

Element	Kommentar
<code>access = x</code>	x: • 'SEQ' ... sequentieller Dateizugriff • 'DIRECT' ... Direktzugriff

<code>action = x</code>	x: • 'READ' • 'WRITE' • 'READWRITE' • 'UNDEFINED'
<code>blank = x</code>	x: • 'NULL' • 'ZERO'
<code>delim = x</code>	x: • 'APOSTROPHE' • 'QUOTE' • 'NONE' • 'UNDEFINED'
<code>direct = x</code>	x: • 'YES' ... Direktzugriff • 'NO' ... kein Direktzugriff für diese Datei erlaubt • 'UNKNOWN' ... unbekannt
<code>err = x</code>	Bei Fehler Sprung zur Anweisungsnummer x
<code>exist = x</code>	x: • .TRUE. ... Datei existiert • .FALSE. ... Datei existiert nicht
<code>form = x</code>	x: • 'FORMATTED' ... Datei geöffnet für formatierte Datensätze • 'UNFORMATTED' ... Datei geöffnet für unformatierte Datensätze • 'UNDEFINED'
<code>formatted = x</code>	x: • 'YES' ... formatierte Datensätze sind erlaubt • 'NO' ... formatierte Datensätze sind nicht erlaubt • 'UNKNOWN' ... unbekannt
<code>iostat = x</code>	x ist 0 wenn OPEN fehlerfrei ausgeführt wurde, ansonsten eine systemabhängige positive Fehlernummer
<code>name = x</code>	Der Dateiname wird der Zeichenketten-Variablen x zugewiesen. Hat die Datei keinen Namen, dann ist das Ergebnis undefiniert.
<code>named = x</code>	x: • .TRUE. ... Datei besitzt Namen • .FALSE. ... Datei besitzt keinen Namen
<code>nextrec = x</code>	x ... Nummer des nächsten Datensatzes
<code>number = x</code>	x ... Nummer der mit einer externen Datei verbundenen I/O-Unit.

opened = x	x: • .TRUE. ... Datei ist geöffnet • .FALSE. ... Datei ist nicht geöffnet
pad = x	x: • 'YES' • 'NO'
position = x	x: • 'REWIND' • 'ASIS' • 'APPEND' • 'UNDEFINED'
read = x	x: • 'YES' • 'NO' • 'UNKNOWN'
readwrite = x	x: • 'YES' • 'NO' • 'UNKNOWN'
recl = x	x ... Datensatzlänge bei Direktzugriff
sequential = x	x: • 'YES' ... sequentiell • 'NO' ... nicht sequentiell • 'UNKNOWN' ... unbekannt
unformatted = x	x: • 'YES' ... unformatierte Datensätze sind erlaubt • 'NO' ... unformatierte Datensätze sind nicht erlaubt • 'UNKNOWN' ... unbekannt
write = x	x: • 'YES' • 'NO' • 'UNKNOWN'

Beispiel: Datei vorhanden? **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  logical :: l
  integer :: st

```

```

inquire (file='/tmp/testdatei.txt', exist=l, iostat=st)

if (st == 0) then
  write (*,*) "Datei existiert?", 1
else
  write(*,*) "Fehler!"
end if
! wenn Datei existiert:      Datei existiert?  T
! wenn Datei nicht existiert: Datei existiert?  F
! wenn aus irgendeinem ein inquire-Fehler auftrat: Fehler!
end program bsp

```

Beispiel: Infos zu einer geöffneten Datei **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  logical      :: ex
  character (15) :: di, fo, ac, se
  integer      :: nu, st

  open (25, file='/tmp/testdatei.txt', status='old', iostat=st)

  if(st /= 0) stop "open-Fehler!"

  inquire (25, exist = ex, direct = di, sequential = se, formatted = fo, &
    access = ac, number = nu, iostat=st)

  if(st == 0) then
    write (*,*) 'EXIST? ', ex
    write (*,*) 'DIRECT? ', di
    write (*,*) 'SEQUENTIAL? ', se
    write (*,*) 'FORMATTED? ', fo
    write (*,*) 'ACCESS? ', ac
    write (*,*) 'NUMBER? ', nu
  else
    write (*,*) "inquire-Fehler!"
  end if

  close(25)

! Ausgabe, z.B.
!   EXIST?  T
!   DIRECT? YES
!   SEQUENTIAL? YES
!   FORMATTED? YES
!   ACCESS? SEQUENTIAL
!   NUMBER?          25
end program bsp

```

30.5.8. Interne Dateien

- Interne Dateien sind vom Datentyp **character** (Zeichen oder Zeichenketten)
- Das Lesen aus bzw. das Schreiben in interne Dateien erfolgt immer sequentiell

Beispiel: Schreiben in eine interne Datei **Fortran 90/95-Code (free source form)**

```

program bsp
  character(15) :: ch

```



```
real          :: r = 12.5678

! Interne Datei "ch"
write (ch, *) r

write (*,*) 'r lexikalisch groesser als Buchstabe "A"? ', lge(ch, 'A')
end program bsp
```

Beispiel: Lesen aus einer internen Datei **Fortran 90/95-Code (free source form)**

```
program bsp
character(15) :: ch = '12.5678'
real          :: r

! Interne Datei "ch"
read (ch, '(F15.5)') r

write (*,*) 'r    = ', r
write (*,*) 'r**2 = ', r**2
end program bsp
```

31. Zeiger

31.1. Was sind Zeiger?

w:Zeiger (Informatik)⁵⁵ Ein Zeiger (Pointer) ist eine Variable, die Adresse und (Daten-)Typ enthält – ähnlich wie bei einem Array, das *rank* und *shape* enthält. Der Zeiger (Pointer) charakterisiert Position, Länge und Organisation des Speicherabschnitts, an dem die Variable (Target) gehalten wird.

31.2. Zeiger in Fortran 95

In Fortran 95 werden Zeiger durch Zufügen des Attributes `pointer` bei der Deklaration von Variablen erzeugt.

datentyp, pointer :: variable

Ein so deklarerter Zeiger kann auf andere Zeiger oder auf mittels `target` gekennzeichnete Variablen verweisen.

datentyp, target :: variable

Die Zeigerzuordnung erfolgt durch das Symbol

=>

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real, pointer :: ptr
  real, target :: trg

  trg = 5.5
  ptr => trg
  write(*,*) ptr
  ! Ausgabe: 5.50000
  ! Zeiger werden bei Bedarf automatisch dereferenziert
end program bsp
```

⁵⁵ <https://de.wikipedia.org/wiki/Zeiger%20%28Informatik%29>

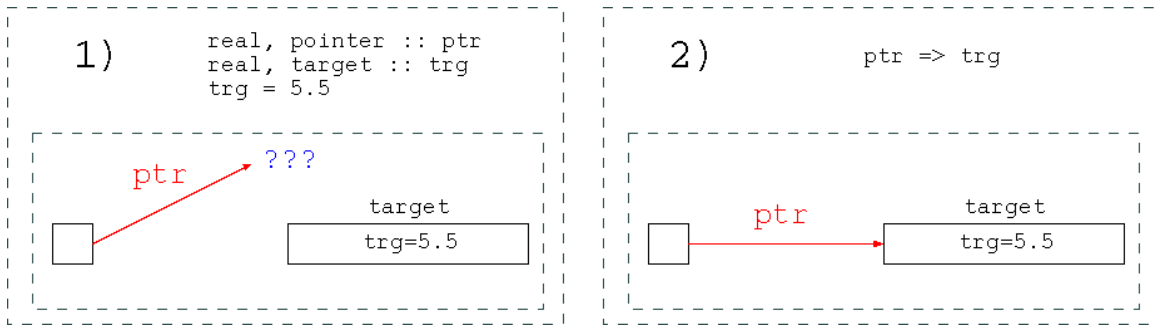


Abb. 35

31.3. Assoziationsstatus

Ein Zeiger kann einen der folgenden Assoziationszustände annehmen:

- undefiniert (dangling)
- nicht zugeordnet (disassociated, null)
- zugeordnet (associated)

Der Zuordnungsstatus eines Zeigers ist unmittelbar nach der Deklaration *undefiniert*. Mittels `zeiger => null()` oder `nullify(zeiger)` kann ein Zeiger auf einen *nicht zugeordneten* Status gesetzt werden. Verweist ein Zeiger auf einen anderen zugeordneten Zeiger oder ein Target, so ist sein Zustand *zugeordnet*.

Der Assoziationsstatus eines Zeigers lässt sich über die Funktion

```
associated (zeiger [, ziel])
```

abfragen. Sinnvoll ist eine derartige Abfrage nur dann, wenn der Zuordnungsstatus nicht undefiniert ist.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  integer, pointer      :: ptr1 => null()
  character(20), pointer :: ptr2
  character(20), target :: str

  str = "Hallo, Welt!"
  ptr2 => str

  write(*,*) associated(ptr1)
  ! Ausgabe: F

  write(*,*) associated(ptr2)
  ! Ausgabe: T

  write(*,*) associated(ptr2, ptr1)
  ! Ausgabe: F
```

```

    write(*,*) associated(ptr2, str)
    ! Ausgabe: T
end program bsp

```

31.4. Speicherplatz dynamisch reservieren und freigeben

Für normale Variablen läuft die Speicherplatzverwaltung automatisch ab. Bisher wurden Zeiger immer solchen normalen (Target)Variablen, für die bereits Speicherplatz reserviert war, zugeordnet. Aber auch für Zeiger selbst kann Speicherplatz reserviert werden. Bei der Zeigerdeklaration ist der Zeigerstatus undefiniert oder nicht zugeordnet. Eine Wertzuweisung an eine solche Zeigervariable würde zur Laufzeit einen Speicherzugriffsfehler ergeben. Die Funktion

```
allocate (zeiger1, [zeiger2, ...] [,stat=integervar])
```

reserviert in Abhängigkeit des Zeiger-Datentyps Speicherplatz für die einzelnen Zeiger. Die Funktion

```
deallocate (zeiger1, [zeiger2, ...] [,stat=integervar])
```

gibt diesen Speicherplatz wieder frei.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  integer, pointer :: ptr1 => null(), ptr2 => null()
  integer :: status

  allocate(ptr1, stat = status)
  ptr1 = 2222

  write (*,*) "Status = ", status
  ! Wenn status = 0, dann wurde erfolgreich Speicherplatz reserviert

  write (*,*) ptr1
  ! Ausgabe: 2222

  ptr2 => ptr1
  ptr1 = 5555

  write (*,*) ptr1, ptr2
  ! Ausgabe: 5555 5555

  deallocate(ptr1)
end program bsp

```

31.5. Zeiger und Felder

Beispiel: **Fortran 90/95-Code** (free source form)

```
program bsp
  implicit none

  integer, dimension(:), pointer    :: ptr => null()
  integer, dimension(5:10), target :: arr = (/55, 66, 77, 88, 99, 111/)

  ptr => arr
  write(*,*) ptr
  ! Ausgabe: 55    66    77    88    99    111

  ptr => arr(7:)
  write(*,*) ptr
  ! Ausgabe: 77    88    99    111
end program bsp
```

Beispiel: "ragged array"

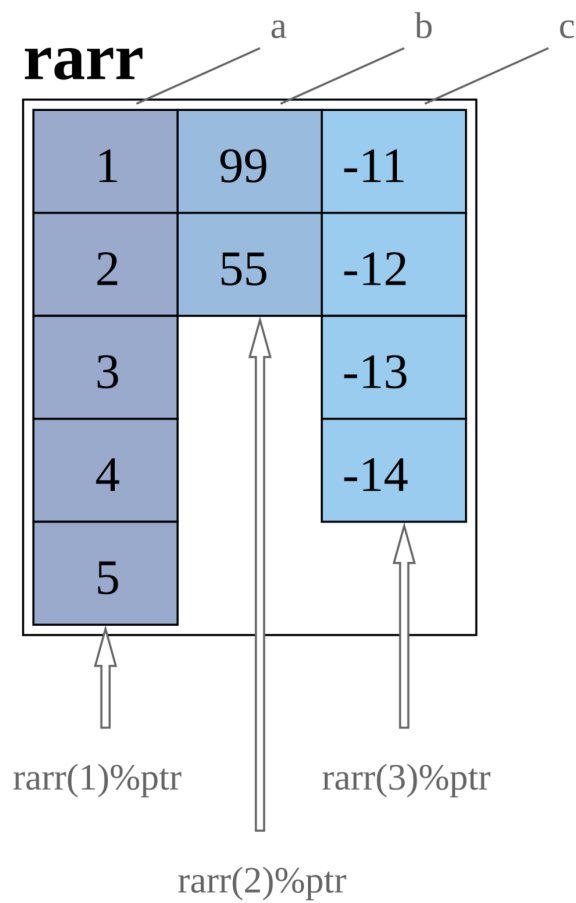


Abb. 36

Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  type element
    integer, dimension(:), pointer :: ptr
  end type element

  integer, dimension(5), target :: a = (/ 1, 2, 3, 4, 5 /)
  integer, dimension(2), target :: b = (/ 99, 55 /)
  integer, dimension(4), target :: c = (/ -11, -12, -13, -14 /)
  integer :: i
  type( element ), dimension(3) :: rarr

  rarr(1)%ptr => a
  rarr(2)%ptr => b

```

```
rarr(3)%ptr => c

do i = 1,3
  write (*,*) "rarr(", i, "): ", rarr(i)%ptr
end do

! Ausgabe
!   rarr( 1 ):  1 2 3 4 5
!   rarr( 2 ): 99 55
!   rarr( 3 ): -11 -12 -13 -14
end program bsp
```

31.6. Verkettete Listen

In der Regel ist die Größe einer Liste vor ihrer Erstellung (zum Bsp. durch Einlesen einer Datei) nicht bekannt, was die Verwendung von Array erschwert. Dies lässt sich durch Pointer vereinfachen. Man unterscheidet dabei viele verschiedene Listentypen:

- linear und einfach verkettet: Diese Liste lässt sich nur in eine Richtung durchlaufen und das Ende hat keinen Bezug zum Anfang.
- zyklisch und einfachverkettet: Nur eine Durchlaufrichtung, aber das Ende ist mit dem Anfang verpointert. Nach dem vollständigen Durchlaufen beginnt man wieder am Anfang.
- linear und doppelt verkettet: Lineare Liste, in der jedes Element auf das nächste UND das vorherige zeigt. Der Durchlauf der Liste ist so in zwei Richtungen möglich. Der Anfang ist dabei nicht mit dem Ende verpointert.
- zyklisch und doppelt verkettet: Zyklische Liste innerhalb derer zwei Durchlaufrichtungen zur Verfügung stehen.

Die Wahl des Listentyps richtet sich nach dem Verwendungszweck. Einfach verkettete Listen sind auch einfach in der Implementierung. Für den Zugriff entgegen der Durchlaufrichtung muss die Liste jedoch einmal vollständig durchlaufen werden, was den Rechenaufwand und vor allem die Rechenzeit erhöht. Sie eignen sich daher gut für einfache Aufgaben mit gerichteter Abarbeitung. Doppelt verkettete Listen lassen mehr Freiraum hinsichtlich ihrer Durchlaufrichtung. Sie sind jedoch komplexer in ihrer Implementierung und erhöhen den Speicheraufwand.

31.6.1. Anfügen von Listenelementen

Beispiel: Lineare, einfach verkettete Liste (LIFO) **Fortran 90/95-Code (free source form)**

```
module m1
  implicit none

  type node
    integer :: id
    character(5) :: value
    type(node), pointer :: next => null()
  end type

  type(node), pointer :: first => null()

  private ! Auf alle nachfolgenden Anweisungen kann von aussen nicht zugegriffen
```

werden

```
public :: add_node, write_all, free_all ! Auf die Subroutinen add_node,
write_all und free_all
                                     ! kann von aussen explizit zugegriffen werden,
                                     ! jedoch nicht auf innerhalb der Subroutinen
                                     ! deklarierte Datentypen
```

contains

```
subroutine add_node(id, str)
  implicit none

  integer, intent(in) :: id
  character(5), intent(in) :: str

  type(node), pointer :: new, tmp

  ! Speicher reservieren
  allocate(new)

  ! Werte setzen
  new%id = id
  new%value = str

  ! Am Beginn der Liste einfügen
  if (.not. associated(first)) then
    first => new
  else
    tmp => first
    first => new
    first%next => tmp
  end if
```

```
end subroutine add_node
```

```
subroutine write_all()
  implicit none

  type(node), pointer :: tmp

  tmp => first

  do
    if (associated(tmp) .eqv. .FALSE.) exit

    write(*,*)tmp%id, tmp%value
    tmp => tmp%next
  end do
end subroutine write_all
```

```
subroutine free_all()
  implicit none

  type(node), pointer :: tmp

  do
    tmp => first

    if (associated(tmp) .eqv. .FALSE.) exit

    first => first%next
    deallocate(tmp)
  end do
end subroutine free_all
```

```
end module m1
```



```

program bsp
  use m1
  implicit none

  call add_node (1, "AAAAA")
  call add_node (2, "BBBBB")
  ! ...
  call add_node (150, "ZZZZZ")

  call write_all
  ! Ausgabe:
  ! 150 ZZZZZ
  ! 2 BBBB
  ! 1 AAAAA

  call free_all ! Die verkettete Liste wird wieder freigegeben
end program bsp

```

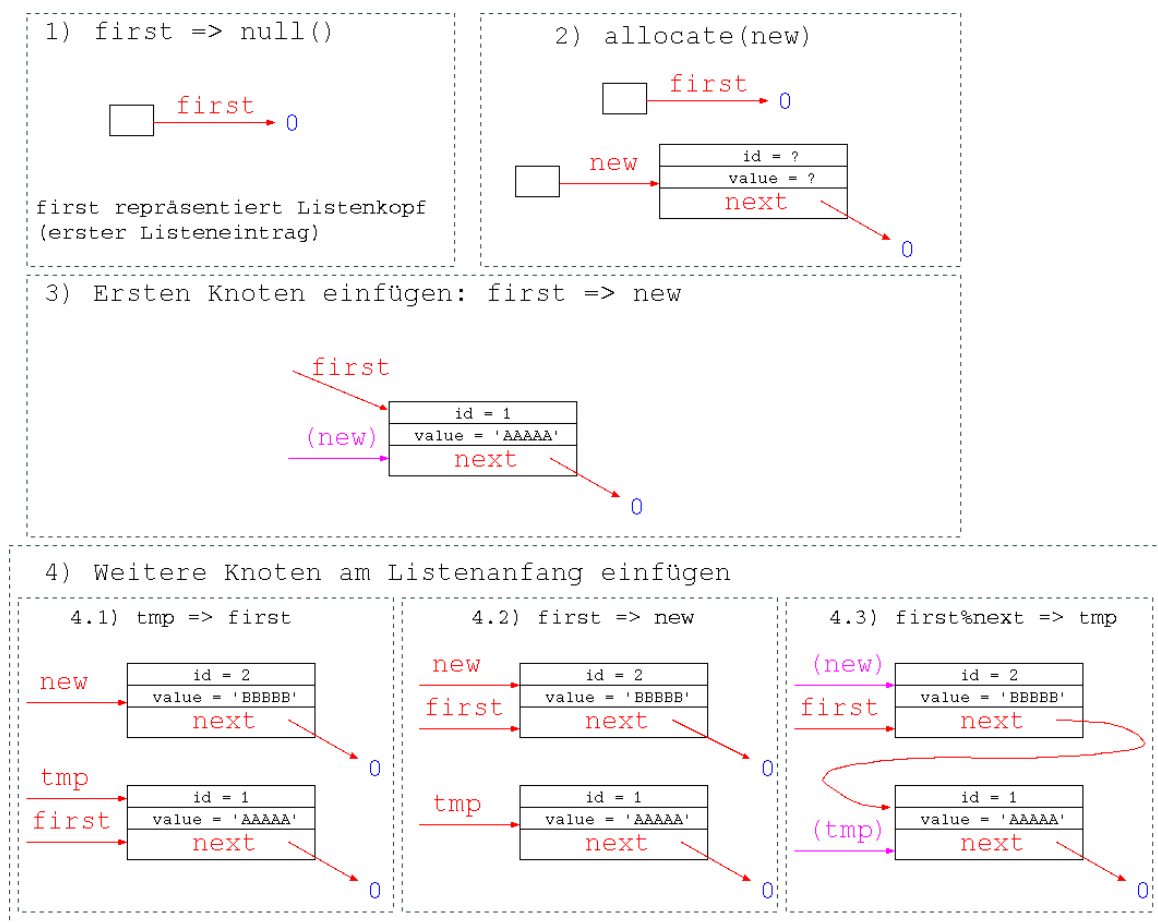


Abb. 37

Beispiel: Zyklische einfach verkettete Liste: **Fortran 90/95-Code (free source form)**

```

module m1
  implicit none
  private
  public: ...

  type node
    integer :: id
    character(5) :: value
    type(node), pointer :: next => null()
  end type

  type(node), pointer :: last => null() !externer Zeiger auf die zyklische
  Liste

... ! Vergleich oben, subroutine add_node(id, str)

subroutine add_node(id, str)
  implicit none

  integer, intent(in) :: id
  character(5), intent(in) :: str

  type(node), pointer :: new, tmp

  ! Speicher reservieren
  allocate(new)

  ! Werte setzen
  new%id = id
  new%value = str

  ! Listen Elemente werden in Reihenfolge des Einlesen verpointert
  if (.not. associated(last)) then
    last => new
    new%next => new                ! das erste Element muss auf sich
selbst zeigen für den zkylus
  else
    tmp => last
    last => new
    new%next => tmp%next
    tmp%next => new
  end if

end subroutine
...

```

31.6.2. Löschen von Listenelementen

Für das Löschen eines Elementes aus einer Liste lässt sich eine kurze Subroutine schreiben, welche die Zeiger neu setzt und das Element löscht. Man kann dabei in einfach verketteten Listen nicht ohne größeren Aufwand das Element löschen, welches soeben in der Liste betrachtet wird, sondern nur das nachfolgende Element.

Fortran 90/95-Code (free source form)

```

SUBROUTINE del_next(pntr)
  TYPE(liste),POINTER :: pntr                !Subroutine bekommt einen
externen Pointer übergeben
  TYPE(liste),POINTER :: tmp => NULL()

```

```
tmp => pntr          ! pntr wird in tmp zwischengespeichert
pntr => pntr%next    ! pntr wird um zwei Positionen weitergesetzt
pntr => pntr%next

DEALLOCATE(tmp%next) ! tmp%next wird gelöscht
tmp%next => pntr      ! Verbindung wird neu gesetzt
END SUBROUTINE
```

32. Vektoren- und Matrizenrechnung

In Fortran95 können einige elementare Vektoren- und Matrizenoperationen sehr einfach ausgeführt werden.

Beispiel: Addition und Subtraktion von Vektoren (Matrizen) **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real, dimension(3) :: a = (/3, 2, -5/), b = (/1, -3, -1/)

  write(*,*) "a+b: ", a+b
  ! Ausgabe: a+b:  4. -1. -6.

  write(*,*) "a-b: ", a-b
  ! Ausgabe: a-b:  2. 5. -4.
end program bsp
```

Beispiel: Multiplikation eines Vektors (einer Matrix) mit einem Skalar **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real, dimension(3) :: a = (/3, 2, -5/)

  write(*,*) "3.5*a: ", 3.5*a
  ! Ausgabe: 3.5*a:  10.5 7. -17.5
end program bsp
```

Beispiel: Skalarprodukt **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  real, dimension(3) :: a = (/3, 2, -5/), b = (/1, -3, -1/)
  real                :: dot_product

  write(*,*) "a.b: ", dot_product(a, b)
  ! Ausgabe: 2
end program bsp
```

Beispiel: Euklidische Norm eines Vektors $|\mathbf{x}| = \sqrt{\mathbf{x}^2} = \sqrt{\sum_{i=1}^n x_i^2}$ **Fortran 90/95-Code (free source form)**

```
program bsp
```

```

implicit none

real, dimension(3) :: x = (/3, 2, -5/)
real                :: x_n

! Norm des Vektors x
x_n = sqrt(sum(x**2))

write(*,*) "Die Norm des Vektors (", x, ") beträgt: ", x_n
! Ausgabe: Die Norm des Vektors ( 3.0  2.0 -5.0 ) beträgt:  6.164414
end program bsp

```

Beispiel: Matrizenmultiplikation **Fortran 90/95-Code** (free source form)

```

program bsp
  implicit none

  real, dimension(3,2) :: A = reshape( (/3., 2., 1., 1., 1., 2.5/), (/3, 2/) )
  real, dimension(2,2) :: B = reshape( (/1., -3., -1., 5./), (/2, 2/) )
  real, dimension(3,2) :: C

  C = matmul(A, B)

  write(*,*) "Matrix C ="
  write(*,*) C(1, :)
  write(*,*) C(2, :)
  write(*,*) C(3, :)
  ! Ausgabe: Matrix C =
  !           0.000000    2.000000
  !           -1.000000    3.000000
  !           -6.500000   11.500000
end program bsp

```

Beispiel: Transponierte Matrix **Fortran 90/95-Code** (free source form)

```

program bsp
  implicit none

  real, dimension(2,2) :: A = reshape( (/3., 2., 1., -1.5/), (/2, 2/) ), AT

  AT = transpose(A)

  write(*,*) "A ="
  write(*,*) A(1, :)
  write(*,*) A(2, :)
  ! Ausgabe: A =
  ! 3.000000    1.000000
  ! 2.000000   -1.500000

  write(*,*) "AT ="
  write(*,*) AT(1, :)
  write(*,*) AT(2, :)
  ! Ausgabe: AT =
  ! 3.000000    2.000000
  ! 1.000000   -1.500000
end program bsp

```

33. Systemroutinen

33.1. Datum und Zeit

Die Subroutine

```
date_and_time(datum,zeit)
```

liefert die Systemzeit in der Form YYYYMMTT und HHMMSS.SSS zurück. **datum** und **zeit** sind Character von mindestens 8 bzw. 10 Zeichen Länge.

Die vom Programm verbrauchte Rechenzeit (Prozessorzeit) in Sekunden liefert die Subroutine (**zeit** ist vom Typ Real)

```
cpu_time(zeit)
```

33.2. Zufallszahlen

Die Subroutine

```
random_number(r)
```

schreibt gleichverteilte Zufallszahlen im Intervall [0,1) in eine Variable **r** vom Typ Real (Skalar oder Feld). Mit der Subroutine

```
random_seed()
```

kann der Zufallszahlengenerator (zufällig) initialisiert werden.

33.3. Kommandozeilenargumente

Nicht unbedingt Standard, aber bei etlichen Fortran-Compilern doch als Standardfunktion implementiert, sind die Prozeduren **iargc** und **getarg** zum Erfragen der beim Programmstart mitgegebenen Kommandozeilenargumente.

Die Funktion

```
i = iargc()
```

liefert die Anzahl der Kommandozeilenargumente. Der Programmname selbst wird dabei nicht mitgezählt.

Die Subroutine

```
getarg(i, c)
```

liefert den Wert eines bestimmten Kommandozeilenargumentes. *i* gibt die Position vor (0 ... Programmname, 1 ... 1.Argument, etc.). Der Parameter *c* ist vom Typ Character. Dort findet sich nach Abarbeitung der Subroutine der zu *i* gehörende Wert des Kommandozeilenargumentes.

34. Von der modularen zur objektorientierten Programmierung

34.1. Module im Detail

Die Bezeichnung des Schlüsselworts `module` weist schon darauf hin, dass Fortran 90/95 eine modulare Softwareentwicklung ermöglicht.

34.1.1. Modulare Programmierung

Was ist modulare Programmierung? w:Modulare Programmierung⁵⁶

Das Modul-Konzept in Fortran 90/95 unterstützt diesen Ansatz vollständig. Das `module`-Konstrukt gliedert sich schematisch so

```
module ...
```

```
    Datenbereich
```

```
    Methodenbereich
```

```
end module ...
```

Vor dem Datenbereich können noch einige Deklarationen (`implicit none`, `save`, etc.) eingefügt sein. Der Methodenbereich wird durch das Schlüsselwort `contains` angekündigt oder als Interface deklariert. Aber prinzipiell gilt, dass zusammengehörende Daten und die dazugehörigen Methoden (Unterprogramme) in einem Modul zusammengefasst werden.

34.1.2. Zugriffsteuerung

Durch Angabe des Schlüsselworts `private` lässt sich die Sichtbarkeit von Datenelementen einschränken. Auf solcherart deklarierte Variablen lässt sich außerhalb des Moduls nicht zugreifen. `public` erlaubt den Zugriff auf entsprechend deklarierte Variablen auch von außerhalb. Zweiteres ist Standardverhalten und das Schlüsselwort `public` muss somit nicht explizit angegeben werden.

Beispiel: **Fortran 90/95-Code (free source form)**

⁵⁶ <https://de.wikipedia.org/wiki/Modulare%20Programmierung>


```
module mod_bsp
  implicit none
  save

  ! Datenbereich
  real, private :: x = 1.2
  real          :: y = 9.8

  contains
  ! Methodenbereich
  real function addX (a)
    real, intent (in) :: a

    addX = x + a
  end function addX
end module mod_bsp

program bsp
  use mod_bsp
  implicit none

  write (*,*) "Ergebnis1 = ", addX (2.1)
  write (*,*) "Ergebnis2 = ", y + 2.1

  ! Ausgabe:
  !   Ergebnis1 =    3.300000
  !   Ergebnis2 =   11.900000

  ! Folgendes geht nicht -> Fehlermeldung:
  ! write (*,*) "Ergebnis3 = ", x + 2.1
  !   1) x ist private und somit außerhalb des Moduls nicht bekannt
  !   2) auch im Hauptprogramm selbst ist kein x deklariert (implicit none,
  !       es wäre ohnehin nicht die gleiche Variable wie im Modul mod_bsp)
end program bsp
```

Rein formal läßt sich der Zugriff auf die Daten und/oder Methoden eines Moduls (module procedures) auch auf andere Arten einschränken, z.B.

```
module mod_bsp
  implicit none
  save

  private :: x ! , ...

  real :: x = 1.2, y = 9.8
  ! ...
```

oder

```
module mod_bsp
  implicit none
  save

  private
  public :: y, addX ! , ...

  real :: x = 1.2, y = 9.8
  ! ...
```

34.1.3. Datenkapselung, COMMONS-Ersatz: Module als Datenbereich

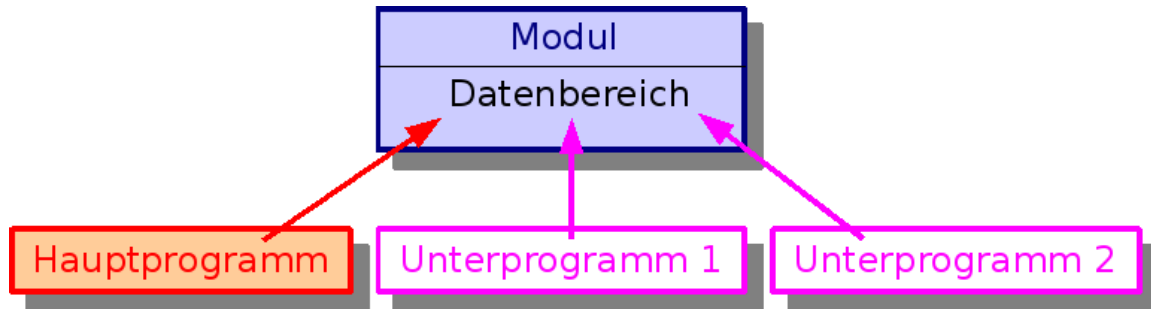


Abb. 38

Module als reinen Datenbereich zu nutzen ist vor allem dann interessant, wenn aus mehreren Programmeinheiten auf die gleichen Daten zugegriffen werden muss, jedoch die zugreifenden Unterprogramme nicht wirklich modulspezifisch sind und dementsprechend nicht als Bestandteil des Moduls angelegt werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```

module konstanten
  real, parameter :: PI = 3.141593
  real, parameter :: E = 2.718282
end module konstanten

! Hauptprogramm
program bsp
  use konstanten
  implicit none

  real :: kreisflaeche

  write(*,*) 'PI = ', PI
  write(*,*) 'E = ', E
  write(*,*) 'Kreisflaeche fuer r=2.1 = ', kreisflaeche(2.1)
  call calcPiMalE
end program bsp

! Unterprogramm 1
real function kreisflaeche(r)
  use konstanten
  implicit none

  real, intent (in) :: r

  kreisflaeche = r**2 * PI
end function kreisflaeche

! Unterprogramm 2
subroutine calcPiMalE()
  use konstanten
  implicit none

  write (*,*) 'PI * E = ', PI * E
end subroutine calcPiMalE
  
```

```
! Ausgabe:
!  PI =    3.141593
!  E =    2.718282
!  Kreisflaeche fuer r=2.1 =    13.85442
!  PI * E =    8.539735
```

34.1.4. Datenabstraktion: Zusammenfassung von Daten und Methoden in einem Modul

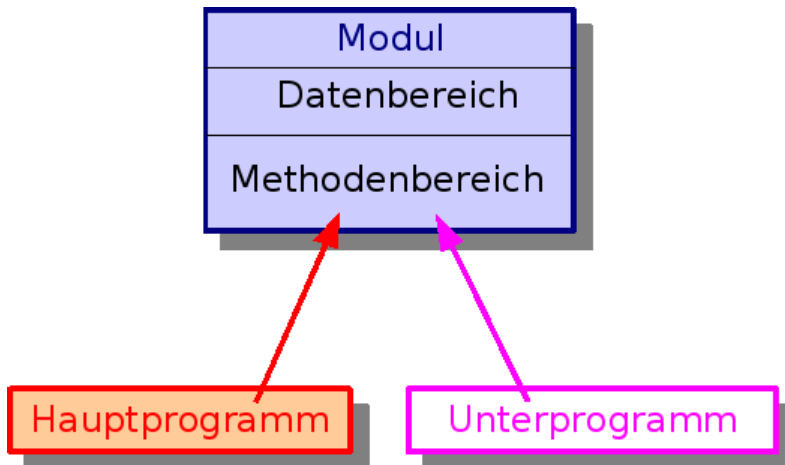


Abb. 39

Sind Daten und Unterprogramme als zusammengehörend zu betrachten, so ist es sinnvoll diese auch gemeinsam in einem Modul abzulegen. Ein Vorteil dabei ist, dass somit die Zugriffssteuerung gezielt eingesetzt werden kann. Moduldaten, die nicht von außerhalb des Moduls geändert werden dürfen, werden als `private` deklariert. Der Zugriff auf diese Daten kann dann nur noch mittels Methoden des Moduls erfolgen.

Beispiel: **Fortran 90/95-Code (free source form)**

```
module kreis
  implicit none
  save

  real, parameter :: PI = 3.141593
  real, private  :: r = 0.0

  contains
    subroutine setR(val)
      real, intent(in) :: val
      r = val
    end subroutine setR

    real function getR()
      getR = r
    end function getR

    real function kreisflaeche()
      kreisflaeche = r**2 * PI
    end function kreisflaeche
end module kreis
```

```

! Hauptprogramm
program bsp
  use kreis
  implicit none

  call setR(5.0)
  write (*,*) "r = ", getR()
  write (*,*) "Flaeche = ", kreisflaeche()

  call sub1(5.0)
end program bsp

! Unterprogramm
subroutine sub1(val)
  use kreis
  implicit none

  real, intent(in) :: val

  call setR(val*2.56)
  write (*,*) "r = ", getR()
  write (*,*) "Flaeche = ", kreisflaeche()
end subroutine sub1

! Ausgabe:
!   r =  5.
!   Flaeche =  78.539825
!   r =  12.799999
!   Flaeche =  514.7185

```

34.1.5. Modul und Datenverbund

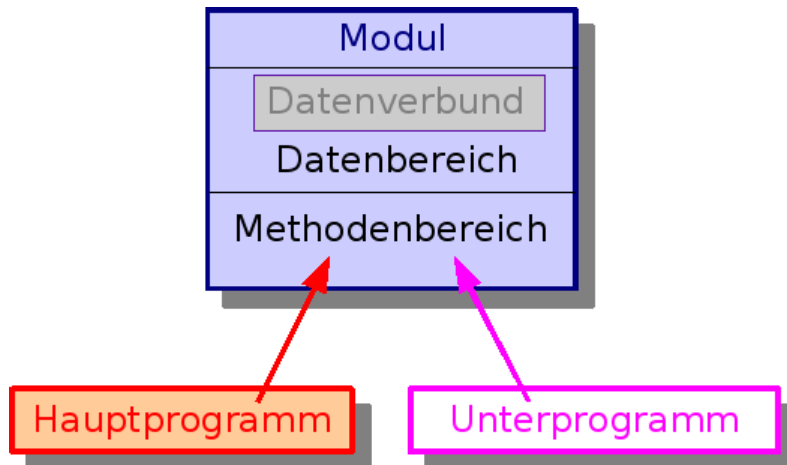


Abb. 40

Das Schlüsselwort `private` darf in einem Datenverbund nur in Verbindung mit einem Modul Anwendung finden. Einerseits kann die Sichtbarkeit aller Variablen im Datenverbund eingeschränkt werden. Andererseits kann auch der Datenverbund selbst als `private` deklariert werden. Ein Zugriff auf derart deklarierte Datenelemente ist dann nur noch durch Unterprogramme des gleichen Moduls möglich. Standardmäßig sind sowohl Datenverbund als auch

seine Einzelkomponenten, so wie alle anderen nicht zugriffsbeschränkten Datenelemente, in einem Modul öffentlich (public).

Beispiel: Öffentlicher Datenverbund **Fortran 90/95-Code (free source form)**

```
module mod1
  save

  type :: tripel
    real:: x, y, z
  end type tripel
end module mod1

program bsp
  use mod1
  implicit none

  type(tripel) :: tr = tripel(10.5, 0.0, -6.5)

  write(*, *) tr
! Ausgabe:
! 10.5  0.0  -6.5
end program bsp
```

Beispiel: Öffentlicher Datenverbund mit privaten Datenelementen **Fortran 90/95-Code (free source form)**

```
module mod1
  save

  type :: tripel
    private
    real:: x, y, z
  end type tripel

  contains
    subroutine createTripel(this, valX, valY, valZ)
      type(tripel)      :: this
      real, intent(in) :: valX, valY, valZ

      this%x = valX
      this%y = valY
      this%z = valZ
    end subroutine createTripel

    subroutine writeTripel(this)
      type(tripel) :: this

      write(*,*) this
    end subroutine writeTripel
end module mod1

program bsp
  use mod1
  implicit none

  type(tripel) :: tr

  call createTripel(tr, 10.5, 0.0, -6.5)
  call writeTripel(tr)
```

```
! Ausgabe:
!   10.50000    0.000000    -6.500000
end program bsp
```

Beispiel: Privater Datenverbund

Fortran 90/95-Code (free source form)

```
module mod1
  save

  type, private :: tripel
    real :: x = 0.0, y=0.0, z=0.0
  end type tripel

  type(tripel), private :: t

  contains
    subroutine changeTripel(valX, valY, valZ)
      real, intent(in) :: valX, valY, valZ
      t = tripel(valX, valY, valZ)
    end subroutine changeTripel

    subroutine writeTripel()
      write (*,*) t
    end subroutine writeTripel
end module mod1

program bsp
  use mod1
  implicit none

  ! Hier könnte z.B. keine Variable vom Typ "triple" angelegt werden, da in dieser
  ! PE nicht
  ! sichtbar (privater Datenverbund des Moduls mod1)

  call writeTripel
  ! Ausgabe:
  !   0.0 0.0 0.0

  call changeTripel(10.5, -5.0, -3.5)

  call writeTripel
  ! Ausgabe:
  !   10.5 -5. -3.5

  call unterprogramm
  ! Ausgabe:
  !   10.5 -5. -3.5
end program bsp

subroutine unterprogramm
  use mod1

  call writeTripel
end subroutine unterprogramm
```

Dieses Beispiel scheint auf den ersten Blick im Gegensatz zu den beiden vorherigen Varianten unnötige Einschränkungen aufzuweisen. Im Haupt- und Unterprogramm können keine Variablen des Typs `tripel` angelegt werden. Im Modul wird immer auf die gleiche Varia-

ble t zugegriffen. Das gewählte Beispiel könnte somit als Singleton⁵⁷ beschrieben werden. Es ist außerhalb des Moduls sichergestellt, dass immer nur ein Tripel existiert. Singletons werden auch in der objektorientierten Softwareentwicklung verwendet. Daneben sind auch andere Situationen vorstellbar, in denen sich das Konzept eines privaten Datenverbunds als nützlich erweisen kann.

34.1.6. Die Schnittstelle: Das Modul als Unterprogrammbibliothek

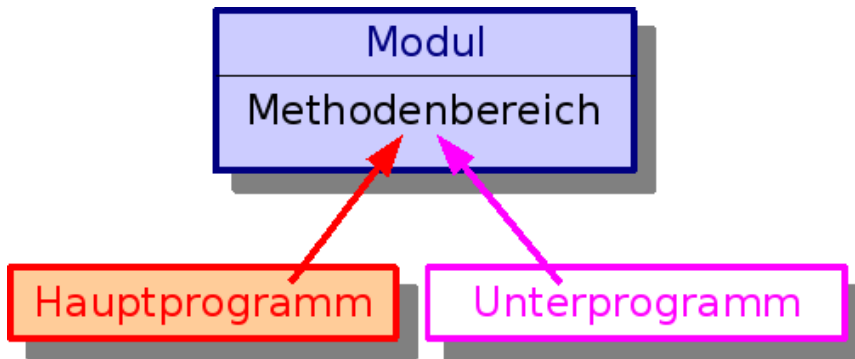


Abb. 41

Ein Modul kann als Sammelstelle für Unterprogramme dienen - quasi eine Bibliothek für häufig und in verschiedenen Kontexten benötigte Prozeduren.

Vorteile:

- Ordnung
- Modul kann in eine eigenständige Datei ausgelagert werden.
- Parameter-Datentypüberprüfung.

Beispiel: Parameter-Datentypüberprüfung

ohne Modul	mit Modul
------------	-----------

⁵⁷ <https://de.wikipedia.org/wiki/Singleton%20%28Mathematik%29>

<pre> program bsp implicit none integer :: zahl = 5 call unterprogramm(zahl) end program bsp subroutine unterprogramm(a) implicit none real, intent(in) :: a write (*,*) a end subroutine unterprogramm </pre>	<pre> module test contains subroutine unterprogramm(a) implicit none real, intent(in) :: a write (*,*) a end subroutine unterprogramm end module test program bsp use test implicit none integer :: zahl = 5 call unterprogramm(zahl) end program bsp </pre>
• <i>gfortran</i> und <i>ifort</i> compilieren dieses Programm ohne Warnhinweis. • Der <i>g95</i> liefert in diesem Fall zumindest eine Warnung, compiliert jedoch auch das Programm. Die Datenausgabe zur Programmlaufzeit liefert jeweils einen fehlerhaften Wert.	Bereits beim Compilieren tritt eine Fehlermeldung auf, z.B. bei <i>g95</i>: In filef90:20 call unterprogramm(zahl) 1 Error: Type mismatch in parameter 'a' at (1). Passing INTEGER(4) to REAL(4)Das Programm lässt sich so nicht compilieren.

Der Schnittstellenblock: interface

Fortran 90/95 kennt das Sprachkonstrukt **interface**, welches sich in vielerlei Hinsicht nutzbringend verwenden lässt. Schnittstellenblöcke sind modulunabhängig verwendbar.

```
interface [name] [interface-Spezifikationsteil] end interface [name]
```

Der *interface-Spezifikationsteil* beinhaltet nur diejenigen Informationen, die für die Deklaration der Schnittstelle relevant sind (z.B. Unterprogrammbezeichnung, Variablendeklaration). Die genaue Festlegung der Unterprogramme (inkl. Ausführungsteil) erfolgt dann außerhalb des Schnittstellenblocks. Ein wesentliches Merkmal des Schnittstellenblocks ist die penible Überprüfung der Unterprogrammparameterdatentypen, so wie das auch beim Einsatz von Modulen geschieht.

Beispiel: Parameter-Datentypüberprüfung

ohne Schnittstellenblock	mit Schnittstellenblock
--------------------------	-------------------------

<pre> program bsp implicit none integer :: zahl = 5 call unterprogramm(zahl) end program bsp subroutine unterprogramm(a) implicit none real, intent(in) :: a write (*,*) a end subroutine unterprogramm </pre>	<pre> program bsp implicit none integer :: zahl = 5 interface subroutine unterprogramm(a) real, intent(in) :: a end subroutine unterprogramm end interface call unterprogramm(zahl) end program bsp subroutine unterprogramm(a) implicit none real, intent(in) :: a write (*,*) a end subroutine unterprogramm </pre>
• <i>gfortran</i> und <i>ifort</i> compilieren dieses Programm ohne Warnhinweis. • Der <i>g95</i> liefert in diesem Fall zumindest eine Warnung, compiliert jedoch auch das Programm. Die Datenausgabe zur Programmlaufzeit liefert jeweils einen fehlerhaften Wert.	Bereits beim Compilieren tritt eine Fehlermeldung auf, z.B. bei <i>gfortran</i>: In filef90:15 call unterprogramm(zahl) 1 Error: Type/rank mismatch in argument 'a' at (1)Das Programm lässt sich so nicht compilieren.

Generische Unterprogrammschnittstelle

Fortran 90/95 ermöglicht generische Methoden. Die im Schnittstellenblock deklarierten Unterprogramme können damit über den gleichen Unterprogrammnamen aufgerufen werden. Intern erfolgt dann der Aufruf des jeweils passenden Unterprogramms durch die Unterschiede der Datentypen der Unterprogrammparameter. Mittel zum Zweck ist der benannte Schnittstellenblock.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  interface gensub
    subroutine writeReal(val)
      real, intent(in) :: val
    end subroutine writeReal

    subroutine writeInteger(val)
      integer, intent(in) :: val
    end subroutine writeInteger

    subroutine writeCharacter(val)
      character, intent(in) :: val
    end subroutine writeCharacter
  end interface gensub

  call gensub(5.5)
  call gensub(3)
  call gensub("H")
  call writeCharacter("X")

```

```

! Ausgabe:
!   Real-Wert =    5.500000
!   Integer-Wert =          3
!   Zeichen = H
!   Zeichen = X
end program bsp

subroutine writeReal(val)
  real, intent(in) :: val
  write (*,*) "Real-Wert = ", val
end subroutine writeReal

subroutine writeInteger(val)
  integer, intent(in) :: val
  write (*,*) "Integer-Wert = ", val
end subroutine writeInteger

subroutine writeCharacter(val)
  character, intent(in) :: val
  write (*,*) "Zeichen = ", val
end subroutine writeCharacter

```

Operatorüberladung

Fortran 90/95 verwendet von Haus aus Operatorüberladung. So können z.B. Variablen arithmetischen Datentyps einfach miteinander addiert werden:

```
c = a + b
```

Dieser Ausdruck funktioniert unabhängig davon, ob die Variablen vom Typ Ganzzahl oder Gleitkommzahl sind. Und auch für Felder funktioniert diese einfache Form der Addition. Die einzelnen Feldkomponenten werden korrekt addiert. Das ist in anderen Programmiersprachen nicht selbstverständlich. Zusätzlich kann in Fortran 90/95 auch der Programmierer mit Hilfe von Schnittstellenblöcken sogenannte *defined operations* festlegen.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  implicit none

  interface operator (.PLUS.)
    function charAdd(c1, c2)
      character, intent(in) :: c1, c2
      character(len=2)      :: charAdd
    end function charAdd
  end interface operator (.PLUS.)

! "Addition" mittels definiertem .PLUS.-Operator
write (*,*) "c1 .PLUS. c2 = ", "A" .PLUS. "B"

! oder auch mittels Funktion
write (*,*) "charAdd () = ", charAdd("A", "B")

! Ausgabe:
!   c1 .PLUS. c2 = AB
!   charAdd () = AB
end program bsp

```

```
function charAdd(c1, c2)
  implicit none

  character, intent(in) :: c1, c2
  character(len=2)      :: charAdd

  charAdd = c1 // c2
end function charAdd
```

Beispiel: nur mit *gfortran* compilierbar, nicht mit *g95* oder *ifort* **Fortran 90/95-Code (free source form)**

```
module mod
  type :: tripel
    real x, y, z
  end type
end module mod

program bsp
  use mod
  implicit none

  interface operator (*)
    function tripelMult(t1, t2)
      type(tripel), intent(in) :: t1, t2
      type(tripel)              :: tripelMult
    end function tripelMult
  end interface operator (*)

  ! Multiplikation mittels überladenem *-Operator
  write (*,*) "t1 * t2 =", tripel(2.0, 3.0, 4.0) * tripel(1.5, 0.5, 2.0)

  ! oder auch mittels Funktion
  write (*,*) "tripelMult () =", tripelMult(tripel(2.0, 3.0, 4.0), tripel(1.5,
    0.5, 2.0))

  ! Ausgabe:
  !   t1 * t2 =   3.000000   1.500000   8.000000
  !   tripelMult () = 3.000000   1.500000   8.000000
end program bsp

function tripelMult(t1, t2)
  use mod
  implicit none

  type(tripel), intent(in) :: t1, t2
  type(tripel)              :: tripelMult

  tripelMult = tripel(t1%x * t2%x, t1%y*t2%y, t1%z*t2%z)
end function tripelMult
```

Bei der Verwendung von *defined operations* sind jedoch einige Bedingungen zu beachten.

Neben den *defined operations* (*interface operator*) gibt es auch noch *defined assignments* (*interface assignment*) für die Überladung des Zuweisungsoperators.

Schnittstellenblock und Modul

Beim vorigen Beispiel bestand das Problem, dass nur einer der getesteten Compiler eine ausführbare Datei zu Stande brachte. Abhilfe schaffen kann die Verlagerung des Schnittstellenblocks in das Modul unter Zuhilfenahme von `module procedure`.

Beispiel: Diesen Programmcode schlucken sowohl *gfortran*, *g95* als auch *ifort* problemlos
Fortran 90/95-Code (free source form)

```
module mod
  type :: tripel
    real x, y, z
  end type

  interface operator (*)
    module procedure tripelMult
  end interface operator (*)

  contains
    function tripelMult(t1, t2)
      implicit none

      type(tripel), intent(in) :: t1, t2
      type(tripel)              :: tripelMult

      tripelMult = tripel(t1%x * t2%x, t1%y*t2%y, t1%z*t2%z)
    end function tripelMult
end module mod

program bsp
  use mod
  implicit none

  ! Multiplikation mittels überladenem *-Operator
  write (*,*) "t1 * t2 =", tripel(2.0, 3.0, 4.0) * tripel(1.5, 0.5, 2.0)

  ! oder auch mittels Funktion
  write (*,*) "tripelMult () =", tripelMult(tripel(2.0, 3.0, 4.0), tripel(1.5,
    0.5, 2.0))

  ! Ausgabe:
  !   t1 * t2 =   3.000000      1.500000      8.000000
  !   tripelMult () =   3.000000      1.500000      8.000000
end program bsp
```

Unterprogramme: Die Übergabe optionaler und benannter Parameter

Der Einsatz optionaler Parameter wurde bereits im Kapitel Unterprogramme⁵⁸ abgehandelt. Zusätzlich unterstützt Fortran 90/95 auch die Verwendung von benannten Parametern (argument keywords). Dazu ist es erforderlich, dass das Unterprogramm ein explizites Interface aufweist, wie das z.B. bei Einbindung von Unterprogrammen in Module automatisch der Fall ist.

Beispiel: **Fortran 90/95-Code (free source form)**

```
module m1
```

⁵⁸ Kapitel 29.6 auf Seite 179

```
contains
  subroutine abc( var1, var2 )
    implicit none

    integer, intent( in ) :: var1, var2

    write( *, * ) var1, var2
  end subroutine abc
end module m1

program bsp
  use m1
  implicit none

  call abc( 12, 99 )
  call abc( var2 = 99, var1 = 12 )
  call abc( var2 = 12, var1 = 99 )
  call abc( 12, var2 = 99 )
! call abc( var1 = 12, 99 )      ! so funktioniert das nicht

! Ausgabe:
!      12          99
!      12          99
!      99          12
!      12          99
end program bsp
```

34.1.7. Sonstiges

- use *modulbezeichnung*, only : *modulelemente*
- use *modulbezeichnung* => *bezeichnung*

34.2. Objektorientierte Programmierung

Modulares Programmieren reißt heutzutage niemanden mehr vom Hocker und selbst objektorientierte Konzepte sind seit spätestens Ende der 80-Jahre des vergangenen Jahrhunderts Stand der Technik. Im akademischen Bereich waren diese Dinge natürlich schon früher bekannt. Fortran hat in diesem Bereich also nie eine Vorreiterrolle inne gehabt. Nach jeweils kurzen Nachdenkphasen haben aber die Fortran-Leute unerschrocken auch in diesem Bereich nachgezogen.

Fortran bildet also aufgrund seiner langen Geschichte ein weites Spektrum der Programmierparadigmen ab: vom prozeduralen Programmieren (FORTRAN 77) über das modulare Programmieren (Fortran 90/95) hin zum objektorientierten Programmieren (Fortran 2003).

Fortran 90/95 ist keine typisch objektorientierte Sprache. Trotzdem lässt sich mit den modularen Spracheigenschaften von Fortran 90/95 bereits in weiten Bereichen Objektorientierung simulieren beziehungsweise nachbauen. Aber erst mit Fortran 2003 lässt sich wirklich komfortabel objektorientiert programmieren. Diese Fortran-Version wirbt dann auch „offiziell“ mit diesem Merkmal.

Welche Eigenschaften verbindet man typischerweise mit Objektorientierung? Da wären

1. Klassen - Datenkapselung und Datenabstraktion
2. Objekte

3. Vererbung
4. Statische Polymorphie: Überladen von Funktionen (und Operatoren)
5. Run-Time-Polymorphie

Welche dieser Merkmale bietet Fortran 90/95 von Haus aus und was lässt sich relativ einfach nachbauen?

34.2.1. Klassen - Datenkapselung und Datenabstraktion

Was ist eine Klasse?

- w:Klasse (objektorientierte Programmierung)⁵⁹
- w:Klasse (UML)⁶⁰

Ein Fortran 90/95-Modul sieht sehr ähnlich aus wie eine Klasse in der objektorientierten Programmierung und ist dementsprechend auch Ausgangspunkt für den objektorientierten Ansatz in Fortran 90/95.

Klasse	Fortran 90/95
Klassenbezeichner	module ... ! Datenbereich contains ! Methodenbereich end module ...
Datenbereich (Attribute)	
Methodenbereich (Operationen)	

Einen grundlegenden Mechanismus zum Schreiben objektorientierter Programme stellt Fortran 90/95 somit in Form der Module zur Verfügung. Jetzt stellt sich aber die Frage, wie diese Klasse instantiiert werden soll. Das wird im Abschnitt *Objekte* gezeigt.

34.2.2. Objekte

Was sind Objekte? w:Objekt (Programmierung)⁶¹

Ein einfaches Modul mit ein paar konventionellen Datenelementen und Prozeduren ist noch keine Klasse, aus der Objekte zu erzeugen wären. Dazu bedarf es noch eines Unterscheidungskriteriums zwischen den einzelnen Objekten. Dieses Unterscheidungskriterium kann, wie auch schon früher im Teilkapitel *Modul und Datenverbund*⁶² gezeigt, mittels Datenverbund hergestellt werden. Die Objekte werden somit nicht über den Modulnamen als Klasse angesprochen, sondern über die Datenverbund-Bezeichnung. Auch das ist keine neue Erkenntnis, sondern eine geschickte Ausnutzung eines Mechanismus, den Fortran 90/95 standardmäßig mit Modulen und Datenverbund zur Verfügung stellt.

Beispiel:

⁵⁹ <https://de.wikipedia.org/wiki/Klasse%20%28objektorientierte%20Programmierung%29>

⁶⁰ <https://de.wikipedia.org/wiki/Klasse%20%28UML%29>

⁶¹ <https://de.wikipedia.org/wiki/Objekt%20%28Programmierung%29>

⁶² Kapitel 34.1.5 auf Seite 219

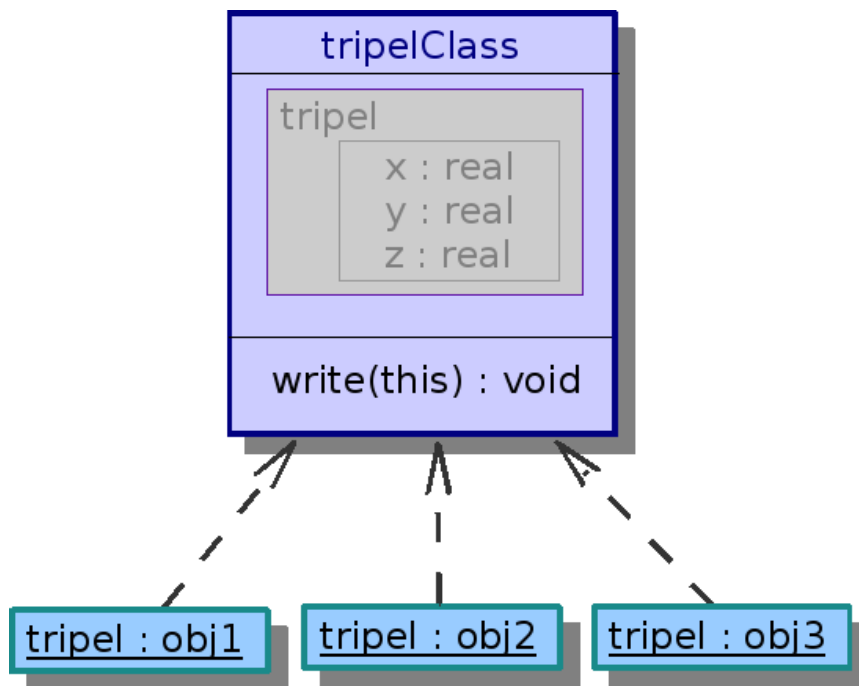


Abb. 42

Fortran 90/95-Code (free source form)

```
module tripelClass
  implicit none
  save

  type :: tripel
    real:: x, y, z
  end type tripel

  contains
    subroutine write(this)
      type(tripel), intent(in) :: this

      write(*, *) "***** Tripel *****"
      write(*, *) this
    end subroutine
end module tripelClass

program bsp
  use tripelClass
  implicit none

  type(tripel) :: obj1 = tripel(1.5, 0.0, -6.5)
  type(tripel) :: obj2 = tripel(2.5, 1.0, -4.5)
  type(tripel) :: obj3 = tripel(3.5, 2.0, -2.5)

  call write(obj1)
  call write(obj2)
  call write(obj3)

  ! Ausgabe:
```

```

! ***** Tripel *****
!      1.500000      0.000000      -6.500000
! ***** Tripel *****
!      2.500000      1.000000      -4.500000
! ***** Tripel *****
!      3.500000      2.000000      -2.500000
end program bsp

```

Weitere wichtige Elemente beim Erzeugen und Zerstören von Objekten sind in der OOP die sogenannten Konstruktoren und Destruktoren.

Was sind Konstruktoren und Destruktoren? w:Konstruktoren und Destruktoren⁶³

Konstruktoren und Destruktoren kennt Fortran 90/95 natürlich nicht. Im Bedarfsfall sind diese Elemente also mittels konventioneller Unterprogramme nachzubilden und dann jeweils explizit in den entsprechenden Programmabschnitten, nach Erzeugung beziehungsweise beim Abbau des jeweiligen Objektes, manuell aufzurufen.

34.2.3. Vererbung

Was ist Vererbung? w:Vererbung (Programmierung)⁶⁴

Das OOP-Prinzip „Vererbung“ ist in Fortran 90/95 nur über Umwege realisierbar. Fortran 90/95 kennt konzeptionsbedingt für diesen Zweck keinen einfachen programmtechnischen Mechanismus. Nachfolgend wird anhand eines kleinen und überschaubaren Beispiels eine mögliche Lösung demonstriert.

Beispiel:

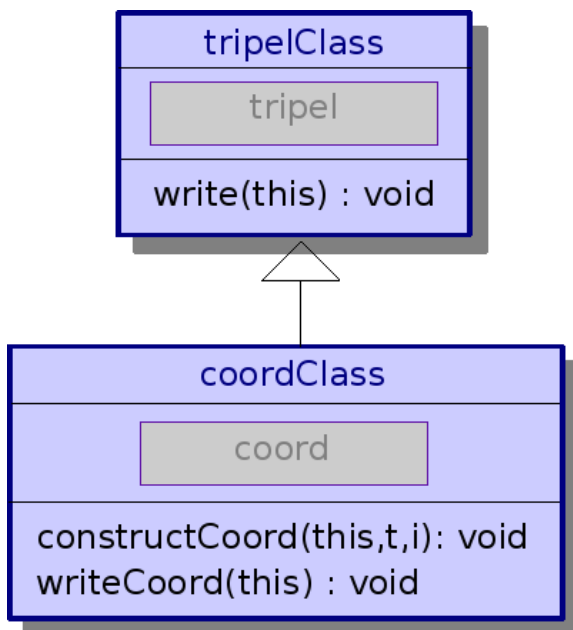


Abb. 43

⁶³ <https://de.wikipedia.org/wiki/Konstruktoren%20und%20Destruktoren>

⁶⁴ <https://de.wikipedia.org/wiki/Vererbung%20%28Programmierung%29>

Fortran 90/95-Code (free source form)

```
module tripelClass
  implicit none
  save

  type :: tripel
    real:: x, y, z
  end type tripel

  contains
    subroutine write(this)
      type(tripel), intent(in) :: this

      write(*, *) "***** Tripel *****"
      write(*, *) this
    end subroutine
end module tripelClass

module coordClass
  use tripelClass
  implicit none
  save

  type :: coord
    type(tripel) :: tr
    integer      :: id
  end type

  contains
    subroutine constructCoord(this, t, i)
      implicit none

      type(coord), intent(out) :: this
      type(tripel), intent(in) :: t
      integer              :: i

      this%tr = t
      this%id = i
    end subroutine constructCoord

    subroutine writeCoord(this)
      implicit none

      type(coord), intent(in) :: this

      write(*, "(A, I5, A)") "***** KOORDINATE ", this%id, "
*****"
      call write(this%tr)
    end subroutine writeCoord
end module coordClass

program bsp
  use coordClass
  implicit none

  type(coord) :: obj1 = coord(tripel(1.5, 0.0, -6.5), 1005)
  type(coord) :: obj2 = coord(tripel(2.5, 1.0, -4.5), 1006)
  type(coord) :: obj3 = coord(tripel(3.5, 2.0, -2.5), 1007)

  call writeCoord(obj1)
  call writeCoord(obj2)
  call writeCoord(obj3)
```

```

! Ausgabe:
! ***** KOORDINATE 1005 *****
! ***** Tripel *****
!      1.500000      0.000000      -6.500000
! ***** KOORDINATE 1006 *****
! ***** Tripel *****
!      2.500000      1.000000      -4.500000
! ***** KOORDINATE 1007 *****
! ***** Tripel *****
!      3.500000      2.000000      -2.500000
end program bsp

```

34.2.4. Statische Polymorphie: Überladen von Funktionen (und Operatoren)

Das Überladen von Funktionen und Operatoren unterstützt Fortran 90/95 wiederum standardmäßig. Diese Konzepte wurden schon in den Abschnitten

- Generische _Unterprogrammschnittstelle⁶⁵
- Operatorüberladung⁶⁶ und
- Schnittstellenblock und Modul⁶⁷

kurz erläutert.

34.2.5. Run-Time-Polymorphie

Polymorphie zur Laufzeit eines Programmes ist mit den Mitteln von Fortran 90/95 nur einigermaßen aufwendig und kompliziert nachzubauen, jedoch prinzipiell möglich. Es sei zu diesem Thema auf die weiterführende Literatur verwiesen.

34.3. Ausblick

Sind die Methoden zur Nachbildung objektorientierter Mechanismen auch in Fortan 2003 verwendbar? Prinzipiell schon, jedoch bietet Fortran 2003 bessere und erweiterte Möglichkeiten zur Behandlung dieses Themas. Dort wurde nämlich das Prinzip des Datenverbunds wesentlich erweitert. Ein `type`-Block kann dann neben den Datenelementen auch Unterprogramme beinhalten, das Schlüsselwort `extends` zwecks Vererbungmechanismus ist vorhanden, etc. Das in Fortran 90/95 als Datenbund/Struktur verwendbare `type`-Konstrukt wurde also in Fortran 2003 zu einer Klasse, ähnlich wie es in anderen Programmiersprachen unter der Bezeichnung `class` bekannt ist, aufgerüstet.

⁶⁵ Kapitel 34.1.6 auf Seite 224

⁶⁶ Kapitel 34.1.2 auf Seite 215

⁶⁷ Kapitel 34.1.6 auf Seite 227

34.4. Literatur

34.4.1. Quellen

- Decyk: Scientific Computing with Fortran⁶⁸
- Gray, Roberts: Object-Based Programming in Fortran 90⁶⁹

34.4.2. Weiterführende Literatur

- Decyk, Norton, Szymanski: Introduction to Object-Oriented Concepts Using Fortran90, 1996⁷⁰
- Decyk, Norton, Szymanski: How to Express C++ Concepts in Fortran90. 1997⁷¹
- Decyk, Norton, Szymanski: How to Support Inheritance and Run-Time Polymorphism in Fortran90, 1998⁷²
- Decyk, Norton: A Simplified Method for Implementing Run-Time Polymorphism in Fortran95, 2004⁷³
- Decyk, Gardner: Object-Oriented Design Patterns in Fortran95, 2006⁷⁴

68 <http://fizyka.umk.pl/~jkob/x5dhf/trunk/docs/scf95.lect1.pdf>
69 <https://www.semanticscholar.org/paper/Object-based-Programming-in-Fortran-90-Gray-and-in-Gray-Roberts/3b2ef0a00991a55f1ec112c17c5d0eb0f830491d>
70 http://www.cs.rpi.edu/~szymansk/00F90/F90_Objects.html
71 <https://www.hindawi.com/journals/sp/1997/570746/>
72 <https://www.semanticscholar.org/paper/How-to-support-inheritance-and-run-time-in-Fortran-Decyk-Norton/456d79a8193f49a06b95fcbbe928ce0f0e55d8cf>
73 https://www.researchgate.net/publication/220061013_A_Simplified_Method_for_Implementing_Run-Time_Polymorphism_in_Fortran95
74 <https://www.semanticscholar.org/paper/Object-Oriented-Design-Patterns-in-Fortran-90%2F95-Decyk-Gardner/33416542f32d03debf5b8eb762b7086c722db8c0>

35. Offizielle Fortran 95-Erweiterungen

Hier sind jene ISO/IEC-Standards angeführt, die nicht Teil des Original-Fortran 95-Standards sind, sondern erst später als offizielle Fortran 95-Erweiterungen formuliert wurden. Etliche Fortran 95-Compiler bieten diese Features derzeit noch nicht oder nur teilweise.

35.1. Floating-point Exceptions

Im Fortran 2003-Standard sind Module betreffend IEEE 754-Standard (IEEE Standard for Binary Floating-Point Arithmetic) enthalten. Näheres dazu wird im Fortran 2003-Kapitelabschnitt Die intrinsischen IEEE-Module⁷⁵ beschrieben.

35.2. Allocatable Components

Diese Erweiterung bezieht sich auf das Schlüsselwort `allocatable`. In Standard-Fortran 90/95 ist die dynamische Allokation von Speicherplatz für Felder mit dem Attribut `allocatable` eigentlich nur in einem lokalen Kontext möglich. Der *Technical Report 15581* fordert, dass solche Felder darüber hinaus auch in den Anwendungsbereichen

- Rückgabewert von Funktionen
- Unterprogrammparameter
- Feldelemente in Datenverbunden

uneingeschränkt verwendbar sein sollen.

Die *Enhanced Data Type Facilities* werden bereits standardmäßig vom aktuellen *gfortran*- und *ifx*-Compiler unterstützt (Fortran 2003 und spätere, Stand: August 2025). Mit dem `-std=f95`-Compiler-Schalter funktionieren folgende Programme mit *gfortran* nicht.

Beispiel: Rückgabewert **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  write ( *, * ) fun_all( 5 )
  write ( *, * ) fun_all( 7 )

! Ausgabe:
!   1 2 3 4 5
!   1 2 3 4 5 6 7

contains
  function fun_all( n )
```

⁷⁵ Kapitel 41.4 auf Seite 266

```

implicit none

integer, dimension( : ), allocatable :: fun_all
integer, intent( in )                :: n
integer                              :: j, st

allocate( fun_all( n ), stat = st )

if( st == 0 ) then
  forall( j = 1 : n )
    fun_all(j) = j
  end forall
else
  write( *, * ) "Allocate-Fehler"
end if
end function fun_all
end program bsp

```

Beispiel: Unterprogrammparameter **Fortran 90/95-Code** (free source form)

```

program bsp
  implicit none

  integer, dimension( : ), allocatable :: dynarr
  integer                              :: j

  allocate( dynarr( 3 ) )

  forall( j = 1 : 3 )
    dynarr( j ) = j * 2
  end forall

  call fun_all( dynarr )

  write( *, * ) "Out: ", dynarr

  deallocate( dynarr )
  allocate( dynarr( 5 ) )

  forall( j = 1 : 5 )
    dynarr( j ) = j * 3
  end forall

  call fun_all( dynarr )

  deallocate( dynarr )

! Ausgabe:
!   Argument: 2 4 6
!   (De)Allocate im UP: 88 99
!   Out: 88 99
!   Argument: 3 6 9 12 15
!   (De)Allocate im UP: 88 99

contains
  subroutine fun_all( a )
    implicit none

    integer, dimension( : ), allocatable, intent( inout ) :: a

    write( *, * ) "Argument:", a

```

```

        deallocate( a )
        allocate( a (2) )
        a(1) = 88
        a(2) = 99

        write( *, * ) "(De)Allocate im UP:", a
    end subroutine fun_all
end program bsp

```

Mit Standard-Fortran 90/95 könnten sie zwar das allozierte Feld als Parameter aus dem Hauptprogramm an das Unterprogramm übergeben, dort wäre es aber nicht als **allocatable** kennzeichenbar und somit im Unterprogramm nicht in der gezeigten Art und Weise (de)allozierbar. Mittels Standardkonformitäts-Compileroptionen, z.B.

- *Intel Fortran Compiler*: **-stand**
- *gfortran*: **-std=f95**

ist überprüfbar, welche Möglichkeiten Standard-Fortran bietet und welche Eigenschaften den Erweiterungen zuschreibbar sind.

Beispiel: *allocatable array* im Datenverbund **Fortran 90/95-Code (free source form)**

```

program bsp
    implicit none

    type struktur
        integer                :: nr
        integer, dimension( : ), allocatable :: arr
    end type struktur

    type( struktur ) :: a1, a2

    allocate( a1%arr( 5 ) )
    allocate( a2%arr( 2 ) )

    a1%nr = 9453
    a1%arr(1) = 1
    a1%arr(5) = 5

    a2%nr = 9454
    a2%arr(1) = 11
    a2%arr(2) = 22

    write ( *, * ) "a1 =", a1%nr, a1%arr
    write ( *, * ) "a2 =", a2%nr, a2%arr

    ! Ausgabe:
    !   a1 = 9453 1 0 0 0 5
    !   a2 = 9454 11 22
end program bsp

```


Teil V.

Fortran 2003

36. Programmaufbau

36.1. Programmaufbau und Zeilenformat

Der grundlegende Programmaufbau und das Zeilenformat von *Fortran 90/95* wurden in *Fortran 2003* beibehalten. Neben der *free source form* ist aus Kompatibilitätsgründen auch noch immer die alte, aus *FORTTRAN 77* bekannte, *fixed source form* gültig.

Eine Zeile darf auch in *Fortran 2003* bei Verwendung der *free source form* standardmäßig maximal 132 Zeichen beinhalten. Ein symbolischer Name darf nun höchstens 63 Zeichen lang sein. Eine Anweisung darf sich maximal über 256 Zeilen erstrecken. Das Zeilenfortsetzungszeichen ist wie in *Fortran 90/95* das Kaufmanns-Und: &. Mit Fortran 2023 wurden diese Beschränkungen deutlich aufgeweicht. Eine Zeile darf nun 10000 Zeichen enthalten. Eine Anweisung darf nun insgesamt aus 1 Mio. Zeichen bestehen.

36.2. Zeichenvorrat

Der *Fortran 2003*-Zeichenvorrat wurde gegenüber *Fortran 90/95* erweitert:

Großbuchstaben:	A bis Z
Kleinbuchstaben:	a bis z
Ziffern:	0 bis 9
FORTTRAN 77-Sonderzeichen:	+ - * / = () : , . ' \$ und Leerzeichen
Fortran 90/95-Sonderzeichen:	_ ! ? " & ; < >
Fortran 2003-Sonderzeichen:	\ [] { } ~ ' ^ # @

FORTTRAN 77

neu bei Fortran 90/95

neu bei Fortran 2003

36.3. Anwendungsgebiet der neu hinzugekommenen Zeichen

Von den in *Fortran 2003* neu zugefügten Zeichen haben nur die eckigen Klammern einen konkreten Anwendungsbereich als Kennzeichnung von Feldkonstruktoren. Ein Feldkonstruktor darf neben der aus *Fortran 95* bekannten Form

(/ *werte* /)

nun auch mit eckigen Klammern geschrieben werden

```
[ werte ]
```

Beispiel: **Fortran 2003-Code**

```
! bsp.f03

program bsp
  implicit none

  integer, dimension( 3 ) :: a = [ 20, 33, 55 ]
  integer, dimension( 2 ) :: b = (/ 44, 55 /)

  write( *, *) a
  write( *, *) b

! Ausgabe:
!   20 33 55
!   44 55
end program bsp
```

Kompilieren, linken:

```
gfortran -o bsp bsp.f03
```

36.4. Der Block

Das Block-Konstrukt kam mit *Fortran 2008* hinzu.

Beispiel: **Fortran 2003-Code**

```
program block
  implicit none
  real :: a
  a = 3.1415

  block
    real :: a
    a = 2.7182
    print *, a
  end block

  print *, a
end program block
```

Ausgabe:

```
2.71819997
3.14150000
```

37. Datentypen

37.1. Intrinsische Datentypen

Die aus Fortran 90/95 bekannten intrinsischen Datentypen sind weiterhin uneingeschränkt gültig.

Datentyp	Kommentar
integer	Ganzzahlen
real	Reelle Zahlen einfacher Genauigkeit
(double precision)	Reelle Zahlen doppelter Genauigkeit
complex	Komplexe Zahlen
logical	Logischer Datentyp (.true., .false.)
character	Zeichen(kette)

Im Umfeld des `character`-Datentyps gab es einige Ergänzungen:

- Unterstützung internationaler Zeichensätze in Rahmen der ISO 10646-Norm (w:Universal Coded Character Set⁷⁶). Im Kapitel `iso_fortran_env`⁷⁷ ist ein kleines Beispiel zu diesem Thema zu finden.
- Die Funktion `selected_char_kind` ist neu. Ihre Funktionsweise ist äquivalent zu den bereits aus Fortran 90/95 bekannten Funktionen `selected_int_kind` und `selected_real_kind`.
- Zeichenkette (String) mit variabler Länge von Zeichen: `character(:)`, `allocatable`

Ein Beispiel mit einem String variabler Länge:

Fortran 2003-Code

```
program bsp
  implicit none

  character(:), allocatable :: str

  str = "Hallo"
  print *, str
  print *, "Laenge = ", len(str), " Zeichen"
  ! Ausgabe:
  ! Hallo
  ! Laenge =          5  Zeichen

  str = "Hallo, Welt"
  print *, str
  print *, "Laenge = ", len(str), " Zeichen"
```

⁷⁶ <https://de.wikipedia.org/wiki/Universal%20Coded%20Character%20Set>

⁷⁷ Kapitel 41.4.1 auf Seite 266

```
! Ausgabe:
! Hallo, Welt
! Laenge =          11  Zeichen
end program bsp
```

Ab Fortran 2008 kann auf den Real- und Imaginärteil von komplexen Zahlen mittels %-Datenverbund-Zeichen zugegriffen werden:

Fortran 2003-Code

```
program bsp
  implicit none

  complex :: img

  img%re = 1.5
  img%im = img%re + 2.5

  print *, img

  ! Ausgabe
  !      (1.50000000,4.00000000)
end program bsp
```

Dies gilt uneingeschränkt für Intels *ifx* (2025.2), dem AOCC-Compiler (*flang*, 2024_09_24) und *gfortran* (14.2.0, 2024).

37.2. Enumeration

Mit Fortran 2003 sind auch in dieser Programmiersprache Enumerationen (Aufzählungstypen) möglich. Die Werte in einer solchen Enumeration besitzen einen **integer**-Datentyp.

```
enum, bind( C ) enumerator :: wert(e) // ... end enum
```

Die von C-Enumerationen bekannten Eigenschaften gelten gleichermaßen für Fortran-Enumerationen, z.B.:

- ohne explizite Zuweisung von Werten wird mit dem Wert 0 gestartet.
- ohne explizite Zuweisung von Werten wird in der Anordnungsreihenfolge der Elemente sukzessiv immer um 1 hochgezählt.
- Wurde dem Vorgängerelement eine Ganzzahl zugewiesen, dem Element jedoch nicht, so ist der Wert dieses Elementes die dem Vorgängerelement zugeordnete Ganzzahl + 1.

Beispiel: **Fortran 2003-Code**

```
program bsp
  implicit none

  integer :: wert

  enum, bind( C )
    enumerator :: MON, DIE, MIT = 3, DON, FRE = 5, SAM = 66, SON = 77
  end enum
```

```

write( *, * ) MON
write( *, * ) MIT
write( *, * ) SAM

wert = 4

if( wert == MIT + 1 ) then
    write( *, * ) "Donnerstag"
endif

! Ausgabe:
!   0
!   3
!  66
!  Donnerstag
end program bsp

```

Obwohl der Fortran 2003-Working Draft J3/04-007 durch das `bind(c)` eine zwingende Anbindung an einen Aufzählungstyp in der Programmiersprache C vorgaukelt, ist dies nicht der Fall. Fortran-Enumerationen funktionierten auch ohne entsprechendes C-Gegenstück. Sie werden im Fortran 2003-Working Draft auch nicht im Abschnitt *Interoperability with C* geführt, sondern direkt im Abschnitt über die Fortran-Datentypen.

37.3. Derived Type

Der *Derived Type* wurde in Fortran 2003 wesentlich erweitert. Er ist nun kein reiner Datenverbund wie noch in Fortran 90/95, sondern eine richtige Klasse. Genauer beschrieben wird dieser Typ später im Kapitel zur OOP, hier zunächst nur einige einfache Erweiterungsmerkmale gegenüber Fortran 90/95.

37.3.1. Benannte Parameter

Beispiel:

Fortran 2003-Code

```

program bsp
    implicit none

    type test
        integer      :: i
        character( 20) :: str
    end type

    type( test ) :: t1 = test ( i = 1, str = "Wiesenfeld" )

    write( *, *) t1

! Ausgabe:
!   1 Wiesenfeld
end program bsp

```

Die Verwendung von benannten Parametern bei der Konstruktion einer *Derived Type*-Variable kann vor allem in Verbindung mit Vorgabewerten sinnvoll sein.

37.3.2. Vorgabewerte

Ab *Fortran 2003* dürfen Variablen in einem *Derived Type* auch mit Vorgabewerten (default values) belegt werden.

Beispiel: **Fortran 2003-Code**

```
program bsp
  implicit none

  type test
    integer      :: i = -1
    character( 20) :: str = "NN"
  end type

  type( test ) :: t1 = test ()
  type( test ) :: t2 = test ( str = "Wiesenfeld" )
  type( test ) :: t3 = test ( str = "Walddorf", i = 1 )

  write( *, *) t1
  write( *, *) t2
  write( *, *) t3

  ! Ausgabe:
  !   -1 NN
  !   -1 Wiesenfeld
  !    1 Walddorf
end program bsp
```

37.4. Felder

Ein mehrdimensionales Feld kann ab Fortran 2008 maximal 15-dimensional gestaltet werden.

Ein benanntes konstantes Feld kann wie folgt initialisiert werden. Die obere Feldgrenze wird durch den Stern (*) ersetzt. Durch den *array constructor* wird die Feldgröße automatisch festgelegt:

Fortran 2003-Code

```
program bsp
  implicit none

  integer, parameter, dimension(1:*) :: arr1 = [1, 2, 3, 4, 5]
  integer, parameter                :: arr2(0:*) = [0, 1, 2]

  print *, arr1
  print *, arr1(2)

  print *, arr2
  print *, arr2(2)

  ! Ausgabe:
  !      1      2      3      4      5
  !      2
  !      0      1      2
  !      2
end program bsp
```

38. Zeiger

38.1. Prozedurenzeiger

38.1.1. Einführung

Mit dem 2003er-Standard unterstützt auch Fortran Zeiger auf Prozeduren (procedure pointer, Funktionszeiger). Diese spielen auch eine wichtige interne Rolle bei der Realisierung objektorientierter Mechanismen und beim C-Binding. Schematisch wird ein Prozedurenzeiger so deklariert:

```
procedure( [name] ), pointer :: zeigername
```

Die Angabe von `name` ist optional. Wird an dieser Stelle ein Name, z.B. ein Unterprogrammbezeichner angegeben, so bedeutet dies, dass der Prozedurenzeiger mit allen Unterprogrammen, die das gleiche Interface aufweisen, kompatibel ist. Prozedurenzeiger können wie normale Zeiger gehandhabt werden.

Beispiel: **Fortran 2003-Code**

```
program bsp
  implicit none

  procedure(up), pointer :: pptr => null()

  pptr => oops

  call pptr

! Ausgabe:
!   oops

  contains
    subroutine oops()
      write( *, * ) "oops"
    end subroutine oops
    subroutine up()
    end subroutine up
end program bsp
```

38.1.2. Prozedurenzeiger mit implizitem Interface

Bei der Deklaration eines Prozedurenzeigers muss keine explizite Schnittstelle angegeben werden. Im Folgenden wird dies am Beispiel eines mit `pptr2` benannten Prozedurenzeigers demonstriert.

Beispiel: **Fortran 2003-Code**


```
program bsp
  implicit none

  procedure( up ) , pointer :: pptr1 => null()
  procedure( )   , pointer :: pptr2 => null()
  procedure( add ), pointer :: pptr3 => null()

  pptr1 => oops
  pptr2 => oops
  call pptr1
  call pptr2

  ! Ausgabe:
  !   oops
  !   oops

  pptr3 => add
  write( *, * ) pptr3( 5 , 12 )

  ! Ausgabe:
  !   17

  ! Folgende auskommentierte Zuordnung waere nicht erlaubt:
  ! pptr1 => add

  contains
    subroutine up()
    end subroutine up
    subroutine oops()
      write( *, * ) "oops"
    end subroutine oops

    function add( a, b )
      integer          :: add
      integer, intent( in ) :: a, b

      add = a + b
    end function add
end program bsp
```

38.1.3. Abstraktes Interface

Das bei der Deklaration eines Prozedurenzeigers als Schnittstelle genannte Unterprogramm muss nicht real implementiert sein. Es können statt dessen auch abstrakte Interfaces Verwendung finden. Diese sind gleich wie konventionelle Interface-Blöcke aufgebaut, mit dem Unterschied, dass sie als **abstract interface** gekennzeichnet sind. Ein mit einem abstrakten Interface deklarerter Prozedurzeiger passt dann für jedes Unterprogramm, welches mit identer Schnittstelle ausgestattet ist.

Fortran 2003-Code

```
program bsp
  implicit none

  abstract interface
    function afunc( x, y )
      integer          :: afunc
```

```

        integer, intent( in ) :: x, y
    end function afunc
end interface

procedure( afunc ), pointer :: pptr1 => null()
procedure( add ) , pointer :: pptr2 => null()

pptr1 => add
write( *, * ) pptr1( 5 , 12 )
! Ausgabe: 17

pptr1 => mult
write( *, * ) pptr1( 3 , 2 )
! Ausgabe: 6

! Folgendes funktioniert uebrigens auch, da add() und mult() das gleiche
Interface
! aufweisen:
pptr2 => mult
write( *, * ) pptr2( 5 , 5 )
! Ausgabe: 25

contains
    function add( a, b )
        integer                :: add
        integer, intent( in ) :: a, b

        add = a + b
    end function add

    function mult( a, b )
        integer                :: mult
        integer, intent( in ) :: a, b

        mult = a * b
    end function mult
end program bsp

```

38.2. Zeiger und das intent-Attribut

Nun ist bei der Übergabe von Zeigern an Unterprogramme auch die Angabe eines `intent`-Attributs möglich. Das war mit *Fortran 90/95* noch nicht erlaubt. Diese `intent`-Angaben beziehen sich aber nicht auf die Variablenwerte an sich, sondern beschränken nur die Möglichkeiten zur Zeigerzuordnung im Unterprogramm selbst.

Beispiel: **Fortran 2003-Code**

```

program bsp
    implicit none

    integer, target :: x = 15
    integer, pointer :: ptr1 => null(), ptr2 => null()

    ptr1 => x
    ptr2 => x

    call mult( ptr1, ptr2)

```

```

write( *, *) "Zuordnungsstatus ptr1:", associated( ptr1 )
write( *, *) "Zuordnungsstatus ptr2:", associated( ptr2 )
write( *, *) "Wert ptr1:", ptr1
write( *, *) "Wert x:", x

! Ausgabe:
!   Zuordnungsstatus ptr1: T
!   Zuordnungsstatus ptr2: F
!   Wert ptr1: 45
!   Wert x: 45

contains
  subroutine mult( a, b )
    integer, pointer, intent( in )    :: a
    integer, pointer, intent( inout ) :: b
    integer, target :: val = 3

! Folgendes waere nun nicht erlaubt, da a nur intent( in )
!   a => null()

! Das auch nicht:
!   a => val

! Das allerdings ist erlaubt:
    a = a * val

! b ist mit intent( inout ) spezifiziert, also ist hier eine Zeigerzuordnung
! erlaubt:
    b => null()

  end subroutine mult
end program bsp

```

38.3. Zeiger und Felder

Auch im Zusammenspiel von Zeigern mit Feldern bringt der *Fortran 2003*-Standard einige Ergänzungen.

39. Ein- und Ausgabe

39.1. Streams

Fortran 2003 bietet zusätzlich zum altbekannten datensatzbasierten I/O nun auch Datei-eingabe und -ausgabe in Form von Streams, wie das z.B. in der Programmiersprache C seit jeher üblich ist.

39.1.1. Unterschied zum alten I/O-Konzept?

- Streams ermöglichen das Lesen und Schreiben von Binärdateien, ohne sich mit den auf Datensätzen aufbauenden Strukturen der konventionellen Fortran-I/O herumschlagen zu müssen.
- Bei der Ein-/Ausgabe mit Streams wird die Datei als kontinuierliche Byte-Sequenz betrachtet.
- Stream-I/O ist logischerweise nicht für interne Dateien gedacht.

39.1.2. Anwendung

Auch wenn Stream-I/O in *Fortran 2003* ein neues Konzept ist, so sind die altbekannten `open`-, `read`-, `write`- und `close`-Befehle dafür zuständig.

- Öffnen eines Streams:

```
open( ..., access = "STREAM", ... )
```

- Lesen und Schreiben:

```
read( ... ) ... write( ... ) ...
```

- Schließen eines Streams:

```
close( ... )
```

39.1.3. Unformatierte Stream-I/O

Beispiel: **Fortran 2003-Code**

```
program bsp
  implicit none

  real          :: a = 55.678
  real          :: b
```

```

character( len = 3 ) :: str

open( 50, file = "test", access = "STREAM", status = "REPLACE")

write( 50 ) "Hallo Welt"
write( 50 ) "Hello World"
! Ausgabe in Datei: siehe Bild 1

write( 50, pos = 100 ) "Greetings"
! Ausgabe in Datei: siehe Bild 2

write( 50, pos = 60 ) a
! Ausgabe in Datei: siehe Bild 3

read( 50, pos = 60 ) b
write( *, * ) b
! Ausgabe:
!   55.678

read( 50, pos = 8 ) str
write( *, * ) str
! Ausgabe:
!   elt

close( 50 )
end program bsp

```

Da es sich um eine unformatierte Ein-/Ausgabe handelt, darf natürlich kein Formatspezifizierer bei den `read`- und `write`-Anweisungen angegeben werden, auch kein `*`. Mittels `pos`-Spezifizierer kann an eine bestimmte Position in der Datei gesprungen werden.

Bild 1	0000:0000 48 61 6c 6c 6f 20 57 65 6c 74 48 65 6c 6c 6f 20	Hallo WeltHello
	0000:0010 57 6f 72 6c 64	World
Abb. 44		
Bild 2	0000:0000 48 61 6c 6c 6f 20 57 65 6c 74 48 65 6c 6c 6f 20	Hallo WeltHello
	0000:0010 57 6f 72 6c 64 00 00 00 00 00 00 00 00 00 00	World.....
	0000:0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0000:0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0000:0040 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0000:0050 00 00 00 00 00 00 00 00 00 00 00 00 00 00	Greetings
	0000:0060 00 00 00 47 72 65 65 74 69 6e 67 73	...Greetings
Abb. 45		
Bild 3	0000:0000 48 61 6c 6c 6f 20 57 65 6c 74 48 65 6c 6c 6f 20	Hallo WeltHello
	0000:0010 57 6f 72 6c 64 00 00 00 00 00 00 00 00 00 00	World.....
	0000:0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0000:0030 00 00 00 00 00 00 00 00 00 00 46 b6 5e 42 00	F [^] B.
	0000:0040 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0000:0050 00 00 00 00 00 00 00 00 00 00 00 00 00 00	Greetings
	0000:0060 00 00 00 47 72 65 65 74 69 6e 67 73	...Greetings
Abb. 46		

39.1.4. Formatierte Stream-I/O

Beispiel: **Fortran 2003-Code**

```

program bsp

```

```

implicit none

real                :: a = 55.678
real                :: b = 13.9876
character( len = 20 ) :: str1, str2, str3
integer             :: fposition

open( 50, file = "test", access = "STREAM", form = "FORMATTED", status =
"REPLACE")

write( 50, "(2A20)" ) "Hallo Welt", "Hello World"

inquire( 50, pos = fposition )
write( *, * ) fposition

write( 50, * ) a, new_line( "x" ), b, " abcdef"

read( 50, *, pos = fposition ) a
write( *, * ) a

read( 50, *, pos = 1 ) str1, str2, str3
write( *, * ) str1, str2, str3

read( 50, * ) str1, a
write( *, * ) str1, a

! Ausgabe in Datei:
!      Hallo Welt      Hello World
!    55.678
!   13.9876  abcdef

! Ausgabe auf Bildschirm:
!    42
!   55.678
!   Hallo      Welt      Hello
!   55.678      13.9876

close( 50 )
end program bsp

```

39.2. Asynchrone I/O

Eine asynchrone I/O ermöglicht es einem Programm, Lese- oder Schreiboperationen durchzuführen und während dessen andere Anweisungen auszuführen, ohne auf den Abschluss der Ein-/Ausgabe zu warten.

Beispiel: **Fortran 2003-Code**

```

program bsp
  implicit none

  integer :: X(500)

  X = 10

  print *, "Foo1"

  open(1, asynchronous='yes', file="tmp.tmp")
  write(1, asynchronous='yes', fmt="(i9)") X

```

```
print *, "Foo2"

wait(1)

close(1)
end program bsp
```

39.3. Rekursive I/O

Innerhalb eines rekursiv aufgerufenen Unterprogramms kann eine Lese- oder Schreiboperation durchgeführt wird. Dies kann teilweise problematisch sein.

Beispiel: **Fortran 2003-Code**

```
recursive subroutine baum(n, z)
  integer, intent(in) :: n, z
  integer :: i

  if (n <= 0) return

  do i = 1, z
    write(*,'(A)', advance="no") "  "
  end do
  print *, n

  call baum(n - 1, z + 1)
end subroutine baum

program bsp
  call baum(3, 0)
end program bsp
```

Ausgabe:

```
      3
     2
    1
```

39.4. Sonstiges

- flush
- iomsg
- round
- decimal
- sign
- ...

Siehe vorerst z.B. The New Features of Fortran 2003 (John Reid) (PDF)⁷⁸

⁷⁸ <https://wg5-fortran.org/N1601-N1650/N1648.pdf>

40. Intrinsische Funktionen und Subroutinen

Die *Fortran 90/95*-Funktionen und -Subroutinen sind natürlich auch in *Fortran 2003* uneingeschränkt gültig. Einige Unterprogramme wurden neu aufgenommen, andere in ihrer Funktionalität etwas erweitert. Mit Fortran 2008 wurde eine Reihe von Funktionen neu aufgenommen (z.B. für die Bit-Verarbeitung, Bessel-, Gamma- und Fehlerfunktion, euklidische Vektornormen). Neu in Fortran 2023 sind u.a. etliche Winkel- und Arkusfunktionen, sowie String-Unterprogramme.

40.1. Neu

40.1.1. Datentypfunktionen

Funktion	Beschreibung
<code>i = selected_char_kind (c)</code>	Gibt den kind-Wert des Parameters zurück. Der Rückgabewert ist von Datentyp <code>integer</code> . Parameter: • <code>name</code> Rückgabewert: • <code>name = "DEFAULT"</code>: der Default-Wert für Zeichen wird zurückgegeben • <code>name = "ASCII"</code>: der Wert für ASCII-Zeichen wird zurückgegeben • <code>name = "ISO_10646"</code>: der Wert für "ISO 10646"-Zeichen wird zurückgegeben • <code>-1 ...</code> Zeichentyp wird nicht unterstützt Beispiel: <code>i = selected_char_kind("ASCII") i => 1</code>

40.1.2. Kommandozeile und Environment

Funktion	Beschreibung
<code>i = command_argument_count ()</code>	Anzahl der übergebenen Kommandozeilenargumente (der Programmname selbst wird nicht mitgezählt). Der Rückgabewert ist vom Typ <code>integer</code> . Beispiel: Programmaufruf mit: <code>./a.out opt1 opt2 i = command_argument_count() i => 2</code>

Subroutine	Beschreibung
------------	--------------

get_command ([c, i, i])	Übergebene Kommandozeilenargumente (ohne Programmname)Parameter: 1. command: Kommandozeilenargumente als Zeichenkette, intent(out), optional 2. length: Länge der Zeichenkette, intent(out), optional 3. status: Status, intent(out), optional • 0 ... OK • -1 ... command-Argument existiert und ist kürzer als length • andere Zahl ... Fehler Beispiel: Programmaufruf mit: ./a.out opt1 opt2 call get_command(str, len, st) str => opt1 opt2 len => 9 st => 0
get_command_argument (i, [c, i, i])	Ein bestimmtes Kommandozeilenargument (inkl. Programmname)Parameter: 1. number: Nummer des gewünschten Kommandozeilenargumentes beginnend bei 0, intent(in) 2. value: Wert des Kommandozeilenargumentes, intent(out), optional 3. length: Länge der Zeichenkette, intent(out), optional 4. status: Status, intent(out), optional • 0 ... OK • -1 ... Argument existiert und ist kürzer als length • andere Zahl ... Fehler Beispiel: Programmaufruf mit: ./a.out opt1 opt2 call get_command_argument(1, str, len, st) str => opt1 len => 4 st => 0

<code>get_environment_variable (c1, [c2, i, i, l])</code>	Wert einer bestimmten Umgebungsvariable. Parameter: 1. name: Name der Umgebungsvariable, <code>intent(in)</code> 2. value: Wert der Umgebungsvariablen, <code>intent(out)</code>, optional 3. length: Länge der Zeichenkette c2, <code>intent(out)</code>, optional 4. status: Status, <code>intent(out)</code>, optional 5. trim_name: <code>intent(in)</code>, optional • 0 ... OK • -1 ... Umgebungsvariable existiert, Wert ist kürzer als length • 1 ... Umgebungsvariable existiert nicht • 2 ... keine Unterstützung von Umgebungsvariablen • andere Zahl ... sonstiger Fehler Beispiel: <code>call get_environment_variable("PWD", str, len, st, .TRUE.)</code> <code>str => /usr/bin</code> <code>len => 8</code> <code>st => 0</code>
---	---

40.1.3. Mathematische Funktionen

Trigonometrische Funktionen

Funktion	Beschreibung
<code>r = acosd (r)</code>	Ab Fortran 2023. Berechnet den Arkus-Cosinus. Gibt den berechneten Wert in Grad (und nicht in Radian wie bei <code>acos</code>) zurück.
<code>r = asind (r)</code>	Ab Fortran 2023. Berechnet den Arkus-Sinus. Gibt den berechneten Wert in Grad (und nicht in Radian wie bei <code>asin</code>) zurück.
<code>r = atand (r)</code>	Ab Fortran 2023. Berechnet den Arkus-Tangens. Gibt den berechneten Wert in Grad (und nicht in Radian wie bei <code>atan</code>) zurück.
<code>r = atan2d (r1, r2)</code>	Ab Fortran 2023. Berechnet den Arkus-Tangens. Gibt den berechneten Wert in Grad (und nicht in Radian wie bei <code>atan</code>) zurück.
<code>r = atand (r1, r2)</code>	Ab Fortran 2023. Berechnet den Arkus-Tangens. Gleich wie <code>atan2d(r1, r2)</code> .

$r = \text{cosd} (r)$	Ab Fortran 2023. Berechnet den Cosinus. Das Argument ist in der Einheit Grad anzugeben (und nicht in Radiant wie bei cos).
$r = \text{sind} (r)$	Ab Fortran 2023. Berechnet den Sinus. Das Argument ist in der Einheit Grad anzugeben (und nicht in Radiant wie bei sin).
$r = \text{tand} (r)$	Ab Fortran 2023. Berechnet den Tangens. Das Argument ist in der Einheit Grad anzugeben (und nicht in Radiant wie bei tan).

Fehler- und Gamma-Funktion

Funktion	Beschreibung
$r = \text{erf} (r)$	Ab Fortran 2008. Berechnet die Fehler-Funktion $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$
$r = \text{erfc} (r)$	Ab Fortran 2008. Berechnet $1 - \text{erf}(x)$
$r = \text{erfc_scaled} (r)$	Ab Fortran 2008. Berechnet $e^{x^2}(1 - \text{erf}(x))$
$r = \text{gamma} (r)$	Ab Fortran 2008. Berechnet die Gamma-Funktion $\Gamma(x) = \int_{0+}^{\infty} e^{-t} t^{x-1} dt$ für $x > 0$
$r = \text{log_gamma} (r)$	Ab Fortran 2008. Berechnet den natürlichen Logarithmus der Gamma-Funktion

Besselsche Funktion

Die Bessel-Funktion J_ν erster Art ν -ter Ordnung ist definiert als (Details siehe w:Bessel-Funktion⁷⁹)

$$J_\nu(x) = \sum_{r=0}^{\infty} \frac{(-1)^r \left(\frac{x}{2}\right)^{2r+\nu}}{\Gamma(\nu+r+1)r!}$$

Funktion	Beschreibung
$r = \text{bessel_j0} (r)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 1. Art und 0. Ordnung

⁷⁹ <https://de.wikipedia.org/wiki/Bessel-Funktion>

$r = \text{bessel_j1} (r)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 1. Art und 1. Ordnung
$r = \text{bessel_jn} (i, r)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 1. Art und i . Ordnung ($i \in \mathbb{N}_0$)

Die Bessel-Funktion Y_n zweiter Art n -ter Ordnung ist definiert als

$$Y_n(x) = \lim_{\nu \rightarrow n} \frac{J_\nu(x) \cos(\nu\pi) - J_{-\nu}(x)}{\sin(\nu\pi)}.$$

Funktion	Beschreibung
$r = \text{bessel_y0} (r1)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 2. Art und 0. Ordnung ($r1 \in \mathbb{R}^+$)
$r = \text{bessel_y1} (r1)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 2. Art und 1. Ordnung ($r1 \in \mathbb{R}^+$)
$r = \text{bessel_yn} (i, r1)$	Ab Fortran 2008. Berechnet die Bessel-Funktion 2. Art und i . Ordnung ($r1 \in \mathbb{R}^+$, $i \in \mathbb{N}_0$)

Euklidische Vektornormen

Funktion	Beschreibung
$r = \text{norm2} (ra)$	Ab Fortran 2008. Gibt die euklidische Vektornorm ($r = \sqrt{ra_1^2 + ra_2^2 + \dots + ra_n^2}$) zurück. ra sei ein Array mit reellen Elementen. Bsp.: <code>program main implicit none print *, norm2([2., 3., 1.])</code> end program main Ausgabe: 3.741657
$r = \text{hypot} (r1, r2)$	Ab Fortran 2008. Gibt die euklidische Vektornorm ($r = \sqrt{r1^2 + r2^2}$) zurück. $r1$ und $r2$ seien reelle Zahlen. <code>hypot</code> steht vermutlich als Abkürzung für "Hypotenuse"

40.1.4. Bit-Verarbeitung

Funktion	Beschreibung
----------	--------------

<code>l = bge (i1, i2)</code>	Ab Fortran 2008. Gibt den Wert true zurück, wenn i1 bitweise größer oder gleich i2 ist, z.B. <code>program main implicit none print *, bge(B'101110110001', B'11001') print *, bge(B'101', B'11001') print *, bge(10, -1) end program main</code> Ausgabe: T F F Warum ist -1 bitweise größer als 10? Antwort: -1 entspricht binär B'11111111111111111111111111111111' und 10 entspricht binär B'1010' (siehe auch w:Zweierkomplement ⁸⁰)
<code>l = bgt (i1, i2)</code>	Ab Fortran 2008. Gibt den Wert true zurück, wenn i1 bitweise größer i2 ist
<code>l = ble (i1, i2)</code>	Ab Fortran 2008. Gibt den Wert true zurück, wenn i1 bitweise kleiner oder gleich i2 ist
<code>l = blt (i1, i2)</code>	Ab Fortran 2008. Gibt den Wert true zurück, wenn i1 bitweise kleiner i2 ist
<code>i = leadz (i1)</code>	Ab Fortran 2008. Gibt die Anzahl der führenden Null-Bits in i1 zurück. Bsp.: <code>program main implicit none print *, leadz(20) end program main</code> Ausgabe: 27 Die Dezimalzahl 20 entspricht der binären Zahl B'10100'. D.h. bei einer 32-bit-Zahl sind 27 führende Null-Bits vorhanden ($27 + 5 = 32$).
<code>i = popcnt (i1)</code>	Ab Fortran 2008. Gibt die Anzahl der 1-Bits in i1 zurück . Bsp.: <code>program main implicit none print *, popcnt(25) end program main</code> Ausgabe: 3

40.1.5. Zeichenketten-Verarbeitung

Folgende Subroutinen wurden mit Fortran 2023 neu eingeführt:

- `split`
- `tokenize`

40.2. Erweitert

- `system_clock(i1, ir, i2)` ... Das zweite Argument (`count_rate`) darf nun vom Datentyp `integer` oder `real` sein.
- `max`, `maxloc`, `maxval`, `min`, `minloc`, `minval` ... Funktionieren nunmehr auch für Werte vom Datentyp `character`.

⁸⁰ <https://de.wikipedia.org/wiki/Zweierkomplement>

- `atan2(r1, r2)`, `log(rx)`, `sqrt(rx)` ... Unterscheidung von positiven und negativen Nullen im Argument.

41. Intrinsische Module

41.1. Grundlegendes

Module gab es bereits mit *Fortran 90/95*. Neu in *Fortran 2003* sind die sogenannten *”intrinsischen Module”*. Das sind jene Module, die bereits standardmäßig von Fortran-2003-Compilern bereitgestellt werden. Werden Datenelemente oder Funktionen aus solchen *intrinsischen Modulen* benötigt, so ist das entsprechende Modul mittels

```
use, intrinsic :: modulname
```

in die jeweilige Programmeinheit einzubinden.

Der Unterschied zu konventionellen (nonintrinsischen) Modulen ist das Wörtchen *intrinsic*, das dem Compiler mitteilt, dass er das Modul bereits mitbringt und nicht irgendwo extern danach suchen soll. Wird nach dem *use*-Schlüsselwort kein entsprechendes Attribut oder das *non_intrinsic*-Attribut angegeben, so zeigt dies an, dass ein nonintrinsisches Modul benutzt wird.

41.2. Das intrinsische Modul *iso_fortran_env*

Das *iso_fortran_env*-Modul enthält einige Fortran-umgebungsspezifische Konstanten.

Beispiel:

Fortran 2003-Code

```
program bsp
  use, intrinsic :: iso_fortran_env
  implicit none

  write( *, * ) INPUT_UNIT
  write( *, * ) OUTPUT_UNIT
  write( *, * ) ERROR_UNIT
  write( *, * ) IOSTAT_END
  write( *, * ) IOSTAT_EOR
  write( *, * ) NUMERIC_STORAGE_SIZE
  write( *, * ) CHARACTER_STORAGE_SIZE
  write( *, * ) FILE_STORAGE_SIZE

  ! Ausgabe, z.B.:
  !   5
  !   6
  !   0
  !  -1
  !  -2
  !  32
  !   8
```



```
! 8
end program bsp
```

Erläuterung:

Konstante	Anmerkung
INPUT_UNIT	Standard-Eingabeeinheit (entspricht <code>unit=*</code> bei <code>read</code>)
OUTPUT_UNIT	Standard-Ausgabeeinheit (entspricht <code>unit=*</code> bei <code>write</code>)
ERROR_UNIT	Standard-Fehlerausgabeeinheit
IOSTAT_END	end-of-file (EOF)
IOSTAT_EOR	end-of-record (EOR)
NUMERIC_STORAGE_SIZE	Speicherplatzbedarf (in bits)
CHARACTER_STORAGE_SIZE	Speicherplatzbedarf (in bits)
FILE_STORAGE_SIZE	Speicherplatzbedarf (in bits)

All diese Konstanten sind Skalare vom Datentyp `integer`.

Des Weiteren können mittels `INTEGER_KINDS`, `REAL_KINDS`, `LOGICAL_KINDS` und `CHARACTER_KINDS` die auf dem jeweiligen System verfügbaren `kind`-Parameter ermittelt werden.

Beispiel: **Fortran 2003-Code**

```
program bsp
  use, intrinsic :: iso_fortran_env
  implicit none

  print *, "integer_kinds = ", INTEGER_KINDS
  print *, "real_kinds = ", REAL_KINDS
  print *, "logical_kinds = ", LOGICAL_KINDS
  print *, "character_kinds = ", CHARACTER_KINDS

  ! Ausgabe, z.B.:
  ! integer_kinds =          1          2          4          8
  ! real_kinds =           4          8         10        16
  ! logical_kinds =         1          2          4          8
  ! character_kinds =       1          4

end program bsp
```

Beispiel (Ausgabe eines Unicode-Zeichens, siehe auch Programming in Modern Fortran - Unicode⁸¹): **Fortran 2003-Code**

```
program bsp
  use, intrinsic :: iso_fortran_env, only: output_unit
  implicit none
```

81 <https://cyber.dabamos.de/programming/modernfortran/unicode.html>

```

integer, parameter :: iso_10646 = selected_char_kind('ISO_10646')
character(kind=iso_10646) :: c

! Das €-Zeichen
c = iso_10646_'\u20AC'

open (output_unit, encoding='UTF-8')
print *, c

! Ausgabe:
!  €
end program bsp

```

Compiler-Aufruf:

```
gfortran.exe bsp.f08 -fbackslash
```

Die Compiler-Option `-fbackslash` übersetzt u.a. `\xnn`, `\unnnn` und `\Unnnnnnnnn` (n sind hexadezimale Ziffern) in Unicode-Zeichen. Steht diese Option in einem Compiler nicht zur Verfügung, so gibt es auch noch andere Möglichkeiten Unicode-Zeichen zu verarbeiten. Details dazu siehe unter obenstehenden Link.

Ab Fortran 2008 können auch Compiler-Eigenschaften abgefragt werden:

Fortran 2003-Code

```

program comp
  use, intrinsic :: iso_fortran_env
  implicit none

  print *, compiler_version()
  print *, compiler_options()
end program comp

```

Ausgabe (Programm kompiliert mit gfortran), z.B.:

```

GCC version 7.5.0
-mtune=generic -march=x86-64

```

41.3. Das intrinsische Modul `iso_c_binding`

Das `iso_c_binding`-Modul liefert die Konstanten und Unterprogramme, die für die Einbindung von C-Bibliotheken in Fortran-Programme erforderlich sind. Näheres dazu findet sich im Kapitel Fortran 2003 und C⁸².

41.4. Die intrinsischen IEEE-Module

Die bereits aus dem dem TR 15580 : 1998 (floating-point exception handling) bekannten Module

- `ieee_exceptions`
- `ieee_arithmetic`
- `ieee_features`

wurden in *Fortran 2003* in Form von intrinsischen Modulen aufgenommen. Diese Module decken den IEEE 754-1985-Standard (auch IEC 559:1989) ab.

41.4.1. Wovon handelt der IEEE 754-1985-Standard?

Im Bereich der Gleitkommazahlen (`real`, `float`, ...) herrschte bis in die 1980er-Jahre Anarchie. Es gab keine verbindlichen Regeln wie Gleitkommazahlen repräsentiert werden, wie gerundet wird, wie Under- und Overflows gehandhabt werden, wie mit Unendlich und NaN verfahren wird, etc. Das führte dazu, dass das gleiche Computerprogramm auf unterschiedlichen Rechnerarchitekturen und mit verschiedenen Compilern unterschiedliche Resultate liefern konnte. Um diesem Manko zu begegnen wurde in den frühen 1980er-Jahren eine Standardisierung angestrebt. Resultat war die Verabschiedung des IEEE 754-1985-Standards (IEEE Standard for Binary Floating-Point Arithmetic for microprocessor systems).

Dieser Standard regelt im Wesentlichen

- die Repräsentation von Gleitkommazahlen:

Darstellung: $zahl = (-1)^s \cdot m \cdot b^e$

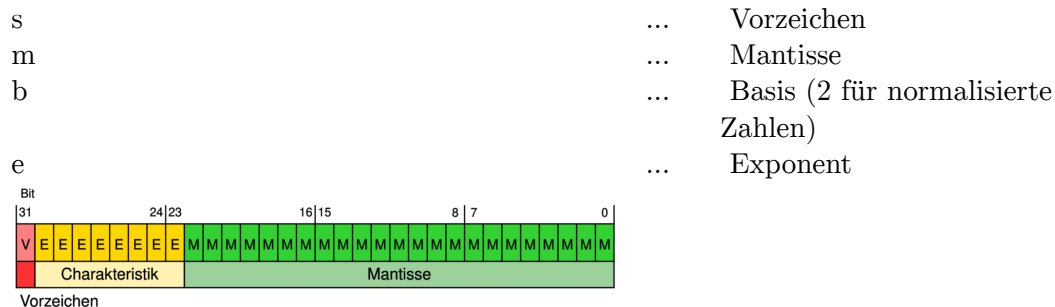


Abb. 47

Zahlenformate:

single	...	4 Bytes
double	...	8 Bytes
double-extended	...	≥10 Bytes, optional

- die Darstellung normalisierter und denormalisierter Zahlen:
 - Normalisierte Zahlen: w:Gleitkommazahlen#Normalisierung⁸³

⁸³ <https://de.wikipedia.org/wiki/Gleitkommazahlen%23Normalisierung>

- Denormalisierte Zahlen: Bereich zwischen der kleinsten darstellbaren normalisierten Zahl und Null (Exponent hat einen reservierten Wert, führende Bit der Mantisse ist 0)
- NaN (Not a Number), $\pm\infty$, ± 0
- Rundungen (zur nächstgelegenen darstellbaren Zahl, in Richtung Null, in Richtung $+\infty$ oder in Richtung $-\infty$)
- das Verhalten verschiedener Operationen (Grundrechenarten, Wurzelberechnung, Konvertierung Gleitkommazahl $\rightarrow$ Ganzzahl, Binär-Dezimal-Konvertierung, Vergleiche mit Nan und ∞ , etc.)
- Exception-Handling (Overflow, Underflow, Division by Zero, Inexact, Invalid)

Weiterführende Weblinks:

- w:IEEE 754⁸⁴
- Kahan, W.: Why do we need a floating-point arithmetic standard?, UC Berkeley, 1981, <http://http.cs.berkeley.edu/~wkahan/ieee754status/why-ieee.pdf>
- Kahan, W.: Lecture Notes on the Status of IEEE Standard 754 for Binary Floating-Point Arithmetic, UC Berkeley, 1996, <http://http.cs.berkeley.edu/~wkahan/ieee754status/IEEE754.PDF>

41.4.2. Implementierung in Fortran 2003

Wie bereits erwähnt, besitzt Fortran 2003 intrinsische Module, mit denen der Zugriff auf bestimmte IEEE-Eigenschaften erfolgen kann. Dazu stehen eine Reihe von Funktionen, Subroutinen, Verbundtypen und Konstanten zur Verfügung. Merkmal ist, dass diese immer mit dem Präfix `ieee` beginnen.

`ieee_arithmetic`

Das aus Programmiersicht umfangreichste Modul ist sicherlich `ieee_arithmetic`. Dieses enthält zahlreiche Funktionen, Subroutinen und Konstanten.

Abfragefunktionen für die Unterstützung bestimmter IEEE-Elemente, z.B.:

<code>l = ieee_support_datatype([x])</code>	Prüft ob die IEEE-Arithmetik für einen speziellen <code>real</code> -Datentyps, charakterisiert durch <code>x</code> (Zahl oder Feld), unterstützt wird. Wird kein Argument angegeben, so wird geprüft, ob die IEEE-Arithmetik für alle <code>real</code> -Datentypen unterstützt wird.
<code>l = ieee_support_nan([x])</code>	Prüft, ob NaN-Werte unterstützt werden

⁸⁴ <https://de.wikipedia.org/wiki/IEEE%20754>

<code>l = ieee_support_rounding(round_value, [x])</code>	Prüft, ob ein bestimmter IEEE-Rundungsmodus unterstützt wird. Mögliche Rundungsmodi sind: • <code>ieee_nearest</code> • <code>ieee_to_zero</code> • <code>ieee_up</code> • <code>ieee_down</code> • <code>ieee_other</code>
--	---

Elementare Funktionen, z.B.:

<code>l = ieee_is_finite(x)</code>	Prüft, ob der Wert <code>x</code> endlich ist
<code>r = ieee_next_after(x, y)</code>	Liefert die nächste darstellbare Zahl von <code>x</code> in Richtung <code>y</code>
<code>r = ieee_rint(x)</code>	Rundet gemäß eingestelltem Rundungsmodus zu einer Ganzzahl und liefert diese Zahl mit dem Datentyp von <code>x</code> zurück.

Die Kind-Funktion:

<code>r = ieee_selected_real_kind([p, r])</code>	Liefert einen kind-Wert
--	-------------------------

Nichtelementare Subroutinen, z.B.:

<code>ieee_get_underflow_mode(gradual)</code>	Liefert den aktuellen Underflow-Modus
<code>ieee_set_rounding_mode(round_value)</code>	Setzt den IEEE-Rundungsmodus, mögliche Werte für <code>round_value</code> • <code>ieee_nearest</code> • <code>ieee_to_zero</code> • <code>ieee_up</code> • <code>ieee_down</code> • <code>ieee_other</code>

ieee_exceptions

Das `ieee_exceptions`-Modul enthält zwei Funktionen, mit denen abgefragt werden kann, welche Exceptions unterstützt werden bzw. inwieweit IEEE-Halting unterstützt wird:

<pre>l = ieee_support_flag(flag, [x]) l = ieee_support_halting(flag)</pre>
--

Mögliche Flags sind

- `ieee_invalid`
- `ieee_overflow`
- `ieee_divide_by_zero`
- `ieee_underflow`
- `ieee_inexact`

Des Weiteren sind in diesem Modul einige Subroutinen zum Setzen bzw. Abfragen diverser Flags enthalten:

```
ieee_get_status( status_value )
ieee_set_flag( flag, flag_value )
ieee_set_halting_mode( flag, halting )
ieee_set_status( status_value )
```

Bei Einbindung des `ieee_arithmetic`-Moduls ist auch automatisch Zugriff auf die `public`-Elemente des Moduls `ieee_exceptions` gegeben.

`ieee_features`

Das `ieee_features`-Modul liefert einige benannte Konstanten, z.B. `ieee_datatype`, `ieee_inf`, `ieee_sqrt`.

Beispiel: Rundungsmodus

IEEE-Subroutinen:

```
ieee_get_rounding_mode( val )
ieee_set_rounding_mode( flag )
```

Mögliche Wert für `flag` sind:

- | | | |
|-----------------------------|-----|---|
| • <code>ieee_nearest</code> | ... | default, Rundung zur nächstgelegenen Zahl (wenn das nicht eindeutig möglich ist, dann Rundung zur nächstgelegenen geraden Zahl) |
| | ... | Rundung in Richtung 0 |
| • <code>ieee_to_zero</code> | | |
| | ... | Rundung Richtung $-\infty$ |
| • <code>ieee_down</code> | | |
| | ... | Rundung Richtung $+\infty$ |
| • <code>ieee_up</code> | | |

Fortran 2003-Code

```
program bsp
```

```

use, intrinsic :: ieee_arithmetic

implicit none

real, dimension(6)    :: a = (/ -1.5, -0.5, 0.5, 1.5, 2.5, 3.5 /)

! Standard-Fortran-Rundungsfunktion
write( *, * ) anint( a )

! IEEE-Rundungsfunktion (default)
write( *, * ) ieee_rint( a )

! IEEE-Rundungsfunktion mit Flag ieee_round_type = ieee_nearest
call ieee_set_rounding_mode( ieee_nearest )
write( *, * ) ieee_rint( a )

! IEEE-Rundungsfunktion mit Flag ieee_round_type = ieee_to_zero
call ieee_set_rounding_mode( ieee_to_zero )
write( *, * ) ieee_rint( a )

! IEEE-Rundungsfunktion mit Flag ieee_round_type = ieee_down
call ieee_set_rounding_mode( ieee_down )
write( *, * ) ieee_rint( a )

! IEEE-Rundungsfunktion mit Flag ieee_round_type = ieee_up
call ieee_set_rounding_mode( ieee_up )
write( *, * ) ieee_rint( a )
end program bsp

```

Ausgabe:

Standard-Fortran	IEEE-Default	ieee_nearest	ieee_to_zero	ieee_down	ieee_up
-2.0 -1.0 1.0 2.0 3.0 4.0	-2.0 0.0 0.0 2.0 2.0 4.0	-2.0 0.0 0.0 2.0 2.0 4.0	-1.0 0.0 0.0 1.0 2.0 3.0	-2.0 -1.0 0.0 1.0 2.0 3.0	-1.0 0.0 1.0 2.0 3.0 4.0

Beispiel: Halting-Modus

Dieser Modus bestimmt, ob nach einer Exception das Programm angehalten oder fortgesetzt wird. Die in diesem Beispiel eingesetzten IEEE-Unterprogramme sind:

- `ieee_support_halting( flag )` ... prüft, ob auf dem System das IEEE-Halting für das angegebenen Flag überhaupt unterstützt wird (Rückgabewert: `.true.`). Mögliche Flags:
 - `ieee_invalid`
 - `ieee_overflow`
 - `ieee_divide_by_zero`
 - `ieee_underflow`
 - `ieee_inexact`
- `ieee_set_halting_mode( flag, mode )` ... setzt den Halting-Modus für ein bestimmtes Flag.
 - `flag` ... wie bei `ieee_support_halting( flag )`
 - `mode`:
 - `.true.` ... anhalten
 - `.false.` ... Programm weiter ausführen

Fortran 2003-Code

```

program main

```

```

use, intrinsic :: ieee_arithmetic
implicit none

real :: a, b

if( ieee_support_halting( ieee_divide_by_zero ) ) then
  call ieee_set_halting_mode( ieee_divide_by_zero, .false. )

  print *, "Gib a und b ein:"
  read( *, * ) a, b
  write( *, * ) "Resultat: ", a / b
  write( *, * ) "Programm wird fortgesetzt ..."
else
  write( *, * ) "IEEE-Halting wird nicht unterstuetzt"
end if
end program main

```

Eingabe:

```
10.0 0.0
```

Ausgabe (bei Programmerstellung mit dem Intel *ifx*, 2025.2):

```

Resultat:      Infinity
Programm wird fortgesetzt ...

```

Wird

```
call ieee_set_halting_mode( ieee_divide_by_zero, .true. )
```

gesetzt, so ist die Ausgabe eine Fehlermeldung mit Programmabbruch

```

forrtl: error (73): floating divide by zero
Image                PC                Routine          Line        Source
libc.so.6            00007FC62B457900  Unknown          Unknown     Unknown
a.out                000000000040461B  Unknown          Unknown     Unknown
...
Abgebrochen (Speicherabzug geschrieben)

```

Bei Programmerstellung mit *gfortran* (14.2.0, 2024) ist das Verhalten identisch.

Weitere Beispiele zum Thema "Exceptions and IEEE arithmetic" sind im Fortran 2003-Working Draft J3/04-007 ab Seite 386 enthalten.

Teil VI.

Bibliotheken

Eine Programmbibliothek bezeichnet in der Programmierung eine Sammlung von Programmfunktionen für zusammengehörende Aufgaben. Bibliotheken sind im Unterschied zu Programmen keine eigenständig lauffähigen Einheiten, sondern Hilfsmodule, die Programmen zur Verfügung gestellt werden.

41.5. Quelltextbibliotheken

Quelltextbibliotheken enthalten Sammlungen von Wertdefinitionen, Deklarationen, Funktionen, Klassen, generischen Bestandteilen, usw.

41.5.1. API

Eine Programmierschnittstelle ist eine Schnittstelle die von einem Softwaresystem anderen Programmen zur Anbindung an das System zur Verfügung gestellt wird. Oft wird dafür die Abkürzung *API* (für engl. *application programming interface*, deutsch: *Schnittstelle zur Anwendungsprogrammierung*) verwendet. Im Gegensatz zu einer Binärschnittstelle (ABI) definiert ein API nur die Verwendung der Schnittstellen auf Quelltextebene.

Neben dem Zugriff auf Datenbanken, die Hardware wie Festplatte oder Grafikkarte kann ein API auch das Erstellen von Komponenten der grafischen Benutzeroberfläche ermöglichen oder vereinfachen.

Im weiteren Sinne wird die Schnittstelle jeder Bibliothek (Library) als API bezeichnet.

41.6. Statische Bibliotheken

Statische Bibliotheken werden nach dem Kompilervorgang durch einen so genannten Linker oder Binder in einem eigenen Schritt mit dem ausführbaren Programm verbunden.

Der Linker sucht aus den Bibliotheksdateien Unterprogramme heraus, für die es im Programm keine Implementierung gibt. Diese werden dann aus den Dateien extrahiert und an das Programm gebunden, d.h. der Unterprogrammcode wird an den Programmcode angefügt und die Aufrufverweise werden auf die Unterprogrammadressen gerichtet.

41.7. Dynamische Bibliotheken

Dynamische Bibliotheken werden erst bei Bedarf in den Arbeitsspeicher geladen und durch den sogenannten Lader mit dem ausführbaren Programm verbunden. Dadurch muss eine Bibliothek, die von mehreren Programmen genutzt wird, nur einmal im Speicher gehalten werden.

Dies ist beispielsweise bei Multitasking-Systemen vorteilhaft, wenn die Bibliotheken insgesamt sehr groß sind und von vielen Prozessen gleichzeitig verwendet werden. Dort wird eine Bibliotheksdatei bei ihrer ersten Verwendung in den Speicher geladen. Trifft ein Programm auf den Verweis zu einem Unterprogramm, das noch nicht eingebunden wurde, dann wird

ein Laufzeitbinder aktiviert. Dieser sucht das Unterprogramm in den im Speicher vorhandenen Bibliotheken, fügt die Adresse am Aufrufpunkt ein und führt das Unterprogramm erstmalig aus.

Bei jedem weiteren Aufruf des Unterprogramms ist dann die Adresse vorhanden, so dass das Unterprogramm direkt aufgerufen wird. Die Ausführungszeit, insbesondere die Startzeit eines Programms, ist hier geringfügig erhöht. Dies wird in Kauf genommen, da der Programmcode der Bibliotheksfunktionen von allen Prozessen geteilt wird. Der Speicherbedarf aller Programme zusammen ist daher in der Regel kleiner als beim statischen Linken.

Unterstützt das Betriebssystem virtuellen Speicher, so entfällt das Laden der gesamten Bibliothek bei der ersten Verwendung. Stattdessen wird die Bibliothek in den Speicherbereich jedes sie verwendenden Prozesses eingeblendet. Die virtuelle Speicherverwaltung lädt danach nur tatsächlich benötigte Teile der Bibliothek bei Bedarf von der Festplatte in den Arbeitsspeicher.

41.8. Bibliotheken in verschiedenen Programmiersprachen

Bibliotheken in Programmiersprachen enthalten Leistungen, die nicht im Compiler implementiert sind, sondern in der Sprache selbst programmiert sind und mit dem Compiler zusammen oder völlig von ihm getrennt dem Programmierer zur Verfügung stehen. Im ersten Fall ist die Bibliothek meist in der Sprachbeschreibung festgelegt. Im zweiten Fall spricht man von einer externen Bibliothek.

41.9. Bibliotheken bei verschiedenen Betriebssystemen

41.9.1. Windows

Bei den Betriebssystemen Windows und auch bei OS/2 wird eine Bibliotheksdatei, die dynamisch bindet, als Dynamic Link Library (DLL) bezeichnet. Entsprechend haben diese Dateien meist die Dateiendung *.dll*. Ihr Dateiformat ist *Portable Executable*.

Problematisch ist bei Windows 95, Windows 98 und Windows Me, dass durch unzureichende Schutzmaßnahmen die DLLs nicht kontrolliert werden - jedes Programm darf sie austauschen und kann dem Betriebssystem damit möglicherweise Schaden zufügen. Windows 2000 und Windows XP hingegen verfügen über einen Systemschutz, der auch die DLLs einbezieht.

Vorteile

- Außer Code können auch Daten (z. B. Dialog-Ressourcen) von mehreren Prozessen gemeinsam genutzt werden.
- DLLs werden häufig statisch gelinkt, können aber auch dynamisch (daher der Name) gelinkt werden. Dynamisch heißt hier, dass die DLL explizit vom Programm zur Laufzeit geladen wird und die Funktionen, die sich in der DLL befinden, „per Hand“ mit dem Programm verbunden werden. Dadurch wird es möglich, durch Austauschen der DLL die Funktionalität des Programms zur Laufzeit zu verändern.

- DLLs können unabhängig vom Hauptprogramm gewartet werden. D. h. Funktionen in der DLL können ohne Wissen des Programms verändert werden. Danach wird die DLL einfach ausgetauscht (die alte DLL-Datei wird überschrieben), ohne dass das Hauptprogramm verändert werden muss.
- Da die DLL als unabhängige Datei dem Hauptprogramm beiliegen muss, können Anbieter von Programmcode besser sicherstellen, dass Programmierer, die die Funktionen ihrer DLL nutzen, dafür auch bezahlen. Die Funktionalität der DLL verschwindet so nicht (wie bei einer Library) im Code des Programms. Dieser Vorteil wird von Befürwortern freier Software als Nachteil gesehen.

Nachteile

Änderungen in DLLs ziehen oft auch Änderungen im Programm mit sich. Dadurch kommt es leicht zu Versionskonflikten, die oft nur sehr schwer aufzuspüren sind.

Eine der Grundideen der DLLs war, Programmcode zwischen mehreren Programmen zu teilen, um so kostbaren Speicher zu sparen. In der Praxis ist es jedoch dazu gekommen, dass viele Programme bei der Installation DLLs in das Windows-Systemverzeichnis schreiben, die außer diesem speziellen Programm kein anderes benutzen kann.

Außerdem ist die Entwicklung und insbesondere die Anbindung im Vergleich aufwändiger als zur statischen Bibliothek.

Quintessenz

DLLs sollte man nur benutzen, wenn man ihre spezielle Funktionalität benötigt und man ausschließlich unter Windows arbeitet. Sind statische Bibliotheken für den Zweck ausreichend, sollte man diese vorziehen. In der Praxis ergeben sich keinerlei Einsparungen bei der Größe des Codes.

41.9.2. Unix-artige

Auf Unix-artigen Betriebssystemen ist für dynamische Bibliotheken die Bezeichnung *shared library* (englisch *shared*, geteilt) gebräuchlich.

Für diese Dateien hat sich die Endung *.so* (*shared object*) eingebürgert. In der Regel folgt dem Bibliotheksnamen noch eine Versionsnummer.

42. Grafik und GUI

42.1. DISLIN

w: Dislin⁸⁵

⁸⁵ <https://de.wikipedia.org/wiki/%20Dislin>

43. Allgemeines

Die *DISLIN Scientific Plotting Software* ist eine Bibliothek für die grafische Datendarstellung. Auch für die Gestaltung grafischer Benutzeroberflächen läßt sich *DISLIN* verwenden. *DISLIN* greift zu diesem Zwecke auf die *Motif*-Bibliothek zu. Die *DISLIN*-Bibliothek ist für mehrere Programmiersprachen konzipiert, so auch auch für die Programmiersprache Fortran.

44. Beispiele

44.1. Beispiel 1: Strings und Zahlen

Fortran 90/95-Code (free source form)

```
program dbsp1
  implicit none

  real, parameter :: PI = 3.1415926

  ! *** Initialisierung ***
  call setpag ("DA4P")           ! DIN-A4 Hochformat
  call metafl ("CONS")          ! Ausgabe auf Konsole (Bildschirm)
  call disini                   ! DISLIN initialisieren

  ! *** Zeichnen ***
  call messag ("Hallo, Welt!", 50,50) ! Message schreiben
  call number (PI, 4, 50, 150)      ! 3.1416 schreiben

  ! *** Aufräumen ***
  call disfin                   ! DISLIN beenden
end program dbsp1
```

Programm erstellen:

- Variante 1:

```
gfortran -c dateiname.f95
dlink dateiname
```

- Variante 2:

```
gfortran -o dateiname dateiname.f95 -ldislin
```

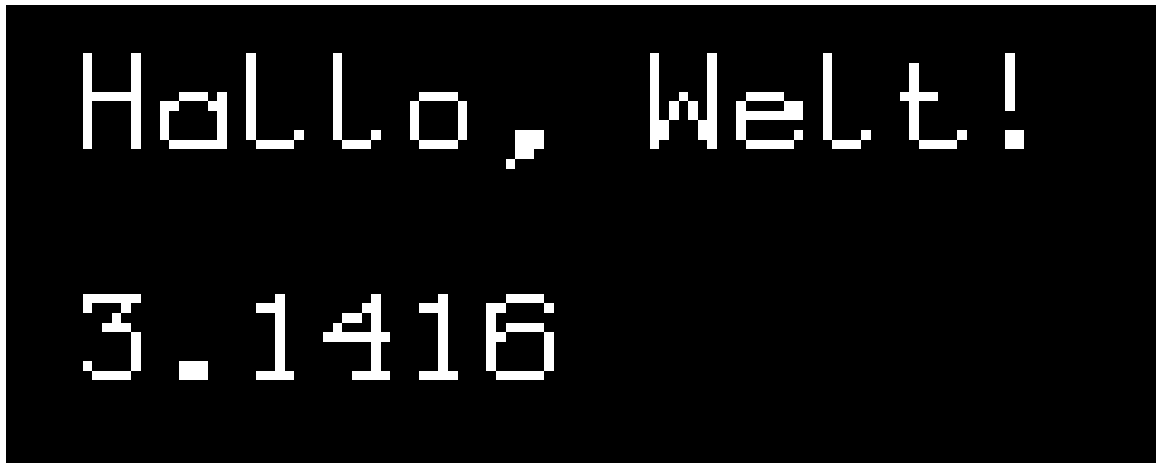


Abb. 48

Eine Auswahl von möglichen aktuellen Parametern für die Subroutine `metafl`:

- "CONS" ... Konsole (Bildschirm)
- "XWIN" ... X-Window (Bildschirm)
- "EPS" ... Encapsulated Postscript-Datei
- "PNG" ... PNG-Datei
- "SVG" ... SVG-Datei
- "PDF" ... PDF-Datei

44.2. Beispiel 2: Zeichnen von Kurven und Funktionen

Fortran 90/95-Code (free source form)

```

program dbsp2
  implicit none

  real, dimension(0:99) :: x, y
  integer :: i, setrgb

  do i = 0, 99
    x(i) = i / 20.0
    y(i) = sin(x(i))
  end do

  ! *** Initialisierung ***
  call setpag ("DA4P")           ! DIN-A4
  call metafl ("PNG")           ! Ausgabe in eine
                                ! PNG-Datei
  call disini                    ! DISLIN initialisieren

  ! *** Zeichnen ***
  call pagfll (255)              ! Hintergrundfarbe auf
                                ! weiß setzen
  call color (setrgb (0., 0., 0.)) ! Vordergrundfarbe auf
                                ! schwarz setzen
  call graf (0.0, 5.0, 0.0, 0.5, -1.0, 1.0, -1.0, 0.1) ! 2D-Koordinatensystem
                                ! setzen
  call curve (x, y, 100)        ! Graphen zeichnen

```

```

! *** Aufräumen ***
call disfin
end program dbsp2
! DISLIN beenden

```

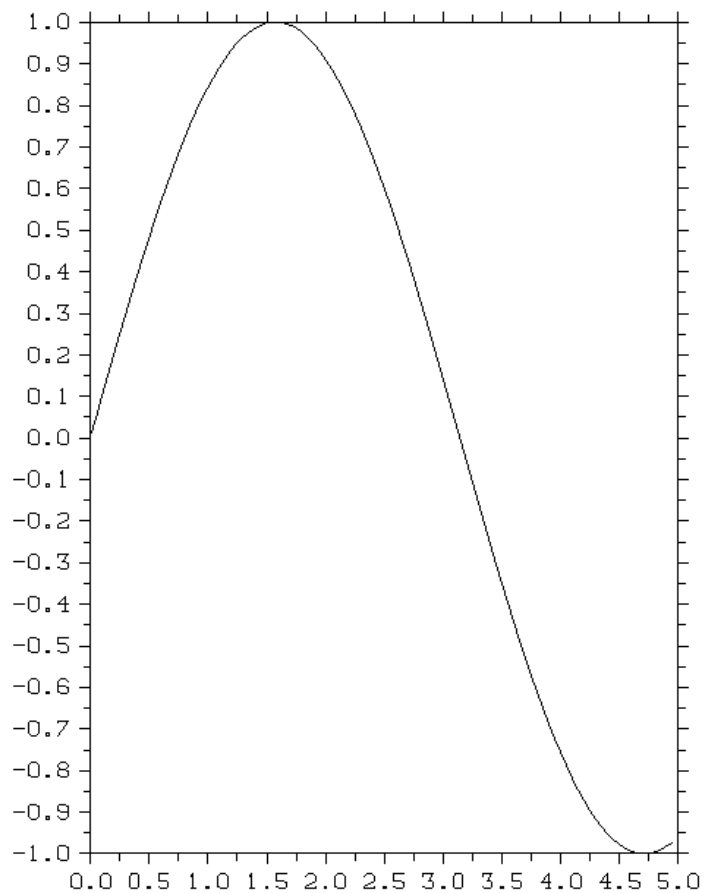


Abb. 49

44.3. Beispiel 3: Ein Pie-Chart

Fortran 90/95-Code (free source form)

```

program dbsp3
  implicit none

  real, dimension(3) :: part = (/5.5, 2.5, 1.0/)
  integer, dimension(3) :: partcol1 = (/10, 100, 150/)
  integer, dimension(3) :: partcol2 = (/10, 100, 150/)

  integer :: setrgb

  ! *** Initialisierung ***
  call setpag ("DA4P")
  call metafl ("CONS")
  call disini
  ! DIN-A4
  ! Console (Bildschirm)
  ! DISLIN initialisieren

```

```

! *** Zeichnen ***
call pagfll (255)                ! Hintergrundfarbe auf
weiß setzen                      ! Vordergrundfarbe auf
call color (setrgb (0., 0., 0.)) ! Shadingpattern (16 =
schwarz setzen                  ! Farbe und
call shdpat (16)                ! Teilfarben
voll)                           ! 3D
call chnpie ("NONE")            ! Pie-Chart zeichnen
Shadingpattern
call pieclr (partcol1, partcol2, 3)
call pietyp ("3D")
call piegrf ("Hallo", 0 , part, 3)

! *** Aufräumen ***
call disfin                      ! DISLIN beenden
end program dbbsp3

```

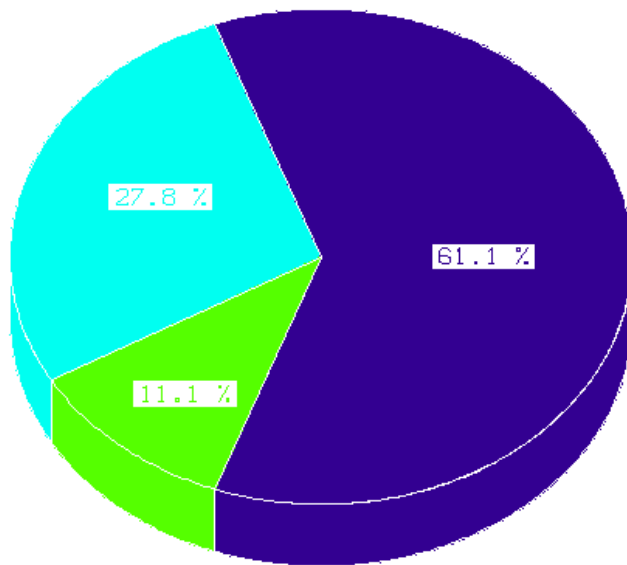


Abb. 50

44.4. Beispiel 4: Ein Meldungsfenster

Fortran 90/95-Code (free source form)

```

program dbbsp4
  implicit none

  call disini
  call dwgmsg ("Hallo, Welt")
  call disfin
end program dbbsp4

```



Abb. 51

Weitere (auch komplexere) Beispiele finden sich im ausführlichen *DISLIN*-Manual. Dieses ist auf der unten genannten Webpräsenz abrufbar.

45. Weblinks

- Max-Planck-Gesellschaft: DISLIN⁸⁶

45.1. f03gl

45.2. Allgemeines

f03gl ist ein Fortran 2003-Interface für OpenGL⁸⁷, speziell für GLUT⁸⁸, freeglut und OpenGLUT. f03gl kann als der Nachfolger von f90gl⁸⁹ betrachtet werden.

45.3. Beispiel

Fortran 2003-Code

```
module bsp_ogl
contains
  subroutine display()
    use opengl_gl
    use opengl_glut
    implicit none

    call glclear(GL_COLOR_BUFFER_BIT + GL_DEPTH_BUFFER_BIT)
    call glColor3f( 0.2, 1.0, 0.3 )

    call glutSolidTeapot( 50.0_gldouble )

    call glutswapbuffers
  end subroutine display

  subroutine gfxinit
    use opengl_gl
    use opengl_glu
    implicit none

    real( glfloat ), dimension( 4 ) :: pos      = (/ 100.0, 100.0, 200.0, 1.0
/)

    call glEnable( GL_LIGHTING )
```

⁸⁶ <https://www.dislin.de>

⁸⁷ <https://de.wikipedia.org/wiki/OpenGL>

⁸⁸ <https://de.wikipedia.org/wiki/OpenGL%20Utility%20Toolkit>

⁸⁹ <http://math.nist.gov/f90gl/>

```

    call glenable( GL_LIGHT0 )
    call glenable( GL_DEPTH_TEST )
    call glenable( GL_COLOR_MATERIAL )
    call glenable( GL_NORMALIZE )
    call glenable( GL_POLYGON_SMOOTH )

    call glLightfv( GL_LIGHT0, GL_POSITION, pos )
    call glClearColor( 0.7, 0.7, 0.7, 0.0 )

    call glMatrixMode( GL_PROJECTION )
    call glOrtho( -100.0_gldouble, 100.0_gldouble, -100.0_gldouble,
100.0_gldouble, &
                -100.0_gldouble, 100.0_gldouble );

    call glMatrixMode( GL_MODELVIEW )
    call glRotatef( 35.0, 1.0, 0.0, 0.0 )
    call glRotatef( -25.0, 0.0, 1.0, 0.0 )
    end subroutine gfxinit
end module bsp_ogl

program bsp
  use opengl_glut
  use bsp_ogl
  use, intrinsic :: iso_c_binding
  implicit none

  integer :: i

  call glutinit()
  call glutinitdisplaymode( GLUT_DOUBLE + GLUT_RGB + GLUT_DEPTH )
  i = glutcreatewindow( "Beispiel" // c_null_char )
  call gfxinit
  call glutdisplayfunc( display )
  call glutmainloop
end program bsp

```

Um das Beispiel nutzen zu können, muss ein Software-Paket, z.B. von der Webseite f03gl⁹⁰, herunter geladen werden (Stand: März 2026). Dieses zip-Paket muss natürlich ausgepackt werden. Gebaut werden kann die f03gl-Software dann mit (w:CMake⁹¹):

```

mkdir build
cd build
cmake ..
make
make test
sudo make install

```

Ggf. muss in der Datei *CMakeCache.txt* im build-Verzeichnis der Compilername von lfortran z.B. auf gfortran geändert werden.

Kompilieren und Linken des obigen Beispielprogrammes *bsp.f90*:

⁹⁰ <https://github.com/MehdiChinoune/f03gl>

⁹¹ <https://de.wikipedia.org/wiki/CMake>

```
gfortran bsp.f90 -lGL -lGLU -lglut -lf03gl -I/usr/local/include/  
-L/usr/local/lib/
```

Die GL-, GLU-Libraries usw. müssen natürlich auf dem Rechner installiert sein. Ansonsten hagelt es Fehlermeldungen.

Nach der Eingabe von:

```
./a.out
```

sollte als Ausgabe die Utah-Teekanne⁹² erscheinen:

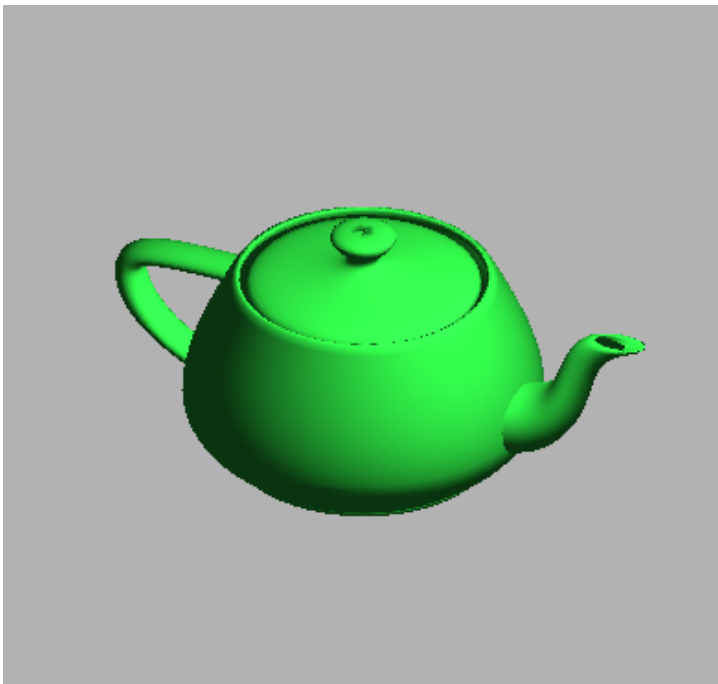


Abb. 52

Eventuell ist noch der `LD_LIBRARY_PATH` um das Verzeichnis `/usr/local/lib` (für die *libf03gl*-Bibliotheks-Datei) zu erweitern. Die beschriebene Vorgehensweise gilt für Linux (konkret OpenSUSE Leap 15.6 und Tumbleweed).

45.4. Besonderheiten

Die Routinen für OpenGL und die GLUT sind in C geschrieben und auch die meisten Anwendungen von OpenGL und GLUT erfolgen in C. Daraus ergeben sich einige Besonderheiten für die Einbindung von OpenGL und GLUT unter Fortran.

⁹² <https://de.wikipedia.org/wiki/Utah-Teekanne>

In C lässt sich folgender Code erzeugen:

```
int main( int argc, char* argv[])
{
    ...
    glutReshapeFunc(ChangeSize);
    ...
    return 0;
}
```

```
void ChangeSize(GLsizei w, GLsizei h)
{
    ...
}
```

Dabei ist *GLsizei* ein von OpenGL spezifizierter Datentyp vom Typ *integer*.

In Fortran 2003 sieht das selbe Code-Fragment unter Verwendung der C-Interoperabilität von Fortran 2003 zum Beispiel folgenderweise aus:

Fortran 2003-Code

```
module simple_opengl_things
  use opengl_gl
  use opengl_glu
  use opengl_glut
  contains
    ...
    subroutine ChangeSize(w, h) bind(c)
      implicit none
      integer( kind=GLsizei ), value :: w
      integer( kind=GLsizei ), value :: h
      ...
    end subroutine ChangeSize
    ...
end module simple_opengl_things

program simple_opengl
  use opengl_glut
  use simple_opengl_things
  ...
  call glutReshapeFunc( ChangeSize )
  ...
end program simple_opengl
```

45.5. Weblinks

- OpenGL⁹³
- freeglut⁹⁴
- OpenGLUT⁹⁵

93 <http://www.opengl.org/>

94 <http://freeglut.sourceforge.net/>

95 <http://openglut.sourceforge.net/>

- Manual für GLUT version 3⁹⁶

45.6. gtk-fortran

Eine relativ neue Möglichkeit GUIs zu erstellen ist gtk-fortran. Dieses Paket stellt das C-Binding für die w:GTK (Programmbibliothek)⁹⁷ zur Verfügung. Es verwendet den Fortran 2008-Standard.

- Für die Homepage generell siehe <https://github.com/vmagnin/gtk-fortran/wiki>.
- Eine Downloadmöglichkeit ist unter <https://github.com/vmagnin/gtk-fortran> gegeben.
- Für die Installationsanweisungen siehe <https://github.com/vmagnin/gtk-fortran/wiki/Installation>. Ggf. muss in der Datei CMakeCache.txt im build-Verzeichnis der Compilername von lfortran z.B. auf gfortran geändert werden.
- Für die Tutorials siehe <https://github.com/vmagnin/gtk-fortran/wiki/Tutorials>.

Ein Beispiel wird nachfolgend aus dem gtk-fortran-Paket, gekürzt und geringfügig überarbeitet, zitiert. Der Code steht unter der w:GNU General Public License⁹⁸, Copyright (C) 2020 The gtk-fortran team. Der Originalcode findet sich im heruntergeladenen gtk-fortran-Paket unter dem Verzeichnis `.../gtk-fortran-gtk4/examples` in der Datei `gtkhello.f90`.

Fortran 2003-Code

```
module handlers
  use, intrinsic :: iso_c_binding, only: c_ptr, c_int, c_funloc, c_null_char
  use gtk, only: FALSE, gtk_window_set_default_size, &
    & gtk_window_set_title, gtk_window_destroy, &
    & g_signal_connect, g_signal_connect_swapped, &
    & gtk_widget_show, gtk_application_window_new, &
    & gtk_box_new, gtk_box_append, gtk_window_set_child, &
    & GTK_ORIENTATION_VERTICAL, GTK_ORIENTATION_HORIZONTAL, &
    & gtk_button_new_with_label, gtk_button_new_with_mnemonic
  implicit none

contains
  subroutine activate(app, gdata) bind(c)
    type(c_ptr), value, intent(in) :: app, gdata
    type(c_ptr) :: window
    type(c_ptr) :: box
    type(c_ptr) :: button1, button2

    window = gtk_application_window_new(app)
    call gtk_window_set_title(window, "Hello GLib & GTK world!"//c_null_char)

    box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10_c_int)

    call gtk_window_set_child(window, box)

    button1 = gtk_button_new_with_label("I say hello"//c_null_char)
    call gtk_box_append(box, button1)
    call g_signal_connect(button1, "clicked"//c_null_char, &
```

⁹⁶ <https://www.opengl.org/resources/libraries/glut/spec3/spec3.html>

⁹⁷ <https://de.wikipedia.org/wiki/GTK%20%28Programmbibliothek%29>

⁹⁸ <https://de.wikipedia.org/wiki/GNU%20General%20Public%20License>

```

        & c_funloc(button1clicked))
    call g_signal_connect(button1, "clicked"//c_null_char, c_funloc(hello))

    button2 = gtk_button_new_with_mnemonic("I don't know why you say _goodbye"&
        & //c_null_char)
    call gtk_box_append(box, button2)
    call g_signal_connect_swapped(button2, "clicked"//c_null_char, &
        & c_funloc(gtk_window_destroy), window)
    call gtk_widget_show(window)
end subroutine activate

subroutine hello(widget, gdata) bind(c)
    type(c_ptr), value, intent(in) :: widget, gdata

    print *, "So I say Hello GTK World!"
end subroutine hello

subroutine button1clicked(widget, gdata) bind(c)
    type(c_ptr), value, intent(in) :: widget, gdata

    print *, "Button 1 clicked!"
end subroutine button1clicked
end module handlers

program bsp
    use, intrinsic :: iso_c_binding, only: c_ptr, c_funloc, c_null_char,
    c_null_ptr
    use gtk, only: gtk_application_new, G_APPLICATION_FLAGS_NONE
    use g, only: g_application_run, g_object_unref
    use handlers

    implicit none
    integer(c_int) :: status
    type(c_ptr) :: app

    app = gtk_application_new("gtk-fortran.examples.gtkhello"//c_null_char, &
        & G_APPLICATION_FLAGS_NONE)
    call g_signal_connect(app, "activate"//c_null_char, c_funloc(activate), &
        & c_null_ptr)
    status = g_application_run(app, 0_c_int, [c_null_ptr])

    print *, "You have exited the GLib main loop, bye, bye..."

    call g_object_unref(app)
end program bsp

```

Compiliert und gelinkt wird das Ganze so:

```
gfortran bsp.f90 $(pkg-config --cflags --libs gtk-4-fortran)
```

Nach dem Programmstart

```
./a.out
```

öffnet sich ein Fenster mit zwei Buttons. Der eine Button schreibt etwas in die Konsole, der andere schließt das Programm.

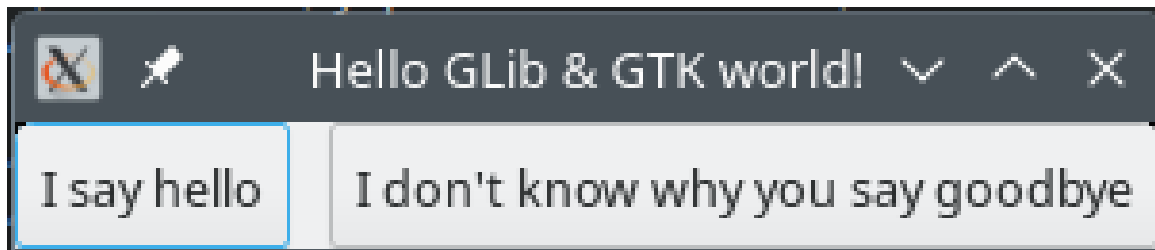


Abb. 53

Getestet wurde das Ganze mit openSUSE Tumbleweed und gfortran 15.2.1 (Stand: März 2026).

46. Mathematik

46.1. BLAS und ATLAS

w: Basic Linear Algebra Subprograms⁹⁹

⁹⁹ <https://de.wikipedia.org/wiki/%20Basic%20Linear%20Algebra%20Subprograms>

47. Allgemeines

Die Basic Linear Algebra Subprograms (BLAS) stellen eine Sammlung von Unterprogrammen für die Vektor- und Matrizenrechnung dar.

- Level 1: Skalar-Vektor-, Vektor-Vektor-Operationen
- Level 2: Matrix-Vektor-Operationen
- Level 3: Matrix-Matrix-Operationen

Die Automatically Tuned Linear Algebra Software (ATLAS) ist ein um einige LAPACK-Funktionen erweitertes BLAS-Paket und bietet die Möglichkeit, automatisiert eine rechneroptimierte Algebra-Bibliothek zu erzeugen.

48. Installation von BLAS

BLAS lässt sich unter OpenSUSE Leap 15.6 einfach mittels dem Paket-Management-System YaST installieren.

49. Beispiele

49.1. Beispiel: Die Level 1-Funktionen *sdot* und *dnrm2*

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real, dimension(3) :: a = (/2.,1.,-1./), b = (/5., -2., 1.5/)
  real                :: c, sdot
  real(kind=8)        :: d, dnrm2

  ! *** Skalarprodukt ***
  ! sdot: s ... REAL, dot ... Skalarprodukt (inneres Produkt)
  ! 1. Argument ... Dimension des Vektors
  ! 2. und 4. A. ... die Vektoren
  ! 3. und 5. A. ... Inkrement (hier 1)
  c = sdot(3, a, 1, b, 1)
  write(*,*) c
  ! Ausgabe: 6.500000

  ! *** Norm des Vektors ***
  ! dnrm2: d ... DOUBLE PRECISION, nrm2 ... (euklidische) Norm
  ! 1. Argument: Dimension des Vektors
  ! 2. A.:      Vektor
  ! 3. A.:      Inkrement (hier 1)
  d = dnrm2(3, dble(a), 1)
  write(*,*) d
  ! Ausgabe: 2.44948974278318
end program bsp
```

Kompilieren und Linken:

```
gfortran bsp.f95 -lblas
```

49.2. Beispiel: Die Level 2-Subroutine *sger*

sger steht für:

- s ... REAL
- ge ... general matrix
- r ... rank 1 operation

Mathematisch ist damit folgende Operation gemeint:

$$A \leftarrow \alpha \mathbf{x} \mathbf{y}^T + A$$

wobei A eine $m \times n$ -Matrix ist, x und y stellen Vektoren dar.

Das nachfolgende Beispiel führt konkret folgende Rechnung aus: $A = \begin{bmatrix} 2.0 \\ 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \end{bmatrix} + \begin{bmatrix} 1.0 & -1.0 \\ 2.0 & 7.0 \end{bmatrix}$

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real, dimension(2) :: x = (/2., 1./), y = (/1., 0./)
  real, dimension(2,2) :: a = reshape ( (/1., 2., -1., 7./), (/2, 2/) )

  call sger (2, 2, 1., x, 1, y, 1, a, 2)
  write(*,*) a
  ! Ausgabe:  3.000000    3.000000   -1.000000    7.000000
end program bsp
```

50. Weblinks

- BLAS (Basic Linear Algebra Subprograms)¹⁰⁰
- Automatically Tuned Linear Algebra Software (ATLAS)¹⁰¹

50.1. FGSL

w: Gnu Scientific Library¹⁰²

50.2. Allgemeines

Bei der "GNU Scientific Library" (GSL) handelt es sich um eine in C geschriebene Bibliothek. Diese bietet Funktionen für ein weites Spektrum der Mathematik. Beispielhaft seien folgende Bereiche genannt:

- Komplexe Zahlen
- Lineare Algebra
- Polynome
- Statistik
- Fast Fourier Transformation (FFT)
- Numerische Differentiation
- Numerische Integration
- Gewöhnliche Differentialgleichungen
- IEEE Floating-Point-Arithmetik

FGSL ist ein Fortran-Interface für diese GSL-Bibliothek. FGSL selbst deckt nicht die komplette Funktionspalette von GSL ab, inzwischen sind aber auch lineare Algebra und die FFT-Funktionen Bestandteil von FGSL, auch wenn zu diesem Zweck der Einsatz der optimierten Bibliotheken LAPACK und FFTW empfohlen wird. Für einige Teilbereiche werden nicht alle in C implementierten Datentypen unterstützt.

FGSL wurde unter Verwendung einiger *Fortran 2003*-Sprachmerkmale erstellt. Die Einbindung von FGSL in eigene Programme setzt aus diesem Grunde einen entsprechenden Fortran-Compiler voraus.

¹⁰⁰ <https://www.netlib.org/blas/>

¹⁰¹ <https://math-atlas.sourceforge.net/>

¹⁰² <https://de.wikipedia.org/wiki/%20Gnu%20Scientific%20Library>

50.3. Beispiele

50.3.1. Beispiel: Datentypen, Potenzierung und mathematische Konstanten

C	Fortran
<pre>#include <stdio.h> #include <gsl/gsl_math.h> int main (void) { double a; double d = 5.0; a = gsl_pow_2 (d) * M_PI_4; printf ("Kreisflaeche = %f\n", a); return 0; }</pre>	<pre>program bsp use fgsl implicit none real(kind = fgsl_double) :: a real(kind = fgsl_double) :: d = 5.0_fgsl_double a = d ** 2 * m_pi_4 write(*, *) "Kreisflaeche = ", a end program bsp</pre>
Kompilieren, Linken: gcc bsp.c -I/usr/local/include -L/usr/local/lib -lgsl -lgslcblas -lm	Kompilieren, Linken: gfortran bsp.f90 -L/usr/local/lib64 -lgsl -lfgsl -lgslcblas -I/usr/local/include

Der kind-Wert `fgsl_double` entspricht dem `c_double` aus dem `iso_c_binding`-Modul. FGSL kennt die speziellen `pow`-Funktionen aus GSL nicht, da Fortran ohnehin über einen eigenen Potenzierungsoperator verfügt. Neben der `m_pi_4`-Konstante ($= \pi/4$) kennt FGSL noch eine ganze Reihe anderer mathematischer Konstanten, z.B.:

<code>m_e</code>	...	e, Eulersche Zahl, 2,714...
<code>m_euler</code>	...	Eulersche Konstante, 0,577...
<code>m_pi</code>	...	π
<code>m_pi_2</code>	...	$\pi/2$
<code>m_sqrt2</code>	...	$\sqrt{2}$

Auch jede Menge physikalische Konstanten kennt FGSL, z.B.:

<code>fgsl_const_mkxa_speed_of_light</code>	...	Lichtgeschwindigkeit im Vakuum, 2.9979... x 10 ⁸ m / s
<code>fgsl_const_mkxa_molar_gas</code>	...	Allgemeine Gaskonstante, 8.314472 J / (K · mol)
<code>fgsl_const_mkxa_inch</code>	...	0.0254 m
<code>fgsl_const_mkxa_torr</code>	...	133.322... Pa
<code>fgsl_const_num_giga</code>	...	10 ⁹

50.3.2. Beispiel: Lösen einer quadratischen Gleichung

Gesucht ist die Lösung der quadratischen Gleichung $x^2 + 12x + 37 = 0$

C	Fortran
---	---------

<pre> #include <stdio.h> #include <gsl/gsl_complex.h> #include <gsl/gsl_poly.h> int main (void) { double a, b, c; gsl_complex z1, z2; int i; a = 1.0; b = 12.0; c = 37.0; i = gsl_poly_complex_solve_quadratic(a, b, c, &z1, &z2); if(i == 1) { /* nur eine Lsg.*/ printf("z = (%f, %f)\n", z1.dat[0], z1.dat[1]); } else { /* 2 Lsg., reell oder komplex */ printf("z1 = (%f, %f)\n", z1.dat[0], z1.dat[1]); printf("z2 = (%f, %f)\n", z2.dat[0], z2.dat[1]); } return 0; /* Ausgabe: z1 = (-6.000000, -1.000000) z2 = (-6.000000, 1.000000) */ } </pre>	<pre> program bsp use fgsl implicit none real(kind = fgsl_double) :: a, b, c complex(fgsl_double), dimension(2) :: z integer(kind = fgsl_int) :: i a = 1.0_fgsl_double b = 12.0_fgsl_double c = 37.0_fgsl_double i = fgsl_poly_complex_solve_quadratic(a, b, c, z(1), z(2)) if(i == 1) then ! nur eine Lsg. write(*, *) " z =", z(1) else ! 2 Lsg., reell oder komplex write(*, *) " z1,2 =", z end if ! Ausgabe: ! z1,2 = (-6.,-1.) (-6.,1.) end program bsp </pre>
---	--

50.3.3. Beispiel: Numerische Integration

Gesucht ist die Lösung des Integrals $I = \int_0^1 \frac{\sin x}{x} dx$

C

Fortran

<pre> #include <stdio.h> #include <math.h> #include <gsl/gsl_integration.h> double f(double x, void *params) { return(sin(x) / x); } int main () { double result, error; size_t neval; gsl_function func; func.function = &f; func.params = 0; gsl_integration_qng (&func, 0.0, 1.0, 1e- 9, 1e-9, &result, &error, &neval); printf ("Ergebnis = %f\n", result); return 0; /* Ausgabe: Ergebnis = 0.946083 */ } </pre>	<pre> module integral implicit none contains function f(x, params) bind(c) use, intrinsic :: iso_c_binding implicit none real(kind = c_double) :: f real(kind = c_double), value :: x type(c_ptr), value :: params f = sin(x) / x end function f end module integral program bsp use fgsl use integral use, intrinsic :: iso_c_binding implicit none real(kind = fgsl_double) :: result, error integer(kind = fgsl_size_t) :: neval integer(kind = fgsl_int) :: i type(fgsl_function) :: func func = fgsl_function_init(f, c_null_ptr) i = fgsl_integration_qng (func, 0.0_fgsl_double, 1.0_fgsl_double, 1e- 9_fgsl_double, & 1e- 9_fgsl_double, & result, error, neval) write(*, *) "Ergebnis =", result ! Ausgabe: ! Ergebnis = 0.946083070367183 end program bsp </pre>
---	--

(F)GSL stellt verschiedene Möglichkeiten der numerischen Integration zur Verfügung. Hier wurde die Funktion für den QNG-Algorithmus (non-adaptive Gauss-Kronrod) gewählt.

50.3.4. Beispiel: IEEE-Floating-Point-Arithmetik

Darstellung einer Fließkommazahl nach IEEE 754-Standard:

$$(-1)^s(1.ffff\dots)2^E$$

Beispielsweise wird eine 32-bit-Fließkommazahl binär so aufgegliedert:

```
seeeeeeeeefffffffffffffffffffff
```

- s ... sign, 1 bit
- e, E ... exponent, 8 bit ($2^8 = 256$; $E_{\min} = -127$; $E_{\max} = 128$)
- f ... fraction, 23 bit

Näheres zum IEEE 754-Standard findet sich z.B. bei w:IEEE 754¹⁰³

FGSL bietet Subroutinen, um Fließkommazahlen anschaulich entsprechend dem IEEE 754-Standard auszugeben:

- `fgsl_ieee_printf( x )` ... Ausgabe der Zahl x im IEEE-Format auf stdout
- `fgsl_ieee_fprintf( str, x )` ... Ausgabe der Zahl x im IEEE-Format. `str` ist ein C-Zeiger (C: FILE *, Fortran: type(c_ptr)).

C	Fortran
<pre>#include <gsl/gsl_ieee_utils.h> int main() { float a = 1.5; double b = -2.6666666666666666; gsl_ieee_printf_float(&a); gsl_ieee_printf_double(&b); }</pre>	<pre>program bsp use fgsl implicit none real(kind = fgsl_float) :: a = 1.5 real(kind = fgsl_double) :: b = -2.6666666666666666d0 call fgsl_ieee_printf(a) call fgsl_ieee_printf(b) end program bsp</pre>
Ausgabe: 1.10000000000000000000000000000000*2 ⁰ - 1.01*2 ¹	

Mit der Subroutine `fgsl_ieee_env_setup()` lassen sich über die Environment-Variable `GSL_IEEE_MODE` einige nützliche Attribute (Rundungsmodus etc.) festlegen.

C	Fortran
<pre>#include <stdio.h> #include <gsl/gsl_ieee_utils.h> int main() { float a = 1.5; int i; gsl_ieee_env_setup(); for(i = 0; i < 5; i++) { a /= 0.11; printf("%f\n", a); } }</pre>	<pre>program bsp use fgsl implicit none real :: a = 1.5 integer :: i call fgsl_ieee_env_setup() do i = 0, 4 a = a / 0.11 write(*, *) a end do end program bsp</pre>

¹⁰³ <https://de.wikipedia.org/wiki/IEEE%20754>

Programmaufruf: <code>GSL_IEEE_MODE="round-to-nearest" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-to-nearest,trap-common"</code> 13.636364 123.966942 1126.972168 10245.201172 93138.195312	Programmaufruf: <code>GSL_IEEE_MODE="round-to-nearest" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-to-nearest,trap-common"</code> 13.636364 123.96695 1126.9723 10245.203 93138.21
Programmaufruf: <code>GSL_IEEE_MODE="round-up" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-up,trap-common"</code> 13.636364 123.966949 1126.972290 10245.203125 93138.210938	Programmaufruf: <code>GSL_IEEE_MODE="round-up" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-up,trap-common"</code> 13.636364 123.96695 1126.9723 10245.203 93138.21
Programmaufruf: <code>GSL_IEEE_MODE="round-down" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-down,trap-common"</code> 13.636363 123.966934 1126.972046 10245.200195 93138.179688	Programmaufruf: <code>GSL_IEEE_MODE="round-down" ./a.out</code> Ausgabe: <code>GSL_IEEE_MODE="round-down,trap-common"</code> 13.636363 123.966934 1126.972 10245.2 93138.18

50.4. Weblinks

- [GSL - GNU Scientific Library](https://www.gnu.org/software/gsl/)¹⁰⁴
- [FGSL: A Fortran interface to the GNU Scientific Library](https://doku.lrz.de/f gsl-a-fortran-interface-to-the-gnu-scientific-library-10746505.html)¹⁰⁵

50.5. LAPACK

w: [LAPACK](https://de.wikipedia.org/wiki/%20LAPACK)¹⁰⁶

50.6. Allgemeines

LAPACK steht für "Linear Algebra Package". LAPACK ist eine Bibliothek zwecks Lösung von

- linearen Gleichungssystemen
- LLS-Aufgaben
- Eigenwertproblemen
- Singulärwertproblemen

LAPACK ist in FORTRAN 77 geschrieben und nutzt weitgehend Funktionen von BLAS Level 3. Falls keine für einen bestimmten Prozessor optimierte Version von BLAS, z.B. ATLAS, bereits installiert ist, kann LAPACK mit der eigenen BLAS-Implementierung kompiliert werden. Aus FORTRAN 77 resultierende Namensbeschränkung auf eine maximale Länge von 6 Zeichen führt zu sehr kryptischen Unterprogrammbezeichnungen, z.B.

	D	G	E	T	R	F
Zeichenposition:	D1	M1	M2	O1	O2	O3

Erläuterung der Zeichenpositionen:

D1	Datentyp:	S	...	real
		D	...	double precision
		C	...	complex
		Z	...	double complex

¹⁰⁴ <https://www.gnu.org/software/gsl/>

¹⁰⁵ <https://doku.lrz.de/f gsl-a-fortran-interface-to-the-gnu-scientific-library-10746505.html>

¹⁰⁶ <https://de.wikipedia.org/wiki/%20LAPACK>

M1, M2	Matrixtyp, z.B.	GE	...	generell
		DI	...	diagonal
		OR	...	orthogonal (reelle Zahlen)
		SY	...	symmetrisch
O1, O2, (O3)	Operation, z.B.	TRF	...	faktoriere

Eine detailliertere und umfassendere Beschreibung des Funktionsumfangs und der Anwendungsmöglichkeiten der LAPACK-Bibliothek bietet der LAPACK Users' Guide¹⁰⁷. Die einzelnen Subroutinen inklusive Unterprogrammparameter sind zudem auch in den LAPACK-Sourcecode-Dateien ausführlich dokumentiert.

50.7. Beispiel: Lösen eines einfachen Gleichungssystems

Gegeben ist folgendes Gleichungssystem:

$$2x + y = 5$$

$$3x + y = 6$$

bzw. in Matrixschreibweise:

$$\begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix} \Leftrightarrow A\mathbf{x} = B$$

Gesucht sind die Unbekannten x und y :

Fortran 90/95-Code (free source form)

```

program bsp
  implicit none

  integer :: info
  real, dimension(2,2) :: a = reshape ( (/2.,3.,1.,1./), (/2,2/) )
  real, dimension(2) :: b = (/5., 6./), ipiv

! SUBROUTINE SGESV(N, NRHS, A, LDA, IPIV, B, LDB, INFO)
! s ... real, ge ... general matrix type, sv ... solver
  call sgesv(2, 1, a, 2, ipiv, b, 2, info)

  write(*,*) "Lösung (x, y): ", b

  if(info == 0) then
    write(*,*) "Ergebnis OK"
  else
    write(*,*) "Ergebnis NOK"
  end if

! Ausgabe: Lösung (x, y):      1.000000      3.000000
! Ausgabe: Ergebnis OK
end program bsp
    
```

¹⁰⁷ <https://www.netlib.org/lapack/lug/index.html>

Kompilieren, Linken:

```
gfortran bsp.f95 -llapack -lblas
```

50.8. Beispiel: Inverse Matrix

Gegeben ist eine 3x3-Matrix

$$A = \begin{pmatrix} 3 & -2 & 1 \\ -3 & 5 & 0 \\ 2 & -1 & 2 \end{pmatrix}$$

die invertiert werden soll. Die Zahlenwerte dieser Matrix A entsprechen einem Beispiel aus *Bartsch: Mathematische Formeln, 21. Auflage, VEB Fachbuchverlag Leipzig, 1986, Seite 109*, ebenfalls zum Thema "Inverse Matrix".

Verwendet werden hierzu die beiden LAPACK-Subroutinen:

- SGETRF
 - S ... Datentyp: real
 - GE ... Matrixtyp: general
 - TRF ... Operation: LU-Faktorisierung (Dreiecksform)
- SGETRI
 - S ... Datentyp: real
 - GE ... Matrixtyp: general
 - TRI ... Operation: Invertierung einer LU-faktorierten Matrix

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real, dimension( 3, 3 ) :: A
  integer, dimension( 3 ) :: ipiv
  real, dimension( 3 )    :: work
  integer                  :: m = 3, n = 3, lda = 3, lwork = 3, info

  A = reshape( (/ 3.0, -3.0, 2.0, -2.0, 5.0, -1.0, 1.0, 0.0, 2.0 /), &
               shape( A ) )

  ! LU-Faktorisierung (Dreieckszerlegung) der Matrix A
  call sgetrf( m, n, A, lda, ipiv, info )

  ! Inverse der LU-faktorierten Matrix A
  call sgetri( n, A, lda, ipiv, work, lwork, info )

  write( *, * ) "Inverse Matrix Ai =", A
  write( *, * ) "Testweise wie im Bartsch-Beispiel, Ai = 1/11 * (", A * 11, ")"

  if( info == 0 ) then
    write( *, * ) "OK"
  else
    write( *, * ) "Nicht OK"
  end if
```

```
! Ausgabe:
!   Inverse Matrix Ai = 0.909091 0.54545456 -0.63636374 0.2727273
!   0.36363637 -0.090909116 -0.45454553 -0.2727273 0.81818193
!   Testweise wie im Bartsch-Beispiel, Ai = 1/11 * ( 10.000001 6. -7.000001
!   3.0000005 4. -1.0000002 -5.000001 -3.0000005 9.000001 )
!   OK

end program bsp
```

50.9. Weblinks

- LAPACK - Linear Algebra Package¹⁰⁸
- LAPACK Search Engine¹⁰⁹
- LAPACK95 - Fortran95 interface to LAPACK¹¹⁰
- Erstellen der LAPACK-Bibliothek mit *gfortran*¹¹¹

108 <http://www.netlib.org/lapack/>

109 <http://www.cs.colorado.edu/~jessup/lapack/>

110 <http://www.netlib.org/lapack95/>

111 <http://gcc.gnu.org/wiki/GfortranBuild>

51. Parallele Programmierung

51.1. OpenMP

w: OpenMP¹¹²

51.2. Was ist OpenMP

OpenMP ist die Abkürzung für "*Open* specifications for *Multi Processing*" und ist eine API für Fortran und C/C++, die zum Zwecke der parallelen Programmierung¹¹³ mittels Shared-Memory-Ansatz für Mehrprozessor-Systeme erschaffen wurde.

Durch Anwendung von Compiler-Direktiven und spezieller Unterprogramme wird die Abarbeitung von bestimmten Programmkonstrukten auf mehrere Threads aufgeteilt.

Der "Master Thread" mit der Nummer 0 ist in einem OpenMP-Programm standardmäßig immer aktiv. Der Programmierer bestimmt im Programmcode, wann eine Gabelung (*fork*) in mehrere Threads gefordert wird und wann das Ganze wieder in einen einzelnen Thread vereint werden soll (*join*).

OpenMP wird schon von vielen Fortran-Compilern unterstützt.

51.3. Ein einfaches Beispiel

Fortran 90/95-Code (free source form)

```
program bsp
  use omp_lib
  implicit none

  ! fork
  !$omp parallel num_threads(3)

    ! Das nur 1x ausgeben (beim Master Thread)
    if( omp_get_thread_num() == 0 ) then
      write( *, * ) 'Insgesamt gibt es ', omp_get_num_threads(), 'Thread(s)'
    end if

    ! Das bei jedem Thread ausgeben
    write( *, * ) 'Thread ', omp_get_thread_num(), 'ist aktiv'
  ! join
  !$omp end parallel
```

¹¹² <https://de.wikipedia.org/wiki/%20OpenMP>

¹¹³ <https://de.wikipedia.org/wiki/Parallele%20Programmierung>

```
! Ausgabe:
!   Insgesamt gibt es           3 Thread(s)
!   Thread                   0 ist aktiv
!   Thread                   1 ist aktiv
!   Thread                   2 ist aktiv
end program bsp
```

Unter Umständen muss das Modul `omp_lib` eingebunden werden. Dieses Modul enthält die `interface` für die OpenMP-Routinen. Eine mögliche Form des OpenMP-Modus ist am Ende dieses Abschnittes angegeben.

Kompilieren und Linken des Beispielsprogramms:

```
gfortran:                gfortran -fopenmp -o bsp bsp.f90
Intel Fortran Compiler:   ifx -qopenmp -o bsp bsp.f90
```

Erläuterung:

- OpenMP-Direktiven werden als Kommentare gekapselt. Bei Verwendung der "free source form" lautet der erste Direktiven-Abschnitt (en. *sentinel*, dt. *Wächter*) immer `!$omp`. Groß-/Kleinschreibung spielt keine Rolle. Es folgt die Anweisung, dass sich nun das Programm gabeln soll (`parallel`). Die Anzahl der gewünschten Threads wird hier explizit mittels der Option `num_threads()` festgelegt.
- Verwendete OpenMP-Funktionen:
 - `omp_get_thread_num()` ... Aktuelle Thread-Nummer
 - `omp_get_num_threads()` ... Anzahl der Threads
- Beendet wird der parallele Programmteil mit `!$omp end parallel`

51.4. Thread-Erzeugung: Die `parallel`-Direktive

Wie im vorigen Beispiel bereits angedeutet, wird ein "fork" (die Threaderzeugung) immer mit der Direktive

```
!$omp parallel [optionen]
```

eingeleitet und mit

```
!$omp end parallel
```

beendet. Es kann hier auch eine Reihe von optionalen Steueranweisungen angegeben werden (siehe vorheriges Beispiel und nachfolgende Beispiele).

51.5. Thread-Anzahl bestimmen

- Festlegung im Rahmen der OpenMP-Direktive `!$omp parallel` über die Option `num_threads( nr )`
- Mittels OpenMP-Subroutinenaufruf vor dem *fork*: `call omp_set_num_threads( nr )`

- Festlegung in der Kommandozeile vor Ausführung des Programmes, z.B.: `export OMP_NUM_THREADS=nr`
- Default (normalerweise 1 Thread pro CPU)
- Dynamische Anpassung zur Programmlaufzeit per Run-Time-Environment: `call omp_set_dynamic( .true. )`

51.6. Sichtbarkeit/Gültigkeit von Daten

Aufgrund des Shared-Memory-Ansatzes werden Daten standardmäßig zwischen den Threads geteilt. Dieses Verhalten kann aber auch optional geändert werden. Mögliche Varianten für die `parallel`-Direktive:

- **shared** ... Solche Daten sind explizit in allen Threads sichtbar und gültig. Eine Änderung solcher Daten in einem Thread wirkt sich auf alle anderen Threads aus.
- **private** ... Solche Daten sind nur im aktuellen Thread sichtbar und gültig, sie werden beim Eintritt in den parallelen Programmabschnitt nicht speziell initialisiert. Änderungen dieser Werte wirken sich nicht auf nachfolgende serielle Programmteile aus.
- **firstprivate** ... Ähnlich wie **private**. Der Unterschied zu **private** ist, dass solcherart markierte Daten mit dem letztgültigen Wert aus dem vorhergehenden seriellen Programmabschnitt initialisiert werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  use omp_lib
  implicit none

  integer :: a, b, c, tnr

  a = 123
  b = 123
  c = 123

  ! Seriell
  write( *, * ) 'Seriell:'
  write( *, * ) 'a = ', a
  write( *, * ) 'b = ', b
  write( *, * ) 'c = ', c
  write( *, * ) '-----'

  call omp_set_num_threads( 3 )

  !$omp parallel shared( a ) private( b ) firstprivate( c )
    write( *, * ) 'Parallel:'

    tnr = omp_get_thread_num() ! Aktuelle Threadnummer

    if( tnr == 0 ) then
      a = a + 5
      b = b + 5
      c = c + 5
    end if

    write( *, * ) 'a = ', a
    write( *, * ) 'b = ', b
    write( *, * ) 'c = ', c
    write( *, * ) '-----'
  end parallel

```

```
!$omp end parallel

! Seriell
write( *, * ) 'Seriell:'
write( *, * ) 'a = ', a
write( *, * ) 'b = ', b
write( *, * ) 'c = ', c
end program bsp
```

Ausgabe:

```
Seriell:
a =      123
b =      123
c =      123
-----
Parallel:      0
a =      128
b =         5
c =      128
-----
Parallel:      1
a =      128
b =         0
c =      123
-----
Parallel:      2
a =      128
b =         0
c =      123
-----
Seriell:
a =      128
b =      123
c =      123
```

Für andere OpenMP-Direktiven sind auch noch andere Sichtbarkeits- und Gültigkeitsbereiche möglich (z.B. `lastprivate`).

51.7. Die do-Direktive

Innerhalb eines `parallel`-Blocks können auch `do`-Schleifen parallelisiert werden. Die Schleifendurchläufe werden auf die einzelnen Threads bzw. CPUs aufgeteilt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  use omp_lib
  implicit none

  integer :: i, tnr

  call omp_set_num_threads( 3 )

  !$omp parallel private( i )
    !$omp do
      do i = 1, 20
        tnr = omp_get_thread_num() ! Aktuelle Threadnummer
        write( *, * ) 'Thread', tnr, ':', i
      end do
    end do
  end parallel
```

```

        end do
    !$omp end do
!$omp end parallel
end program bsp

```

Ausgabe:

Thread	0 :	1
Thread	0 :	2
Thread	0 :	3
Thread	0 :	4
Thread	0 :	5
Thread	0 :	6
Thread	0 :	7
Thread	1 :	8
Thread	1 :	9
Thread	1 :	10
Thread	1 :	11
Thread	1 :	12
Thread	1 :	13
Thread	1 :	14
Thread	2 :	15
Thread	2 :	16
Thread	2 :	17
Thread	2 :	18
Thread	2 :	19
Thread	2 :	20

Die Zuweisung der Schleifendurchläufe an die Threads kann gesteuert werden. Dazu wird der do-Direktive eine `schedule`-Anweisung mit dem Argument Typ und ev. auch mit dem Argument Chunk-Größe beigefügt. Als Typen sind möglich

- `static`
- `dynamic`
- `guided`
- `runtime`

Diese Bezeichnungen beziehen sich auf die Art der Thread-Erzeugung.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  use omp_lib
  implicit none

  integer :: i, tnr

  call omp_set_num_threads( 3 )

  !$omp parallel private( i )
    !$omp do schedule(static, 3)
      do i = 1, 20
        tnr = omp_get_thread_num() ! Aktuelle Threadnummer
        write( *, * ) 'Thread', tnr, ':', i
      end do
    !$omp end do
  !$omp end parallel
end program bsp

```


Ausgabe:

```
Thread      0 :      1
Thread      0 :      2
Thread      0 :      3
Thread      0 :     10
Thread      0 :     11
Thread      0 :     12
Thread      0 :     19
Thread      0 :     20
Thread      1 :      4
Thread      1 :      5
Thread      1 :      6
Thread      1 :     13
Thread      1 :     14
Thread      1 :     15
Thread      2 :      7
Thread      2 :      8
Thread      2 :      9
Thread      2 :     16
Thread      2 :     17
Thread      2 :     18
```

Eine `do while`-Schleife kann nicht auf diese Art und Weise mittels OpenMP-Direktive parallel ausgeführt werden.

51.8. Die sections-Direktive

Auch die Festlegung, dass bestimmte Programmabschnitte auf je einen Thread verteilt werden sollen, ist möglich. Dazu wird das Konstrukt

```
!$omp sections [optionen]
!$omp section
  block
!$omp section
  block
...
!$omp end sections
```

innerhalb eines `parallel`-Blocks eingesetzt.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  use omp_lib
  implicit none

  integer :: a, b

  a = 20
  b = 30

  call omp_set_num_threads( 3 )

  !$omp parallel shared( a, b )
    !$omp sections
      !$omp section
```

```

        write( *, * ) omp_get_thread_num(), a
        write( *, * ) omp_get_thread_num(), "---"

        !$omp section
        write( *, * ) omp_get_thread_num(), b
        write( *, * ) omp_get_thread_num(), "----"
        !$omp end sections
    !$omp end parallel

! Ausgabe (ifort):
!   0           20
!   0 ---
!   1           30
!   1 ----
end program bsp

```

51.9. Weitere Direktiven

- workshare
- single

51.10. Kombinierte Direktiven

Unmittelbar aufeinanderfolgende Einzeldirektiven können auch in einer einzigen Direktive zusammengefasst werden. Möglich sind

- parallel do
- parallel sections
- parallel workshare

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  use omp_lib
  implicit none

  integer :: i, tnr

  call omp_set_num_threads( 3 )

  !$omp parallel do private(i) schedule(static, 3)
    do i = 1, 20
      tnr = omp_get_thread_num() ! Aktuelle Threadnummer
      write( *, * ) 'Thread', tnr, ':', i
    end do
  !$omp end parallel do
end program bsp

```

Eine zusammengehörende OpenMP-Direktive darf auch auf mehrere Zeilen verteilt werden.

Beispiel: **Fortran 90/95-Code (free source form)**

```

program bsp
  use omp_lib
  implicit none

```

```
integer :: i, tnr

call omp_set_num_threads( 3 )

!$omp parallel do &
!$omp private(i) &
!$omp schedule(static, 3)
do i = 1, 20
    tnr = omp_get_thread_num() ! Aktuelle Threadnummer
    write( *, * ) 'Thread', tnr, ': ', i
end do
!$omp end parallel do
end program bsp
```

51.11. Synchronisation

Bei der parallelen Programmierung können Situationen auftreten, die bei einer seriellen Programmausführung nie passieren würden, z.B. race conditions¹¹⁴. Damit es nicht soweit kommt, bietet OpenMP einige Direktiven zur Synchronisation der Threadausführung.

51.11.1. master-Direktive

```
!$omp master
...
!$omp end master
```

Der eingeschlossene Programmblock wird nur vom Master Thread ausgeführt und von den anderen Threads ignoriert.

51.11.2. critical-Direktive

```
!$omp critical
...
!$omp end critical
```

Dieser Programmteil wird zwar von allen Threads ausgeführt, allerdings ist sichergestellt, dass dies nicht gleichzeitig erfolgt.

51.11.3. atomic-Direktive

```
!$omp atomic
```

Ähnlich zu `critical`. Allerdings gilt dies Direktive nur für eine einzelne unmittelbar nachfolgende spezielle Programmanweisung.

¹¹⁴ <https://de.wikipedia.org/wiki/Race%20Condition>

51.11.4. barrier-Direktive

```
!$omp barrier
```

Sobald ein Thread eine solche Barriere erreicht, wartet er bis alle andere Threads diese Barriere auch erreicht haben. Erst dann geht's weiter.

51.11.5. flush-Direktive

```
!$omp flush
```

Erstellung eines konsistenten Speicherbildes.

51.11.6. ordered-Direktive

```
!$omp do ordered
do ...
...
!$omp ordered
...
!$omp end ordered
...
end do
!$omp end do
```

”Geordnete Ausführung” von do-Schleifen in der gleichen Reihenfolge einer seriellen Abarbeitung.

Beispiel: **Fortran 90/95-Code (free source form)**

```
program bsp
  use omp_lib
  implicit none

  integer :: i, tnr

  call omp_set_num_threads( 3 )

  !$omp parallel private( i )
  !$omp do ordered schedule(static, 3)
  do i = 1, 20
    !$omp ordered
    tnr = omp_get_thread_num() ! Aktuelle Threadnummer
    write( *, * ) 'Thread', tnr, ':', i
    !$omp end ordered
  end do
  !$omp end do
  !$omp end parallel
end program bsp
```

Ausgabe:

Thread	0 :	1
Thread	0 :	2
Thread	0 :	3
Thread	1 :	4
Thread	1 :	5
Thread	1 :	6
Thread	2 :	7
Thread	2 :	8
Thread	2 :	9
Thread	0 :	10
Thread	0 :	11
Thread	0 :	12
Thread	1 :	13
Thread	1 :	14
Thread	1 :	15
Thread	2 :	16
Thread	2 :	17
Thread	2 :	18
Thread	0 :	19
Thread	0 :	20

51.12. Das Modul `omp_lib`

Das Modul `omp_lib` enthält die `interface` für die Routinen von OpenMP. Eine mögliche Form des Modules ist nachfolgend abgebildet. In dem Modul wird die `import`-Anweisung verwendet, die Teil des Standards *Fortran 2003* ist.

Fortran 90/95-Code (free source form)

```

module omp_lib
!
! OpenMP Fortran API v2.5
!
  implicit none

  integer, parameter, private :: sgl = kind( 0.0 )
  integer, parameter, private :: dbl = kind( 0.0d0 )

  integer, parameter, private :: omp_real_kind = dbl
  integer, parameter, private :: omp_integer_kind = sgl
  integer, parameter, private :: omp_logical_kind = sgl
  integer, parameter, private :: omp_lock_kind = dbl
  integer, parameter, private :: omp_nest_lock_kind = dbl

  interface
    subroutine omp_destroy_lock ( var )
      import :: omp_lock_kind
      integer ( kind=omp_lock_kind ), intent(inout) :: var
    end subroutine omp_destroy_lock

    subroutine omp_destroy_nest_lock ( var )
      import :: omp_nest_lock_kind
      integer ( kind=omp_nest_lock_kind ), intent(inout) :: var
    end subroutine omp_destroy_nest_lock

    function omp_get_dynamic ()
      import :: omp_logical_kind
      logical ( kind=omp_logical_kind ) :: omp_get_dynamic
    end function omp_get_dynamic

    function omp_get_max_threads ()

```

```

    import :: omp_integer_kind
    integer ( kind=omp_integer_kind ) :: omp_get_max_threads
end function omp_get_max_threads

function omp_get_nested ()
    import :: omp_logical_kind
    logical ( kind=omp_logical_kind ) :: omp_get_nested
end function omp_get_nested

function omp_get_num_procs ()
    import :: omp_integer_kind
    integer ( kind=omp_integer_kind ) :: omp_get_num_procs
end function omp_get_num_procs

function omp_get_num_threads ()
    import :: omp_integer_kind
    integer ( kind=omp_integer_kind ) :: omp_get_num_threads
end function omp_get_num_threads

function omp_get_thread_num ()
    import :: omp_integer_kind
    integer ( kind=omp_integer_kind ) :: omp_get_thread_num
end function omp_get_thread_num

function omp_get_wtick ()
    import :: omp_real_kind
    real ( kind=omp_real_kind ) :: omp_get_wtick
end function omp_get_wtick

function omp_get_wtime ()
    import :: omp_real_kind
    real ( kind=omp_real_kind ) :: omp_get_wtime
end function omp_get_wtime

subroutine omp_init_lock ( var )
    import :: omp_lock_kind
    integer ( kind=omp_lock_kind ), intent(out) :: var
end subroutine omp_init_lock

subroutine omp_init_nest_lock ( var )
    import :: omp_nest_lock_kind
    integer ( kind=omp_nest_lock_kind ), intent(out) :: var
end subroutine omp_init_nest_lock

function omp_in_parallel ()
    import :: omp_logical_kind
    logical ( kind=omp_logical_kind ) :: omp_in_parallel
end function omp_in_parallel

subroutine omp_set_dynamic ( enable_expr )
    import :: omp_logical_kind
    logical ( kind=omp_logical_kind ), intent(in) :: enable_expr
end subroutine omp_set_dynamic

subroutine omp_set_lock ( var )
    import :: omp_lock_kind
    integer ( kind=omp_lock_kind ), intent(inout) :: var
end subroutine omp_set_lock

subroutine omp_set_nest_lock ( var )
    import :: omp_nest_lock_kind
    integer ( kind=omp_nest_lock_kind ), intent(inout) :: var
end subroutine omp_set_nest_lock

subroutine omp_set_nested ( enable_expr )
    import :: omp_logical_kind
    logical ( kind=omp_logical_kind ), intent(in) :: enable_expr
end subroutine omp_set_nested

```

```

subroutine omp_set_num_threads ( number_of_threads_expr )
  import :: omp_integer_kind
  integer ( kind=omp_integer_kind ), intent(in) ::
number_of_threads_expr
  end subroutine omp_set_num_threads

function omp_test_lock ( var )
  import :: omp_logical_kind, omp_lock_kind
  logical ( kind=omp_logical_kind ) :: omp_test_lock
  integer ( kind=omp_lock_kind ), intent(inout) :: var
  end function omp_test_lock

function omp_test_nest_lock ( var )
  import :: omp_integer_kind, omp_nest_lock_kind
  integer ( kind=omp_integer_kind ) :: omp_test_nest_lock
  integer ( kind=omp_nest_lock_kind ), intent(inout) :: var
  end function omp_test_nest_lock

subroutine omp_unset_lock ( var )
  import :: omp_lock_kind
  integer ( kind=omp_lock_kind ), intent(inout) :: var
  end subroutine omp_unset_lock

subroutine omp_unset_nest_lock ( var )
  import :: omp_nest_lock_kind
  integer ( kind=omp_nest_lock_kind ), intent(inout) :: var
  end subroutine omp_unset_nest_lock
end interface

end module omp_lib

```

51.13. Literatur

- Rohit Chandra, Leonardo Dagum, Dave Kohr, Dror Maydan, Jeff McDonald, Ramesh Menon, *Parallel Programming in OpenMP*, Morgan Kaufmann Publishers, 2001, ISBN-13: 978-1-55860-671-5, ISBN-10: 1-55860-671-8

51.14. Weblinks

- w:OpenMP¹¹⁵
- OpenMP-Homepage¹¹⁶
- OpenMP Fortran Summary¹¹⁷

¹¹⁵ <https://de.wikipedia.org/wiki/OpenMP>

¹¹⁶ <http://www.openmp.org>

¹¹⁷ http://studies.ac.upc.edu/FIB/MP/openmp_f.pdf

Teil VII.

Fortran in Kombination mit anderen Programmiersprachen

52. Fortran und Tcl

Tcl/Tk kann im Zusammenhang mit Fortran zwecks Erstellung einer Tk-Benutzeroberfläche für Fortran-Programme interessant sein. Die zeitkritischen oder mathematisch orientierten Programmteile werden mittels Fortran-Code realisiert. Der Programmcode für die Benutzerschnittstelle wird mittels Tcl/Tk-Skript zur Verfügung gestellt.

52.1. Beispiel

52.1.1. Prinzipskizze

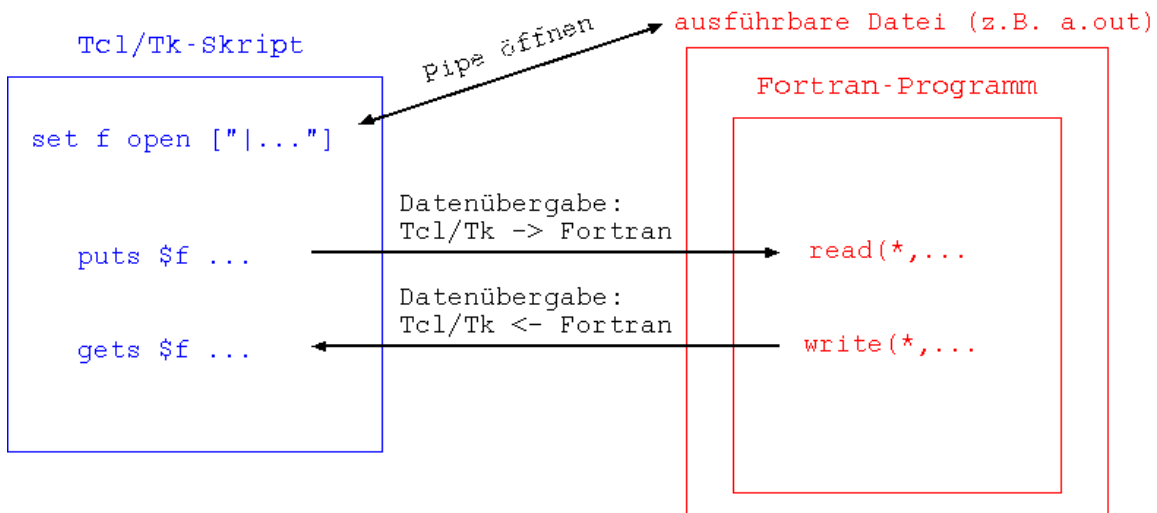


Abb. 54

52.1.2. Tcl/Tk-Code

<syntaxhighlight lang=tcl>

```
1. !/usr/bin/wish  
  
wm title . Sinus ;# Fenstertitel  
  
entry .e1 ;# Eingabefeld button .b1 -text "Hier drücken" -command fcall ;# Schaltfläche  
label .l1 -bg green ;# Textfeld  
  
pack .e1 -padx 10 -pady 5 ;# Widgets packen pack .b1 -padx 10 -pady 5 pack .l1 -padx 10  
-pady 5
```

proc fcall { } { ;# Kommunikation mit Fortran, Ergebnis schreiben

```
    set f [open "|./a.out" r+]                ;# a.out ist das kompilierte und
gelinkte Fortran-Programm
    set val [.e1 get]

    puts $f $val
    flush $f

    gets $f wert
    close $f

    .ll config -text $wert
```

} </syntaxhighlight>

52.1.3. Fortran-Code

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real :: val, sin

  read (*,*) val
  write(*,'(A12,F6.3)') "Ergebnis = ", sin(val)
end
```

52.1.4. Programmausführung

Das Fortran-Programm muss selbstverständlich vorab einmal kompiliert und gelinkt werden. Im Beispielsfall muss die exekutierbare Ausgabedatei *a.out* heißen. Unter Linux wird das Tcl-Skript vor dem ersten Start als ausführbar (-> mittels *chmod*-Befehl) markiert. Der Programmaufruf erfolgt über das Tcl-Skript, das wie ein normales Programm durch Eingabe des Programmnamens gestartet wird.

52.1.5. Ergebnis



Abb. 55

52.2. Alternativen

Tcl/Tk bietet eine Schnittstelle zur Programmiersprache C (`tcl.h`, `tk.h`, `libtcl*.so`, `libtk*.so`). Mit dem im nächsten Kapitel behandelten Fortran-C-Binding kann auf diese C-Funktionen zugegriffen werden. Die Ftcl-Bibliothek nutzt diesen Mechanismus.

53. Weblinks

- Tcl/Tk-Homepage¹¹⁸
- Programming:Tcl (englischsprachiges Wikibook)¹¹⁹
- Ftcl: Using Tcl in Fortran programs and vice versa¹²⁰

¹¹⁸ <http://www.tcl.tk/>

¹¹⁹ <http://en.wikibooks.org/wiki/Programming:Tcl>

¹²⁰ <http://ftcl.sourceforge.net/>

54. Fortran und C

54.1. Interface

Ein Interface ist die Schnittstellendefinition zu einem externen Unterprogramm. Das externe Unterprogramm kann, muss aber nicht in Fortran geschrieben sein.

interface Spezifikation der externen Unterprogramme end interface

Beispiel:

```
interface
! subroutine x(a, b, c) sei ein externes Unterprogramm
subroutine x(a, b, c)
  real, intent(in) :: a
  integer          :: b, c
end subroutine x
end interface

call x(2.5, 1, 3)
```

54.2. Fortran 90/95

In Fortran 90/95 ist der Zugriff auf C-Funktionen nicht standardisiert. Derartige Zugriffe sind compilerabhängig zu lösen. Es sind je nach verwendetem Fortran-Compiler verschiedene Compilerdirektiven zu setzen, Optionen beim Compileraufruf und ähnliches zu beachten. Fortran 2003 bietet eine standardisierte Schnittstelle für den Zugriff auf C-Funktionen.

54.3. gfortran und gcc

54.3.1. Beispiel: "call by value" und "call by reference"

Fortran übergibt die Argumente eines Unterprogrammes standardmäßig "call by reference". Im Zusammenspiel mit C besteht nun das Problem, dass in C Argumente "call by reference" via Pointer oder "call by value" übergeben werden. In *gfortran* ist die Funktion *%val* inkludiert, mit der auch in Fortran der "call by value"-Mechanismus nachgebildet werden kann.

Fortran-Code *bsp.f90*: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none
```



```
interface
  subroutine zahl(a, b)
    integer :: a, b
  end subroutine zahl
end interface

call zahl(%val(5), 7)
end program bsp
```

C-Code *bsp.c*: Programmcode

```
#include <stdio.h>

void zahl(int a, int *b)
{
  printf("%s%d\n", "Ergebnis = ", a * *b);
}
```

Compilieren und Linken:

```
gcc -c -o bsp1.o bsp.c
gfortran -c -fno-underscoring -o bsp2.o bsp.f90
gfortran bsp1.o bsp2.o
```

Ausgabe:

```
Ergebnis = 35
```

54.3.2. Beispiel: Übergabe von Zeichenketten

Fortran Code *bsp.f90*: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  interface
    subroutine hallo(str)
      character(*) :: str
    end subroutine
  end interface

  call hallo("Hallo, Sonne" // char(0))      ! char(0) -> Nulltermination für
C
  call hallo("Hallo, Protuberanz" // char(0)) ! char(0) -> Nulltermination für
C
  call hallo("Hallo, Mond")                  ! keine Nulltermination
! Ausgabe:
! Hallo, Sonne
! Hallo, Protuberanz
! Hallo, Mond
end program bsp
```

C-Code *bsp.c*: Programmcode

```
#include <stdio.h>

void hallo(char *str)
{
    printf("%s\n", str);
}
```

Compilieren und Linken:

```
gcc -c -o bsp1.o bsp.c
g9fortran -c -fno-underscoring -o bsp2.o bsp.f90
gfortran bsp1.o bsp2.o
```

54.3.3. Beispiel: Rückgabewert

Die Rückgabe eines Wertes einfachen Datentyps aus einer C-Funktion nach Fortran stellt kein Problem dar. Es ist einzig zu beachten, dass der Datentyp in C und Fortran übereinstimmt.

Fortran-Code *bsp.f90*: **Fortran 90/95-Code (free source form)**

```
program bsp
    implicit none

    interface
        function zahl(x, y)
            integer :: zahl
            integer :: x, y
        end function
    end interface

    integer :: res

    res = zahl(%val(76), %val(32))
    write(*,*) res
    ! Ausgabe: 108
end program bsp
```

C-Code *bsp.c*: **Programmcode**

```
int zahl(int a, int b)
{
    return (a+b);
}
```

54.4. ifx und gcc

54.4.1. Beispiel: "call by value" und "call by reference"

Fortran-Code *bsp.f90*: **Fortran 90/95-Code (free source form)**

```
program bsp
  implicit none

  interface
    subroutine zahl(a, b)
      !dec$ attributes c :: zahl
      !dec$ attributes reference :: b
      integer :: a, b
    end subroutine zahl
  end interface

  call zahl(5, 7)
end program bsp
```

C-Code *bsp.c*: Programmcode

```
#include <stdio.h>

void zahl(int a, int *b)
{
  printf("%s%d\n", "Ergebnis = ", a * *b);
}
```

Compilieren, Linken:

```
gcc -c -o bsp1.o bsp.c
ifx -c -o bsp2.o bsp.f90
ifx bsp1.o bsp2.o
```

Ausgabe:

```
Ergebnis = 35
```

54.4.2. Beispiel: Übergabe von Zeichenketten

Fortran-Code *bsp.f90*: Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  interface
    subroutine hallo(str)
      !dec$ attributes c :: hallo
      !dec$ attributes reference :: str
      character(*) :: str
    end subroutine
  end interface

  call hallo("Hallo, Sonne" // char(0)) ! char(0) -> Nulltermination
  call hallo("Hallo, Protuberanz"C)      ! C      -> Nulltermination (bei
Intel-Fortran-Compiler)
  call hallo("Hallo, Mond")              ! keine explizite Nulltermination
! Ausgabe:
!  Hallo, Sonne
!  Hallo, Protuberanz
```

```
! Hallo, MondHallo, Sonne
end program bsp
```

C-Code *bsp.c*: Programmcode

```
#include <stdio.h>

void hallo(char *str)
{
    printf("%s\n", str);
}
```

Compilieren und Linken:

```
gcc -c -o bsp1.o bsp.c
ifx -c -o bsp2.o bsp.f90
ifx bsp1.o bsp2.o
```

54.4.3. Beispiel: Rückgabewert

Fortran-Code *bsp.f90*: Fortran 90/95-Code (free source form)

```
program bsp
    implicit none

    interface
        function zahl(x, y)
            !dec$ attributes c :: zahl
            integer :: zahl
            integer :: x, y
        end function
    end interface

    integer :: res

    res = zahl(76, 32)
    write(*,*) res
    ! Ausgabe: 108
end program bsp
```

C-Code *bsp.c*: Programmcode

```
int zahl(int a, int b)
{
    return (a+b);
}
```

54.5. Fortran 2003

Ab *Fortran 2003* ist es viel einfacher auf *C* zu zugreifen, als in *Fortran 95*. Es wurde im *Fortran 2003*-Standard ein intrinsisches Modul namens `iso_c_binding` vorgesehen, das die

zum Zugriff auf C-Programme nötigen Elemente enthält. Fortran 2018 und 2023 brachten einige Erweiterungen zu diesem Thema. Darauf wird aber vorerst hier nicht eingegangen.

Folgende Beispiele wurden unter dem Betriebssystem OpenSUSE Leap 15.6 getestet. Zum Einsatz kamen der gfortran-Compiler (7.5.0, 2017) und Intels ifx (2025.2.0).

54.6. Ein einfaches Beispiel

Fortran 2003-Code: bsp.f90 **Fortran 2003-Code**

```
program bsp
  implicit none

  interface
    function addition( a, b ) bind( c )
      use, intrinsic :: iso_c_binding
      real( kind = c_float ), value :: a
      real( kind = c_float ), value :: b
      real( kind = c_float )          :: addition
    end function addition
  end interface

  write (*,*) addition( 2.5, 3.3 )

! Ausgabe:  5.8
end program bsp
```

C-Code: bsp.c **Programmcode**

```
float addition(float a, float b)
{
  return (a + b);
}
```

Makefile: **Programmcode**

```
FC = gfortran # oder ifx
CC = gcc

bsp: bsp_c.o bsp_f90.o
    $(FC) -o bsp bsp_c.o bsp_f90.o

bsp_c.o: bsp.c
    $(CC) -c -o bsp_c.o bsp.c

bsp_f90.o: bsp.f90
    $(FC) -c -o bsp_f90.o bsp.f90

.PHONY: clean
clean:
    rm *.o
```

Was ist neu gegenüber Fortran 95?

- `bind( c )` ... bind-Attribut, stellt unter anderem die Interoperabilität mit C bezüglich Prozedurnamenskonventionen nach der Übersetzung sicher
- `use, intrinsic :: iso_c_binding` ... Einbindung des intrinsischen Moduls `iso_c_binding`
- `real( kind = c_float )` ... C-Datentyp `float`
- `value` ... call by value

54.7. Datentyp-Zuordnung

Das `iso_c_binding`-Modul stellt benannte Konstanten zur Verfügung, die bei Fortran-Datentypen als `kind`-Wert zu verwenden sind, um den jeweiligen C-Datentyp zu charakterisieren. Weist eine solche Konstante einen negativen Wert auf, dann ist keine Entsprechung von Fortran-Datentyp zu C-Datentyp vorhanden.

Fortran-Datentyp	Benannte <code>iso_c_binding</code> -Konstante (kind-Wert)	C-Datentyp
integer	<code>c_int</code>	<code>int</code>
	<code>c_short</code>	<code>short int</code>
	<code>c_long</code>	<code>long int</code>
	<code>c_long_long</code>	<code>long long int</code>
	<code>c_signed_char</code>	<code>signed char, unsigned char</code>
	<code>c_size_t</code>	<code>size_t</code>
	<code>c_int8_t</code>	<code>int8_t</code>
	<code>c_int16_t</code>	<code>int16_t</code>
	<code>c_int32_t</code>	<code>int32_t</code>
	<code>c_int64_t</code>	<code>int64_t</code>
	<code>c_int_least8_t</code>	<code>int_least8_t</code>
	<code>c_int_least16_t</code>	<code>int_least16_t</code>
	<code>c_int_least32_t</code>	<code>int_least32_t</code>
	<code>c_int_least64_t</code>	<code>int_least64_t</code>
	<code>c_int_fast8_t</code>	<code>int_fast8_t</code>
	<code>c_int_fast16_t</code>	<code>int_fast16_t</code>
	<code>c_int_fast32_t</code>	<code>int_fast32_t</code>
	<code>c_int_fast64_t</code>	<code>int_fast64_t</code>
	<code>c_intmax_t</code>	<code>intmax_t</code>
	<code>c_intptr_t</code>	<code>intptr_t</code>
real	<code>c_float</code>	<code>float</code>
	<code>c_double</code>	<code>double</code>
	<code>c_long_double</code>	<code>long double</code>

com- plex	c_float_complex	float _Complex
	c_double_complex	double _Complex
	c_long_double_complex	long double _Complex
logical	c_bool	_Bool
character	c_char	char
Quelle: J3/04-007 Fortran 2003 Working Draft		

Das `iso_c_binding`-Modul stellt keine speziellen kind-Werte für `unsigned`-Integer-Datentypen zur Verfügung. Im Bedarfsfall sind die entsprechenden kind-Werte für die `signed`-Datentypen zu verwenden.

Nicht jeder Fortran-Compiler unterstützt alle genannten C-Datentypen und die unterstützten Datentypen können sich compilerspezifisch in der Byteanzahl unterscheiden.

54.8. „call by value“ vs. „call by reference“

Im einführenden Beispiel¹²¹ wurden die Funktionsargumente „call by value“ übergeben. Das Variablenattribut `value` stellt dieses Verhalten sicher. Wird dieses Attribut nicht gesetzt, so gilt „call by reference“.

Beispiel:

Fortran-Code *bsp.f90*: **Fortran 2003-Code**

```
program bsp
  implicit none

  interface
    subroutine zahl( a, b ) bind( c )
      use, intrinsic :: iso_c_binding
      integer( kind=c_int ), value :: a
      integer( kind=c_int )      :: b
    end subroutine zahl
  end interface

  call zahl(5, 7)

  ! Ausgabe:
  !   Ergebnis = 35
end program bsp
```

C-Code *bsp.c*: **Programmcode**

```
#include <stdio.h>

void zahl(int a, int *b)
{
  printf("%s%d\n", "Ergebnis = ", a * *b);
}
```

¹²¹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran_und_C%23Ein_einfaches_Beispiel%20

54.9. Globale C-Variablen

Muss in Fortran auf globale C-Variablen zugegriffen werden, so sind diese im Gültigkeitsbereich eines Fortran-Modul zu spezifizieren.

54.10. Felder

Interoperabilität zwischen Fortran und C ist nur mit Feldern definierter Größe gegeben. Allozierbare Felder oder Zeigerfelder sind nicht erlaubt.

Beispiel:

Fortran 2003-Code

```
program bsp
  implicit none

  integer, dimension( 3 ) :: a = (/ 1, 2, 3 /)

  interface
    subroutine feld1( f ) bind( c )
      use, intrinsic :: iso_c_binding

      integer( c_int ), dimension(*) :: f
    end subroutine feld1
  end interface

  write (*,*) "Feld a vorher: ", a
  call feld1( a )
  write (*,*) "Feld a nachher: ", a

  ! Ausgabe:
  !   Feld a vorher:  1 2 3
  !   Feld a nachher: 999 2 3
end program bsp
```

Programmcode

```
void feld1(int f[])
{
    f[0] = 999;
}
```

54.11. Übergabe von Zeichenketten

Beispiel:

Fortran-Code *bsp.f90*: Fortran 2003-Code

```
program bsp
  use, intrinsic :: iso_c_binding
  implicit none

  interface
```



```
subroutine string1( str_in ) bind( c )
  use, intrinsic :: iso_c_binding

  character( kind=c_char, len=1 ) :: str_in ! Ungültiges Fortran 2003 da nur
len=1 erlaubt ist
  end subroutine string1
end interface

call string1( c_char_"Greetings from Fortran" // c_null_char )

! Ausgabe:
!  "Greetings from Fortran"
end program bsp
```

C-Code *bsp.c*: Programmcode

```
#include <stdio.h>

void string1(char str_in[])
{
  printf("%s \n", str_in);
}
```

Für `len=1` funktioniert das Beispiel mit allen angeführten Compilern. Das `len`-Attribut kann im Übrigen auch weggelassen werden; einige unterstützen als compilerspezifische Erweiterungen auch andere Längen. Im Fortran 2003-Working-Draft wird in „Note 15.23“ eine andere Möglichkeit für die Übergabe von Zeichenketten angeführt. Diese folgt im wesentlichen der Annahme:

C-Zeichenketten sind `char`-Felder mit einem terminierenden `\0`-Zeichen und können deshalb in Fortran als Felder vom Datentyp `character` mit einem `kind`-Wert `c_char` angesprochen werden.

Daher wird im Interface der Dummy-Parameter in Form eines Feldes aus Zeichen des Typs `c_char` deklariert. Das entspräche auch genau der aufzurufenden C-Funktion. Allerdings ist solcherart verfasster Code dann nicht mit allen Compilern übersetzbar.

Beispiel:

Fortran-Code *bsp.f90*: Fortran 2003-Code

```
program bsp
  use, intrinsic :: iso_c_binding
  implicit none

  interface
    subroutine string1( str_in ) bind( c )
      use, intrinsic :: iso_c_binding

      character( kind=c_char ), dimension(*) :: str_in
    end subroutine string1
  end interface

  call string1( c_char_"Greetings from Fortran" // c_null_char )

! Ausgabe:
!  "Greetings from Fortran"
end program bsp
```

Benannte `iso_c_binding`-Konstanten für Zeichen mit spezieller Bedeutung in C:

Benannte Konstante	Wert
<code>c_null_char</code>	<code>'\0'</code>
<code>c_alert</code>	<code>'\a'</code>
<code>c_backspace</code>	<code>'\b'</code>
<code>c_form_feed</code>	<code>'\f'</code>
<code>c_new_line</code>	<code>'\n'</code>
<code>c_carriage_return</code>	<code>'\r'</code>
<code>c_horizontal_tab</code>	<code>'\t'</code>
<code>c_vertical_tab</code>	<code>'\v'</code>

54.12. Enumerationen

Mit Fortran 2003 sind auch in dieser Programmiersprache Enumerationen (Aufzählungstypen) möglich. Die Werte in einer solchen Enumeration besitzen einen `integer`-Datentyp. Der `kind`-Wert ist nicht festgelegt, wird jedoch so gewählt, dass im Rahmen der jeweiligen Möglichkeiten alle Enumeratoren erfasst sind.

Die von C-Enumerationen bekannten Eigenschaften gelten gleichermaßen für Fortran-Enumerationen, z.B.:

- ohne explizite Zuweisung von Werten wird mit dem Wert 0 gestartet.
- ohne explizite Zuweisung von Werten wird in der Anordnungsreihenfolge der Elemente sukzessiv immer um 1 hochgezählt.
- Wurde dem Vorgängerelement eine Ganzzahl zugewiesen, dem Element jedoch nicht, so ist der Wert dieses Elementes die dem Vorgängerelement zugeordnete Ganzzahl + 1.

Beispiel:

Fortran-Code *bsp.f90*: **Fortran 2003-Code**

```

program bsp
  implicit none

  enum, bind(c)
    enumerator :: M0=1, DI=2, MI=3, D0=4, FR=5, SA=6, S0=7
  end enum

  interface
    subroutine tag( w ) bind( c )
      use, intrinsic :: iso_c_binding

      integer( c_int ), value :: w
    end subroutine tag
  end interface

  call tag(MI);

! Ausgabe:
!   Mittwoch
end program bsp

```

C-Code *bsp.c*: **Programmcode**

```
#include <stdio.h>

typedef enum {
    MO=1, DI=2, MI=3, DO=4, FR=5, SA=6, SO=7
} wochentag;

void tag(wochentag w)
{
    switch(w)
    {
        case MO:
            printf("Montag\n");
            break;
        case DI:
            printf("Dienstag\n");
            break;
        case MI:
            printf("Mittwoch\n");
            break;
        case DO:
            printf("Donnerstag\n");
            break;
        case FR:
            printf("Freitag\n");
            break;
        case SA:
            printf("Samstag\n");
            break;
        case SO:
            printf("Sonntag\n");
            break;
        default:
            printf("Kein Tag\n");
    }
}
```

54.13. Zeiger

Für das C-Zeiger-Handling stellt Fortran 2003 im `iso_c_binding`-Modul den Datenverbund `c_ptr` und einige Unterprogramme bereit.

UP	Beschreibung
<code>l = c_associated (c_ptr1 [, c_ptr2])</code>	Prüft den Assoziationsstatus von <code>c_ptr1</code> . Diese Funktion ermittelt also, ob <code>c_ptr1</code> überhaupt assoziiert ist, oder ob <code>c_ptr1</code> mit <code>c_ptr2</code> assoziiert ist.
<code>c_ptr = c_loc (x)</code>	Gibt die Adresse von <code>x</code> zurück.
<code>c_f_pointer (c_ptr, fptr [, shape])</code>	Wandelt einen C-Zeiger <code>c_ptr</code> in einen Fortran-Zeiger <code>fptr</code> um. Die Vorgabe von <code>shape</code> ist nur dann erforderlich und möglich, wenn <code>fptr</code> ein Feld ist. <code>fptr</code> ist <code>intent(inout)</code> .

Beispiel:

Fortran-Code *bsp.f90*: Fortran 2003-Code

```

module cglob
  use, intrinsic :: iso_c_binding

  type( c_ptr ), bind( c )          :: ptr1, ptr2, ptr3
  real( kind=c_float ), target, bind( c ) :: a, b
end module cglob

program bsp
  use cglob
  implicit none

  real, pointer :: p => null()

! *** Zuordnungsstatus ***
  write (*,*) "Ist ptr1 assoziiert? ", c_associated( ptr1 )
  write (*,*) "Ist ptr2 assoziiert? ", c_associated( ptr2 )
  write (*,*) "Ist ptr2 mit ptr3 assoziiert? ", c_associated( ptr2, ptr3 )
  write (*,*) "Ist ptr2 mit &a assoziiert? ", c_associated( ptr2, c_loc(a) )

! *** Wert von ptr2? ***
  call c_f_pointer( ptr2, p )
  write (*,*) "Wert von ptr2? ", p

! *** ptr1 neu setzen ***
  ptr1 = c_loc( b )

! *** Wert von ptr1 ***
  call c_f_pointer( ptr1, p )
  write (*,*) "Wert von ptr1? ", p

! Ausgabe:
!   Ist ptr1 assoziiert?  F
!   Ist ptr2 assoziiert?  T
!   Ist ptr2 mit ptr3 assoziiert?  F
!   Ist ptr2 mit &a assoziiert?  T
!   Wert von ptr2 (bzw. p)?  5555.66
!   Wert von ptr1 (bzw. p)?  -12.3
end program bsp

```

C-Code *bsp.c*: Programmcode

```

float a = 5555.66;
float b = -12.3;

float *ptr1 = 0;
float *ptr2 = &a;
float *ptr3 = &b;

```

Nun ist die Zeiger-Verwendung wie im obigen Beispiel skizziert, eher die Ausnahme als die Regel. Wesentlich häufiger trifft man Zeiger in C-Bibliotheken im Zusammenhang mit Funktionen an. Dort dienen sie aus Anwendersicht als **return**-Werte oder der Parameterübergabe „call-by-reference“. Oft sind dabei auch Zeiger auf Strukturen im Spiel. Näheres dazu folgt im nächsten Abschnitt.

Für C-Zeiger auf den Wert `NULL` bietet das ISO-C-Binding-Modul die Konstante `C_NULL_PTR`.

54.14. Datenverbund

Die einzelnen Datenelemente der Struktur / des Datenverbundes müssen in Fortran und C selbstverständlich äquivalenten Datentyp aufweisen. Die Bezeichnungen der jeweiligen Datenelemente müssen nicht beibehalten werden. Sehr wohl müssen aber die Positionen der Einzelelemente im Fortran-Datenverbund der nachzubildenden C-Struktur entsprechen.

Für die Gewährleistung der Interoperabilität darf ein an C gebundener Datenverbund in Fortran (**struct**) **keine** Zeiger mit dem **pointer**-Attribut oder allozierbaren Felder als Datenelemente enthalten. Sind in der C-Struktur Zeiger vorhanden, so sind diese im entsprechenden Fortran-Datenverbund mittels **type(c_ptr)** zu beschreiben.

Für Bitfelder oder Unions gibt es in Fortran keine entsprechenden Gegenstücke.

Beispiel:

Fortran 2003-Code

```
module cglob
  use, intrinsic :: iso_c_binding

  type, bind( c ) :: verbund
    integer( kind=c_int ) :: a
    real( kind=c_float ) :: b
  end type verbund

  type(verbund), bind( c ) :: v
end module cglob

program bsp
  use cglob
  implicit none

  interface
    subroutine set_v( a_in, b_in) bind( c )
      use, intrinsic :: iso_c_binding

      integer( c_int ), value :: a_in
      real( c_float ), value :: b_in
    end subroutine set_v

    subroutine print_v() bind( c )
    end subroutine print_v
  end interface

  call set_v( 5, -10.9)
  call print_v()
  write (*,*) "Fortran-Ausgabe: ", v
  v%a = 99
  call print_v()
  write (*,*) "Fortran-Ausgabe: ", v

  ! Ausgabe:
  !   C-Ausgabe: 5  -10.900000
  !   Fortran-Ausgabe: 5 -10.9
  !   C-Ausgabe: 99 -10.900000
  !   Fortran-Ausgabe: 99 -10.9
end program bsp
```

Programmcode

```
#include <stdio.h>

typedef struct {
    int a;
    float b;
} verbund;

verbund v;

void set_v(int a_in, float b_in)
{
    v.a = a_in;
    v.b = b_in;
}

void print_v()
{
    printf("C-Ausgabe: %i  %f\n", v.a, v.b);
}
```

Beispiel: Struktur als Rückgabewert und Argument einer Funktion - Teil 1

Bekannt seien zwei Funktionsprototypen in der Programmiersprache C:

```
XYZ *get_xyz();
```

und

```
void set_xyz(XYZ *x);
```

XYZ sei ein C-**struct**, dessen Inhalt hier nicht näher interessiert.

Die Schnittstelle, mit dem in Fortran die C-Anbindung der Funktionen realisiert wird, könnte so aussehen:

```
interface
  function get_xyz() bind( c )
    use, intrinsic :: iso_c_binding
    type( c_ptr ) :: get_xyz
  end function get_xyz

  subroutine set_xyz( x ) bind( c )
    use, intrinsic :: iso_c_binding
    type( c_ptr ), value :: x
  end subroutine set_xyz
end interface
```

Und schon können diese Funktionen auch in Fortranroutinen direkt Anwendung finden, z.B:

```
type( c_ptr ) :: x56

x56 = get_xyz()
call set_xyz( x56 )
```

Wie man anhand dieses Beispiels erkennt, ist die interne Struktur von `XYZ` in diesem Fall vollkommen irrelevant. Es ist nicht nötig, in der API-Dokumentation oder in den C-Headerdateien nachzuforschen, wie der C-`struct` konkret aufgebaut ist. Für den Fortran-Programmierer ist dies immer ein `type( c_ptr )` (natürlich nur, solange Zeiger auf Strukturen gefordert sind). Wird ein Zeiger des Typs `c_ptr` nur via Fortranprogramm zwischen verschiedenen C-Funktionen übergeben oder zurückgeliefert, so ist auch keine Umwandlung in einen Fortran-Zeiger erforderlich und wäre mangels genauer Kenntnis des Aufbaus von `XYZ` hier auch nicht möglich. Erst dann, wenn von Fortran aus auf einzelne Elemente eines C-`struct` zugegriffen werden soll oder mit Kopien der Struktur hantiert wird, muss diese Struktur in Fortran nachgebaut werden.

Beispiel: Struktur als Rückgabewert und Argument einer Funktion - Teil 2

Fortran 2003-Code

```
module test
  use, intrinsic :: iso_c_binding
  implicit none

  type, bind( c ) :: A
  ! Das funktioniert nicht mit allen Kompilern; versuchen Sie ansonsten:
  !   type A
  !     sequence
  !     integer( c_int ) :: xc, yc
  !     type( c_ptr )    :: str
  end type

  interface
    function get_a() bind( c )
      use, intrinsic :: iso_c_binding
      type( c_ptr ) :: get_a
    end function get_a

    subroutine print_a( x ) bind( c )
      use, intrinsic :: iso_c_binding
      type( c_ptr ), value :: x
    end subroutine print_a
  end interface
end module test

program main
  use test
  implicit none

  type( c_ptr )           :: x
  type( A ), pointer      :: fptr
  character( len=9 ), pointer :: strptr

  x = get_a()

  ! C-Ausgabe
  call print_a( x )
```

```

!  Ausgabe:
!    x = 5
!    y = 997
!    str = Irgendwas

! Fortran-Ausgabe
call c_f_pointer( x, fptr)
call c_f_pointer( fptr%str, strptr ) ! <--- Fehlerquelle bei gfortran
write(*,*) fptr%xc, fptr%yc, strptr
!  Ausgabe
!    5 997 Irgendwas
end program main

```

Programmcode

```

#include <stdlib.h>
#include <stdio.h>

typedef struct
{
    int x;
    int y;
    const char *str;
} A;

A *get_a()
{
    A *a = ( A * )malloc( sizeof( A ) );

    a->x = 5;
    a->y = 997;
    a->str = "Irgendwas";

    return a;
}

void print_a( A *v )
{
    printf("x = %d\n", v->x);
    printf("y = %d\n", v->y);
    printf("str = %s\n", v->str);
}

```

Bei der Umwandlung des C-String-Zeigers in einen Fortran-Zeiger ist die genaue Stringlänge erforderlich. Ist diese zu klein gewählt, so ist nur ein Teil des C-Strings über den Fortran-Zeiger sichtbar. Wird diese Stringlänge zu groß gewählt, dann wird diese Länge auch voll ausgenutzt und nach dem eigentlich gewünschten String folgen noch ein Menge x-beliebige Zeichen, da in Fortran der C-String-Begrenzer `\0` nicht die Bedeutung wie in C besitzt.

Im obigen Beispiel wurde die Speicherplatzreservierung und Wertebelegung für eine Variable des Typs `A` in einer C-Funktion erledigt. Nun soll diese Aufgabe im Fortran-Programm wahrgenommen werden und dann diese in Fortran mit Werten belegte Variable an die `print_a`-Funktion übergeben werden. Kein Problem, möchte man im ersten Augenblick meinen. Doch der `c_ptr`-Typ im Datenverbund für den Zeiger auf eine Zeichenkette macht Probleme. Der `g95`-Compiler duldet bspw. in der `c_loc`-Funktion keine Zeichenketten mit einer Länge größer als 1. Der Compiler "castet" auch nicht von selbst im Datenverbundkon-

struktur eine Zeichenkette in den Typ `c_ptr`. Eine Möglichkeit, diese Probleme zu umgehen, besteht darin, einfach einer C-Funktion einen String zu übergeben und die Adresse dieses Strings zurückgeben zu lassen. Diese Variante wird auch im folgenden Beispiel verwendet.

Beispiel: Struktur als Rückgabewert und Argument einer Funktion - Teil 3

Fortran 2003-Code

```
module test
  use, intrinsic :: iso_c_binding
  implicit none

  type, bind( C ) :: A
    integer( c_int ) :: xc, yc
    type( c_ptr )    :: str
  end type

  type( A ), target :: a1

  interface
    subroutine print_a( x ) bind( c )
      use, intrinsic :: iso_c_binding
      type( c_ptr ), value :: x
    end subroutine print_a

    function c_string_addr( s ) bind( c )
      use, intrinsic :: iso_c_binding
      character( c_char ) :: s
      type( c_ptr )        :: c_string_addr
    end function c_string_addr
  end interface
end module test

program main
  use test
  implicit none

  character( len = 30 ) :: str

  str = "Das ist ein Beispiel" // C_NULL_CHAR

  a1 = A( 14, 56, c_string_addr( str ) )

  call print_a( c_loc( a1 ) )
!  Ausgabe:
!    x = 14
!    y = 56
!    str = Das ist ein Beispiel
end program main
```

Programmcode

```
#include <stdlib.h>
#include <stdio.h>

typedef struct
{
  int x;
  int y;
  const char *str;
} A;
```

```

void print_a( A *v )
{
    printf("x = %d\n", v->x);
    printf("y = %d\n", v->y);
    printf("str = %s\n", v->str);
}

char *c_string_addr( char *str )
{
    return str;
}

```

54.15. Problematische Kamelhöckerschreibweise?

C ist case-sensitive, Fortran nicht. Oft sind Funktionen in C-Bibliotheken in der w:Binnenmajuskel¹²² vorzufinden, z.B.

```
void writeHallo();
```

Die Einbindung dieser C-Funktion in ein Fortran-Programm mit

```

interface
    subroutine writeHallo() bind(c)
    end subroutine writeHallo
end interface

```

könnte beim Linken eine Fehlermeldung der Art

```
undefined reference to `writehallo'
```

liefern.

Was ist passiert? Wie schon angedeutet, ist Fortran case-insensitive. Für einen Fortran-Compiler ist `writeHallo` gleich `writehallo` oder `WRITEHaLLo`. Für einen C-Compiler wären das alles unterschiedliche Funktionsbezeichner. Also wandelt der Fortran-Compiler die "unnütze" Groß-/Kleinschreibung in eine einheitliche Schreibweise um und der C-Compiler nicht. Dementsprechend findet der Linker auch keine Funktion `writehallo`. Es gibt eben nur die nicht-identische C-Funktion `writeHallo`.

Aber auch für dieses Problem gibt es in Fortran eine Lösung. Mit dem `bind`-Attribut können zusätzlich auch noch Labels (Benennungen) vergeben werden, z.B.:

```
bind( c, name="xyz" )
```

¹²² <https://de.wikipedia.org/wiki/Binnenmajuskel>

Solche Labels in Stringform sind nicht von der "Kopf ab"-Strategie des Fortran-Compilers betroffen. Das ist keine C-Binding-Spezialität, sondern trifft generell für alle Zeichenketten zu. Ein

```
interface
  subroutine writeHallo() bind(c, name="writeHallo")
  end subroutine writeHallo
end interface
```

sollte das beschriebene Link-Problem lösen.

54.16. Sonstiges

- Auch C-Funktionszeiger kennt das `iso_c_binding`-Modul. Zu diesem Zwecke gibt es den Datenverbund `c_funptr`, einige Umwandlungsfunktionen ähnlich jenen im Abschnitt Zeiger¹²³ behandeln und die Konstante `C_NULL_FUNPTR`.

¹²³ Kapitel 38 auf Seite 247

55. Fortran und Python

F2PY

F2py (Fortran to Python interface generator) ist ein Wrapper, der Fortran-Module, -Subroutinen oder -Funktionen in sogenannte SharedObject-Dateien kompiliert, die dann in Python als normales Modul importiert werden können. Einsatzgebiete sind vor allem numerische Berechnungen, in denen sowohl die Geschwindigkeit der Fortranroutinen, aber auch die Flexibilität von Pythonscripten genutzt werden soll.

55.1. Voraussetzungen

Erforderlich für die Installation von f2py sind einerseits eine Python-Version ab 2.0 zusammen mit den Erweiterungsmodulen NumPy und SciPy und außerdem ein Fortran-Compiler. Genaue Informationen finden Sie auf der Homepage des f2py-Projekts, die unten angegeben ist. Es ist sehr wichtig, dass Sie zunächst alle Komponenten installiert haben, bevor Sie fortfahren. Die Erfahrung hat gezeigt, dass dies manchmal gar nicht so einfach ist. Um zu überprüfen, ob Ihr f2py funktionsfähig ist, tippen Sie einfach *f2py* in die Konsole ein. Wünschenswerterweise sollte dies einige Informationen über das Programm auf Ihrem Bildschirm anzeigen.

Die aktuelle Spyder-IDE (ein Entwicklungspaket für Python, erhältlich bei <https://www.spyder-ide.org>, siehe z.B. aber auch Ing_Mathematik:_Python#Spyder_IDE¹²⁴) bringt f2py von Haus aus mit (zumindest die Variante für das Linux-Betriebssystem).

55.2. Beispiel

Als Testbeispiel wählen wir ein kleines Programm namens punktrechnung.f90. Es enthält ein Modul und eine Subroutine zur Berechnung von Division und Multiplikation. Wir wollen diese Routinen später von Python aus aufrufen können und die Funktionalitäten nutzen.

55.2.1. Fortran Code

Fortran 90/95-Code (free source form)

```
!File: punktrechnung.f90
module punktrechnung
  real,parameter::pi=3.14
```

¹²⁴ https://de.wikibooks.org/wiki/Ing_Mathematik%3A_Python%23Spyder_IDE

```
contains

subroutine division(x,y)
  integer::x,y
  print*, "x/y = ",x/y
end subroutine division

end module punktrechnung

subroutine multi(x,y)
  integer::x,y
  print*, "x*y= ",x*y
end subroutine multi
```

55.2.2. Wrappen

Mit dem Befehl `f2py -c -m meinmodul punktrechnung.f90` erstellen wir eine Objekt-Datei mit der Endung `.so`, die von Python als Modul importiert werden kann. Das Modul trägt den Namen "meinmodul" und es beinhaltet alle Funktionalitäten der Datei `punktrechnung.f90`.

55.2.3. Zugriff

Im interaktiven Modus von Python können wir dieses Modul nun importieren und die Funktionen nutzen:

```
>>> import meinmodul
>>> meinmodul.multi(5,4)
x*y=      20
>>> meinmodul.punktrechnung.division(36,6)
x/y =      6
```

55.3. Weblinks

- F2PY - Calling Fortran routines from Python¹²⁵
- F2PY Users Guide and Reference Manual¹²⁶

¹²⁵ <https://www.numfys.net/howto/F2PY/>

¹²⁶ <https://numpy.org/doc/stable/f2py/>

Teil VIII.

Fortran-„Dialekte“

56. F

56.1. Was ist F?

F ist ein Fortran 90/95-Subset. Das bedeutet auch, dass sich F-Programmcodes immer mit Fortran 90/95-Compilern übersetzen lassen sollte. Umgekehrt ist das natürlich oft nicht möglich. Grob gesagt, wurde bei F auf die bedingungslose Kompatibilität mit FORTRAN 77-Code gepfiffen und nur die moderneren Fortran 90/95-Sprachmittel erlaubt. Die überbordenden Möglichkeiten und Schreibweisen zur Erreichung ein und desselben Ziels wurden also deutlich eingebremst. F ist somit ein bisschen leichter erlernbar als Fortran 90/95 in seiner Gesamtheit und führt durch restriktivere Regeln automatisch zu einem etwas stringenteren Programmierstil. Und das war bei der Einführung von F auch das Ziel: Einsatz von F in der Ausbildung und Lehre, wenngleich natürlich von den F-Schöpfern der Einsatz in praktischen Projekten auch gerne gesehen worden wäre.

56.2. Einige Unterschiede zu Fortran 90/95

Im Nachfolgenden werden Fortran 90/95- und F-Beispiele gegenüber gestellt. Das soll aber nicht bedeuten, dass die Fortran 90/95-Beispiele zwingend so formuliert werden müssen oder optimaler Fortran 90/95-Code sind. Es soll nur gezeigt werden, welche Freiheiten Fortran 90/95 im Gegensatz zu F erlaubt. F ist ein Subset von Fortran 90/95 und die als F-Code deklarierten Beispiele enthalten somit auch immer gültigen Fortran 90/95-Code.

56.2.1. Diverses

- Strengere Regeln bei **end**-Statements für Hauptprogramm, Unterprogramme und Module:

Fortran 90/95	F
<pre>program bsp ! ... end</pre>	<pre>program bsp ! ... end program bsp</pre>

56.2.2. Datentypen

- In F gibt es den Datentyp **double precision** nicht. Er kann aber problemlos wie in Fortran 90 /95 mittels **real** und **kind**-Attribut ersetzt werden.
- Die alte (FORTRAN 77)-Form ohne Doppel-Doppelpunkt zwecks Variablendeklaration ist in F nicht erlaubt:

Fortran 90/95	F
<pre> program bsp real r r = 12.5 ! ... end program bsp </pre>	<pre> program bsp real :: r r = 12.5 ! ... end program bsp </pre>

- In F gibt es keine impliziten Datentypvereinbarungen. Der Datentyp muss immer explizit festgelegt werden:

Fortran 90/95	F
<pre> program bsp i = 12 ! ... end program bsp </pre>	<pre> program bsp integer :: i = 12 ! ... end program bsp </pre>

56.2.3. Ein- / Ausgabe

- Die vereinfachte Schreibweise `write( *, * )` und `read( *, * )` funktioniert bei F nicht. F akzeptiert Apostrophe nicht als Zeichenkettenbegrenzer:

Fortran 90/95	F
<pre> program bsp write(*, *) 'Hallo Welt!' ! Alternativ auch: print *, 'Hallo Welt!' end program bsp </pre>	<pre> program bsp write(unit = *, fmt = *) "Hallo Welt!" ! Alternativ auch: print *, "Hallo Welt!" end program bsp </pre>

- Kein `format`, keine Marken:

Fortran 90/95	F
<pre> program bsp real :: a = 30.0 write(*, fmt = 999) a 999 format(F10.3) end program bsp </pre>	<pre> program bsp real :: a = 30.0 write(unit = *, fmt = "(F10.3)") a end program bsp </pre>

56.2.4. Schleifen

- F kennt keine `do-while`-Schleife.

56.2.5. Funktionen und Subroutinen

- Egal, ob die Parameterliste leer ist oder nicht, in F müssen immer die Klammern im Anschluss an den Unterprogrammnamen geschrieben werden

- Externe Unterprogramme sind nicht erlaubt.

Fortran 90/95	F
<pre> program bsp call schreibe end program bsp subroutine schreibe write(*, *) "Hallo" end subroutine schreibe </pre>	<pre> program bsp call schreibe() contains subroutine schreibe() write(unit = *, fmt = *) "Hallo" end subroutine schreibe end program bsp </pre>

56.2.6. Module

F ist restriktiver bei der Verwendung von Modulen als Fortran 90/95. Für Modulvariablen und -unterprogramme muss immer ein Zugriffsspezifizierer `public` oder `private` vorgegeben werden.

Fortran 90/95	F
<pre> module modu real :: pi = 3.1416 contains subroutine schreibe() write(*, *) "Hallo" end subroutine schreibe end module modu program bsp use modu write(*, *) pi call schreibe() end program bsp </pre>	<pre> module modu private real, public :: pi = 3.1416 public :: schreibe contains subroutine schreibe() write(unit = *, fmt = *) "Hallo" end subroutine schreibe end module modu program bsp use modu write(unit = *, fmt = *) pi call schreibe() end program bsp </pre>

Weitere Unterschiede sind mittels angegebener Weblinks auffindbar bzw. dem F-Manual zu entnehmen.

56.3. Weblinks

- F Programming Language Homepage¹²⁷
- The F language¹²⁸

¹²⁷ <https://web.archive.org/web/20150109130310/http://www.fortran.com/F/index.html>

¹²⁸ <https://web.archive.org/web/20140424064256/https://www.cisl.ucar.edu/zine/96/fall/articles/2.F.language.html>

57. Ratfor

57.1. Allgemeines

Ratfor (**R**ational **F**ortran) ist zum Großteil eine Mischung aus C und Fortran. Es entstand aus der Begeisterung Brian W. Kernighans¹³⁰ für die Programmiersprache C. In einer Zeit, als C seinen Siegeszug antrat und Programmierer mit den zum Teil noch archaischen Sprachmerkmalen von FORTRAN 66 und 77 konfrontiert waren, hatte er die fixe Idee, dass auch Fortran-Programmierer mit der Syntax von C beglückt werden müssen. Ratfor-Code wurde dann per Präprozessor in Fortran-Code umgewandelt, um danach den Gang alles Fortran-irdischen zu gehen und per schnödem Fortran-Compiler in Maschinencode übersetzt zu werden. Ratfor dürfte sich einer gewissen Popularität erfreut haben. Darauf deutet auch hin, dass sogar für Fortran 90 ein entsprechender Ratfor-Präprozessor in Form eines Perl-Skripts geschaffen wurde. Gerüchten zufolge wurde in den '80er Jahren des vergangenen Jahrhunderts als möglicher Nachfolger von Ratfor ein Ratfiv¹³² realisiert.

Hier nun einige der wesentlichen Ratfor-Sprachmerkmale:

- Die fixe Zeilenform aus FORTRAN 77 entfällt
- Anweisungen werden mit Strichpunkt abgeschlossen
- Blöcke werden in geschweifte Klammern eingefasst
- `for`-Schleifen, `if`-Verzweigungen, etc. nach Art der Programmiersprache C
- `switch`-Verzweigung (nur Ratfor 77, ohne `break`)
- Kommentare werden mit `#` eingeleitet
- `<`, `<=`, `...`, `!=` anstelle von `.LT.`, `.LE.`, `...`, `.NE.`
- etc.

Nachfolgend je ein kleineres Beispiel, um einen kurzen Einblick in Ratfor 77 und Ratfor 90 und die aus diesen Ratfor-Codes per Präprozessor generierten Fortran-Codes zu geben.

57.2. Ratfor 77

Ratfor	FORTRAN 77
--------	------------

¹³⁰ w:Brian W. Kernighan ^{<https://de.wikipedia.org/wiki/Brian%20W.%20Kernighan>}

¹³² w:en:Ratfiv ^{<https://en.wikipedia.org/wiki/Ratfiv>}

<pre>program bsp i = 2 switch(i){ case 1: write(*, *) "Fall 1"; case 2: write(*, *) "Fall 2" default: write(*, *) "Default" } end</pre>	<pre>C Output from Public domain Ratfor, version 1.0 program bsp i = 2 I23000=(i) goto 23000 23002 continue write(*, *) "Fall 1" goto 23001 23003 continue write(*, *) "Fall 2" goto 23001 23004 continue write(*, *) "Default" goto 23001 23000 continue if (I23000.eq.1)goto 23002 if (I23000.eq.2)goto 23003 goto 23004 23001 continue end</pre>
--	---

57.3. Ratfor 90

Ratfor	Fortran 90
<pre># Ratfor 90 - Beispiel program bsp integer :: i real :: x = 10.5 if(x /= 10.0) { for(i = 0; i < 3; i = i + 1) { write(*,*) 'IF Hallo ', i; } } else { write(*,*) 'ELSE Hallo'; } end program bsp</pre>	<pre>! Ratfor 90 - Beispiel program bsp implicit none integer :: i real :: x = 10.5 if (x .ne. 10.0) then i=0 do if(.not. (i<3)) then exit end if write(*,*) 'IF Hallo ', i i=i+1 end do else write(*,*) 'ELSE Hallo' end if end program bsp</pre>

57.4. Weblinks

- Brian W. Kernighan: RATFOR – A Preprocessor for a Rational Fortran¹³³

133 <http://wolfram.schneider.org/bsd/7thEdManVol2/ratfor/ratfor.html>

Teil IX.

Sonstiges

58. Cray Pointer

58.1. Einleitung

Cray Pointer (auch *Integer Pointer* genannt) waren und sind teilweise heute noch in Fortran ein Ersatz für die aus der Programmiersprache C bekannten Zeiger. Sie sind nicht Teil der Fortran-Standards, werden jedoch von einigen Compilerherstellern als Erweiterung des Fortran-Sprachumfangs angeboten. *Cray Pointer* haben nicht viel mit den unter Fortran 90/95 neu eingeführten Zeigern gemeinsam und können Portabilitätsprobleme bereiten. Die Bezeichnung *Cray Pointer* deutet auf den "Erfinder" dieser Erweiterung hin - *Cray Research*¹³⁵ (firmiert aktuell als *Cray Inc.*).

58.2. Grundlagen

58.2.1. Pärchenbildung: Speicheradresse ↔ Objekt

```
pointer( pointeradr, pointee ) [, ...]
```

- **pointeradr** ... integer-Skalar (Speicheradresse)
- **pointee** ... Skalar oder Vektor

Der Datentyp für **pointeradr** muss groß genug gewählt werden, damit die Speicheradresse auch aufgenommen werden kann. Dies stellt schon das erste Problem dar, da diese "Zeigergröße" systemabhängig ist. In weitere Folge muss dieser Speicherplatzbedarf je Zeiger auch bei der Verwendung von "zeigerarithmetischen Kunststücken" beachtet werden. Um portabel zu bleiben darf **pointeradr** gar nicht deklariert werden.

58.2.2. Speicheradresse ermitteln

```
i = loc( var )
```

Diese Funktion gibt die Speicheradresse von **var** zurück.

58.2.3. Speicherplatz anfordern

```
ptradr = malloc( bytes )
```

Fordert **bytes** Speicherplatz an und gibt die erste dazugehörige Speicheradresse zurück.

¹³⁵ w:Cray ^{<https://de.wikipedia.org/wiki/Cray>}

58.2.4. Speicherplatz freigeben

```
free( ptradr )
```

58.2.5. Sonstiges

- *Cray Pointer* können auch auf Unterprogramme zeigen.

58.3. Beispiel

Fortran 90/95-Code (free source form)

```
program bsp
  implicit none

  real, dimension(4) :: arr = (/ 12.5, -3.3, -55.0, -144.9 /)
  real                :: pointee

! ***** Pärchenbildung und loc() *****
! iptradr wird nicht explizit deklariert, der Compiler ermittelt automatisch
! den erforderlichen Datentyp (kind-Wert) -> 4 Byte-Integer bei
! 32bit-Systemen, 8 Byte-Integer bei 64bit-Systemen, ...
pointer( iptradr, pointee(2, 2) )

  iptradr = loc( arr(1) )

  write( *,* ) pointee
! Ausgabe:
!   12.5      -3.3      -55.0      -144.9

  write( *,* ) pointee(:, 2)
! Ausgabe:
!   -55.0      -144.9

  iptradr = loc ( arr(2) )

  write( *,* ) pointee
! Ausgabe:
!   -3.3      -55.0      -144.9      1.4012985E-45

! ***** Zeigerarithmetik, Ergebnis systemabhängig *****
  iptradr = iptradr + 4      ! + 4 Bytes
  write( *,* ) pointee(1, 1)
! Ausgabe:
!   -55.0

! ***** malloc() und free() *****
  iptradr = malloc( 16 )
  pointee( 1, 1 ) = 99.9
  pointee( 2, 1 ) = 999.9

  write( *,* ) pointee
! Ausgabe:
!   99.9      999.9      0.0E+0      0.0E+0

  call free( iptradr )
end program bsp
```

Kompilieren, Linken:

- Intel (2025.2.0): `ifx bsp.f90`
- gfortran (7.5.0 und 14.2.0): `gfortran bsp.f90 -fcray-pointer`

58.4. Weblinks

- gfortran - Cray Pointers¹³⁶

¹³⁶ <http://gcc.gnu.org/onlinedocs/gfortran/Cray-pointers.html>

Teil X.

Anhang

59. Eine Gegenüberstellung von grundlegenden Fortran- und C-Sprachelementen

Nachfolgend werden einige grundlegende Sprachelemente von Fortran95 und C tabellarisch gegenübergestellt. Die Gegenüberstellung erfolgt nur schematisch, da hier nur ein Überblick über die Gemeinsamkeiten und Unterschiede der beiden Programmiersprachen aufgezeigt werden soll.

Tabellenlegende:

- — : nicht vorhanden
- ... : nicht näher bestimmt, aber irgendwas Sinnvolles und Zulässiges
- *kursiv* Geschriebenes steht für beliebige (aber natürlich nur sinnvolle und zulässige) Variablennamen, Zahlen, etc.

60. Programmgrundstruktur

60.1. Groß-, Kleinschreibung

Fortran95

Fortran ist case-insensitiv

C

C ist case-sensitiv

60.2. Kommentare

Fortran95

! xyz

C

/* xyz */

60.3. Das Ende einer Anweisung

Fortran95

1. Jede Anweisung steht grundsätzlich in einer eigenen Zeile.
2. Die Zeilenlänge ist begrenzt.
3. Mehrere Anweisungen dürfen auch in einer Zeile stehen, wenn sie durch Semikolon ; voneinander getrennt sind.
4. Eine Zeilenfortsetzung muss durch ein Kaufmanns-Und & am Ende der fortzusetzenden Zeile angezeigt werden.
5. Die Anzahl der erlaubten Fortsetzungzeilen ist begrenzt.

C

1. Einzelne Anweisungen sind immer durch Semikolon ; zu trennen.
2. Eine Anweisung kann sich über mehrere Zeilen erstrecken.

60.4. Hauptprogramm

Fortran95

program bsp ... end program bsp

C

int main(int argc, char **argv) { ... }

60.5. Unterprogramme

Fortran95

call sub1(...) ... subroutine sub1(...) ... end subroutine sub1
var = fun1(...) ... function fun1(...) *datentyp* :: fun1 ... fun1 = ...
end function fun1

C

void sub1(...); ... sub1(...); ... void sub1(...) { ... }
datentyp fun1(...); ... *var* = fun1(...); ... *datentyp* fun1(...) {
... return ...; }

- Prinzipiell werden Parameter per "call by reference" übergeben.
- Die Datenübertragungsrichtung lässt sich mittels *intent*-Angabe steuern.
- Prinzipiell werden Parameter per "call by value" übergeben.
- "call by reference" wird mittels Übergabe von Zeigern realisiert.

61. Einfache Datentypen

Fortran95	C	Bytes
—	unsigned-Datentypen	...
integer	int	4
integer(kind=1)	signed char	1
integer(kind=2)	short int	2
integer(kind=4)	int	4
integer(kind=8)	long int	8
real	float	4
double precision	double	8
real(kind=4)	float	4
real(kind=8)	double	8
complex	— (ab ISO C99 -> <i>complex.h</i> -Bibliothek)	8
logical	— (-> int-Zahlen, ab ISO C99 -> <i>stdbool.h</i> -Bibliothek)	4
character	unsigned char	1
character(len=...)	unsigned char[...]	...

(Zu beachten ist, dass die Byteanzahl der Datentypen teilweise maschinenabhängig ist. Die *kind*-Angaben sind zudem auch compilerabhängig.)

62. Variablendeklaration

Fortran95

datentyp :: var1, var2, ...

C

datentyp var1, var2, ...

63. Konstanten

Fortran95

'Hallo' oder "Hallo"

'A', "A"

Arithmetische Konstanten höherer Genauigkeit sind zwingend mit einer speziellen Datentypendung zu versehen

C

"Hallo"

Character: 'A', Zeichenkette:
"A"

—

64. Benannte Konstanten

Fortran95

datentyp, parameter :: konstante = *wert*

C

const *datentyp* konstante = *wert*
oder
#define konstante *wert*

65. Operatoren

65.1. Arithmetische Operatoren

Fortran95	C
+	+
-	-
*	*
/	/
**	—— (-> Bibliotheksfunktion)
— (-> intrinsic functions)	% (Modulo)

65.2. Vergleichsoperatoren

Fortran95	C
==	==
<	<
<=	<=
>	>
>=	>=
/=	!=

65.3. Logische Operatoren

Fortran95	C
.AND.	&&
.OR.	
.NOT.	!
.EQV.	——
.NEQV.	——

65.4. Stringverknüpfung

Fortran95	C
//	—— (-> Bibliotheksfunktion)

65.5. Bitoperatoren

Fortran95

—— (-> intrinsic functions)

C

«, », &, |, ^, ~

65.6. Weitere Operatoren

Fortran95

——

C

++, -, +=, -=, etc.

66. Zusammengesetzte Datentypen

Fortran95

```
type :: namesequence integer :: a ... end type name... type(  
name) :: var var%a = wert
```

C

```
struct name { int a; ... }; ... struct name var; var.a  
= wert;
```


67. Felder

Fortran95

```
datentyp, dimension(zahl) :: var  
datentyp, dimension(zahl1, zahl2, ...) :: var;
```

- Ein Feldindex startet standardmäßig bei 1. Dieser Indexstartwert kann aber vom Programmierer verschoben werden.
- Ein mehrdimensionales Feld wird spaltenweise gespeichert.
- Fortran95 kennt standardmäßig viele Möglichkeiten mit Feldern (Vektoren und Matrizen) zu hantieren (z.B. Feldkonstruktor, *reshape*, *where*, Zugriff auf eine Teilmenge eines Feldes, etc.).

```
var(zahl) = wert  
var(zahl1, zahl2) = wert  
var = wert
```

```
var(zahl1:zahl2) = wert
```

C

```
datentyp var[zahl]  
datentyp var[zahl1][zahl2]...;
```

- Ein Feldindex startet immer bei 0.
- Ein mehrdimensionales Feld wird zeilenweise gespeichert.

```
var[zahl] = wert;  
var[zahl1][zahl2] = wert;  
— (-> mittels Schleifendurchlauf über das ganze Feld)  
— (-> mittels Schleifendurchlauf über die Feldteilmenge)
```


68. Zeichenketten

Fortran95

`character(len=zahl) ::
var`
—

C

`char var[zahl]`

Zeichenketten werden (intern) mit einem `\0`-Zeichen terminiert. Dies muss bei der Variablendeklaration hinsichtlich der Feldlänge berücksichtigt werden.

69. Blöcke, Verzweigungen, Schleifen

69.1. Block

Fortran95	C
...	{ ... }

69.2. Verzweigungen

Fortran95	C
if (<i>bedingung</i>) <i>anweisung</i> if (<i>bedingung</i>) then ... end if if (<i>bedingung</i>) then ... else ... end if — select case (...) case (...) ... case (...) ... case default ... end select	if (<i>bedingung</i>) <i>anweisung</i> ; if (<i>bedingung</i>) { ... } if (<i>bedingung</i>) { ... } else { ... } ... ? ... : ... switch (...) { case ...: ... break; case ...: ... break; ... default: ... }

69.3. Schleifen

Fortran95	C
do <i>schleifenvar</i> = <i>anfangswert</i> , <i>endwert</i> , <i>schrittweite</i> ... end do do while (...) ... end do do ... if (...) exit end do	for (<i>anfangsbedingung</i> ; <i>endbedingung</i> ; <i>durchgangsanweisung</i>) { ... } while (...) { ... } do { ... } while (...);

69.4. Sonstiges

Fortran95	C
stop	#include <stdlib.h> exit(<i>status</i>); oder im Hauptprogramm return;
exit	break;
cycle	continue;

70. Ein-, Ausgabe

Fortran95

`write(*, format) ...`

`read(*, format) ...`

C

`#include <stdio.h> printf(...);`

`#include <stdio.h> scanf(...);`

71. Dateien

Fortran95

```
open (unit=nr, ...)write (nr, format) ... read (nr, format) ... ...  
close(unit=nr)
```

C

```
#include <stdio.h> ... FILE *f = fopen(...); fprintf (f, ...);  
fscanf(f, ...); ... fclose(f);
```


72. Zeiger

Fortran95

```
datatype, pointer :: var  
... datatype, pointer :: var1 => null(); datatype, target :: var2 =  
wert; ... var1 => var2 write(*,*) var1
```

C

```
datatype *var;  
... datatype *var1 = 0; datatype var2 = wert; ... var1 = &var2;  
printf("%d", *var1);
```


73. Anwendungsbeispiele

73.1. Dreiecksberechnung

74. Aufgabe

Es sollen einige charakteristische Dreieckswerte berechnet werden. Der Programmanwender gibt die Koordinatenwerte (x, y) für die Dreieckseckpunkte P_1 , P_2 und P_3 vor.

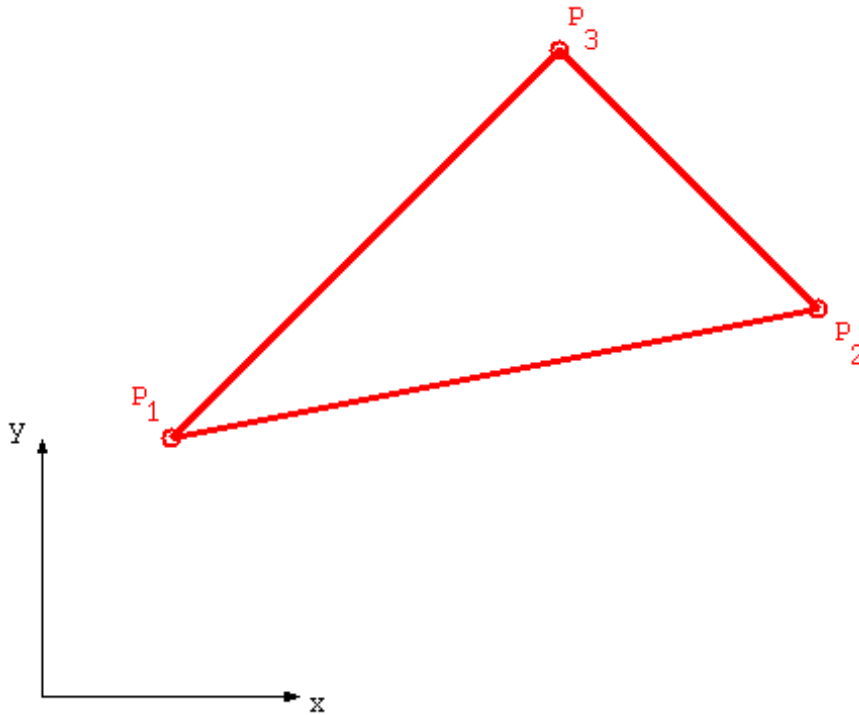


Abb. 56

Das Programm berechnet u.a. folgende Werte und übermittelt diese an die Standardausgabe:

- Längen der Dreiecksschenkel und Dreiecksumfang
- Innenwinkel
- Fläche
- Umkreis (Mittelpunkt und Radius)
- Inkreis (Mittelpunkt und Radius)
- Schwerpunkt

75. Grundlagen

Die Dreiecksberechnung erfolgt in diesem Anwendungsbeispiel hauptsächlich mittels Vektorrechnung.

Näheres zu Dreiecken und zur Vektorrechnung ist folgenden Enzyklopädieartikeln und Büchern zu entnehmen:

- Wikipedia: Dreieck¹³⁷
- Wikipedia: Vektor¹³⁸
- Formelsammlung Mathematik: Geometrie¹³⁹
- Formelsammlung Mathematik: Trigonometrie¹⁴⁰
- Mathematik: Schulmathematik¹⁴¹
- Mathematik: Lineare Algebra¹⁴²
- Ing Mathematik: Vektoren¹⁴³

¹³⁷ <https://de.wikipedia.org/wiki/Dreieck%20>

¹³⁸ <https://de.wikipedia.org/wiki/Vektor%20>

¹³⁹ <https://de.wikibooks.org/wiki/Formelsammlung%20Mathematik%3A%20Geometrie>

¹⁴⁰ <https://de.wikibooks.org/wiki/Formelsammlung%20Mathematik%3A%20Trigonometrie>

¹⁴¹ <https://de.wikibooks.org/wiki/Mathematik%3A%20Schulmathematik>

¹⁴² <https://de.wikibooks.org/wiki/Mathematik%3A%20Lineare%20Algebra>

¹⁴³ <https://de.wikibooks.org/wiki/Ing%20Mathematik%3A%20Vektoren>

75.1. Koordinatenwerte —> Richtungsvektoren

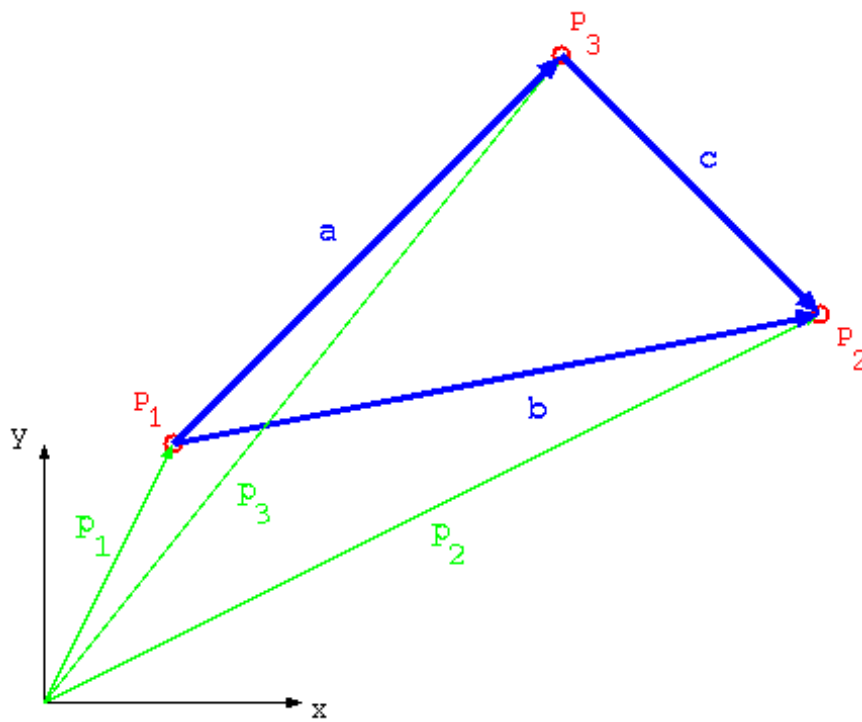


Abb. 57

$$\mathbf{a} = \mathbf{p}_3 - \mathbf{p}_1$$

$$\mathbf{b} = \mathbf{p}_2 - \mathbf{p}_1$$

$$\mathbf{c} = \mathbf{b} - \mathbf{a}$$

75.2. Seitenlängen und Umfang

$$a_{norm} = |\mathbf{a}|$$

$$b_{norm} = |\mathbf{b}|$$

$$c_{norm} = |\mathbf{c}|$$

$$U = a_{norm} + b_{norm} + c_{norm}$$

Bedingung: $a_{norm} \neq 0$, $b_{norm} \neq 0$, $c_{norm} \neq 0$

75.3. Winkel

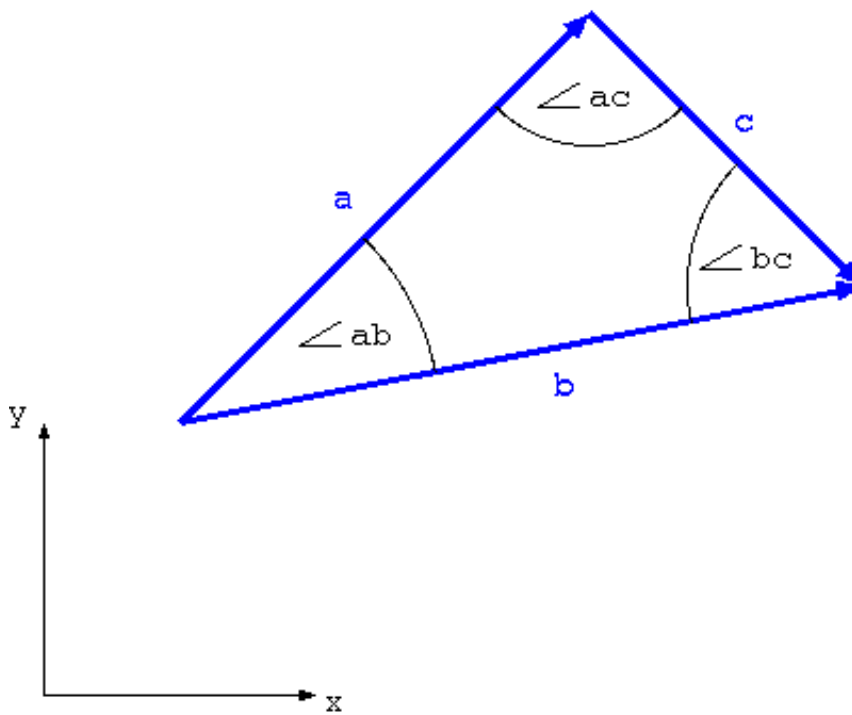


Abb. 58

$$\angle ab = \arccos \frac{\mathbf{a} \cdot \mathbf{b}}{a_{\text{norm}} b_{\text{norm}}}$$

$$\angle bc = \arccos \frac{\mathbf{b} \cdot \mathbf{c}}{b_{\text{norm}} c_{\text{norm}}}$$

$$\angle ac = \pi - \angle ab - \angle bc$$

Bedingung: $\angle ab \neq 0$, $\angle ac \neq 0$, $\angle bc \neq 0$

75.4. Fläche

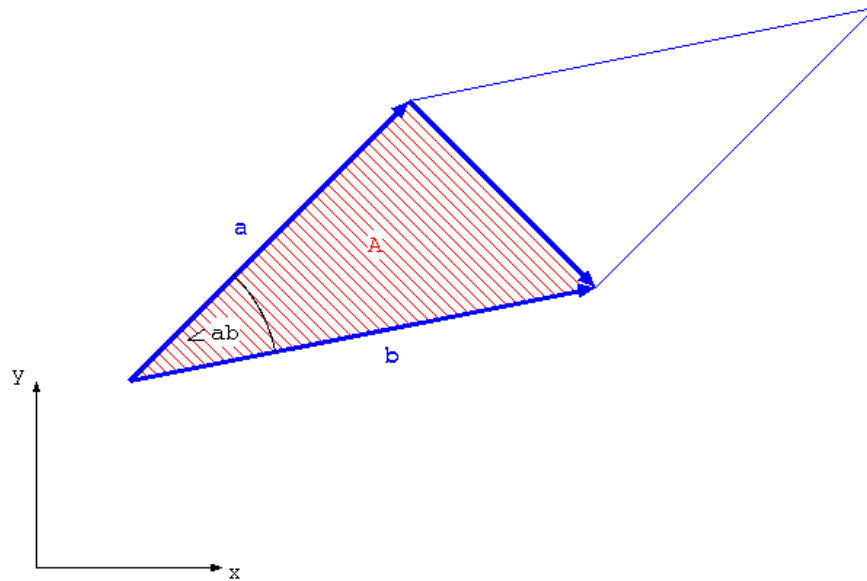


Abb. 59

Es gilt

$$A_{\text{Parallelogramm}} = |\mathbf{a} \times \mathbf{b}| = a_{\text{norm}} b_{\text{norm}} \sin \angle ab$$

und somit

$$A = \frac{A_{\text{parallelogramm}}}{2} = \frac{a_{\text{norm}} b_{\text{norm}} \sin \angle ab}{2}$$

75.5. Umkreis

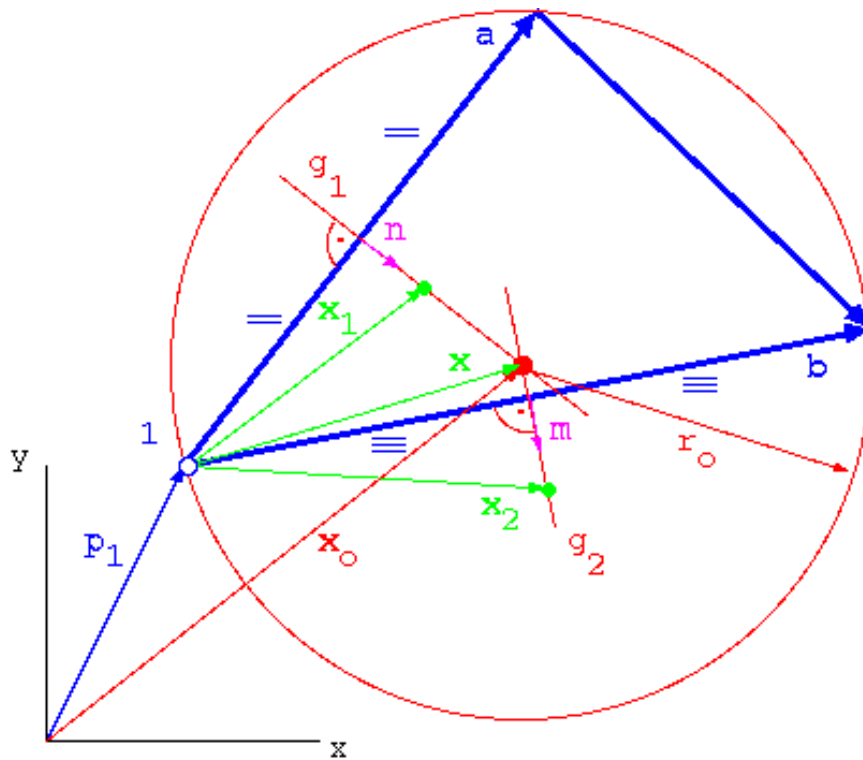


Abb. 60

Normalen:

$$\mathbf{a} \cdot \mathbf{n} = 0$$

$$\mathbf{b} \cdot \mathbf{m} = 0$$

$$|\mathbf{n}| = 1$$

$$|\mathbf{m}| = 1$$

$\Rightarrow$

$$n_1 = -a_2 \sqrt{\frac{1}{a_1^2 + a_2^2}}$$

$$n_2 = a_1 \sqrt{\frac{1}{a_1^2 + a_2^2}}$$

$$m_1 = -b_2 \sqrt{\frac{1}{b_1^2 + b_2^2}}$$

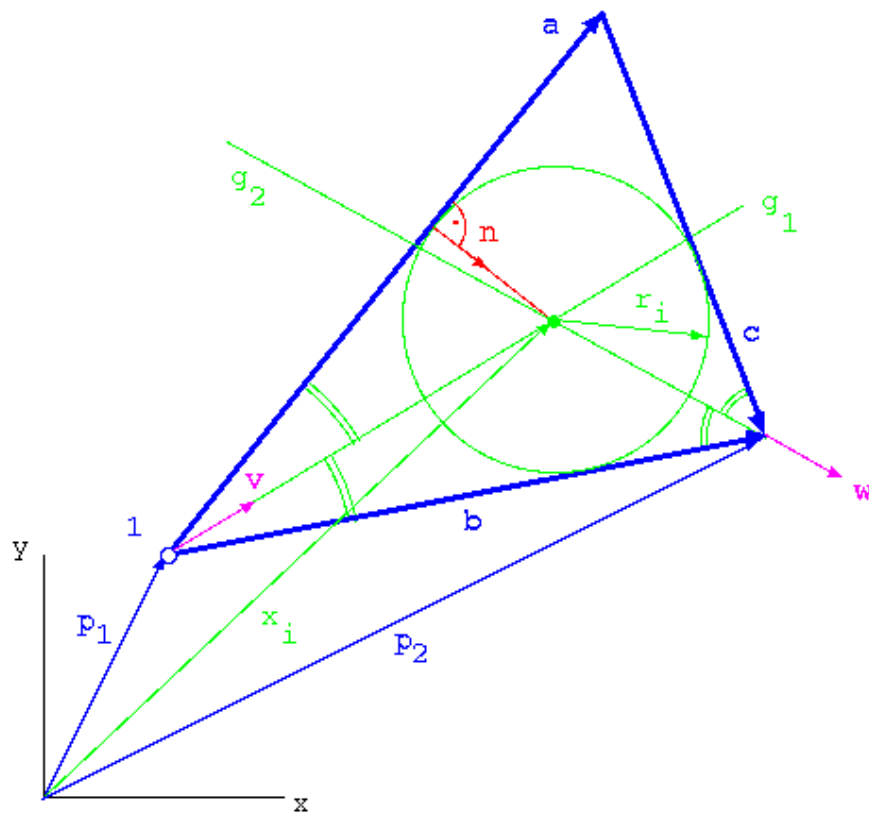
$$m_2 = b_1 \sqrt{\frac{1}{b_1^2 + b_2^2}}$$

$$g1: \mathbf{x}_1 = \frac{\mathbf{a}}{2} + t_1 \mathbf{n}$$

Umkreismittelpunkt und -rad

$$\mathbf{x}_o = \mathbf{p}_1 + \frac{\mathbf{b}}{2} + t_2 \mathbf{m}$$

$$r_o = \left| \frac{\mathbf{b}}{2} + t_2 \mathbf{m} \right|$$



$$\mathbf{v} = \frac{1}{2} \left(\frac{\mathbf{a}}{a_{norm}} + \frac{\mathbf{b}}{b_{norm}} \right)$$

$$\mathbf{w} = \frac{1}{2} \left(\frac{\mathbf{b}}{b_{norm}} + \frac{\mathbf{c}}{c_{norm}} \right)$$

$$g1: \mathbf{x}_1 = t_1 \mathbf{v}$$

$$g2: \mathbf{x}_2 = \mathbf{b} + t_2 \mathbf{w}$$

Der Inkreismittelpunkt ergibt sich als Schnittpunkt dieser beiden Geraden: $\mathbf{x} = \mathbf{x}_1 = \mathbf{x}_2$

$\Rightarrow$

$$t_2 = \frac{b_1 v_2 - b_2 v_1}{w_2 v_1 - w_1 v_2}$$

Bedingung: $w_2 v_1 - w_1 v_2 \neq 0$

Inkreismittelpunkt:

$$\mathbf{x}_i = \mathbf{p}_2 + t_2 \mathbf{w}$$

Inkreisradius:

$$r_i = \frac{2A}{a_{norm} + b_{norm} + c_{norm}}$$

75.7. Schwerpunkt

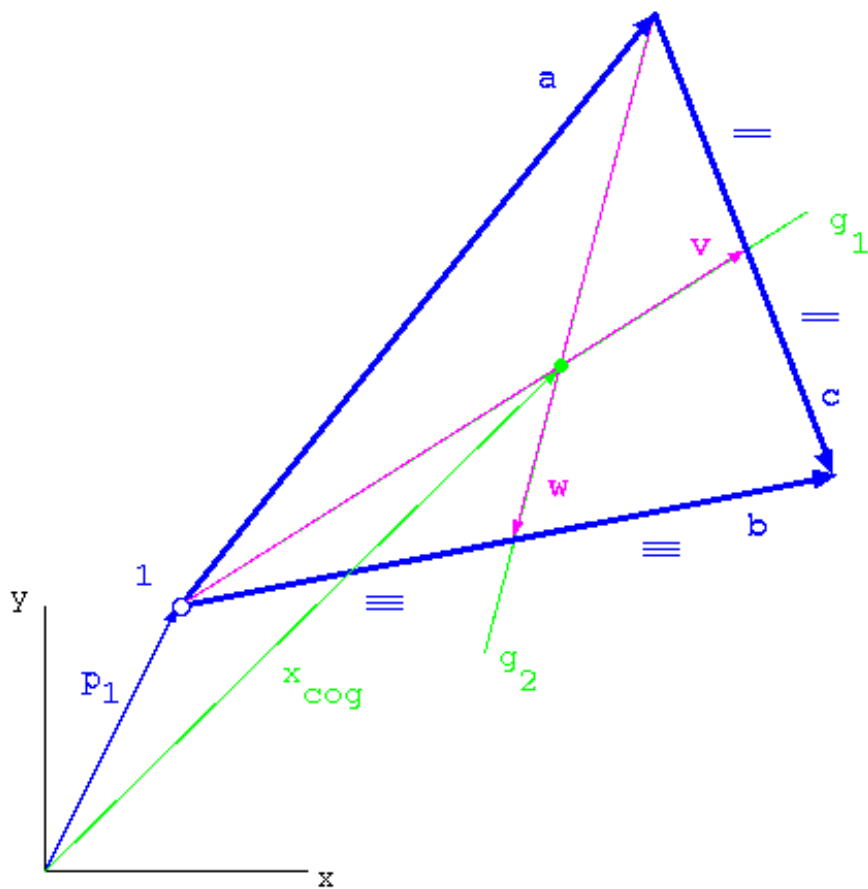


Abb. 62

$$\mathbf{v} = \mathbf{a} + \frac{\mathbf{c}}{2}$$

$$\mathbf{w} = \frac{\mathbf{b}}{2} - \mathbf{a}$$

$$g1: \mathbf{x}_1 = t_1 \mathbf{v}$$

$$g2: \mathbf{x}_2 = \mathbf{a} + t_2 \mathbf{w}$$

Der Dreiecksschwerpunkt ergibt sich als Schnittpunkt dieser beiden Geraden: $\mathbf{x} = \mathbf{x}_1 = \mathbf{x}_2$

$$t_2 = \frac{a_1 v_2 - a_2 v_1}{w_2 v_1 - w_1 v_2}$$

Bedingung: $w_2 v_1 - w_1 v_2 \neq 0$

$\Rightarrow$

$$\mathbf{x} = \mathbf{a} + t_2 \mathbf{w}$$

$$\mathbf{x}_{cog} = \mathbf{p}_1 + \mathbf{x}$$

76. Code

- Makefile¹⁴⁴
- triangle.f90¹⁴⁵
- modul1.f90¹⁴⁶

¹⁴⁴ https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Dreiecksberechnung%3A_Makefile%20%20

¹⁴⁵ https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Dreiecksberechnung%3A_Triangle.f90%20

¹⁴⁶ https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Dreiecksberechnung%3A_Modul1.f90%20

77. Screenshots

```

***** DREIECKSBERECHNUNG *****

Eingabe:
Punkt 1 (x y): 10.5 20.56
Punkt 2 (x y): 100.6 70.5
Punkt 3 (x y): 110 15.5

*** Ergebnisse ***
Eingabe:
P1          =      10.50000      20.56000
P2          =      100.6000      70.50000
P3          =      110.0000      15.50000
Berechnet:
a           =      99.50000      -5.059999
b           =      90.10000      49.94000
c           =     -9.400002      55.00000
Länge a     =      99.62858
Länge b     =     103.0146
Länge c     =     55.79749
Umfang      =     258.4407
Winkel ab   =     0.5569285      rad
Winkel ab   =     31.90965      °
Winkel ac   =     1.350712      rad
Winkel ac   =     77.39011      °
Winkel bc   =     1.233952      rad
Winkel bc   =     70.70023      °
Fläche      =     2712.468      ^2
Umkreismittelpunkt =     61.13598      35.45197
Umkreisradius =     52.78043
Inkreismittelpunkt =     84.89420      37.79489
Inkreisradius =     20.99103
Schwerpunkt =     73.70000      35.52000

```

Abb. 63

77.1. Kettenlinie

77.2. Aufgabe

Der Programmbenutzer gibt die Kettenlänge L , die Abstände der beiden Abhängepunkte in x - und y -Richtung (b und h), sowie das spezifische Kettengewicht q vor. Aus diesen Angaben werden diverse Daten (Seilparameter, Kräfte, Durchhang, ...) für die Kette berechnet. Zusätzlich wird die aus den Angaben resultierende Kettenlinie grafisch am Bildschirm ausgegeben. Diese grafische Ausgabe erfolgt mit Hilfe der DISLIN-Grafikbibliothek.

77.3. Grundlagen

- Stereostatik¹⁴⁷
- siehe Nullstellenbestimmung nach Newton-Raphson¹⁴⁸

77.4. Code

- Makefile¹⁴⁹
- catenary.f90¹⁵⁰
- catmod1.f90¹⁵¹
- catdisl.f90¹⁵²
- dislin.f90 (diese Datei ist Bestandteil der DISLIN-Bibliothek)

147 <https://de.wikibooks.org/wiki/Stereostatik%3A%20Seile%20und%20Ketten%20>

148 <https://de.wikipedia.org/wiki/Newton-Verfahren%20>

149 https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Kettenlinie%3A_Makefile%20%20

150 https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Kettenlinie%3A_Catenary.f90%20

151 https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Kettenlinie%3A_Catmod1.f90%20

152 https://de.wikibooks.org/wiki/Fortran%3A_Anhang_B%3A_Kettenlinie%3A_Catdisl.f90%20

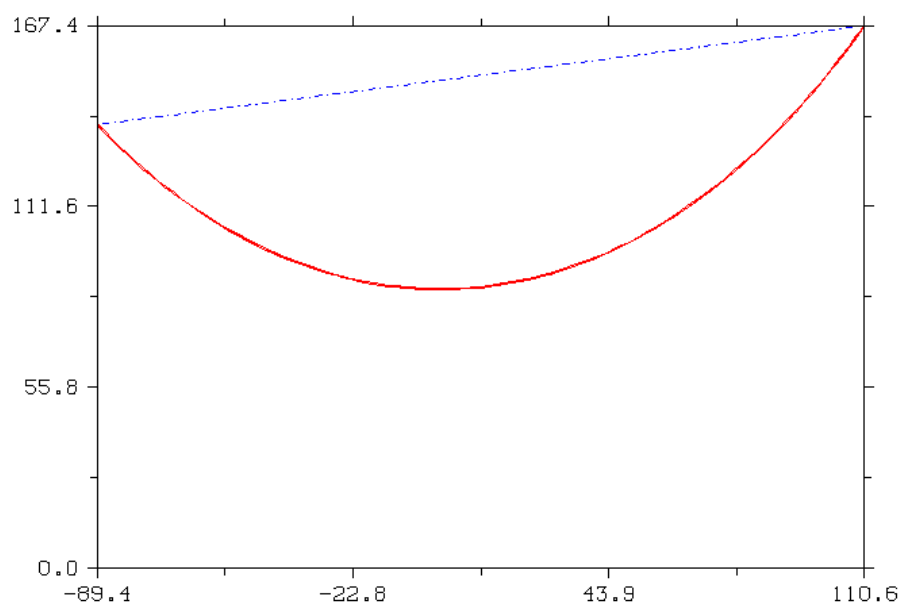


Abb. 65

78. Debugger

Werden Programme länger und komplexer, so kann der Einsatz eines Debuggers das Auffinden von logischen Fehlern erleichtern.

79. Der GNU Debugger

Der *gdb* (GNU Debugger) wird besonders im Open Source-Bereich im Zusammenspiel mit der GCC verwendet. Die Anwendung im Zusammenhang mit *gfortran*-compilierten Programmen ist relativ einfach, wenn auch nicht so problemlos wie bei C- oder C++-Programmen.

79.1. gdb und gfortran

Das Programm muss mit dem Optionsschalter *-g* erstellt werden. *gfortran* kennt noch andere erweiterte Optionen, die spezialisiertere Ergebnisse liefern. Für einfache Beispiele wie sie in diesem Buch vorzufinden sind reicht aber die *-g*-Option vollkommen aus.

```
gfortran -g -o bsp bsp.f90
```

Aufruf des Debuggers mittels

```
gdb bsp
```

Zu beachten ist, dass nach dem Start des Debuggers ein Breakpoint

```
b MAIN__
```

gesetzt wird, ansonsten findet der Debugger das zu debuggende Fortran-Programm nicht. Danach wird der Debuggerlauf mit

```
run
```

gestartet.

Einige wichtige *gdb*-Befehle:

<i>b <zahl></i>	Breakpoint in der Zeile <i><zahl></i> setzen
<i>list</i>	Codelisting
<i>next</i>	Zur nächsten Zeile springen
<i>step</i>	Sprung in eine Subroutine
<i>print <var></i>	Gibt den Wert der Variablen <i><var></i> aus
<i>c</i>	Das Programm fortsetzen (continue)
<i>until <zahl></i>	Programm bis zur Zeile <i><zahl></i> ausführen
<i>quit</i>	Debugger beenden

79.2. gdb und Intels ifx

Es gilt im Prinzip das Gleiche wie im vorigen Abschnitt.

80. Quellcodedokumentation

80.1. ROBODoc

ROBODoc ist freie Software unter der GNU General Public License. Dieses Tool verwendet die normalen Kommentarzeichen der jeweiligen Programmiersprache und nutzt für die Dokumentationsgenerierung eigene Auszeichnungstags innerhalb dieser Kommentare.

80.2. Ein einfaches Beispiel

80.2.1. Beispielcode

Fortran 90/95-Code (free source form)

```
!****h* Beispielprogramm/circle
! NAME
!   circle
!
! DESCRIPTION
!   Modul fuer Kreisfunktionen
!
! AUTHOR
!   Intruder
!
! CREATION DATE
!   04.08.2007
!*****
module circle
  implicit none

  !****d* circle/pi
  ! NAME
  !   pi
  !
  ! DESCRIPTION
  !   pi = 3.14
  !*****
  real, parameter :: pi = 3.14

contains
  !****f* circle/area
  ! NAME
  !   area
  !
  ! DESCRIPTION
  !   Berechnet Kreisflaeche
  !
  ! INPUTS
  !   r ... Radius (real)
  !
```



```

! RESULT
!   Kreisflaeche (real)
!*****
real function area( r )
  implicit none

  real, intent( in ) :: r

  area = r ** 2 * pi
end function area
end module circle

```

Fortran 90/95-Code (free source form)

```

!*****h* Beispielpogramm/main
! NAME
!   main
!
! DESCRIPTION
!   Hauptprogramm
!
! AUTHOR
!   Intruder
!
! CREATION DATE
!   04.08.2007
!*****
program main
  use circle
  implicit none

  real          :: r, a

  read( *, * ) r
  a = area( r )
  write( *, * ) "Flaeche = ", a
end program main

```

80.2.2. Erstellen der Dokumentation

Erstellen der Dokumentation im HTML-Format als Multidokument mit Index:

```
robodoc --src . --doc doc --multidoc --html --index
```

80.2.3. Screenshot

		Table of Contents
		Sourcefiles
		Index
		Definitions
		Functions
		Modules
Beispielprogramm/circle [Modules]		
[Top] [Modules]		
NAME		
circle		
DESCRIPTION		
Modul fuer Kreisfunktionen		
AUTHOR		
Intruder		
CREATION DATE		
04.08.2007		
circle/area [Functions]		
[Top] [circle] [Functions]		

Abb. 66

80.2.4. Kurze Erläuterung

ROBODoc filtert die erforderlichen Angaben für die Dokumentation aus den Fortran-Kommentaren heraus. Zu diesem Zweck sind in den Kommentaren *Header* einzubauen. Diese *Header* bestehen aus

- begin marker
- items
- end marker

Ein *begin marker* beginnt immer mit 4 Sternchen, dann kommt ein Buchstabe als Elementkennzeichner. Es folgt ein Stern und dann Angaben zur Stellung des Elementes in der Dokumentationshierarchie.

In diesem Beispiel wurden folgende Elementkennzeichner verwendet:

- h ... Modul
- f ... Funktion
- d ... Konstante (Definition)

Der Hierarchiebaum wird durch die strikte Angabe von

übergeordnetes Element/aktuelles Element
--

erstellt.

Die verschiedenen Items sind dann unterhalb dieses *begin markers* durch Schlüsselworte gekennzeichnet:

- NAME
- DESCRIPTION
- AUTHOR
- etc.

Sie dienen zur konkreten Beschreibung des jeweiligen Elementes.

Abgeschlossen wird ein solcher Dokumentations-Header durch den *end marker*. Dieser wird durch mindestens drei Sternchen gebildet.

Zwecks Erstellung der Dokumentation sind viele Optionen verfügbar. Die unbedingt erforderlichen Angaben sind

- `--src` mit Angabe der Quelldatei bzw. dem Verzeichnis, in dem die Quelldateien liegen
- `--doc` mit einem Vornamen für die Dokumentationsdateien bzw. der Bezeichnung des gewünschten Dokumentationsverzeichnisses
- eine Angabe zu der Dokumentationsform:
 - `--multidoc ...` Die Dokumentation wird in Form mehrerer Einzeldateien in des Dokumentationsverzeichnis geschrieben
 - `--singledoc ...` Der Dokumentationsinhalt wird in eine einzige Datei geschrieben
- die Angabe des Dokumentationsformates:
 - `--html ...` HTML
 - `--rtf ...` RTF
 - `--latex ...` LaTeX
 - `--dbxml ...` XML DocBook

Für dieses Beispielprojekt wird auch noch ein Index erstellt (`--index`).

Detaillierte Informationen zu ROBODoc sind unter dem nachfolgend angeführten Weblink zur ROBODoc-Homepage abrufbar.

80.3. Weblinks

- ROBODoc¹⁵³

80.4. Natural Docs

Auch der Dokumentationsgenerator *Natural Docs* unterstützt bereits Fortran ab dem Standard 90/95 (free source form) in einer Basisvariante. *Natural Docs* ist in der Programmiersprache Perl verfasst. Als Softwarelizenz wurde die *General Public License* gewählt. Zum Zeitpunkt der Kapitererstellung war die Version 1.35 aktuell.

¹⁵³ <https://rfsber.home.xs4all.nl/Robo/>

80.5. Ein einfaches Beispiel

Fortran 90/95-Code (free source form)

```
! Section: Beispielprogramm
!   Autor: Intruder, Datum: 04.08.2007
```

```
! Group: circle
!   Modul fuer Kreisfunktionen
module circle
  implicit none
```

```
! Constant: pi
!   pi = 3.14
real, parameter :: pi = 3.14
```

```
contains
  ! Function: area
  !   Berechnet Kreisflaeche
  !
  ! Parameters:
  !   r ... Radius (real)
  !
  ! Returns:
  !   Kreisflaeche (real)
  real function area( r )
    implicit none

    real, intent( in ) :: r

    area = r ** 2 * pi
  end function area
end module circle
```

```
! Group: main
!   Hauptprogramm
program main
  use circle
  implicit none

  real          :: r, a

  read( *, * ) r
  a = area( r )
  write( *, * ) "Flaeche = ", a
end program main
```

Erstellung der HTML-Dokumentation:

```
NaturalDocs -i . -o HTML bsp -p .
```

Screenshot:

Beispielprogramm

Beispielprogramm (bsp.f90)

INDEX

Everything

Database Indexes

Files

Functions

Constants

Beispielprogramm

Autor: Intruder, Datum: 04.08.2007

Summary

Beispielprogramm

Autor: Intruder, Datum: 04.08.2007

CIRCLE

Modul fuer Kreisfunktionen

pi

pi = 3.14

area

real function area(r)

Berechnet Kreisflaeche

Parameters

Abb. 67

80.6. Kurze Erläuterung

Natural Docs filtert die benötigten Informationen mit Hilfe von Schlüsselwörtern aus den Kommentarbereichen der Fortrandateien. Die im Beispielprogramm verwendeten Schlüsselwörter sind:

- Section
- Group
- Constant
- Function
- Parameters
- Returns

Groß- Kleinschreibung spielt bei diesen Schlüsselwörtern keine Rolle. Abgeschlossen wird ein Schlüsselwort mit einem Doppelpunkt.

Beim Ausgabeschalter (-o) muss neben einem Ausgabeverzeichnis auch noch ein Ausgabeformat angegeben werden. Möglich sind

- HTML
- FramedHTML

Beim Schalter -p ist ein (Projekt)Verzeichnis anzugeben, wo *Natural Docs* einige benötigte Dateien ablegen kann, die nicht direkt zur erstellten Dokumentation gehören.

Für weitergehende Infos zu *Natural Docs* wird auf die unten angegebene Webpräsenz zu dieser Software verwiesen.

80.7. Weblinks

- Natural Docs¹⁵⁴

80.8. Doxygen

80.9. Ein einfaches Beispiel

Fortran 90/95-Code (free source form)

```
!> \file testprogramm.f90
!! \brief Datei fuer geometrische Berechnungen

!> Modul: circle
!> \author Intruder
!> \date 28.10.2007

module circle
  implicit none

  real, parameter :: pi = 3.14  !< PI

  contains
    !> Berechnet Kreisflaeche
    !> \param r ... Radius
    !> \return Kreisflaeche
    real function area( r )
      implicit none

      real, intent( in ) :: r

      area = r ** 2 * pi
    end function area
end module circle

!> Hauptprogramm
program main
  use circle
  implicit none

  real :: r, a

  read( *, * ) r
  a = area( r )
  write( *, * ) "Flaeche = ", a
end program main
```

Erstellen einer Konfigurationsdatei:

¹⁵⁴ <http://www.naturaldocs.org/>

```
doxygen -g Doxyfile
```

Anschließend sollte man die Doxyfile bearbeiten und nach seinen eigenen Wünschen anpassen (Projektname, Outputordner etc.). Wichtig ist, dass das Flag `OPTIMIZE_FOR_FORTRAN` auf YES gesetzt wird.

Erstellen der Dokumentation (standardmäßig im HTML- und LaTeX-Format, optional auch XML, RTF und man-pages):

```
doxygen Doxyfile
```

Screenshot (HTML-Variante):



Abb. 68

80.10. Weblinks

- [doxygen-Homepage](http://www.doxygen.nl)¹⁵⁵

¹⁵⁵ <http://www.doxygen.nl>

81. Make & Co.



Abb. 69

81.1. Einleitung

Hier wird keine Einführung in die Verwendung und Syntax von make- und config-Tools geboten, sondern nur kurz auf einige Spezialitäten hingewiesen, die bei den ersten Einsatzversuchen derartiger Werkzeuge im Zusammenhang mit Fortran zu beachten sind.

Grundsätzlich kann bei der Erstellung von Makefiles und Konsorten eine ähnliche Vorgehensweise wie bei konventionellen C-Programmen gewählt werden. Es ist bei Fortran-Programmen jedoch zu bedenken, dass bei Verwendung von Modulen `mod`-Dateien generiert werden (ab Fortran 90). Diese `mod`-Dateien sind in weiterer Folge für das Kompilieren von moduleinbindenden Quellcodedateien und den Linkvorgang von entscheidender Bedeutung. Somit ist bei hierarchisch verzweigten Quellcodeverzeichnisbäumen Obacht zu geben, dass jeweils auch Zugriff zu diesen `mod`-Dateien gegeben ist. Dies kann geschehen durch

- geeigneten Aufbau der Makefiles,
- durch Verwendung von Tools, die solche Abhängigkeiten automatisch auflösen
- oder auch durch explizite Bekanntgabe der entsprechenden Pfade an Compiler und Linker.

81.2. Explizite Bekanntgabe von Modulpfaden

81.2.1. gfortran

Standardmäßig werden `include`- und `mod`-Dateien im aktuellen Verzeichnis gesucht. Die Suche kann aber mit folgendem Compilerschalter auf andere Pfade ausgedehnt werden:

- `-I...`: Suchpfad für
 - `include`-Dateien
 - `mod`-Dateien

erweitern.

Standardmäßig werden `mod`-Dateien in das aktuelle Verzeichnis geschrieben. Dieses Verhalten kann mit folgendem Schalter geändert werden:

- `-J...`: Legt Verzeichnis fest, in das die `mod`-Dateien geschrieben werden, gleichzeitig auch Suchpfad für `mod`-Dateien.

(Alias für `-M...` um Konflikte mit bisherigen GCC-Optionen zu vermeiden)

81.2.2. ifx

- `-I...`: Suchpfad für
 - `include`-Dateien
 - `mod`-Dateien

erweitern.

81.3. GNU Make

GNU Make erkennt derzeit leider nur FORTRAN 77-Dateien mit der Endung `.f` automatisch. Für "free source form"-Fortran-Programme sind daher einige vorbereitende Arbeiten nötig,

um dann auch die Vorteile (und Nachteile) der impliziten Anwendung von "Pattern Rules" genießen zu dürfen. Werden alle Makeschritte für Fortran-Dateien explizit vorgegeben, dann kann man sich dies natürlich sparen.

81.3.1. Ein einfaches Beispiel

Es sei ein einfaches Beispiel gegeben, das eine FORTRAN 77-, eine Fortran 2003- und eine C-Datei enthält. Diese Dateien liegen im selben Verzeichnis.

Quellcode-Dateien

main.f90: Fortran 2003-Code

```
! Das Hauptprogramm
program main
  implicit none

  interface
    function addition( a, b ) bind( c )
      use, intrinsic :: iso_c_binding
      real( kind = c_float ), value :: a
      real( kind = c_float ), value :: b
      real( kind = c_float )          :: addition
    end function addition

    subroutine sub()
  end subroutine sub
end interface

call sub()
write (*,*) addition( 2.5, 3.3 )

! Ausgabe:
!   Summe =
!       5.8
end program main
```

func.c: Programmcode

```
/* Addiere zwei Zahlen */
float addition(float a, float b)
{
  return (a + b);
}
```

sub.f:

```
0.|1.2.3.4.5.6.7|.8
1234567890123456789012345678901234567890123456789012345678901234567890
```

```
C  Eine einfache FORTRAN 77-Subroutine
    SUBROUTINE SUB
      WRITE( *, * ) 'Summe ='
    END
```

1234567890123456789012345678901234567890123456789012345678901234567890
0.|1.2.3.4.5.6.7|.8

Explizite Vorgabe der Makeschritte

Makefile:

```
FC = gfortran      # oder ifx, ...

prog: main.o func.o sub.o
    $(FC) -o $@ $^

main.o: main.f90
    $(FC) -c $^

func.o: func.c
    $(CC) -c $^

sub.o: sub.f
    $(FC) -c $^
```

Nutzung von "Pattern Rules"

Makefile:

```
FC = gfortran      # oder ifx, ...

%.o: %.f90
    $(FC) -c $<

prog: main.o func.o sub.o
    $(FC) -o $@ $^
```

Die Generierung der Objektdaten aus den Quellcodedateien geschieht hier implizit. Für C- und FORTRAN 77-Dateien sucht sich GNU Make die entsprechenden Regeln aus seiner internen Regel-Datenbank. Für .f90-Dateien wurde der entsprechende Befehl hier von uns explizit durch eine "Pattern Rule" vorgegeben.

Die make-Ausgabe sieht so aus:

```
gfortran -c main.f03
cc      -c -o func.o func.c
gfortran -c -o sub.o sub.f
gfortran -o prog main.o func.o sub.o
```

Solange die Quelldateien im selben Verzeichnis liegen, ist die Erstellung eines Makefiles ziemlich einfach. Wenn aber die Quelldateien gestaffelt in Unterverzeichnissen gespeichert sind und womöglich noch Abhängigkeiten von einem Unterverzeichnis zum anderen gegeben sind, dann kann die ganze Sache ziemlich kompliziert werden. Im Anschluss wird eine einfache nichtrekursive Make-Variante gezeigt.

81.3.2. Nichtrekursive Make-Variante

Vor der Programmerstellung:

```
-- (D) projektverzeichnis
| -- (F) Makefile
| -- (F) module.mk
| -- (F) main.f90
|
| -- (D) kreis
| | -- (F) module.mk
| | -- (F) kreis.f90
| | -- (F) kreissegment.f90
|
| -- (D) quadrat
| | -- (F) module.mk
| | -- (F) quadrat.f90
|
(D) ... directory
(F) ... file
```

Nach der Programmerstellung durch Aufruf von make:

```
-- (D) projektverzeichnis
| -- (F) Makefile
| -- (F) module.mk
| -- (F) main.f90
| -- (F) main.o
| -- (F) prog
| -- (F) kreis.mod
| -- (F) kreissegment.mod
| -- (F) quadrat.mod
|
| -- (D) kreis
| | -- (F) module.mk
| | -- (F) kreis.f90
| | -- (F) kreissegment.f90
| | -- (F) kreis.o
| | -- (F) kreissegment.o
|
| -- (D) quadrat
| | -- (F) module.mk
| | -- (F) quadrat.f90
| | -- (F) quadrat.o
```

Quellcode-Dateien

main.f90: Fortran 90/95-Code (free source form)

```
program main
  use kreis
  use kreissegment
  use quadrat
  implicit none

  call k()
  call q()
  call ks()
```

```
end program main
```

kreis/kreis.f90: **Fortran 90/95-Code (free source form)**

```
module kreis
  implicit none

  private
  public :: k

  contains
    subroutine k()
      print *, "Ich bin ein Kreis!"
    end subroutine k
end module kreis
```

kreis/kreissegment.f90: **Fortran 90/95-Code (free source form)**

```
module kreissegment
  use kreis
  implicit none

  private
  public :: ks

  contains
    subroutine ks()
      call k()
      print *, "Hihi, war nur ein Scherz. Ich bin ein Kreissegment!"
    end subroutine ks
end module kreissegment
```

quadrat/quadrat.f90: **Fortran 90/95-Code (free source form)**

```
module quadrat
  implicit none

  private
  public :: q

  contains
    subroutine q()
      print *, "Ich bin ein Quadrat!"
    end subroutine q
end module quadrat
```

Makefile, Include-Dateien

Makefile:

```
FC      := gfortran  # oder ifx, ...
SRC      :=

OBJ      = $(subst .f90,.o,$(SRC))

%.o: %.f90
    $(FC) -c -o $$@ $<
```

```
include kreis/module.mk
include quadrat/module.mk
include module.mk
```

```
prog: $(OBJ)
    $(FC) -o $@ $~
```

module.mk:

```
SRC += main.f90
```

kreis/module.mk:

```
SRC += kreis/kreis.f90 kreis/kreissegment.f90
```

quadrat/module.mk:

```
SRC += quadrat/quadrat.f90
```

Es gibt nur ein Makefile im Projekthauptverzeichnis. Sämtliche unterverzeichnisspezifischen Details (hier nur die Bekanntgabe der Quellcodedateien) werden in den jeweiligen Unterverzeichnissen in eigenen Include-Dateien (.mk) festgelegt und in das Makefile eingebunden. Da, anders als beim rekursiven Make, nicht in die einzelnen Unterverzeichnisse gewechselt wird, werden allfällige mod-Dateien auch nur in das Projekthauptverzeichnis (= das aktuelle Verzeichnis) geschrieben.

81.3.3. Weiterführende Make-Infos

- GNU Make Manual¹⁵⁶
- Robert Mecklenburg: Managing Projects with GNU Make, O'Reilly Openbook, 2004¹⁵⁷
- Peter Miller: Recursive Make Considered Harmful, 2006¹⁵⁸

81.4. CMake

CMake ist kein Make-Klon, sondern ein moderner Autotools-Ersatz.

Gleiches Beispiel wie bei #Nichtrekursive_Make-Variante¹⁵⁹, es muss in diesem Fall nur eine CMakeLists.txt-Datei im Projekthauptverzeichnis erstellt werden. Makefile und dazugehörige Dateien werden automatisch beim cmake-Aufruf erstellt.

CMakeLists.txt:

```
PROJECT(bsp Fortran)
```

¹⁵⁶ <http://www.gnu.org/software/make/manual/>

¹⁵⁷ <http://www.oreilly.com/catalog/make3/book/index.csp>

¹⁵⁸ https://accu.org/journals/overload/14/71/miller_2004/

¹⁵⁹ Kapitel 81.3.2 auf Seite 435

```
SET(src
    main.f90
    kreis/kreis.f90
    kreis/kreissegment.f90
    quadrat/quadrat.f90
)

ADD_EXECUTABLE(prog ${src})
```

Generieren der Makefiles, etc.:

- `FC=gfortran cmake .`

```
-- Check for working Fortran compiler: /xyz/bin/g95
-- Check for working Fortran compiler: /xyz/bin/g95 -- works
-- Checking whether /xyz/bin/g95 supports Fortran 90
-- Checking whether /xyz/bin/g95 supports Fortran 90 -- yes
-- Configuring done
-- Generating done
-- Build files have been written to: /abc/projektverzeichnis
```

Mittels `FC=...` wird der zu verwendende Fortran-Compiler festgelegt. Wenn irgendein auf dem System installierter Fortran-Compiler verwendet werden soll, kann diese Vorgabe auch entfallen. In der `CMakeLists.txt` muss die Programmiersprache Fortran ausdrücklich aktiviert werden. Entweder wie hier im `PROJECT`-Statement oder alternativ auch über die `ENABLE_LANGUAGE`-Anweisung.

Programmerstellung:

- `make`

```
Scanning dependencies of target prog
[ 25%] Building Fortran object CMakeFiles/prog.dir/kreis/kreis.o
[ 50%] Building Fortran object CMakeFiles/prog.dir/kreis/kreissegment.o
[ 75%] Building Fortran object CMakeFiles/prog.dir/quadrat/quadrat.o
[100%] Building Fortran object CMakeFiles/prog.dir/main.o
Linking Fortran executable prog
```

CMake-Homepage:

- CMake Cross-platform Make¹⁶⁰

81.5. SCons

Die Konfigurations-Dateien von SCons sind Python-Dateien.

81.5.1. Ein einfaches Beispiel

Gegeben seien die weiter oben gelisteten Dateien `kreis/kreis.f90`, `kreis/kreissegment.f90`, `quadrat/quadrat.f90`, `main.f90`. Um dieses Beispiel zu bauen, erstellt man im Quellcode-Verzeichnis zuerst eine `SConstruct`-Datei, z.B.:

¹⁶⁰ <http://www.cmake.org>

```
src_files = Split('kreis/kreis.f90 kreis/kreissegment.f90 quadrat/quadrat.f90
main.f90')
Program('prog', src_files)
```

Dann startet man `scons` in einer Konsole (zuvor natürlich, wenn noch nicht geschehen, in das entsprechende Quellcode-Verzeichnis wechseln) und erhält z.B. folgende Ausgabe:

```
scons: Reading SConscript files ...
scons: done reading SConscript files.
scons: Building targets ...
gfortran -o kreis/kreis.o -c kreis/kreis.f90
gfortran -o kreis/kreissegment.o -c kreis/kreissegment.f90
gfortran -o quadrat/quadrat.o -c quadrat/quadrat.f90
gfortran -o main.o -c main.f90
gfortran -o prog kreis/kreis.o kreis/kreissegment.o quadrat/quadrat.o main.o
scons: done building targets.
```

Gestartet wird das erstellte Programm in diesem Beispiel mittels `./prog`. Ausgabe:

```
Ich bin ein Kreis!
Ich bin ein Quadrat!
Ich bin ein Kreis!
Hihi, war nur ein Scherz. Ich bin ein Kreissegment!
```

Bereinigt werden kann das Verzeichnis (oder können die Verzeichnisse) mit `scons -c`. Als Ausgabe erscheint dann in der Konsole:

```
scons: Reading SConscript files ...
scons: done reading SConscript files.
scons: Cleaning targets ...
Removed kreis/kreis.o
Removed kreis.mod
Removed kreis/kreissegment.o
Removed kreissegment.mod
Removed main.o
Removed quadrat/quadrat.o
Removed quadrat.mod
Removed prog
scons: done cleaning targets.
```

Das wars. Für weiterführende Informationen siehe die nachfolgenden Weblinks.

81.5.2. Weblinks

- [w:SCons](#)¹⁶¹
- [SCons-Homepage](#)¹⁶²
- [SCons Wiki - MultipleDirectoryFortranBuild](#)¹⁶³

¹⁶¹ <https://de.wikipedia.org/wiki/SCons>

¹⁶² <http://www.scons.org/>

¹⁶³ <http://www.scons.org/wiki/MultipleDirectoryFortranBuild?highlight=%28MultipleDirectoryFortranBuild%29>

82. Style Guides, Best Practices

Es gibt viele Arten, moderne Fortran-Programme zu schreiben. Einrückungen, Groß-/Kleinschreibung, Camel-Case/Snake-Case etc. All das kann variiert werden. Es ist deshalb sinnvoll, eine einheitliche Schreibweise (zumindest projektweise) zu etablieren. Zu diesem Zweck gibt es Style-Guides, zu denen nachfolgend einige Weblinks angegeben werden. Diese Guides stellen aber keine normierten Standards dar, sondern sind "Best Practices", also Vorschläge, die sich bewährt haben.

- MODFLOW 6 Fortran style guide¹⁶⁴
- Fortran: Module Design¹⁶⁵
- Fortran Coding Standards for JULES¹⁶⁶
- Fortran Best Practices¹⁶⁷

¹⁶⁴ https://modflow6.readthedocs.io/en/stable/_dev/styleguide.html

¹⁶⁵ <https://matthiasnoback.nl/2025/06/fortran-module-design/>

¹⁶⁶ https://jules-lsm.github.io/coding_standards/guidelines/style.html

¹⁶⁷ <https://www.fortran90.org/src/best-practices.html>

Teil XI.

Weblinks

82.1. Wikis

- Wikipedia zum Thema Fortran¹⁶⁸
- Das englischsprachige Fortran-Wikibook¹⁶⁹
- Fortran Wiki¹⁷⁰

82.2. Standards

- GFortran Standards Documents¹⁷¹
- J3 Fortran Standards Technical Committee¹⁷²
- Fortran Standards¹⁷³

82.3. Skripten, Tutorials, Bücher

82.3.1. Allgemeines

- Intel-Fortran-Compiler Developer Guide and Reference¹⁷⁴

82.3.2. Fortran 2023

- The New Features of Fortran 2023 (John Reid) (PDF)¹⁷⁵

82.3.3. Fortran 2018

- The new features of Fortran 2018 (John Reid) (PDF)¹⁷⁶
- Programming in Modern Fortran (A HyperText-based introduction to programming in Fortran 2003, 2008, and 2018 on Unix)¹⁷⁷

82.3.4. Fortran 2003/2008

- The New Features of Fortran 2003 (John Reid) (PDF)¹⁷⁸
- The New Features of Fortran 2008 (John Reid) (PDF)¹⁷⁹
- Working Draft J3/04-007¹⁸⁰
- Brainerd: Guide to Fortran 2008 Programming; Springer, 2015, ISBN 978-1-4471-6889-8

¹⁶⁸ <http://de.wikipedia.org/wiki/Fortran>

¹⁶⁹ <http://en.wikibooks.org/wiki/Fortran>

¹⁷⁰ <https://fortranwiki.org/fortran/show/HomePage>

¹⁷¹ <https://gcc.gnu.org/wiki/GFortranStandards>

¹⁷² <https://www.j3-fortran.org/>

¹⁷³ <https://www.fortran90.org/>

¹⁷⁴ <https://www.intel.com/content/www/us/en/docs/fortran-compiler/developer-guide-reference/2025-3/overview.html>

¹⁷⁵ <https://wg5-fortran.org/N2201-N2250/N2212.pdf>

¹⁷⁶ <https://wg5-fortran.org/N2101-N2150/N2145.pdf>

¹⁷⁷ <https://cyber.dabamos.de/programming/modernfortran/>

¹⁷⁸ <https://wg5-fortran.org/N1601-N1650/N1648.pdf>

¹⁷⁹ <https://wg5-fortran.org/N1801-N1850/N1828.pdf>

¹⁸⁰ <https://j3-fortran.org/doc/year/04/04-007.pdf>

82.3.5. Fortran 90/95

- Wissenschaftliches Programmieren in Fortran (Meik Hellmund, Uni Leipzig) PDF¹⁸¹
- Programmieren in Fortran 90/95 (Dr. Heidrun Kolinsky, Rechenzentrum der Universität Bayreuth, PDF)¹⁸²
- The Key Features of Fortran 95 (Adams, Brainerd, Martin, Smith - The Fortran Company) PDF¹⁸³
- Numerical Recipes in Fortran 90 Part 2 (PDF)¹⁸⁴

82.3.6. FORTRAN 77

- Fortran 77 Language Reference Manual, Universität Innsbruck (PDF)¹⁸⁵
- FORTRAN 77 Full Language ANSI X3J3/90.4¹⁸⁶
- Numerical Recipes in Fortran 77 (PDF)¹⁸⁷

82.4. Compiler

- GNU Fortran¹⁸⁸
- Intel Fortran Compiler¹⁸⁹
- Open Watcom¹⁹⁰
- Silverfrost FTN95 Compiler¹⁹¹

82.5. Debugger

- Der GNU Debugger¹⁹²

82.6. Amüsantes

- Real Programmers Don't Use PASCAL¹⁹³

181 <https://www.math.uni-leipzig.de/~hellmund/Vorlesung/pres2.pdf>

182 <https://www.univie.ac.at/adg/Teaching/Armueb12/effortran/fortran.pdf>

183 https://monoceros.physics.muni.cz/~jancely/NM/Jazyky/G95_F/doc/Key_Features_of_F95.pdf

184 <http://www.elch.chem.msu.ru/tch/group/FortranBooks/NumericalRecipesinF90.pdf>

185 http://physik.uibk.ac.at/hephy/praktikum/fortran_manual.pdf

186 <https://wg5-fortran.org/ARCHIVE/Fortran77.html>

187 <https://belglas.com/wp-content/uploads/2019/10/numericalrecipesinf77.pdf>

188 <http://gcc.gnu.org/fortran/>

189 <http://software.intel.com/en-us/articles/fortran-compilers/>

190 <https://sourceforge.net/projects/openwatcom/>

191 <https://www.silverfrost.com/>

192 <https://sourceware.org/gdb/>

193 <https://www.ee.torontomu.ca/~elf/hack/realm.html>

82.7. Sonstiges

- Fortran High-performance parallel programming language¹⁹⁴

¹⁹⁴ <https://fortran-lang.org>

Teil XII.

Stichwortverzeichnis

UNKNOWN TEMPLATE Vorlage:Navigationsleiste Alphabet

- ****** (Potenz-Operator)¹⁹⁵
- **+, -, *, /, **** (Arithmetische Operatoren, FORTRAN 77)¹⁹⁶
- **+, -, *, /, **** (Arithmetische Operatoren, Fortran 95)¹⁹⁷
- **=** (Wertzuweisung, FORTRAN 77)¹⁹⁸
- **=** (Wertzuweisung, Fortran 95)¹⁹⁹
- **<, <=, >, >=, ==, /=** (Vergleichsoperatoren, Fortran 95)²⁰⁰

82.8. A

- **abs** (Fortran 95)²⁰¹
- **ABS** (FORTRAN 77)²⁰²
- **Absolutwert** (FORTRAN 77)²⁰³
- **Absolutwert** (Fortran 95)²⁰⁴
- **ACOS** (FORTRAN 77)²⁰⁵
- **adjustl** (Fortran 95)²⁰⁶
- **adjustr** (Fortran 95)²⁰⁷
- **aimag** (Fortran 95)²⁰⁸
- **AIMAG** (FORTRAN 77)²⁰⁹
- **aint** (Fortran 95)²¹⁰
- **AINT** (FORTRAN 77)²¹¹
- **allocate** (Fortran 95)²¹²
- **allocated** (Fortran 95)²¹³
- **ALOG** (FORTRAN 77)²¹⁴

¹⁹⁵ Kapitel 1.2 auf Seite 5

¹⁹⁶ Kapitel 9.1 auf Seite 49

¹⁹⁷ Kapitel 21.1 auf Seite 131

¹⁹⁸ Kapitel 7.4 auf Seite 43

¹⁹⁹ Kapitel 19.4 auf Seite 122

²⁰⁰ Kapitel 23.1 auf Seite 135

²⁰¹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Absolutwert%20

²⁰² Kapitel 14.2.3 auf Seite 70

²⁰³ Kapitel 14.2.3 auf Seite 70

²⁰⁴ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Absolutwert%20

²⁰⁵ Kapitel 14.2.16 auf Seite 73

²⁰⁶ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

²⁰⁷ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

²⁰⁸ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Komplexe%20Zahlen%20

²⁰⁹ Kapitel 14.2.10 auf Seite 71

²¹⁰ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische%20Funktionen%23Rundung%20

²¹¹ Kapitel 14.2.2 auf Seite 70

²¹² Kapitel 31.4 auf Seite 203

²¹³ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Abfragen_von_Feldstatus%2C_Felddaten_und_Feldmetadaten%20

²¹⁴ Kapitel 14.2.13 auf Seite 72

-
- ALOG10 (FORTRAN 77)²¹⁵
 - AMAX0 (FORTRAN 77)²¹⁶
 - AMAX1 (FORTRAN 77)²¹⁷
 - AMIN0 (FORTRAN 77)²¹⁸
 - AMIN1 (FORTRAN 77)²¹⁹
 - AMOD (FORTRAN 77)²²⁰
 - .AND. (FORTRAN 77)²²¹
 - .AND. (Fortran 95)²²²
 - anint (Fortran 95)²²³
 - ANINT (FORTRAN 77)²²⁴
 - any (Fortran 95)²²⁵
 - Arithmetisches IF (FORTRAN 77)²²⁶
 - Arkusfunktionen (FORTRAN 77)²²⁷
 - Arkusfunktionen (Fortran 95)²²⁸
 - Array constructor (Fortran 95)²²⁹
 - asin (Fortran 95)²³⁰
 - ASIN (FORTRAN 77)²³¹
 - ASSIGN (FORTRAN 77)²³²
 - Assigned GOTO (FORTRAN 77)²³³
 - associated (Zeigerfunktion, Fortran 95)²³⁴
 - associated (Assoziationsstatus, Fortran 95)²³⁵
 - Assoziationsstatus (Fortran 95)²³⁶
 - atan (Fortran 95)²³⁷

215 Kapitel 14.2.14 auf Seite 72

216 Kapitel 14.2.8 auf Seite 71

217 Kapitel 14.2.8 auf Seite 71

218 Kapitel 14.2.9 auf Seite 71

219 Kapitel 14.2.9 auf Seite 71

220 Kapitel 14.2.5 auf Seite 71

221 Kapitel 10.1 auf Seite 53

222 Kapitel 22.1 auf Seite 133

223 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Rundung%20

224 Kapitel 14.2.2 auf Seite 70

225 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Abfragen_von_Feldstatus%2C_Felddaten_und_Feldmetadaten%20

226 Kapitel 17.4 auf Seite 107

227 Kapitel 14.2.16 auf Seite 73

228 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Arkusfunktionen%20

229 Kapitel 20.4.1 auf Seite 129

230 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Arkusfunktionen%20

231 Kapitel 14.2.16 auf Seite 73

232 Kapitel 17.5 auf Seite 108

233 Kapitel 17.5 auf Seite 108

234 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Zeigerfunktionen%20

235 Kapitel 31.3 auf Seite 202

236 Kapitel 31.3 auf Seite 202

237 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Arkusfunktionen%20

- ATAN (FORTRAN 77)²³⁸
- atan2 (Fortran 95)²³⁹
- ATAN2 (FORTRAN 77)²⁴⁰
- ATLAS, Automatically Tuned Linear Algebra Software²⁴¹

82.9. B

- Bedingtes GOTO (FORTRAN 77)²⁴²
- Benannte Konstanten (FORTRAN 77)²⁴³
- Benannte Konstanten (Fortran 95)²⁴⁴
- Binärzahl (Fortran 95)²⁴⁵
- bit_size (Fortran 95)²⁴⁶
- Bitfunktionen (Fortran 95)²⁴⁷
- BLAS, Basic Linear Algebra Subprograms²⁴⁸
- btest (Fortran 95)²⁴⁹

82.10. C

- C (Anhang A)²⁵⁰
- C (Fortran und C)²⁵¹
- CABS (FORTRAN 77)²⁵²
- call (Fortran 95)²⁵³
- CALL (FORTRAN 77)²⁵⁴
- CCOS (FORTRAN 77)²⁵⁵
- ceiling (Fortran 95)²⁵⁶
- CEXP (FORTRAN 77)²⁵⁷

238 Kapitel 14.2.16 auf Seite 73

239 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Arkusfunktionen%20

240 Kapitel 14.2.16 auf Seite 73

241 Kapitel 46.1 auf Seite 297

242 Kapitel 13.3 auf Seite 62

243 Kapitel 7.3 auf Seite 42

244 Kapitel 19.3 auf Seite 122

245 Kapitel 19.1.1 auf Seite 119

246 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

247 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

248 Kapitel 46.1 auf Seite 297

249 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

250 Kapitel 59 auf Seite 373

251 https://de.wikibooks.org/wiki/Fortran%3A_Fortran_und_C%20

252 Kapitel 14.2.3 auf Seite 70

253 Kapitel 29.3 auf Seite 174

254 Kapitel 15.3 auf Seite 78

255 Kapitel 14.2.15 auf Seite 72

256 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Rundung%20

257 Kapitel 14.2.12 auf Seite 72

-
- char (Fortran 95)²⁵⁸
 - CHAR (FORTRAN 77)²⁵⁹
 - character (Fortran 95)²⁶⁰
 - CHARACTER (FORTRAN 77)²⁶¹
 - CLOG (FORTRAN 77)²⁶²
 - close (Fortran 95)²⁶³
 - CLOSE (FORTRAN 77)²⁶⁴
 - cmplx (Fortran 95)²⁶⁵
 - CMPLX (FORTRAN 77)²⁶⁶
 - COMMON (FORTRAN 77)²⁶⁷
 - Compiler²⁶⁸
 - complex (Fortran 95)²⁶⁹
 - COMPLEX (FORTRAN 77)²⁷⁰
 - conjg (Fortran 95)²⁷¹
 - CONJG (FORTRAN 77)²⁷²
 - CONTINUE (FORTRAN 77)²⁷³
 - cos (Fortran 95)²⁷⁴
 - COS (FORTRAN 77)²⁷⁵
 - cosh (Fortran 95)²⁷⁶
 - COSH (FORTRAN 77)²⁷⁷
 - cshift (Fortran 95)²⁷⁸
 - CSIN (FORTRAN 77)²⁷⁹
 - CSQRT (FORTRAN 77)²⁸⁰

-
- 258 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23Umwandlung%20in%20CHARACTER%20
 - 259 Kapitel 14.1.5 auf Seite 70
 - 260 Kapitel 19.1.3 auf Seite 120
 - 261 Kapitel 7.1.3 auf Seite 39
 - 262 Kapitel 14.2.13 auf Seite 72
 - 263 Kapitel 30.5.4 auf Seite 192
 - 264 Kapitel 16.4.4 auf Seite 96
 - 265 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23Umwandlung%20in%20COMPLEX%20
 - 266 Kapitel 14.1.4 auf Seite 70
 - 267 Kapitel 15.6 auf Seite 83
 - 268 Kapitel 1.1.1 auf Seite 4
 - 269 Kapitel 19.1.1 auf Seite 119
 - 270 Kapitel 7.1.1 auf Seite 39
 - 271 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Komplexe%20Zahlen%20
 - 272 Kapitel 14.2.10 auf Seite 71
 - 273 Kapitel 13.2 auf Seite 61
 - 274 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Winkelfunktion%20
 - 275 Kapitel 14.2.15 auf Seite 72
 - 276 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Hyperbelfunktionen%20
 - 277 Kapitel 14.2.17 auf Seite 73
 - 278 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Sonstige%20
 - 279 Kapitel 14.2.15 auf Seite 72
 - 280 Kapitel 14.2.11 auf Seite 72

- cycle (Fortran 95)²⁸¹

82.11. D

- DABS (FORTRAN 77)²⁸²
- DACOS (FORTRAN 77)²⁸³
- DASIN (FORTRAN 77)²⁸⁴
- DATA (FORTRAN 77)²⁸⁵
- DATAN (FORTRAN 77)²⁸⁶
- DATAN2 (FORTRAN 77)²⁸⁷
- Datei (FORTRAN 77)²⁸⁸
- Datei (Fortran 95)²⁸⁹
- Datensatz (FORTRAN 77)²⁹⁰
- Datentyp (FORTRAN 77)²⁹¹
- Datentyp (Fortran 95)²⁹²
- Datentyp höherer Genauigkeit (Fortran 95)²⁹³
- Datentypumwandlung (FORTRAN 77)²⁹⁴
- Datenverbund (Fortran 95)²⁹⁵
- DBLE (FORTRAN 77)²⁹⁶
- DCOS (FORTRAN 77)²⁹⁷
- DCOSH (FORTRAN 77)²⁹⁸
- DDIM (FORTRAN 77)²⁹⁹
- deallocate (Fortran 95)³⁰⁰
- Debugger (Anhang C)³⁰¹
- DEXP (FORTRAN 77)³⁰²
- dim (Fortran 95)³⁰³

281 Kapitel 25.6.3 auf Seite 154

282 Kapitel 14.2.3 auf Seite 70

283 Kapitel 14.2.16 auf Seite 73

284 Kapitel 14.2.16 auf Seite 73

285 Kapitel 15.9 auf Seite 86

286 Kapitel 14.2.16 auf Seite 73

287 Kapitel 14.2.16 auf Seite 73

288 Kapitel 16.4 auf Seite 94

289 Kapitel 30.5 auf Seite 190

290 Kapitel 16.4.1 auf Seite 95

291 Kapitel 7.1 auf Seite 39

292 Kapitel 19.1 auf Seite 119

293 Kapitel 26 auf Seite 157

294 Kapitel 14.1 auf Seite 69

295 Kapitel 27 auf Seite 161

296 Kapitel 14.1.3 auf Seite 69

297 Kapitel 14.2.15 auf Seite 72

298 Kapitel 14.2.17 auf Seite 73

299 Kapitel 14.2.7 auf Seite 71

300 Kapitel 31.4 auf Seite 203

301 Kapitel 78 auf Seite 419

302 Kapitel 14.2.12 auf Seite 72

303 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Positive%20Differenz%20

-
- DIM (FORTRAN 77)³⁰⁴
 - DINT (FORTRAN 77)³⁰⁵
 - DLOG (FORTRAN 77)³⁰⁶
 - DISLIN³⁰⁷
 - DLOG10 (FORTRAN 77)³⁰⁸
 - DMAX1 (FORTRAN 77)³⁰⁹
 - DMIN1 (FORTRAN 77)³¹⁰
 - DMOD (FORTRAN 77)³¹¹
 - DNINT (FORTRAN 77)³¹²
 - do-if-exit (Fortran 95)³¹³
 - do-Liste (Fortran 95)³¹⁴
 - DO-Schleife (FORTRAN 77)³¹⁵
 - dot_product (Fortran 95)³¹⁶
 - double precision (Fortran 95)³¹⁷
 - DOUBLE PRECISION (FORTRAN 77)³¹⁸
 - do while (Fortran 95)³¹⁹
 - do-Zählschleife (Fortran 95)³²⁰
 - DPROD (FORTRAN 77)³²¹
 - DSIGN (FORTRAN 77)³²²
 - DSIN (FORTRAN 77)³²³
 - DSINH (FORTRAN 77)³²⁴
 - DSQRT (FORTRAN 77)³²⁵
 - DTAN (FORTRAN 77)³²⁶
 - DTANH (FORTRAN 77)³²⁷
 - Dynamische Speicherallokation (Fortran 95)³²⁸

304 Kapitel 14.2.7 auf Seite 71

305 Kapitel 14.2.1 auf Seite 70

306 Kapitel 14.2.13 auf Seite 72

307 Kapitel 42.1 auf Seite 279

308 Kapitel 14.2.14 auf Seite 72

309 Kapitel 14.2.8 auf Seite 71

310 Kapitel 14.2.9 auf Seite 71

311 Kapitel 14.2.5 auf Seite 71

312 Kapitel 14.2.2 auf Seite 70

313 Kapitel 25.4.1 auf Seite 145

314 Kapitel 25.4.4 auf Seite 148

315 Kapitel 13.5 auf Seite 64

316 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Funktionen_f%C3%BCr_Vektoren_und_Matrizen%20

317 Kapitel 19.1.1 auf Seite 119

318 Kapitel 7.1.1 auf Seite 39

319 Kapitel 25.4.3 auf Seite 148

320 Kapitel 25.4.2 auf Seite 146

321 Kapitel 14.2.4 auf Seite 70

322 Kapitel 14.2.6 auf Seite 71

323 Kapitel 14.2.15 auf Seite 72

324 Kapitel 14.2.17 auf Seite 73

325 Kapitel 14.2.11 auf Seite 72

326 Kapitel 14.2.15 auf Seite 72

327 Kapitel 14.2.17 auf Seite 73

328 Kapitel 20.2.2 auf Seite 126

82.12. E

- Ein- und Ausgabe (FORTRAN 77)³²⁹
- Ein- und Ausgabe (Fortran 95)³³⁰
- else-if (Fortran 95)³³¹
- ENTRY (FORTRAN 77)³³²
- epsilon (Fortran 95)³³³
- .EQ. (FORTRAN 77)³³⁴
- .EQV. (FORTRAN 77)³³⁵
- .EQV. (Fortran 95)³³⁶
- Ergebnisdatentyp (FORTRAN 77)³³⁷
- Ergebnisdatentyp (Fortran 95)³³⁸
- exit (Fortran 95)³³⁹
- exp (Fortran 95)³⁴⁰
- EXP (FORTRAN 77)³⁴¹
- Explizite Typzuweisung (FORTRAN 77)³⁴²
- Explizite Typzuweisung (Fortran 95)³⁴³
- Explizite Typumwandlung (FORTRAN 77)³⁴⁴
- Explizite Typumwandlung (Fortran 95)³⁴⁵
- exponent (Fortran 95)³⁴⁶
- Exponentialfunktion (FORTRAN 77)³⁴⁷
- Exponentialfunktion (Fortran 95)³⁴⁸
- external (Fortran 95)³⁴⁹
- EXTERNAL (FORTRAN 77)³⁵⁰

329 Kapitel 16 auf Seite 89

330 Kapitel 30 auf Seite 183

331 Kapitel 25.2.4 auf Seite 142

332 Kapitel 15.7 auf Seite 84

333 [https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_](https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Weitere%20Funktionen%20)
 Weitere%20Funktionen%20

334 Kapitel 11.1 auf Seite 55

335 Kapitel 10.1 auf Seite 53

336 Kapitel 22.1 auf Seite 133

337 Kapitel 9.2 auf Seite 50

338 Kapitel 21.2 auf Seite 131

339 Kapitel 25.6.2 auf Seite 153

340 [https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_](https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Exponentialfunktion%20)
 Mathematische_Funktionen%23Exponentialfunktion%20

341 Kapitel 14.2.12 auf Seite 72

342 Kapitel 7.2.2 auf Seite 42

343 Kapitel 19.2.2 auf Seite 121

344 Kapitel 9.2.3 auf Seite 51

345 Kapitel 21.2.3 auf Seite 132

346 [https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_](https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Weitere%20Funktionen%20)
 Weitere%20Funktionen%20

347 Kapitel 14.2.12 auf Seite 72

348 [https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_](https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Exponentialfunktion%20)
 Mathematische_Funktionen%23Exponentialfunktion%20

349 Kapitel 29.5 auf Seite 178

350 Kapitel 15.5 auf Seite 82

82.13. F

- F³⁵¹
- f2c³⁵²
- Feld (FORTRAN 77)³⁵³
- Feld (Fortran 95)³⁵⁴
- Feld (Parameter, Fortran 95)³⁵⁵
- Feld (Parameter, FORTRAN 77)³⁵⁶
- Feldfunktion (Fortran 95)³⁵⁷
- Feldinitialisierung (Fortran 95)³⁵⁸
- FLOAT (FORTRAN 77)³⁵⁹
- floor (Fortran 95)³⁶⁰
- forall-Schleife (Fortran 95)³⁶¹
- format (Fortran 95)³⁶²
- FORMAT (FORTRAN 77)³⁶³
- Formatierung (FORTRAN 77)³⁶⁴
- Formatierung (Fortran 95)³⁶⁵
- Fortran³⁶⁶
- FORTRAN I³⁶⁷
- FORTRAN II³⁶⁸
- FORTRAN IV³⁶⁹
- FORTRAN 66³⁷⁰
- FORTRAN 77³⁷¹
- FORTRAN 77 (Einleitung)³⁷²
- Fortran 90³⁷³
- Fortran 95³⁷⁴

351 Kapitel 56 auf Seite 359

352 Kapitel 1.1.1 auf Seite 4

353 Kapitel 8 auf Seite 45

354 Kapitel 20 auf Seite 125

355 Kapitel 29.4 auf Seite 177

356 Kapitel 15.4 auf Seite 79

357 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%20

358 Kapitel 20.4 auf Seite 128

359 Kapitel 14.1.2 auf Seite 69

360 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Rundung%20

361 Kapitel 25.5.2 auf Seite 150

362 Kapitel 30.4 auf Seite 187

363 Kapitel 16.3 auf Seite 92

364 Kapitel 16.3 auf Seite 92

365 Kapitel 30.4 auf Seite 187

366 Kapitel 1 auf Seite 3

367 Kapitel 1.1.1 auf Seite 4

368 Kapitel 1.1.1 auf Seite 4

369 Kapitel 1.1.1 auf Seite 4

370 Kapitel 1.1.1 auf Seite 4

371 https://de.wikibooks.org/wiki/Fortran%3A_FORTRAN_77%20

372 Kapitel 1.1.1 auf Seite 4

373 Kapitel 1.1.1 auf Seite 4

374 https://de.wikibooks.org/wiki/Fortran%3A_Fortran_95%20

- Fortran 95 (Einleitung)³⁷⁵
- Fortran 2003³⁷⁶
- Fortran 2003 (Einleitung)³⁷⁷
- Fortsetzungszeile (FORTRAN 77)³⁷⁸
- Free source form (Fortran 95)³⁷⁹
- Freies Zeilenformat (Fortran 95)³⁸⁰
- function (Fortran 95)³⁸¹
- FUNCTION (FORTRAN 77)³⁸²
- Funktionsanweisung (FORTRAN 77)³⁸³

82.14. G

- .GE. (FORTRAN 77)³⁸⁴
- getarg (Fortran 95)³⁸⁵
- GNU Fortran (gfortran)³⁸⁶
- GOTO (FORTRAN 77)³⁸⁷
- GOTO (Assigned, FORTRAN 77)³⁸⁸
- GOTO (Bedingt, FORTRAN 77)³⁸⁹
- Graphical User Interface (DISLIN)³⁹⁰
- .GT. (FORTRAN 77)³⁹¹
- GTK+ (gtk-fortran)³⁹²

82.15. H

- Hauptprogramm (FORTRAN 77)³⁹³
- Hauptprogramm (Fortran 95)³⁹⁴
- Hexadezimalzahl (Fortran 95)³⁹⁵

375 Kapitel 1.1.1 auf Seite 4

376 https://de.wikibooks.org/wiki/Fortran%3A_Fortran_2003%20

377 Kapitel 1.1.1 auf Seite 4

378 Kapitel 6.2 auf Seite 33

379 Kapitel 18.2 auf Seite 113

380 Kapitel 18.2 auf Seite 113

381 Kapitel 29.3 auf Seite 174

382 Kapitel 15.2 auf Seite 76

383 Kapitel 15.1 auf Seite 75

384 Kapitel 11.1 auf Seite 55

385 Kapitel 33.3 auf Seite 213

386 Kapitel 2 auf Seite 13

387 Kapitel 13.1 auf Seite 61

388 Kapitel 17.5 auf Seite 108

389 Kapitel 13.3 auf Seite 62

390 Kapitel 42.1 auf Seite 279

391 Kapitel 11.1 auf Seite 55

392 Kapitel 45.6 auf Seite 293

393 Kapitel 6.3 auf Seite 35

394 Kapitel 18.3 auf Seite 115

395 Kapitel 19.1.1 auf Seite 119

-
- Hollerith (FORTRAN 77)³⁹⁶
 - HPF, High Performance Fortran³⁹⁷
 - Hyperbelfunktion (FORTRAN 77)³⁹⁸
 - Hyperbelfunktionen (Fortran 95)³⁹⁹

82.16. I

- IABS (FORTRAN 77)⁴⁰⁰
- iand (Fortran 95)⁴⁰¹
- iargc (Fortran 95)⁴⁰²
- ibclr (Fortran 95)⁴⁰³
- ibset (Fortran 95)⁴⁰⁴
- ichar (Fortran 95)⁴⁰⁵
- ICHAR (FORTRAN 77)⁴⁰⁶
- IDIM (FORTRAN 77)⁴⁰⁷
- IDINT (FORTRAN 77)⁴⁰⁸
- IDNINT (FORTRAN 77)⁴⁰⁹
- ieor (Fortran 95)⁴¹⁰
- IF (Arithmetisch, FORTRAN 77)⁴¹¹
- IF-Verweigung (FORTRAN 77)⁴¹²
- if-Einzeiler (Fortran 95)⁴¹³
- if-then (Fortran 95)⁴¹⁴
- if-then-else (Fortran 95)⁴¹⁵
- implicit (Fortran 95)⁴¹⁶

³⁹⁶ Kapitel 17.3 auf Seite 106

³⁹⁷ Kapitel 1.1.2 auf Seite 5

³⁹⁸ Kapitel 14.2.17 auf Seite 73

³⁹⁹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Hyperbelfunktionen%20

⁴⁰⁰ Kapitel 14.2.3 auf Seite 70

⁴⁰¹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

⁴⁰² Kapitel 33.3 auf Seite 213

⁴⁰³ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

⁴⁰⁴ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

⁴⁰⁵ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23Umwandlung%20in%20INTEGER%20

⁴⁰⁶ Kapitel 14.1.1 auf Seite 69

⁴⁰⁷ Kapitel 14.2.7 auf Seite 71

⁴⁰⁸ Kapitel 14.1.1 auf Seite 69

⁴⁰⁹ Kapitel 14.2.2 auf Seite 70

⁴¹⁰ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

⁴¹¹ Kapitel 17.4 auf Seite 107

⁴¹² Kapitel 13.4 auf Seite 62

⁴¹³ Kapitel 25.2.1 auf Seite 139

⁴¹⁴ Kapitel 25.2.2 auf Seite 141

⁴¹⁵ Kapitel 25.2.3 auf Seite 141

⁴¹⁶ Kapitel 19.2.1 auf Seite 121

- IMPLICIT (FORTRAN 77)⁴¹⁷
- Implizite Typumwandlung (FORTRAN 77)⁴¹⁸
- Implizite Typumwandlung (Fortran 95)⁴¹⁹
- INCLUDE (FORTRAN 77)⁴²⁰
- index (Fortran 95)⁴²¹
- INDEX (FORTRAN 77)⁴²²
- int (Fortran 95)⁴²³
- INT (FORTRAN 77)⁴²⁴
- integer (Fortran 95)⁴²⁵
- INTEGER (FORTRAN 77)⁴²⁶
- Intel Fortran Compiler⁴²⁷
- intent (Fortran 95)⁴²⁸
- INTERFACE (Fortran und C)⁴²⁹
- intrinsic (Fortran 95)⁴³⁰
- INTRINSIC (FORTRAN 77)⁴³¹
- Intrinsic function (FORTRAN 77)⁴³²
- Intrinsic function (Fortran 95)⁴³³
- ior (Fortran 95)⁴³⁴
- ishft (Fortran 95)⁴³⁵
- ishftc (Fortran 95)⁴³⁶
- ISIGN (FORTRAN 77)⁴³⁷

417 Kapitel 7.2.1 auf Seite 40

418 Kapitel 9.2.2 auf Seite 50

419 Kapitel 21.2.2 auf Seite 132

420 Kapitel 17.2 auf Seite 105

421 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%20

422 Kapitel 14.3.2 auf Seite 73

423 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23Umwandlung%20in%20INTEGER%20

424 Kapitel 14.1.1 auf Seite 69

425 Kapitel 19.1.1 auf Seite 119

426 Kapitel 7.1.1 auf Seite 39

427 Kapitel 3 auf Seite 19

428 Kapitel 29.3 auf Seite 174

429 Kapitel 54.1 auf Seite 335

430 Kapitel 29.5 auf Seite 178

431 Kapitel 15.5 auf Seite 82

432 Kapitel 14 auf Seite 69

433 Kapitel 28 auf Seite 167

434 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

435 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

436 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

437 Kapitel 14.2.6 auf Seite 71

82.17. J

82.18. K

- kind (Fortran 95)⁴³⁸
- kind (Funktion, Fortran 95)⁴³⁹
- Kommandozeilenargument (Fortran 95)⁴⁴⁰
- Kommentar (Fortran 95)⁴⁴¹
- Kommentarzeile (FORTRAN 77)⁴⁴²
- Komplexe Zahlen (FORTRAN 77)⁴⁴³
- Komplexe Zahlen (Fortran 95)⁴⁴⁴

82.19. L

- LAPACK, Linear Algebra Package⁴⁴⁵
- lbound (Fortran 95)⁴⁴⁶
- .LE. (FORTRAN 77)⁴⁴⁷
- len (Fortran 95)⁴⁴⁸
- LEN (FORTRAN 77)⁴⁴⁹
- len_trim (Fortran 95)⁴⁵⁰
- Lexikalische Funktionen (FORTRAN 77)⁴⁵¹
- Lexikalische Funktionen (Fortran 95)⁴⁵²
- lge (Fortran 95)⁴⁵³
- LGE (FORTRAN 77)⁴⁵⁴
- lgt (Fortran 95)⁴⁵⁵

438 Kapitel 26.1 auf Seite 157

439 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23kind-Parameter%20

440 Kapitel 33.3 auf Seite 213

441 Kapitel 18.1 auf Seite 113

442 Kapitel 6.2 auf Seite 33

443 Kapitel 14.2.10 auf Seite 71

444 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Komplexe%20Zahlen%20

445 Kapitel 50.5 auf Seite 310

446 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Abfragen_von_Feldstatus%2C_Felddaten_und_Feldmetadaten%20

447 Kapitel 11.1 auf Seite 55

448 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

449 Kapitel 14.3.1 auf Seite 73

450 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

451 Kapitel 14.3.3 auf Seite 73

452 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Lexikalische%20Funktionen%20

453 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Lexikalische%20Funktionen%20

454 Kapitel 14.3.3 auf Seite 73

455 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Lexikalische%20Funktionen%20

- LGT (FORTRAN 77)⁴⁵⁶
- Lineares Gleichungssystem (LAPACK)⁴⁵⁷
- lle (Fortran 95)⁴⁵⁸
- LLE (FORTRAN 77)⁴⁵⁹
- llt (Fortran 95)⁴⁶⁰
- LLT (FORTRAN 77)⁴⁶¹
- Logarithmus (Fortran 95)⁴⁶²
- Logarithmus (FORTRAN 77)⁴⁶³
- logical (Fortran 95)⁴⁶⁴
- LOGICAL (FORTRAN 77)⁴⁶⁵
- Logische Operatoren (FORTRAN 77)⁴⁶⁶
- .LT. (FORTRAN 77)⁴⁶⁷

82.20. M

- Matrixoperation⁴⁶⁸
- Matrizenrechnung (BLAS und ATLAS)⁴⁶⁹
- Matrizenrechnung (Fortran 95)⁴⁷⁰
- max (Fortran 95)⁴⁷¹
- MAX0 (FORTRAN 77)⁴⁷²
- MAX1 (FORTRAN 77)⁴⁷³
- Maximum (FORTRAN 77)⁴⁷⁴
- Maximum (Fortran 95)⁴⁷⁵
- Mehrdimensionales Feld (FORTRAN 77)⁴⁷⁶
- Mehrdimensionales Feld (Fortran 95)⁴⁷⁷

⁴⁵⁶ Kapitel 14.3.3 auf Seite 73

⁴⁵⁷ Kapitel 50.5 auf Seite 310

⁴⁵⁸ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Lexikalische%20Funktionen%20

⁴⁵⁹ Kapitel 14.3.3 auf Seite 73

⁴⁶⁰ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Lexikalische%20Funktionen%20

⁴⁶¹ Kapitel 14.3.3 auf Seite 73

⁴⁶² https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Logarithmen%20

⁴⁶³ Kapitel 14.2.13 auf Seite 72

⁴⁶⁴ Kapitel 19.1.2 auf Seite 120

⁴⁶⁵ Kapitel 7.1.2 auf Seite 39

⁴⁶⁶ Kapitel 10.1 auf Seite 53

⁴⁶⁷ Kapitel 11.1 auf Seite 55

⁴⁶⁸ Kapitel 1.2 auf Seite 5

⁴⁶⁹ Kapitel 46.1 auf Seite 297

⁴⁷⁰ Kapitel 32 auf Seite 211

⁴⁷¹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Maximum%20

⁴⁷² Kapitel 14.2.8 auf Seite 71

⁴⁷³ Kapitel 14.2.8 auf Seite 71

⁴⁷⁴ Kapitel 14.2.8 auf Seite 71

⁴⁷⁵ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Maximum%20

⁴⁷⁶ Kapitel 8.2 auf Seite 46

⁴⁷⁷ Kapitel 20.3 auf Seite 127

-
- min (Fortran 95)⁴⁷⁸
 - MIN0 (FORTRAN 77)⁴⁷⁹
 - MIN1 (FORTRAN 77)⁴⁸⁰
 - Minimum (Fortran 95)⁴⁸¹
 - Minimum (FORTRAN 77)⁴⁸²
 - mod (Fortran 95)⁴⁸³
 - MOD (FORTRAN 77)⁴⁸⁴
 - Modul (Fortran 95)⁴⁸⁵
 - Modulo (FORTRAN 77)⁴⁸⁶
 - Modulo (Fortran 95)⁴⁸⁷

82.21. N

- .NE. (FORTRAN 77)⁴⁸⁸
- .NEQV. (FORTRAN 77)⁴⁸⁹
- .NEQV. (Fortran 95)⁴⁹⁰
- nint (Fortran 95)⁴⁹¹
- NINT (FORTRAN 77)⁴⁹²
- .NOT. (FORTRAN 77)⁴⁹³
- .NOT. (Fortran 95)⁴⁹⁴
- not (Fortran 95)⁴⁹⁵
- null (Fortran 95)⁴⁹⁶
- nullify (Fortran 95)⁴⁹⁷

478 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Minimum%20

479 Kapitel 14.2.9 auf Seite 71

480 Kapitel 14.2.9 auf Seite 71

481 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Minimum%20

482 Kapitel 14.2.9 auf Seite 71

483 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Modulo%20

484 Kapitel 14.2.5 auf Seite 71

485 Kapitel 34.1.1 auf Seite 215

486 Kapitel 14.2.5 auf Seite 71

487 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Modulo%20

488 Kapitel 11.1 auf Seite 55

489 Kapitel 10.1 auf Seite 53

490 Kapitel 22.1 auf Seite 133

491 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Rundung%20

492 Kapitel 14.2.2 auf Seite 70

493 Kapitel 10.1 auf Seite 53

494 Kapitel 22.1 auf Seite 133

495 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Bitfunktionen%20

496 Kapitel 31.3 auf Seite 202

497 Kapitel 31.3 auf Seite 202

82.22. O

- Oktalzahl (Fortran 95)⁴⁹⁸
- open (Fortran 95)⁴⁹⁹
- OPEN (FORTRAN 77)⁵⁰⁰
- Operatorenpriorität (FORTRAN 77)⁵⁰¹
- Operatorenpriorität (Fortran 95)⁵⁰²
- .OR. (FORTRAN 77)⁵⁰³
- .OR. (Fortran 95)⁵⁰⁴

82.23. P

- parameter (Fortran 95)⁵⁰⁵
- PARAMETER (FORTRAN 77)⁵⁰⁶
- PAUSE (FORTRAN 77)⁵⁰⁷
- pointer (Fortran 95)⁵⁰⁸
- Positive Differenz (FORTRAN 77)⁵⁰⁹
- Prozedur (Parameter, FORTRAN 77)⁵¹⁰
- Prozedur (Parameter, Fortran 95)⁵¹¹

82.24. Q

- Quadratwurzel (FORTRAN 77)⁵¹²

82.25. R

- Ratfor⁵¹³
- read (Fortran 95)⁵¹⁴
- READ (FORTRAN 77)⁵¹⁵

498 Kapitel 19.1.1 auf Seite 119

499 Kapitel 30.5.3 auf Seite 191

500 Kapitel 16.4.3 auf Seite 95

501 Kapitel 9.1.1 auf Seite 49

502 Kapitel 21.1.1 auf Seite 131

503 Kapitel 10.1 auf Seite 53

504 Kapitel 22.1 auf Seite 133

505 Kapitel 19.3 auf Seite 122

506 Kapitel 7.3 auf Seite 42

507 Kapitel 17.1 auf Seite 105

508 Kapitel 31.2 auf Seite 201

509 Kapitel 14.2.7 auf Seite 71

510 Kapitel 15.5 auf Seite 82

511 Kapitel 29.5 auf Seite 178

512 Kapitel 14.2.11 auf Seite 72

513 Kapitel 57 auf Seite 363

514 Kapitel 30.1 auf Seite 183

515 Kapitel 16.1 auf Seite 89

-
- real (Fortran 95)⁵¹⁶
 - real (Funktion, Fortran 95)⁵¹⁷
 - REAL (FORTRAN 77)⁵¹⁸
 - REAL (Funktion, FORTRAN 77)⁵¹⁹
 - Rekursiver Unterprogrammaufruf (Fortran 95)⁵²⁰
 - reshape (Fortran 95)⁵²¹
 - return (Fortran 95)⁵²²
 - RETURN (FORTRAN 77)⁵²³
 - Rundung (FORTRAN 77)⁵²⁴
 - Rundung (Fortran 95)⁵²⁵

82.26. S

- SAVE (FORTRAN 77)⁵²⁶
- scan (Fortran 95)⁵²⁷
- Schleife (FORTRAN 77)⁵²⁸
- Schleife (Fortran 95)⁵²⁹
- select case (Fortran 95)⁵³⁰
- selected_real_kind (Fortran 95)⁵³¹
- Sequentielle Datei (FORTRAN 77)⁵³²
- Sequentielle Datei (Fortran 95)⁵³³
- sign (Fortran 95)⁵³⁴
- SIGN (FORTRAN 77)⁵³⁵
- sin (Fortran 95)⁵³⁶
- SIN (FORTRAN 77)⁵³⁷

516 Kapitel 19.1.1 auf Seite 119

517 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Datentypfunktionen%23Umwandlung%20in%20REAL%20

518 Kapitel 7.1.1 auf Seite 39

519 Kapitel 14.1.2 auf Seite 69

520 Kapitel 29.8 auf Seite 181

521 Kapitel 20.4.2 auf Seite 129

522 Kapitel 29.3 auf Seite 174

523 Kapitel 15 auf Seite 75

524 Kapitel 14.2.2 auf Seite 70

525 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Rundung%20

526 Kapitel 15.8 auf Seite 85

527 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

528 Kapitel 13.5 auf Seite 64

529 Kapitel 25.4 auf Seite 145

530 Kapitel 25.3 auf Seite 144

531 Kapitel 26.2.1 auf Seite 158

532 Kapitel 16.4.5 auf Seite 99

533 Kapitel 30.5.6 auf Seite 195

534 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Vorzeichentransfer%20

535 Kapitel 14.2.6 auf Seite 71

536 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Winkelfunktionen%20

537 Kapitel 14.2.15 auf Seite 72

- sinh (Fortran 95)⁵³⁸
- SINH (FORTRAN 77)⁵³⁹
- size (Fortran 95)⁵⁴⁰
- SNGL (FORTRAN 77)⁵⁴¹
- Spaltenorganisation⁵⁴²
- sqrt (Fortran 95)⁵⁴³
- SQRT (FORTRAN 77)⁵⁴⁴
- Standardfunktion (FORTRAN 77)⁵⁴⁵
- Standardfunktion (Fortran 95)⁵⁴⁶
- stop (Fortran 95)⁵⁴⁷
- STOP (FORTRAN 77)⁵⁴⁸
- Stringfunktionen (FORTRAN 77)⁵⁴⁹
- Stringfunktionen (Fortran 95)⁵⁵⁰
- subroutine (Fortran 95)⁵⁵¹
- SUBROUTINE (FORTRAN 77)⁵⁵²
- Symbolische Namen (FORTRAN 77)⁵⁵³
- Symbolische Namen (Fortran 95)⁵⁵⁴

82.27. T

- tan (Fortran 95)⁵⁵⁵
- TAN (FORTRAN 77)⁵⁵⁶
- tanh (Fortran 95)⁵⁵⁷
- TANH (FORTRAN 77)⁵⁵⁸

-
- 538 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Hyperbelfunktionen%20
 - 539 Kapitel 14.2.17 auf Seite 73
 - 540 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Abfragen_von_Feldstatus%2C_Felddaten_und_Feldmetadaten%20
 - 541 Kapitel 14.1.2 auf Seite 69
 - 542 Kapitel 6.2 auf Seite 33
 - 543 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Quadratwurzel%20
 - 544 Kapitel 14.2.11 auf Seite 72
 - 545 Kapitel 14 auf Seite 69
 - 546 Kapitel 28 auf Seite 167
 - 547 Kapitel 25.6.1 auf Seite 153
 - 548 Kapitel 13.8 auf Seite 66
 - 549 Kapitel 14.3 auf Seite 73
 - 550 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%20
 - 551 Kapitel 29.3 auf Seite 174
 - 552 Kapitel 15.3 auf Seite 78
 - 553 Kapitel 6.5 auf Seite 36
 - 554 Kapitel 18.5 auf Seite 115
 - 555 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Winkelfunktionen%20
 - 556 Kapitel 14.2.15 auf Seite 72
 - 557 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Hyperbelfunktionen%20
 - 558 Kapitel 14.2.17 auf Seite 73

-
- target (Fortran 95)⁵⁵⁹
 - Tcl/Tk (Fortran und Tcl)⁵⁶⁰
 - Teilfeld (Fortran 95)⁵⁶¹
 - transpose (Fortran 95)⁵⁶²
 - trim (Fortran 95)⁵⁶³
 - type (Fortran 95)⁵⁶⁴

82.28. U

- ubound (Fortran 95)⁵⁶⁵
- Unterprogramm (FORTRAN 77)⁵⁶⁶
- Unterprogramm (Fortran 95)⁵⁶⁷
- use (Fortran 95)⁵⁶⁸

82.29. V

- Variable (FORTRAN 77)⁵⁶⁹
- Variable (Fortran 95)⁵⁷⁰
- Vektorrechnung (BLAS und ATLAS)⁵⁷¹
- Vektorrechnung (Fortran 95)⁵⁷²
- Vergleichsoperator (FORTRAN 77)⁵⁷³
- Vergleichsoperator (Fortran 95)⁵⁷⁴
- Verkettete Liste (Fortran 95)⁵⁷⁵
- Verzweigung (FORTRAN 77)⁵⁷⁶
- Verzweigung (Fortran 95)⁵⁷⁷
- Vorzeichentransfer (FORTRAN 77)⁵⁷⁸

559 Kapitel 31.2 auf Seite 201

560 Kapitel 52 auf Seite 329

561 Kapitel 20.5 auf Seite 129

562 Kapitel 32 auf Seite 211

563 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Stringfunktionen%23Sonstige%20

564 Kapitel 34.1.5 auf Seite 219

565 https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Feldfunktionen%23Abfragen_von_Feldstatus%2C_Felddaten_und_Feldmetadaten%20

566 Kapitel 15 auf Seite 75

567 Kapitel 29 auf Seite 171

568 Kapitel 34 auf Seite 215

569 Kapitel 7.2 auf Seite 40

570 Kapitel 19.2 auf Seite 120

571 Kapitel 46.1 auf Seite 297

572 Kapitel 32 auf Seite 211

573 Kapitel 11.1 auf Seite 55

574 Kapitel 23.1 auf Seite 135

575 Kapitel 31.6 auf Seite 206

576 Kapitel 13 auf Seite 61

577 Kapitel 25 auf Seite 139

578 Kapitel 14.2.6 auf Seite 71

- Vorzeichentransfer (Fortran 95)⁵⁷⁹

82.30. W

- where-elsewhere-Schleife (Fortran 95)⁵⁸⁰
- where (Fortran 95)⁵⁸¹
- "While"-Schleife (FORTRAN 77)⁵⁸²
- Winkelfunktion (FORTRAN 77)⁵⁸³
- Winkelfunktion (Fortran 95)⁵⁸⁴
- write (Fortran 95)⁵⁸⁵
- WRITE (FORTRAN 77)⁵⁸⁶

82.31. X

82.32. Y

82.33. Z

- Zeichenvorrat (FORTRAN 77)⁵⁸⁷
- Zeichenvorrat (Fortran 95)⁵⁸⁸
- Zeichenvorrat (Fortran 2003)⁵⁸⁹
- Zeiger (Fortran 95)⁵⁹⁰
- Zeigerfunktionen (Fortran 95)⁵⁹¹
- Zeilenformat (FORTRAN 77)⁵⁹²
- Zeilenformat (Fortran 95)⁵⁹³

⁵⁷⁹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Vorzeichentransfer%20

⁵⁸⁰ Kapitel 25.5.1 auf Seite 149

⁵⁸¹ Kapitel 25.5.1 auf Seite 149

⁵⁸² Kapitel 13.6.1 auf Seite 65

⁵⁸³ Kapitel 14.2.15 auf Seite 72

⁵⁸⁴ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Mathematische_Funktionen%23Winkelfunktionen%20

⁵⁸⁵ Kapitel 30.2 auf Seite 184

⁵⁸⁶ Kapitel 16.2 auf Seite 91

⁵⁸⁷ Kapitel 6.4 auf Seite 35

⁵⁸⁸ Kapitel 18.4 auf Seite 115

⁵⁸⁹ Kapitel 36.2 auf Seite 241

⁵⁹⁰ Kapitel 31 auf Seite 201

⁵⁹¹ https://de.wikibooks.org/wiki/Fortran%3A_Fortran%2095%3A_Standardfunktionen%3A_Zeigerfunktionen%20

⁵⁹² Kapitel 6.2 auf Seite 33

⁵⁹³ Kapitel 18.2 auf Seite 113

83. Autoren

Edits	User
2	1234qwer1234qwer ¹
7	Akkordeon ²
3	Baumfreund-FFM ³
1	Bennibijah ⁴
1	Bodhi-Baum ⁵
1	Chris7791 ⁶
3	CommonsDelinker ⁷
1	DannyS712 ⁸
1	Der Messer ⁹
116	Dirk Huenniger ¹⁰
3	Erik del Toro Streb ¹¹
1	FilePeter ¹²
1	Fortranplus ¹³
1	Giuseppe.cantarella ¹⁴
2	Gunnar.offel ¹⁵
1	Heftigindenafter ¹⁶
7	HerrHartmuth ¹⁷
2	Heuler06 ¹⁸
758	Intruder ¹⁹

```

1  https://de.wikibooks.org/wiki/Benutzer:1234qwer1234qwer4
2  https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Akkordeon&action=edit&redlink=1
3  https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Baumfreund-FFM&action=edit&
   redlink=1
4  https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Bennibijah&action=edit&redlink=
   1
5  https://de.wikibooks.org/wiki/Benutzer:Bodhi-Baum
6  https://de.wikibooks.org/wiki/Benutzer:Chris7791
7  https://de.wikibooks.org/wiki/Benutzer:CommonsDelinker
8  https://de.wikibooks.org/wiki/Benutzer:DannyS712
9  https://de.wikibooks.org/wiki/Benutzer:Der_Messer
10 https://de.wikibooks.org/wiki/Benutzer:Dirk_Huenniger
11 https://de.wikibooks.org/wiki/Benutzer:Erik_del_Toro_Streb
12 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:FilePeter&action=edit&redlink=1
13 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Fortranplus&action=edit&
   redlink=1
14 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Giuseppe.cantarella&action=
   edit&redlink=1
15 https://de.wikibooks.org/wiki/Benutzer:Gunnar.offel
16 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Heftigindenafter&action=edit&
   redlink=1
17 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:HerrHartmuth&action=edit&
   redlink=1
18 https://de.wikibooks.org/wiki/Benutzer:Heuler06
19 https://de.wikibooks.org/wiki/Benutzer:Intruder

```


- 1 JamesP²⁰
- 8 Juetho²¹
- 4 Klaus Eifert²²
- 1 Kyrsjo²³
- 1 Loehden²⁴
- 43 MichaelFrey²⁵
- 3 Minorax²⁶
- 3 Mjchael²⁷
- 1 Mosbatho²⁸
- 1 Philipendula²⁹
- 1 Qualitätsfreund³⁰
- 2 Robin von Stedow³¹
- 1 RolfSander³²
- 1 Salatgurke~dewikibooks³³
- 3 Stephan Kulla³⁴
- 1 Texvc2LaTeXBot³⁵
- 5 ThePacker³⁶
- 19 Thornard³⁷
- 1 WOSlinker³⁸
- 4 Zase Wieder³⁹

-
- 20 <https://de.wikibooks.org/wiki/Benutzer:JamesP>
 - 21 <https://de.wikibooks.org/wiki/Benutzer:Juetho>
 - 22 https://de.wikibooks.org/wiki/Benutzer:Klaus_Eifert
 - 23 <https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Kyrsjo&action=edit&redlink=1>
 - 24 <https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Loehden&action=edit&redlink=1>
 - 25 <https://de.wikibooks.org/wiki/Benutzer:MichaelFrey>
 - 26 <https://de.wikibooks.org/wiki/Benutzer:Minorax>
 - 27 <https://de.wikibooks.org/wiki/Benutzer:Mjchael>
 - 28 <https://de.wikibooks.org/wiki/Benutzer:Mosbatho>
 - 29 <https://de.wikibooks.org/wiki/Benutzer:Philipendula>
 - 30 <https://de.wikibooks.org/wiki/Benutzer:Qualit%25C3%25A4tsfreund>
 - 31 https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Robin_von_Stedow&action=edit&redlink=1
 - 32 <https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:RolfSander&action=edit&redlink=1>
 - 33 <https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:Salatgurke~dewikibooks&action=edit&redlink=1>
 - 34 https://de.wikibooks.org/wiki/Benutzer:Stephan_Kulla
 - 35 <https://de.wikibooks.org/wiki/Benutzer:Texvc2LaTeXBot>
 - 36 <https://de.wikibooks.org/wiki/Benutzer:ThePacker>
 - 37 <https://de.wikibooks.org/wiki/Benutzer:Thornard>
 - 38 <https://de.wikibooks.org/w/index.php%3ftitle=Benutzer:WOSlinker&action=edit&redlink=1>
 - 39 https://de.wikibooks.org/wiki/Benutzer:Zase_Wieder

Abbildungsverzeichnis

- GFDL: Gnu Free Documentation License. <http://www.gnu.org/licenses/fdl.html>
- cc-by-sa-4.0: Creative Commons Attribution ShareAlike 4.0 License. <https://creativecommons.org/licenses/by-sa/4.0/deed.en>
- cc-by-sa-3.0: Creative Commons Attribution ShareAlike 3.0 License. <http://creativecommons.org/licenses/by-sa/3.0/>
- cc-by-sa-2.5: Creative Commons Attribution ShareAlike 2.5 License. <http://creativecommons.org/licenses/by-sa/2.5/>
- cc-by-sa-2.0: Creative Commons Attribution ShareAlike 2.0 License. <http://creativecommons.org/licenses/by-sa/2.0/>
- cc-by-sa-1.0: Creative Commons Attribution ShareAlike 1.0 License. <http://creativecommons.org/licenses/by-sa/1.0/>
- cc-by-1.0: Creative Commons Attribution 1.0 License. <https://creativecommons.org/licenses/by/1.0/deed.en>
- cc-by-2.0: Creative Commons Attribution 2.0 License. <http://creativecommons.org/licenses/by/2.0/>
- cc-by-2.5: Creative Commons Attribution 2.5 License. <http://creativecommons.org/licenses/by/2.5/deed.en>
- cc-by-3.0: Creative Commons Attribution 3.0 License. <http://creativecommons.org/licenses/by/3.0/deed.en>
- cc-by-4.0: Creative Commons Attribution 4.0 License. <https://creativecommons.org/licenses/by/4.0/deed.de>
- GPL: GNU General Public License. <http://www.gnu.org/licenses/gpl-2.0.txt>
- LGPL: GNU Lesser General Public License. <http://www.gnu.org/licenses/lgpl.html>
- PD: This image is in the public domain.
- ATTR: The copyright holder of this file allows anyone to use it for any purpose, provided that the copyright holder is properly attributed. Redistribution, derivative work, commercial use, and all other use is permitted.
- EURO: This is the common (reverse) face of a euro coin. The copyright on the design of the common face of the euro coins belongs to the European Commission. Authorised is reproduction in a format without relief (drawings, paintings, films) provided they are not detrimental to the image of the euro.

- LFK: Lizenz Freie Kunst. <http://artlibre.org/licence/lal/de>
- FAL: Free Art License <https://artlibre.org/licence/lal/en/>
- CFR: Copyright free use.
- EPL: Eclipse Public License. <http://www.eclipse.org/org/documents/epl-v10.php>

Copies of the GPL, the LGPL as well as a GFDL are included in chapter Licenses⁴⁰. Please note that images in the public domain do not require attribution. You may click on the image numbers in the following table to open the webpage of the images in your webbrowser.

⁴⁰ Kapitel 84 auf Seite 479

1	Tiesse, Lizzyd710, BotMultichill, YaCBot, JarektBot, SieBot, Bernard Piette, Gwern, ArnoldReinhold, Ranveig	PD
2	Skippppp ¹ , Shazz ² , Borkowsk ³ , Qwertyus ⁴ , MáneyMánadóttir ⁵ , Skippppp ⁶ , Shazz ⁷ , Borkowsk ⁸ , Qwertyus ⁹ , MáneyMánadóttir ¹⁰	CC-BY-SA-4.0
3	Surachit ¹¹ , Surachit ¹²	GFDL,CC-BY-SA-3.0
4	Intruder ¹³	PD
5	Intruder ¹⁴	PD
6	Intruder ¹⁵	PD
7	Intruder ¹⁶	PD
8	Intruder ¹⁷	PD
9	Intruder ¹⁸	PD
10	Intruder ¹⁹	PD
11	Courtesy of the Naval Surface Warfare Center, Dahlgren, VA., 1988.	PD
12	Arnold Reinhold	CC-BY-SA-2.5
13	Intruder ²⁰	PD
14	Intruder ²¹	PD
15	Art. Lebedev Studio ²²	PD
16	Ratopi ²³	GFDL,CC-BY-SA-3.0
17	User Arpingstone ²⁴ on en.wikipedia ²⁵	PD
18	Jailbird ²⁶ , Jailbird ²⁷	CC-BY-SA-2.0
19	Intruder ²⁸	PD
20	Stephan Baum ²⁹ (recolored by Mzajac ³⁰ , converted to SVG by Booyabazooka ³¹), Stephan Baum ³² (recolored by Mzajac ³³ , converted to SVG by Booyabazooka ³⁴)	GFDL,CC-BY-SA-3.0
21	Intruder ³⁵	PD
22	Intruder ³⁶	PD
23	Intruder ³⁷	PD
24	Intruder ³⁸	PD
25	Intruder ³⁹	PD
26	Intruder ⁴⁰	PD
27	Intruder ⁴¹	PD
28	Intruder ⁴²	PD
29	Intruder ⁴³	PD
30	Intruder ⁴⁴	PD
31	A.Illigen ⁴⁵ , A.Illigen ⁴⁶	CC-BY-SA-4.0
32	Leipnizkeks	GFDL,CC-BY-SA-3.0
33	Migebert ⁴⁷ , Migebert ⁴⁸	CC-BY-SA-4.0
34	User:Petwoe on German Wikipedia	GFDL,CC-BY-SA-3.0
35	Intruder ⁴⁹	GFDL,CC-BY-SA-3.0
36	Intruder ⁵⁰	PD
37	Intruder ⁵¹	GFDL,CC-BY-SA-3.0
38	Intruder ⁵²	PD
39	Intruder ⁵³	PD

40	Intruder ⁵⁴	PD
41	Intruder ⁵⁵	PD
42	Intruder ⁵⁶	PD
43	Intruder ⁵⁷	PD
44	Intruder ⁵⁸	PD
45	Intruder ⁵⁹	PD
46	Intruder ⁶⁰	PD
47	RokerHRO ⁶¹ in der Wikipedia auf Deutsch ⁶² , RokerHRO ⁶³ at German Wikipedia ⁶⁴	PD
48	Intruder ⁶⁵	GFDL,CC-BY-SA-3.0
49	Intruder ⁶⁶	GFDL,CC-BY-SA-3.0
50	Intruder ⁶⁷	GFDL,CC-BY-SA-3.0
51	Intruder ⁶⁸	PD
52	Intruder ⁶⁹	PD
53	Intruder ⁷⁰ , Intruder ⁷¹	CC-BY-4.0
54	Intruder ⁷²	PD
55	Intruder ⁷³	PD
56	Intruder ⁷⁴	PD
57	Intruder ⁷⁵	PD
58	Intruder ⁷⁶	PD
59	Intruder ⁷⁷	PD
60	Intruder ⁷⁸	PD
61	Intruder ⁷⁹	PD
62	Intruder ⁸⁰	PD
63	Intruder ⁸¹	PD
64	Intruder ⁸²	PD
65	Intruder ⁸³	PD
66	Intruder ⁸⁴	PD
67	Intruder ⁸⁵	PD
68	Intruder ⁸⁶	PD
69	MASA	GFDL,CC-BY-SA-3.0

1 <https://commons.wikimedia.org/w/index.php?title=User:Skippppp&action=edit&redlink=1>
2 <https://pl.wikipedia.org/wiki/User:Shazz>
3 <https://pl.wikipedia.org/wiki/User:Borkowsk>
4 <https://commons.wikimedia.org/w/index.php?title=User:Qwertyus&action=edit&redlink=1>
5 <https://commons.wikimedia.org/w/index.php?title=User:M%C3%A1neyM%C3%A1nad%C3%B3ttir&action=edit&redlink=1>
6 <https://commons.wikimedia.org/w/index.php?title=User:Skippppp&action=edit&redlink=1>
7 <https://pl.wikipedia.org/wiki/User:Shazz>
8 <https://pl.wikipedia.org/wiki/User:Borkowsk>
9 <https://commons.wikimedia.org/w/index.php?title=User:Qwertyus&action=edit&redlink=1>
10 <https://commons.wikimedia.org/w/index.php?title=User:M%C3%A1neyM%C3%A1nad%C3%B3ttir&action=edit&redlink=1>
11 <https://commons.wikimedia.org/wiki/User:Surachit>
12 <https://commons.wikimedia.org/wiki/User:Surachit>
13 <https://en.wikibooks.org/wiki/de:User:Intruder>
14 <https://en.wikibooks.org/wiki/de:User:Intruder>
15 <https://en.wikibooks.org/wiki/de:User:Intruder>
16 <https://en.wikibooks.org/wiki/de:User:Intruder>
17 <https://en.wikibooks.org/wiki/de:User:Intruder>
18 <https://en.wikibooks.org/wiki/de:User:Intruder>
19 <https://en.wikibooks.org/wiki/de:User:Intruder>
20 <https://en.wikibooks.org/wiki/de:User:Intruder>
21 <https://en.wikibooks.org/wiki/de:User:Intruder>
22 <http://www.artlebedev.com/>
23 <https://de.wikipedia.org/wiki/User:Ratopi>
24 <https://en.wikipedia.org/wiki/User:Arpingstone>
25 <https://en.wikipedia.org>
26 <https://commons.wikimedia.org/w/index.php?title=User:Jailbird&action=edit&redlink=1>
27 <https://commons.wikimedia.org/w/index.php?title=User:Jailbird&action=edit&redlink=1>
28 <https://en.wikibooks.org/wiki/de:User:Intruder>
29 <https://commons.wikimedia.org/wiki/User:Baumst~commonswiki>
30 <https://en.wikipedia.org/wiki/User:Mzajac>
31 <https://commons.wikimedia.org/wiki/User:Booyabazooka>
32 <https://commons.wikimedia.org/wiki/User:Baumst~commonswiki>
33 <https://en.wikipedia.org/wiki/User:Mzajac>
34 <https://commons.wikimedia.org/wiki/User:Booyabazooka>
35 <https://en.wikibooks.org/wiki/de:User:Intruder>
36 <https://en.wikibooks.org/wiki/de:User:Intruder>
37 <https://en.wikibooks.org/wiki/de:User:Intruder>
38 <https://en.wikibooks.org/wiki/de:User:Intruder>
39 <https://en.wikibooks.org/wiki/de:User:Intruder>
40 <https://en.wikibooks.org/wiki/de:User:Intruder>
41 <https://en.wikibooks.org/wiki/de:User:Intruder>
42 <https://en.wikibooks.org/wiki/de:User:Intruder>
43 <https://en.wikibooks.org/wiki/de:User:Intruder>
44 <https://en.wikibooks.org/wiki/de:User:Intruder>
45 <https://commons.wikimedia.org/w/index.php?title=User:A.Illigen&action=edit&redlink=1>
46 <https://commons.wikimedia.org/w/index.php?title=User:A.Illigen&action=edit&redlink=1>
47 <https://commons.wikimedia.org/wiki/User:Migebert>
48 <https://commons.wikimedia.org/wiki/User:Migebert>
49 <https://en.wikibooks.org/wiki/de:User:Intruder>
50 <https://en.wikibooks.org/wiki/de:User:Intruder>
51 <https://en.wikibooks.org/wiki/de:User:Intruder>
52 <https://en.wikibooks.org/wiki/de:User:Intruder>

53 <https://en.wikibooks.org/wiki/de:User:Intruder>
54 <https://en.wikibooks.org/wiki/de:User:Intruder>
55 <https://en.wikibooks.org/wiki/de:User:Intruder>
56 <https://en.wikibooks.org/wiki/de:User:Intruder>
57 <https://en.wikibooks.org/wiki/de:User:Intruder>
58 <https://en.wikibooks.org/wiki/de:User:Intruder>
59 <https://en.wikibooks.org/wiki/de:User:Intruder>
60 <https://en.wikibooks.org/wiki/de:User:Intruder>
61 <https://en.wikipedia.org/wiki/de:User:RokerHRO>
62 <https://en.wikipedia.org/wiki/de:>
63 <https://en.wikipedia.org/wiki/de:User:RokerHRO>
64 <https://en.wikipedia.org/wiki/de:>
65 <https://en.wikibooks.org/wiki/de:User:Intruder>
66 <https://en.wikibooks.org/wiki/de:User:Intruder>
67 <https://en.wikibooks.org/wiki/de:User:Intruder>
68 <https://en.wikibooks.org/wiki/de:User:Intruder>
69 <https://en.wikibooks.org/wiki/de:User:Intruder>
70 <https://commons.wikimedia.org/w/index.php?title=User:Intruder&action=edit&redlink=1>
71 <https://commons.wikimedia.org/w/index.php?title=User:Intruder&action=edit&redlink=1>
72 <https://en.wikibooks.org/wiki/de:User:Intruder>
73 <https://en.wikibooks.org/wiki/de:User:Intruder>
74 <https://en.wikibooks.org/wiki/de:User:Intruder>
75 <https://en.wikibooks.org/wiki/de:User:Intruder>
76 <https://en.wikibooks.org/wiki/de:User:Intruder>
77 <https://en.wikibooks.org/wiki/de:User:Intruder>
78 <https://en.wikibooks.org/wiki/de:User:Intruder>
79 <https://en.wikibooks.org/wiki/de:User:Intruder>
80 <https://en.wikibooks.org/wiki/de:User:Intruder>
81 <https://en.wikibooks.org/wiki/de:User:Intruder>
82 <https://en.wikibooks.org/wiki/de:User:Intruder>
83 <https://en.wikibooks.org/wiki/de:User:Intruder>
84 <https://en.wikibooks.org/wiki/de:User:Intruder>
85 <https://en.wikibooks.org/wiki/de:User:Intruder>
86 <https://en.wikibooks.org/wiki/de:User:Intruder>

84. Licenses

84.1. GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. <<http://fsf.org/>>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed. Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program: to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow. TERMS AND CONDITIONS 0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion. 1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work. 2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary. 3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures. 4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee. 5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

* a) The work must carry prominent notices stating that you modified it, and giving a relevant date. * b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices". * c) You must license the entire work, as a whole, under the License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it. * d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is considered "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate. 6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

* a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange. * b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge. * c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b. * d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a

different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements. * e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a product that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying. 7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option receive any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

* a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or * b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or * c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or * d) Limiting the use for publicity purposes of names of licensors or authors of the material; or * e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or * f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way. 8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates

your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10. 9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so. 10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it. 11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law. 12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from

conveying the Program. 13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such. 14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

84.2. GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright (C) 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc. <<http://fsf.org/>>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed. 0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference. 1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The Invariant Sections are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardsly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML, or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML, or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version. 15. Disclaimer of Warranty.

THESE ARE THE TERMS AND CONDITIONS OF THE LICENSE. NO WARRANTY IS GIVEN FOR THE PROGRAM. TO THE EXTENT PERMITTED BY APPLICABLE LAW, EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION. 16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. 17. Interpretation of Sections 15 and 16.

A section Entitled XYZ means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as Acknowledgements, "Dedications", Endorsements, or "History".) To "Preserve the Title" such a section when you modify the Document means that it remains a section Entitled XYZ according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties; any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License. 2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies. 3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first one listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

If it is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document. 4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission. * B. List on the Title Page, authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement. * C. State on the Title page the name of the publisher of the Modified Version, as the publisher. * D. Preserve all the copyright notices of the Document. * E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices. * F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below. * G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice. * H. Include an unaltered copy of this License. * I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence. * J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission. * K. For any section Entitled Acknowledgements or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and

of the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

one of each of the contributor acknowledgments and/or dedications given therein. * L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles. * M. Delete any section Entitled Endorsements". Such a section may not be included in the Modified Version. * N. Do not retile any existing section to be Entitled Endorsements" to conflict in title with any Invariant Section. * O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this license give permission to use their names for publicity or to assert or imply endorsement of any Modified Version. 5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled Endorsements". 6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document. 7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an aggregate if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate. 8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title

You should have received a copy of the GNU General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

<program> Copyright (C) <year> <name of author> This program comes with ABSOLUTELY NO WARRANTY; for details type 'show w'. This is free software, and you are welcome to redistribute it under certain conditions; type 'show c' for details.

The hypothetical commands 'show w' and 'show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <<http://www.gnu.org/licenses/>>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <<http://www.gnu.org/philosophy/why-not-lgpl.html>>.

(section 1) will typically require changing the actual title. 9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it. 10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License or any later version applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document. 11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is eligible for relicensing if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing. ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

Copyright (C) YEAR YOUR NAME. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the "with ... Texts." line with this:

with the Invariant Sections being LIST THEIR TITLES, with the Front-Cover Texts being LIST, and with the Back-Cover Texts being LIST.

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

84.3. GNU Lesser General Public License

GNU LESSER GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. <<http://fsf.org/>>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

This version of the GNU Lesser General Public License incorporates the terms and conditions of version 3 of the GNU General Public License, supplemented by the additional permissions listed below.

0. Additional Definitions.

As used herein, “this License” refers to version 3 of the GNU Lesser General Public License, and the “GNU GPL” refers to version 3 of the GNU General Public License.

“The Library” refers to a covered work governed by this License, other than an Application or a Combined Work as defined below.

An “Application” is any work that makes use of an interface provided by the Library, but which is not otherwise based on the Library. Defining a subclass of a class defined by the Library is deemed a mode of using an interface provided by the Library.

A “Combined Work” is a work produced by combining or linking an Application with the Library. The particular version of the Library with which the Combined Work was made is also called the “Linked Version”.

The “Minimal Corresponding Source” for a Combined Work means the Corresponding Source for the Combined Work, excluding any source code for portions of the Combined Work that, considered in isolation, are based on the Application, and not on the Linked Version.

The “Corresponding Application Code” for a Combined Work means the object code and/or source code for the Application, including any data and utility programs needed for reproducing the Combined Work from the Application, but excluding the System Libraries of the Combined Work.

1. Exception to Section 3 of the GNU GPL.

You may convey a covered work under sections 3 and 4 of this License without being bound by section 3 of the GNU GPL.

2. Conveying Modified Versions.

If you modify a copy of the Library, and, in your modifications, a facility refers to a function or data to be supplied by an Application that uses the facility (other than as an argument passed when the facility is invoked), then you may convey a copy of the modified version:

* a) under this License, provided that you make a good faith effort to ensure that, in the event an Application does not supply the function or data, the facility still operates, and performs whatever part of its purpose remains meaningful, or

* b) under the GNU GPL, with none of the additional permissions of this License applicable to that copy.

3. Object Code Incorporating Material from Library Header Files.

The object code form of an Application may incorporate material from a header file that is part of the Library. You may convey such object code under terms of your choice, provided that, if the incorporated material is not limited to numerical parameters, data structure layouts and accessors, or small macros, inline functions and templates (ten or fewer lines in length), you do both of the following:

* a) Give prominent notice with each copy of the object code that the Library is used in it and that the Library and its use are covered by this License.

* b) Accompany the object code with a copy of the GNU GPL and this license document.

4. Combined Works.

You may convey a Combined Work under terms of your choice that, taken together, effectively do not restrict modification of the portions of the Library contained in the Combined Work and reverse engineering for debugging such modifications, if you also do each of the following:

* a) Give prominent notice with each copy of the Combined Work that the Library is used in it and that the Library and its use are covered by this License.

* b) Accompany the Combined Work with a copy of the GNU GPL and this license document.

* c) For a Combined Work that displays copyright notices during execution, include the copyright notice for the Library among these notices, as well as a reference directing the user to the copies of the GNU GPL and this license document.

* d) Do one of the following:

- o 0) Convey the Minimal Corresponding Source under the terms of this License, and the Corresponding Application Code in a form suitable for, and under terms that permit, the user to recombine or relink the Application with a modified version of the Linked Version to produce a modified Combined Work, in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.
- o 1) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (a) uses at run time a copy of the Library already present on the user's computer system, and (b) will operate properly with a modified version of the Library that is interface-compatible with the Linked Version.

* e) Provide Installation Information, but only if you would otherwise be required to provide such information under section 6 of the GNU GPL, and only to the extent that such information is necessary to install and execute a modified version of the Combined Work produced by recombining or relinking the Application with a modified version of the Linked Version. (If you use option 4d0, the Installation Information must accompany the Minimal Corresponding Source and Corresponding Application Code. If you use option 4d1, you must provide the Installation Information in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.)

5. Combined Libraries.

You may place library facilities that are a work based on the Library side by side in a single library together with other library facilities that are not Applications and are not covered by this License, and convey such a combined library under terms of your choice, if you do both of the following:

* a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities, conveyed under the terms of this License.

* b) Give prominent notice with the combined library that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

6. Revised Versions of the GNU Lesser General Public License.

The Free Software Foundation may publish revised and/or new versions of the GNU Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library as you received it specifies that a certain numbered version of the GNU Lesser General Public License “or any later version” applies to it, you have the option of following the terms and conditions either of that published version or of any later version published by the Free Software Foundation. If the Library as you received it does not specify a version number of the GNU Lesser General Public License, you may choose any version of the GNU Lesser General Public License ever published by the Free Software Foundation.

If the Library as you received it specifies that a proxy can decide whether future versions of the GNU Lesser General Public License shall apply, that proxy's public statement of acceptance of any version is permanent authorization for you to choose that version for the Library.